PYTHON DATA SCIENCE · COMPLETE COURSE WORKBOOK

Python Data Science

Your working notebook for the whole course, all 21 sessions. Read the key ideas, study the worked examples, do the guided labs, predict before you run, and check yourself with the self-check quizzes.

21 sessions · 41 lessons | Write-in exercises & self-checks | Name: _____

CONTENTS

S01 **Toolkit and First Arrays**

- 0.1 Your Data Science Toolkit
- 1.1 Arrays vs Lists

S02 **Vectorized Operations and Broadcasting**

- 1.2 Vectorized Operations
- 1.3 Broadcasting

S03 **Randomness and Linear Algebra**

- 1.4 Random Number Generation
- 1.5 Linear Algebra Essentials

S04 **DataFrames and Loading Data**

- 2.1 DataFrames and Series
- 2.2 Loading Data

S05 **Cleaning and Grouping**

- 2.3 Cleaning Data
- 2.4 GroupBy Operations

S06 **Merging and Reshaping**

- 2.5 Merging and Joining
- 2.6 Pivot Tables and Reshaping

S07 **Time Series and Windows**

- 2.7 Time Series
- 2.8 Window Functions

S08 **Features and First Charts**

- 2.9 Feature Engineering Basics
- 3.1 Line, Scatter and Bar Charts

S09 **Composing and Customizing Figures**

- 3.2 Subplots and Composition
- 3.3 Customizing Plots

S10 **Saving Figures; Center and Spread**

- 3.4 Saving Figures
- 4.1 Descriptive Statistics I: Center and Spread

S11 **Shape, Outliers and Distributions**

- 4.2 Descriptive Statistics II: Shape, Position and Outliers
- 4.3 Probability Distributions

S12 **Correlation and Hypothesis Tests**

- 4.4 Correlation Analysis
- 4.5 Hypothesis Testing

S13 **A/B Testing and the EDA Workflow**

- 4.6 A/B Testing
- 4.7 An EDA Workflow

S14 **SQL from Python: Queries and Joins**

- 5.1 Database Connections
- 5.2 Complex Queries

S15 **SQL Windows and Query Speed**

- 5.3 SQL Window Functions
- 5.4 Query Optimization

S16 **Documents and HTTP**

- 5.5 NoSQL with MongoDB
- 6.1 HTTP with Requests

S17 **Scraping Static and Dynamic Pages**

- 6.2 Parsing HTML with Beautiful Soup
- 6.3 Dynamic Pages with Selenium

S18 **APIs: Keys, Tokens and Polite Clients**

6.4 API Authentication

6.5 Rate Limiting and Retries

S19 **Seaborn and Plotly**

7.1 Statistical Graphics with Seaborn

7.2 Interactive Charts with Plotly

S20 **Bokeh and Streamlit**

7.3 Interactive Charts with Bokeh

7.4 Dashboards with Streamlit

S21 **Capstone Studio and Showcase**

8.1 The Pinecrest Report

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 1: Toolkit and First Arrays

Your working notebook for the first session of Python Data Science. First you check that your computer is ready and warm up your Python. Then you put the toolkit to work on 20,302 orders from Pinecrest Outfitters, and meet the NumPy array: the structure underneath pandas, Matplotlib and nearly everything else in this course.

Lessons 0.1–1.1 | Session 1 · 3 hours | Name: _____

HOW TO USE THIS WORKBOOK

Each lesson has **By the end you can** (your goals), **Key ideas** (the page to keep), a **Worked example** (new code with its real output), the **Guided lab** you do in class, **Predict, then run** items, the lesson **project** (started in class, finished at home), a six-question **Self-check** and the **Journal** prompt. Lab files are in `labs/module-01/`: open the `_starter.py` file, never the solution, and run it once before you change anything. Write in this workbook. Bring it every session.

SESSION GOALS

- Show that your computer is ready: `check_environment.py` ends with READY for Python Data Science.
- Run code in a notebook and as a script, and get back to a working `(.venv)` prompt without help.
- Load a real file with a relative path, pick a column, count its values and filter its rows.
- Read a traceback from the bottom and take a sensible next step.
- Create NumPy arrays, read their dtype and shape, and select rows, columns and cells.
- Predict when a slice is a view, and make a copy when you need one.
- Start both lesson projects: `first_look.py` and the list-versus-array benchmark report.

START HERE: IS YOUR TOOLKIT READY?

ENVIRONMENT CHECK

- ☐ I can open a terminal and get to my project folder: `cd pinecrest-analysis`
- ☐ My prompt starts with `(.venv)` after I activate. My activate command is:
  _____
- ☐ `python check_environment.py` ends with `READY for Python Data Science`

- ☐ If it said NOT READY, I copied the numbered fix list here and will do it before Session 2:

The five steps for every time you sit down: **1** open a terminal · **2** `cd pinecrest-analysis` · **3** activate, look for `(.venv)` · **4** start JupyterLab or VS Code from this folder · **5** in a notebook, check that the kernel is the `.venv` one.

DIAGNOSTIC: PREDICT, THEN CHECK

Six questions on the Python this course builds on. Circle your answer before the class reveals it. Nobody collects this. Missed one? The skill and what to review: **D1** slicing (Intro Python: lists and slicing) · **D2** names and copies (Intro: lists are mutable) · **D3** dictionaries (Intro: dictionaries and `.get()`) · **D4** functions (Intro: return values and default arguments) · **D5** comprehensions (Intro: list comprehensions) · **D6** text from files (Intermediate: reading files). For more practice at home, fill in `labs/module-01/l0_1-orientation/diagnostic_starter.py`: it checks itself.

D1

What does this print?

```
days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
print(days[-2:], days[1:3])
```

A) `['Sat', 'Sun'] ['Tue', 'Wed']`

B) `['Sun'] ['Tue', 'Wed', 'Thu']`

C) `['Sat', 'Sun'] ['Mon', 'Tue', 'Wed']`

D) `['Fri', 'Sat'] ['Tue', 'Wed']`

D2

What does this print?

```
a = [1, 2, 3]
b = a
c = a[:]
b.append(4)
print(a, c)
```

A) `[1, 2, 3] [1, 2, 3]`

B) `[1, 2, 3, 4] [1, 2, 3, 4]`

C) `[1, 2, 3, 4] [1, 2, 3]`

D) `[1, 2, 3] [1, 2, 3, 4]`

D3

What does this print?

```
stock = {"tent": 4, "stove": 0}
stock["lamp"] = stock.get("lamp", 0) + 2
print(len(stock), stock["lamp"])
```

A) `2 2`

B) `3 2`

C) A `KeyError`

D) `3 None`

D4

What does this print?

```
def with_tax(price, rate=0.08):
    return round(price * (1 + rate), 2)

print(with_tax(50), with_tax(50, rate=0.10))
```

A) `54.0 54.0`

B) `4.0 5.0`

C) `None None`

D) `54.0 55.0`

D5

What does this print?

```
totals = [434.90, 75.90, 399.80, 26.71]
big = [t for t in totals if t >= 100]
print(len(big), max(big))
```

A) `2 434.9`

B) `4 434.9`

C) `2 399.8`

D) `3 434.9`

D6

A row read from a CSV file with the `csv` module looks like this. What does the code print?

```
row = {"store": "Boise", "revenue": "5962.0"}
print(row["revenue"] * 2)
```

A) `11924.0`

B) `5962.05962.0`

C) A `TypeError`

D) `5962.0 2`

LESSONS

0.1 Your Data Science Toolkit

BY THE END YOU CAN

- Install Python 3.12 or newer and create a virtual environment for the course on Windows, macOS or Linux
- Install the course packages from a `requirements.txt` file and confirm they work
- Run code both in a notebook and as a script, and say when each one suits the job
- Download the course data and load a file from it with a relative path
- Read a Python traceback from the bottom up and take a sensible next step when you are stuck

KEY IDEAS

- **One project, one environment.** `.venv` holds this course's packages and nothing else. Activate it in every new terminal and look for `(.venv)`.
- **Install once, from the file.** `python -m pip install -r requirements.txt`. The `~=` pins keep you on the versions the lessons were tested with.
- **Notebook to explore, script to keep.** A notebook remembers variables between cells; a script starts fresh and shows only what you `print()`.
- **Relative paths start where Python is working.** `"data/orders.csv"` only works when you start from the project folder.
- **Three pandas moves for today:** read a file, pick a column with square brackets, then count (`value_counts()`), summarize (`describe()`) or filter (`orders[condition]`).
- **Read a traceback from the bottom.** Last line: what went wrong. Then find the line in your own file.
- **An unusual value is a question, not automatically a mistake.**

YOU WANT TO...	WRITE
Turn the environment on (Windows)	`.venv\Scripts\Activate.ps1`
Turn the environment on (macOS, Linux)	`source .venv/bin/activate`
Start a notebook	`jupyter lab` (from the project folder)
Run a script	`python first_look.py`
Ask where Python is working	`print(Path.cwd())`

YOU WANT TO...	WRITE
Read a CSV file	`orders = pd.read_csv("data/orders.csv")`
Size of a table	`orders.shape`
One column	`orders["order_total"]`
Count each value	`orders["status"].value_counts()`
Keep some rows	`orders[orders["status"] == "completed"]`

WORKED EXAMPLE: WHO PAYS FOR SHIPPING?

A new question for the three moves. Shipping is charged on some online orders. How many orders paid each amount, and what share of online orders shipped free?

```
import pandas as pd

orders = pd.read_csv("data/orders.csv", parse_dates=["order_date"])
print(orders["shipping_cost"].value_counts())
```

```
shipping_cost
0.00     16624
6.95      2754
14.95      924
Name: count, dtype: int64
```

Three shipping prices: free, $6.95 and $14.95. But 16,624 free orders is more than all the online orders there are, because store orders never have shipping. The question is about online orders, so filter first.

```
online = orders[orders["channel"] == "online"]
free = online["shipping_cost"] == 0

print(len(online), "online orders")
print(free.sum(), "shipped free:", round(free.mean() * 100, 1), "percent")
```

```
12854 online orders
9176 shipped free: 71.4 percent
```

`free` is a column of True and False. Its `.sum()` counts the True values and its `.mean()` is the fraction that is True. Read the result as a sentence: about 71% of online orders shipped free.

GUIDED LAB

Open `labs/module-01/l0_1-orientation/payment_methods_starter.py` in VS Code or JupyterLab (the `# %%` lines are cells). Run it once before you change anything: it should print five TODO lines and no error.

Work through TODO 1 to 5. The question: **how do Pinecrest customers pay, and is one payment method special?** Each TODO uses a move from the worked example above.

You are done when it prints the table size, four payment methods with their counts, the digital-wallet share, the number and average size of gift-card orders, and where cash is used. Write your one-sentence finding about cash here:

Finished early? `labs/module-01/l0_1-orientation/debug_traceback.py` has three bugs. Fix them one traceback at a time: last line first.

◆ PREDICT, THEN RUN

Write what you think each one prints **before** you run it. Being wrong here is useful: it shows you exactly which idea to look at again.

P1

Two numbers. Which one is bigger, and roughly by how much?

```
print(len(orders), len(orders[orders["channel"] == "store"]))
```

P2

The file covers two calendar years. What does this print?

```
print(orders["order_date"].min().year, orders["order_date"].max().year)
```

P3

This line has a mistake in it. Predict the kind of error and its message.

```
orders["Status"]
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Project: `first_look.py`, a summary of the orders file and a chart of completed revenue by month (lesson page: "Project: First Look at Pinecrest").

In class: Step 1 (load, count, first and last date) and Step 2 (count `channel` and `status`). Check: 20,302 orders, 2024-01-01 to 2025-12-31.

At home: Steps 3 to 6. Check yourself: completed revenue $3,624,306.27, best month 2025-06, and a file `monthly_revenue.png` with 24 bars. Then run `python first_look.py` from a terminal.

SELF-CHECK

Six questions on this lesson. Circle or write your answer, then check with the class.

Q1

Your terminal prompt starts with `(.venv)`. What does that tell you?

A) Python is running inside a notebook

B) This terminal will use the project's own Python and the packages installed for it

C) The course packages are being downloaded right now

D) The `data` folder was found

Q2

Which command installs every package the course needs, using the `pip` that belongs to the active environment?

A) `pip install requirements.txt`

B) `python requirements.txt`

C) `python -m venv requirements.txt`

D) `python -m pip install -r requirements.txt`

Q3

The status column of `orders` holds completed 18,762 times, returned 1,143 times and cancelled 397 times. What does this print?

```
print(len(orders[orders["status"] == "returned"]))
```

Q4

A script stops with this traceback. What is the most useful first step?

```
Traceback (most recent call last):
  File "/home/you/pinecrest-analysis/report.py", line 4, in <module>
    print(orders["Order Total"].sum())
  ...
KeyError: 'Order Total'
```

A) Print `orders.columns` and copy the exact column name from there

B) Reinstall pandas, because the error happened inside it

C) Restart the computer and run the script again

D) Change the path to use backslashes

Q5

In a notebook, cell 1 loads `orders` and cell 3 prints `orders.shape`. You restart the kernel and run only cell 3. What happens?

A) Cell 3 prints the shape, because the notebook file remembers the table

B) A `FileNotFoundError`, because the data folder is closed

C) A `NameError`: the name `orders` is not defined

D) Nothing is printed and there is no error

Q6

Python is working in your project folder, which contains `data/orders.csv` (and no file called order.csv). What does this print?

```
from pathlib import Path

print(Path("data/orders.csv").exists(), Path("data/order.csv").exists())
```

JOURNAL

This lesson's prompt (from the lesson page): describe, in your own words, what a virtual environment is and what would go wrong without one. Then write down the exact commands you need each time you sit down to work on this course on your computer. Finally: what is one question about Pinecrest's sales you would like to be able to answer by the end of the course?

1.1 Arrays vs Lists

BY THE END YOU CAN

- Explain how a NumPy array stores numbers differently from a Python list, and why that makes it smaller and faster
- Create arrays from lists, from scratch, and from a CSV file, and read their `dtype`, `shape`, `ndim` and `size`
- Select elements, rows and columns with indexing and slicing in one and two dimensions
- Predict whether an operation gives you a view or a copy, and make a copy when you need one
- Measure memory and speed yourself and choose between a list and an array for a task

KEY IDEAS

- **A list is an address book; an array is one block of values.** The list keeps an address for every item and every item is a full Python object. The array stores the type once, then only the numbers.
- **Three rules:** one type for the whole array (the `dtype`), a fixed size, and operations that act on every element at once.
- ** `* 2` doubles an array; it repeats a list.** The same is true for `+`: arrays add element by element, lists join.
- **Mixed values are converted silently.** One text value makes the whole array text. A decimal stored into an integer array is chopped. When in doubt, print `.dtype`.
- **Rows, comma, columns.** `table[row, col]`; a lone `:` means all of them. Print `.shape` when you are not sure which is which.
- **A slice is a window, not a photocopy.** Writing to a slice changes the original. `.copy()` gives you an independent array; `np.shares_memory(a, b)` settles any doubt.
- **Arrays do not grow.** Collect in a list, convert once, compute on the array.
- ** `nan` means a missing value got into the calculation.** `np.isnan(x).sum()` counts them; `np.nanmean`, `np.nansum`, `np.nanmax` and `np.nanmin` skip them.

YOU WANT TO...	WRITE
Make an array from a list	`week = np.array(week_list)`
Describe an array	`week.dtype`, `week.shape`, `week.ndim`, `week.size`, `week.nbytes`
Build one from scratch	`np.zeros(5)`, `np.arange(0, 10, 2)`, `np.linspace(0, 1, 5)`

YOU WANT TO...	WRITE
Load one CSV column	`np.genfromtxt("data/daily_sales.csv", delimiter=",", skip_header=1, usecols=5)`
Change the type (new array)	`counts.astype(np.float64)`
One cell, one row, one column	`t[0, 2]`, `t[1]`, `t[:, 2]`
Give 28 values a table shape	`revenue[:28].reshape(7, 4)`
An independent copy	`a[1:4].copy()`
Mean that skips missing values	`np.nanmean(revenue)`

WORKED EXAMPLE: TWO YEARS OF HIGH TEMPERATURES AS A TABLE

`data/store_weather.csv` has one row per city per day: 731 days for each of four cities, sorted by city (Boise, Bend, Flagstaff, Asheville), then by date. Column 2 is the day's high in °F. Because every city has the same number of rows and nothing is missing, the whole column fits a 4 by 731 table.

```
import numpy as np

high = np.genfromtxt("data/store_weather.csv", delimiter=",", skip_header=1, usecc
print(high.shape, high.dtype, np.isnan(high).sum())
```

```
(2924,) float64 0
```

2,924 values, all `float64` (that is what `genfromtxt` gives unless told otherwise), none missing.

```
temps = high.reshape(4, 731)        # rows = cities, columns = days
print(temps.shape)
print(temps[0, :7])                 # Boise, first week of January 2024
print(temps[:, 0])                  # every city on 1 January 2024
```

```
(4, 731)
[37. 44. 30. 26. 29. 27. 32.]
[37. 37. 38. 49.]
```

Rows, comma, columns: `temps[0, :7]` is row 0 (Boise) and the first seven columns (days). `temps[:, 0]` is every row, column 0: one value per city. Boise's week matches the temperatures in the lesson's

Exercise 1.

```
print(temps[2].mean().round(1))     # Flagstaff, average high over two years
print(temps.max(), temps.min())     # hottest and coldest high anywhere
print(np.shares_memory(temps, high))
```

```
60.0
112.0 10.0
True
```

`temps[2]` is all of Flagstaff's row. The last line prints `True`: reshaping gave the same memory a new shape, so `temps` is a view of `high`, not a copy.

GUIDED LAB

Lab A (everyone): open `labs/module-01/l1_1-arrays-vs-lists/transactions_week_starter.py`. You load the `transactions` column, describe the array, and slice out the second week of January 2024 as a table of 7 days by 4 stores. Rows are Mon=0 to Sun=6; columns are Asheville=0, Bend=1, Boise=2, Flagstaff=3. You are done when you have printed Saturday's row, Bend's column, Boise on Saturday, the busiest store-day and the bytes used as `float64` and as `int16`.

Lab B (pairs): open `labs/module-01/l1_1-arrays-vs-lists/what_if_copy_starter.py`. A manager asks what week 1 would have looked like with 10 more units a day at Bend. Answer on a copy and prove the real numbers did not change: Bend's real total must print as 868 every time. One of you types; the other predicts each `np.shares_memory` answer before it runs.

Before you type anything in Lab A, fill in this map of the table. Write Mon to Sun down the side and the four stores across the top, then mark what `week2[5]`, `week2[:, 1]` and `week2[5, 2]` select:

Finished early? `labs/module-01/l1_1-arrays-vs-lists/debug_silent_traps.py` prints three wrong answers without a single error message. Find the one line to change in each block.

PREDICT, THEN RUN

Write what you think each one prints **before** you run it. Being wrong here is useful: it shows you exactly which idea to look at again.

P1

What does this print?

```
x = np.array([1, 2, 3])
print(x + x, [1, 2, 3] + [1, 2, 3])
```

P2

What does this print?

```
g = np.arange(6).reshape(2, 3)
row = g[0]
row[:] = 9
print(g)
```

P3

What does this print?

```
z = np.zeros(3, dtype=np.int64)
z[0] = 2.99
z[1] = True
print(z)
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Project: the benchmark report (lesson page: "Project: The Benchmark Report"). You do the same analysis with lists and with arrays and report your own measurements.

In class: Step 1 (load `visitors` and `revenue`; both `(2903,) float64`) and Step 2 (list twins and the memory comparison: 92,952 bytes against 23,224).

At home: Steps 3 to 6. Check yourself: 6 missing days, mean spend per visitor $20.87 by two methods, `week1` has shape `(7, 4)` and Boise's column totals $55,400.22. Your timing line will not match the page: that is expected.

Keep this notebook. Session 2 starts from your `revenue` and `visitors` arrays.

◆ SELF-CHECK

Six questions on this lesson. Circle or write your answer, then check with the class.

Q7

What does this print?

```
a = [1, 2, 3]
b = np.array([1, 2, 3])
print(a * 2, b * 2)
```

A) `[1, 2, 3, 1, 2, 3] [2 4 6]`

B) `[2, 4, 6] [2 4 6]`

C) `[1, 2, 3, 1, 2, 3] [1 2 3 1 2 3]`

D) A `TypeError`

Q8

What does this print?

```
t = np.arange(12).reshape(3, 4)
print(t[1, 2], t[:, 0])
```

Q9

What does this print?

```
a = np.array([5, 6, 7, 8])
b = a[1:3].copy()
b[0] = 0
print(a)
```

A) `[5 0 7 8]`

B) `[5 6 7 8]`

C) `[0 6 7 8]`

D) `[5 0 0 8]`

Q10

What is the dtype of `np.array([1, 2.5, 3])`?

```
print(np.array([1, 2.5, 3]).dtype)
```

A) `int64`

B) `object`

C) `<U32`

D) `float64`

Q11

What does this print?

```
r = np.array([10.0, np.nan, 30.0])
print(r.mean(), np.nanmean(r))
```

Q12

Which job suits a Python list better than a NumPy array?

A) Raising two million prices by 8%

B) Totaling each column of a days-by-stores table

C) Collecting sensor readings one at a time when you do not know how many there will be

D) Storing five million revenue figures in as little memory as possible

JOURNAL

This lesson's prompt (from the lesson page): explain the parking lot and the starting grid in your own words to someone who has never programmed. Then record the memory ratio and the speedup you measured in the project. Did either number surprise you?

EXIT TICKET

Three questions, no notes. Write your answers, hand them in or keep them: they tell you and your instructor what to revisit.

E1

Your prompt does not start with `(.venv)`. What will probably go wrong, and what do you type to fix it?

E2

`a = np.array([1, 2, 3])`, then `b = a[:2]` and `b[0] = 9`. What is `a` now, and what one change would have protected it?

E3

The mean of a revenue array prints `nan`. What does that tell you, and what do you use instead?

NOTES

HOMEWORK CHECKLIST

- ☐ Finish `first_look.py` (Steps 3 to 6) and run it from a terminal. My completed revenue: $ _____
- ☐ Finish the benchmark report (Steps 3 to 6). My memory ratio: _____ x My speedup: _____ x
- ☐ Lesson 0.1 exercises 1 to 3 (4 is a stretch) and Lesson 1.1 exercises 1 to 3 (4 is a stretch)
- ☐ Both Quick Quizzes on the lesson pages
- ☐ Journal: both entries in `journal.md`, including the exact commands I need each time I sit down
- ☐ Before Session 2: load the `transactions` column (column 3) and print its shape, dtype, minimum and maximum
- ☐ Before Session 2: predict `np.array([1, 2, 3]) + np.array([10, 20, 30])`, then run it. My prediction: _____

- ☐ If my check said NOT READY: fix list done and the check run again

BRING TO SESSION 2

- Your laptop, with the environment check saying READY (nothing new to install)
- Your `pinecrest-analysis` folder with `data/` and `labs/` in it
- Your finished Lesson 1.1 project notebook: Session 2 uses its `revenue` and `visitors` arrays
- This workbook and your journal

GLOSSARY

Virtual environment	A folder (`.venv`) holding one project's own copy of Python packages, separate from every other project.
Package	A library someone else wrote that you install and import, such as NumPy or pandas.
requirements.txt	A text file listing the packages a project needs, one per line, usually with version pins.
Notebook	A file of cells (`.ipynb`) that you run one at a time, with each result shown under its cell.
Kernel	The Python process that runs a notebook's cells. It must be the course environment's Python.
Script	A `.py` file that runs from top to bottom, starting fresh every time.
Relative path	A file location given from the folder Python is working in, such as `data/orders.csv`.
Traceback	The report Python prints when an error stops it. The last line names the error.
DataFrame	pandas' table: rows and named columns. Taught properly in Module 2.
Array (ndarray)	NumPy's container: many values of one type in one block of memory.
dtype	The single type shared by every element of an array, such as `float64` or `int64`.
shape	The length of an array along each dimension, as a tuple: `(7,)` or `(3, 4)`.
Vectorization	Doing an operation on a whole array in one expression instead of a Python loop.
View	An array that shares memory with another array. Writing to one changes the other.
Copy	An independent array with its own memory, made with `.copy()`.
nan	"Not a number": NumPy's marker for a missing value. Arithmetic that touches it gives `nan`.

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 2: Vectorized Operations and Broadcasting

Your working notebook for Session 2. In the first half you stop writing loops over numbers: you describe what you want done to a whole array, and answer two-part questions with masks. In the second half you learn the two rules that let NumPy combine a table with a row or a column, so you can compare each store with its own normal in one line.

Lessons 1.2-1.3 | Session 2 · 3 hours | Name: _____

HOW TO USE THIS WORKBOOK

Each lesson has **By the end you can**, **Key ideas**, a **Worked example** (new code with its real output), the **Guided lab** you do in class, **Predict, then run** items, the lesson **project** (started in class, finished at home), a six-question **Self-check** and the **Journal** prompt. Lab files are in `labs/module-02/` : open the `_starter.py` file and run it once before you change anything. Today's habit: say the shape or the count you expect **before** you run a line, and check it after.

SESSION GOALS

- Do arithmetic between arrays that line up by position, and use ufuncs such as `np.maximum` and `np.abs`.
- Build a mask, then count with it, filter with it, and combine two masks with `&`, `|` and `~`.
- Choose values by condition with `np.where` and `np.select`.
- Summarize a table along rows or columns and predict the shape of the result.
- State the two broadcasting rules and predict a result shape, or an error, on paper.
- Combine a table with a row and with a column (`keepdims=True`), and build a grid from two flat arrays.
- Catch the mistakes that raise no error by checking shapes and adding up the pieces.
- Start both lesson projects: the store scorecard and the weekday index.

WARM-UP: WHAT DO YOU REMEMBER FROM SESSION 1?

Five questions, no notes. Write a short answer to each, then check with the class.

W1

Name the three rules that are true of every NumPy array and not of a Python list.

W2

What does `[1, 2, 3] * 2` give? What does `np.array([1, 2, 3]) * 2` give?

W3

`b = a[1:4]`, then `b[0] = 0`. What happened to `a`, and how do you prevent it?

W4

The mean of the revenue array printed `nan`. Why, and what did you use instead?

W5

`t` has shape `(7, 4)`: days by stores. What does `t[:, 2]` select, and what is its shape?

LESSONS

1.2 Vectorized Operations

BY THE END YOU CAN

- Combine arrays element by element with arithmetic and with universal functions such as `np.sqrt`, `np.log10` and `np.maximum`
- Build boolean masks with comparisons, combine them with `&`, `|` and `~`, and use them to count, filter and assign
- Choose values by condition with `np.where` and `np.select`
- Summarize a two-dimensional array along rows or columns with `axis=`, and predict the shape of the result
- Rewrite a Python loop over numbers as an array expression and check that both give the same answer

KEY IDEAS

- **Vectorized** means one operation on a whole array, with the loop running inside NumPy's compiled code.
- **Parallel arrays** line up by position: element 0 of every column describes the same store-day. A mask built from one column can filter another.
- **A comparison makes a mask** of True and False. Its **sum counts**, its **mean is a fraction**, and inside square brackets it **filters**.
- **Combine masks with `&` (and), `|` (or), `~` (not)**, and put parentheses around every comparison. Never `and` / `or`.
- ** **`np.where(cond, a, b)` is an if/else for every element.** `np.select` ** handles three or more outcomes; the first true condition wins.
- **The axis you name is the axis that disappears.** For a days-by-stores table, `axis=0` gives one value per store and `axis=1` one per day. Check the shape of the result.
- ** `argmax` and `argsort` return positions.** Use a position to look something up in a parallel array: `days[week.argmax()]`.
- ** `nan` fails every comparison**, so missing values silently drop out of both groups. Test with `np.isnan` on purpose.
- **A mask gives a copy; a slice gives a view.** Assign through the mask (`x[x > 10] = 10`) to change the array itself.

YOU WANT TO...	WRITE
In a loop: `for x in values: out.append(x * k)`	`values * k`
`if cond: count += 1`	`(cond).sum()`
`if cond: keep.append(x)`	`values[cond]`
`if x > limit: ... else: ...`	`np.where(values > limit, ..., ...)`
`if / elif / else`	`np.select([...], [...], default=...)`
`max(0, x)` for each x	`np.maximum(values, 0)`
Both conditions true	`(a > 1) & (b == 2)`
One total per column / per row	`t.sum(axis=0)` / `t.sum(axis=1)`
Where is the largest?	`x.argmax()`
Tally the distinct values	`np.unique(labels, return_counts=True)`

WORKED EXAMPLE: DO SALE DAYS MEAN BIGGER BASKETS?

On sale days the stores take in almost twice the revenue. Is that because each customer buys more items, or because there are more customers? `units / transactions` is the number of items in an average purchase.

```
units = load(4)
transactions = load(3)
promo = load(7)

per_sale = units / transactions            # items per purchase, every store-day
print(per_sale[:4].round(2))
print(per_sale.mean().round(2), per_sale.min().round(2), per_sale.max().round(2))
```

```
[2.16 2.02 2.18 2.09]
2.11 1.68 2.68
```

One division gives 2,903 answers. A typical purchase is about two items, and the figure never strays far: 1.68 to 2.68.

```
is_promo = promo == 1

print(is_promo.sum(), "sale store-days")
print(per_sale[is_promo].mean().round(2), "items per purchase on sale days")
print(per_sale[~is_promo].mean().round(2), "items per purchase on ordinary days")
```

```
96 sale store-days
2.11 items per purchase on sale days
2.11 items per purchase on ordinary days
```

The mask is built from `promo` and used on `per_sale`: parallel arrays. The two averages are the same to two decimals. Sale days do not make baskets bigger.

```
print(transactions[is_promo].mean().round(1), "purchases on a sale day")
print(transactions[~is_promo].mean().round(1), "purchases on an ordinary day")

large = per_sale >= 2.3
print(large.sum(), "large-basket days;", (large & is_promo).sum(), "of them were s
```

```
155.4 purchases on a sale day
81.5 purchases on an ordinary day
194 large-basket days; 0 of them were sale days
```

What changes is the number of purchases. And of the store-days with unusually large baskets, none at all were sale days. An honest "no difference" is a finding too: the sale brings more buyers, not bigger baskets.

GUIDED LAB

Lab A (everyone): open `labs/module-02/l1_2-vectorized-operations/staffing_load_starter.py`. The question: **how many visitors does each staff member look after, and when is a store stretched?** ("Stretched" means 60 or more visitors per staff member.) Six TODOs: one division, a summary, a mask, two combined masks, three labels with `np.select`, and the date and store of the most stretched day.

Lab B (pairs): open `labs/module-02/l1_2-vectorized-operations/eight_weeks_axis_starter.py`. A table of 56 days by 4 stores. Before each aggregation, the partner who is not typing says the shape of the answer out loud: "one per store" is 4 values, "one per day" is 56.

Fill this in before you run Lab B. The table's shape is `(56, 4)`. `table.sum(axis=0)` has shape _____ and holds one total per _____. `table.sum(axis=1)` has shape _____ and holds one total per _____. `table.argmax(axis=0)` gives _____ positions, each between 0 and _____.

Finished early? `labs/module-02/l1_2-vectorized-operations/debug_masks.py` has four mask bugs: two with error messages, two without.

PREDICT, THEN RUN

Write what you think each one prints **before** you run it. Being wrong here is useful: it shows you exactly which idea to look at again.

P1

What does this print?

```
s = np.array([3, 8, 5, 10])
m = s > 4
print(m, s[m], s[~m])
```

P2

What does this print?

```
g = np.array([[1, 2],
              [3, 4],
              [5, 6]])
print(g.sum(axis=0), g.sum(axis=1), g.sum())
```

P3

What does this print?

```
r = np.array([200.0, np.nan, 50.0])
print((r > 100).sum(), (r == np.nan).sum(), np.isnan(r).sum())
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Project: the store scorecard (lesson page: "Project: The Store Scorecard"): how busy each store is, how well it turns visitors into buyers, what a typical purchase is worth, and what sale weekends do.

In class: Step 1 (load seven arrays; check `2903 store-days, 6 with no revenue`) and Step 2 (three measures). Check: conversion 23.5%, basket value $88.64, units per sale 2.11.

At home: Steps 3 to 6. Check yourself: four store rows; sale-day lift 1.93x; weekend mean $10,073.19; four labels whose counts add up to 2,903; the five biggest days are all at one store.

Keep this notebook. Lesson 1.3's project uses the same columns.

SELF-CHECK

Six questions on this lesson. Circle or write your answer, then check with the class.

Q1

What does this print?

```
v = np.array([4, 9, 2, 7])
print((v > 3).sum(), (v > 3).mean())
```

A) `3 0.75`

B) `20 0.75`

C) `[4 9 7] 3`

D) `3 6.67`

Q2

`stores` (text) and `visitors` (numbers) are parallel arrays. Which line counts the Bend store-days with more than 500 visitors?

A) `(stores == "Bend" and visitors > 500).sum()`

B) `(stores == "Bend" & visitors > 500).sum()`

C) `((stores == "Bend") & (visitors > 500)).sum()`

D) `(stores == "Bend").sum() & (visitors > 500).sum()`

Q3

What does this print?

```
t = np.array([[1, 2, 3],
              [4, 5, 6]])
print(t.max(axis=1), t.sum(axis=0).shape)
```

Q4

What does this print?

```
x = np.array([5.0, np.nan, 12.0])
print((x > 8).sum() + (x <= 8).sum())
```

A) `3`

B) `2`

C) `nan`

D) `1`

Q5

What does this print?

```
p = np.array([40, 95, 60])
print(np.where(p >= 60, "pass", "retry"))
```

Q6

What does this print?

```
names = np.array(["Asheville", "Bend", "Boise", "Flagstaff"])
totals = np.array([2935, 3312, 4347, 2661])
print(names[totals.argmax()], totals.argmin())
```

A) `4347 2661`

B) `Boise Flagstaff`

C) `2 3`

D) `Boise 3`

JOURNAL

This lesson's prompt (from the lesson page): find a loop over numbers in something you wrote before this course (or invent a small one). Write the loop and its array version side by side. Which lines of the loop turned into which piece of the array expression? Then finish this sentence in your own words: "The axis I name is the axis that..."

1.3 Broadcasting

BY THE END YOU CAN

- State the broadcasting rules and use them to predict the shape of a result, or that an operation will fail
- Combine a 2-D table with a row of values and with a column of values
- Reshape an array for broadcasting with `np.newaxis`, `reshape` and `keepdims=True`
- Build a grid of every combination from two 1-D arrays
- Recognize the broadcasting mistakes that raise no error, and catch them by checking shapes

KEY IDEAS

- **Broadcasting** lets NumPy combine arrays of different shapes by reusing the smaller one, like a rubber stamp pressed on every line of a form.
- **Rule 1:** write the shapes one above the other, lined up on the **right**. A missing entry counts as 1.
- **Rule 2:** each pair of lengths must be **equal, or one of them is 1**. The result takes the larger of each pair. Any pair that fails is an error.
- **Per column (per store): a table and a row.** `table - table.mean(axis=0)` just works.
- **Per row (per day): a table and a column.** `table / table.sum(axis=1, keepdims=True)`.
- **A column meeting a row builds a grid** of every combination, whether you wanted one or not.
- **Standardizing** is one line: `(table - table.mean(axis=0)) / table.std(axis=0)`. It puts columns of different sizes on one scale.
- **The dangerous mistakes are silent.** After any operation that mixes shapes, print `.shape`: four values in should mean four values out.

YOU WANT TO...	WRITE
Predict a result shape without computing	`np.broadcast_shapes((7, 4), (4,))`
Each column minus its own mean	`t - t.mean(axis=0)`
Each row divided by its own total	`t / t.sum(axis=1, keepdims=True)`
Flat array to a column	`a[:, np.newaxis]` or `a.reshape(-1, 1)`
Flat array to a row with two dimensions	`a[np.newaxis, :]`
Every combination of two flat arrays	`a[:, np.newaxis] * b`
z-scores per column	`(t - t.mean(axis=0)) / t.std(axis=0)`

YOU WANT TO...	WRITE
Any shape back to flat	`a.ravel()`

WORKED EXAMPLE: HOW MANY STAFF DOES EACH STORE NEED?

Week 1 of visitors as a 7 by 4 table (days by stores: Asheville, Bend, Boise, Flagstaff). Three questions, three shapes: a table and a row, a table and a column, and a column meeting a row.

```
visitors = load(2)
V = visitors[:28].reshape(7, 4)

store_mean = V.mean(axis=0)                 # shape (4,)
print(store_mean.round(1))
print((V - store_mean).round(0)[-2:])       # Saturday and Sunday, against each st
```

```
[239.3 268.4 380.4 207.4]
[[139. 156. 119.  38.]
 [-28.  16.  84.  48.]]
```

`(7, 4)` with `(4,)`: from the right, 4 meets 4, and 7 meets a missing axis. The row of averages is reused on all seven days. Saturday is far above normal at every store.

```
share = V / V.sum(axis=1, keepdims=True)    # (7, 4) with (7, 1)
print(share.round(2)[:2])                   # Monday and Tuesday
print(share.sum(axis=1))
```

```
[[0.23 0.23 0.38 0.16]
 [0.23 0.24 0.33 0.2 ]]
[1. 1. 1. 1. 1. 1. 1.]
```

`(7, 4)` with `(7, 1)`: 4 meets 1, 7 meets 7. Each day is divided by its own total, and the check line shows every row adding up to 1. Without `keepdims=True` the totals would be flat, 4 would meet 7, and NumPy would refuse.

```
per_staff = np.array([40, 50, 60])          # visitors one staff member can look a
needed = np.ceil(store_mean / per_staff[:, np.newaxis])    # (4,) with (3, 1)
print(needed.shape)
print(needed)
```

```
(3, 4)
[[ 6.  7. 10.  6.]
 [ 5.  6.  8.  5.]
 [ 4.  5.  7.  4.]]
```

`(4,)` with `(3, 1)`: 4 meets 1, and a missing axis meets 3. A row meets a column, so the result is a 3 by 4 grid: one row per staffing level, one column per store. At 50 visitors per staff member Boise needs 8 people on an average day and Asheville 5.

GUIDED LAB

Lab A (everyone): open `labs/module-02/l1_3-broadcasting/fair_visitors_starter.py`. Divide each store's visitors by its own average (a table and a row), then each day by its own total (a table and a column). The starter leaves a blank after each pair of shapes: **write your prediction there before you run the line.**

Lab B (pairs): open `labs/module-02/l1_3-broadcasting/target_grid_starter.py`. Predict four result shapes, build a 4 by 3 grid of revenue targets, then find out why a forecast check reports an average error of 1,753 when every forecast was close.

Practice the two rules on paper first. Write the result shape, or "error": `(224, 4)` with `(4,)` gives _____ · `(224, 4)` with `(224,)` gives _____ · `(224, 4)` with `(224, 1)` gives _____ · `(4, 1)` with `(3,)` gives _____ · `(4,)` with `(4, 1)` gives _____ :

Finished early? `labs/module-02/l1_3-broadcasting/debug_shapes.py`: one shape bug with an error message and two without.

PREDICT, THEN RUN

Write what you think each one prints **before** you run it. Being wrong here is useful: it shows you exactly which idea to look at again.

P1

What does this print?

```
t = np.ones((3, 4))
print((t + np.array([1, 2, 3, 4])).shape, (t + np.array([[1], [2], [3]]))[2])
```

P2

What does this print?

```
col = np.array([1, 2, 3])[:, np.newaxis]
row = np.array([10, 20])
print(col + row)
```

P3

This line fails. Predict the two shapes in the error message, then say which pair of lengths is the problem.

```
np.ones((3, 4)) + np.array([1, 2, 3])
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Project: the weekday index (lesson page: "Project: The Weekday Index"): how much a Saturday is worth at each store, on a scale that is fair to stores of different sizes.

In class: Step 1 (a 224-day by 4-store table; check the shape `(224, 4)` and missing values per store `[1 1 0 1]`) and Step 2 (each day divided by its store's mean: store means `[5697.45 6650.89 9108.3 5895.63]`).

At home: Steps 3 to 6. Check yourself: a 7 by 4 index with Saturday near 1.5 at every store; a chart of four lines that nearly overlap; 13 store-days with a z-score above 3; Boise's share about a third.

Keep this notebook. The 224 by 4 table is useful again in Session 3.

◆ SELF-CHECK

Six questions on this lesson. Circle or write your answer, then check with the class.

Q7

An array of shape `(6, 3)` is added to an array of shape `(3,)`. What is the result?

A) A `ValueError`, because the shapes are different

B) An array of shape `(6, 3)`

C) An array of shape `(6,)`

D) An array of shape `(3, 3)`

Q8

`t` has shape `(30, 4)`: 30 days as rows, 4 stores as columns. Which expression subtracts each day's mean from that day's row?

A) `t - t.mean(axis=1)`

B) `t - t.mean(axis=0)`

C) `t - t.mean(axis=1, keepdims=True)`

D) `t - t.mean()`

Q9

What does this print?

```
a = np.array([1, 2, 3])
b = np.array([[10], [20]])
print((a * b).shape, (a * b)[1, 2])
```

Q10

Which pair of shapes can NOT be broadcast together?

A) `(8, 1)` and `(1, 5)`

B) `(8, 5)` and `(5,)`

C) `(8, 5)` and `(1,)`

D) `(8, 5)` and `(8,)`

Q11

What does this print?

```
print(np.broadcast_shapes((4, 1), (3,)))
```

Q12

`sales` has shape `(12,)` and `plan` has shape `(12, 1)`. What does `sales - plan` give?

A) An array of shape `(12, 12)`: every sales value minus every plan value, with no error

B) An array of shape `(12,)` holding the twelve differences

C) A `ValueError`, because the shapes are different

D) An array of shape `(12, 1)` holding the twelve differences

JOURNAL

This lesson's prompt (from the lesson page): write the two broadcasting rules from memory, then check them against the lesson. Next, describe the accidental grid in your own words: what two shapes cause it, why NumPy does not complain, and the one habit that would have caught it. Which of the silent mistakes do you think you are most likely to make?

EXIT TICKET

Three questions, no notes. Write your answers, hand them in or keep them: they tell you and your instructor what to revisit.

E1

`revenue` and `promo` are parallel arrays. Write one line that counts the sale days (`promo` equal to 1) with revenue above 10,000.

E2

A table has shape `(224, 4)`: days by stores. What is the shape of `table.mean(axis=0)`, and what does each value mean?

E3

You want every row of that table divided by its own row total. Write the expression, and say what goes wrong if you leave out the keyword.

NOTES

HOMEWORK CHECKLIST

- ☐ Finish the store scorecard (Steps 3 to 6). My sale-day lift: _____ x
- ☐ Finish the weekday index (Steps 3 to 6). Saturday's index at Boise: _____
- ☐ Read "Sorting and Picking by Position" and "From Loop to Array Expression" in Lesson 1.2
- ☐ Lesson 1.2 exercises 1 to 3 (4 is a stretch) and Lesson 1.3 exercises 1 to 3 (4 is a stretch)
- ☐ Both Quick Quizzes on the lesson pages
- ☐ Journal: both entries, including the two broadcasting rules written from memory
- ☐ Before Session 3: predict what shapes `(8, 1, 3)` and `(5, 1)` give together: _____ then check
- ☐ Before Session 3: two random 30-day totals of Boise's revenue. My guess for how different they are:

_____

BRING TO SESSION 3

- Your laptop with the same working environment (nothing new to install)
- Your `pinecrest-analysis` folder with `data/` and `labs/` in it
- Your finished Lesson 1.3 project notebook (the 224-day by 4-store table)
- Your written guess about the two random 30-day totals
- This workbook and your journal

GLOSSARY

Vectorized operation	One operation applied to a whole array, with the loop running inside NumPy.
Parallel arrays	Arrays of the same length where position k in each describes the same thing (the same store-day).
ufunc (universal function)	A NumPy function that applies itself to every element: `np.sqrt`, `np.abs`, `np.maximum`.
Boolean mask	An array of True and False made by a comparison. Used to count, filter and assign.
** `&` , `|` , `~` **	And, or, not for masks. Each comparison needs its own parentheses.
np.where	A vectorized if/else: `np.where(condition, value_if_true, value_if_false)`.
np.select	A vectorized if/elif/else for three or more outcomes; the first true condition wins.
Aggregation	An operation that boils many values down to one: `sum`, `mean`, `max`, `std`.
axis	The dimension an aggregation collapses. The axis you name is the axis that disappears.
argmax / argsort	The position of the largest value / the positions that would sort the array.
Broadcasting	NumPy's rules for combining arrays of different shapes by reusing the smaller one.
keepdims=True	Keeps the collapsed axis with length 1, so the result is a column (or row) that broadcasts.
np.newaxis	Adds an axis of length 1 when indexing: `a[:, np.newaxis]` turns `(n,)` into `(n, 1)`.
Grid	The table of every combination that results when a column meets a row.

z-score	How many standard deviations a value is above or below its column's mean.

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 3: Randomness and Linear Algebra

Your working notebook for Session 3, the last of the NumPy module. In the first half you make your own numbers: random draws you can reproduce, fair samples of real rows, and simulations that answer "what could happen?". In the second half you meet the working parts of linear algebra, with no proofs: weighted sums, the `@` operator, solving for unknowns, and fitting a line through real data.

Lessons 1.4–1.5 | Session 3 · 3 hours | Name: _____

HOW TO USE THIS WORKBOOK

Each lesson has **By the end you can**, **Key ideas**, a **Worked example** (new code with its real output), the **Guided lab** you do in class, **Predict, then run** items, the lesson **project** (started in class, finished at home), a six-question **Self-check** and the **Journal** prompt. Lab files are in `labs/module-03/`. Two habits for today: in random code, run from the top when you want the same numbers again; in `@` code, write the shapes down before you run the line.

SESSION GOALS

- Create a seeded generator and draw floats, whole numbers and whole tables from it.
- Explain why rerunning a cell changes your numbers, and make a random result reproducible.
- Sample rows fairly (with and without replacement), shuffle, and split rows at random.
- Answer a "what are the chances?" question by simulation, and check the model against real data.
- Compute weighted sums with `@`, predict the shape of a matrix product, and tell `@` from `*`.
- Solve for unknowns with `np.linalg.solve`, and fit a line with `np.linalg.lstsq`.
- Read a coefficient as a careful sentence, and quote a forecast with its typical miss.
- Start both lesson projects: 10,000 possible months, and a first revenue model.

WARM-UP: WHAT DO YOU REMEMBER FROM SESSION 2?

Five questions, no notes. Write a short answer to each, then check with the class.

W1

`big = revenue > 15000`. What does `big.sum()` tell you? And `big.mean()`?

W2

Write a mask for "Boise and a sale day", given the parallel arrays `stores` and `promo`.

W3

`table` has shape `(224, 4)`: days by stores. What is the shape of `table.sum(axis=1)` and what does it hold?

W4

State the two broadcasting rules.

W5

How do you divide every row of that table by its own row total?

LESSONS

1.4 Random Number Generation

BY THE END YOU CAN

- Create a random number generator with `np.random.default_rng` and draw floats, integers and arrays of any shape from it
- Explain what a seed does, and structure your code so its random results can be reproduced
- Sample from an array with and without replacement, shuffle it, and split rows at random
- Draw from the normal, Poisson and binomial distributions and say what kind of quantity each one models
- Estimate a probability by simulation, and judge how far to trust the estimate and the model behind it

KEY IDEAS

- **Create one generator, once, with a seed:** `rng = np.random.default_rng(42)`. Draw everything from it.
- **A seed fixes where the sequence starts.** The generator then moves forward with every draw, like a bookmark. Reproducible means: same code, run from the top, same result.
- ** `rng.integers(low, high)` leaves out the upper end.** A die is `integers(1, 7)`. `size` can be a shape.
- ** `rng.choice` draws from your own values.** With replacement by default; `replace=False` gives different items; `p` sets the chances.
- **Sample row positions, then use them on every column**, so parallel arrays stay matched.
- ** `rng.permutation` returns a shuffled copy. `rng.shuffle` works in place and returns `None`.**
- **Normal** for measurements around a typical value, **Poisson** for counts of events, **binomial** for successes out of n tries.
- **Simulation: model, repeat, count.** Describe one case with random draws, simulate many at once, take the mean of a mask.
- **More simulations make an estimate more precise. Only a better model makes it more true.** Check the model against real data.

YOU WANT TO...	WRITE
Floats from 0 up to 1	`rng.random(5)`
Ten dice	`rng.integers(1, 7, size=10)`

YOU WANT TO...	WRITE
A table of draws	`rng.integers(0, 100, size=(3, 4))`
Draw from your own values	`rng.choice(values, size=6)`
Different items only	`rng.choice(values, size=3, replace=False)`
Five different row positions	`rng.choice(n, size=5, replace=False)`
A shuffled copy / shuffle in place	`rng.permutation(x)` / `rng.shuffle(x)`
A random 80/20 split	`order = rng.permutation(n); order[:cut], order[cut:]`
Counts of events / successes in n tries	`rng.poisson(lam, size)` / `rng.binomial(n, p, size)`
A chance, from a simulation	`(sim > 130).mean()`

WORKED EXAMPLE: HOW GOOD IS AN AVERAGE FROM ONLY 40 DAYS?

Suppose Asheville's door counter worked on only 40 days. How far from the truth could a 40-day average of visitors be? You have all 727 real days, so you can find out by sampling.

```
import numpy as np

def load(col, dtype=float):
    """Read one column of daily_sales.csv into an array."""
    return np.genfromtxt("data/daily_sales.csv", delimiter=",",
                         skip_header=1, usecols=col, dtype=dtype)

stores = load(1, dtype=str)
visitors = load(2)
asheville = visitors[stores == "Asheville"]
print(asheville.size, "days | true average", asheville.mean().round(1))
```

```
727 days | true average 274.9
```

The true average over all 727 days is 274.9 visitors. Normally you would not know it; here it lets you judge the samples.

```
rng = np.random.default_rng(21)

sample = rng.choice(asheville, size=40, replace=False)     # 40 different days
print(sample[:8])
print("estimate from 40 days:", sample.mean().round(1))
```

```
[238. 234. 173. 178. 403. 282. 199. 286.]
estimate from 40 days: 276.0
```

One sample of 40 different days gives one estimate. It is close, and a different sample would give a different number. How different?

```
many = rng.choice(asheville, size=(10_000, 40))           # 10,000 samples of 40
estimates = many.mean(axis=1)
low, high = np.percentile(estimates, [5, 95])
print(many.shape, estimates.shape)
print(f"90% of 40-day estimates fall between {low:.0f} and {high:.0f}")
```

```
(10000, 40) (10000,)
90% of 40-day estimates fall between 253 and 299
```

Ten thousand samples at once: `size=(10_000, 40)` is one sample per row, and `mean(axis=1)` collapses the 40 days. Nine times in ten, a 40-day average lands between 253 and 299: within about 23 visitors of the truth.

```
bigger = rng.choice(asheville, size=(10_000, 160)).mean(axis=1)
low, high = np.percentile(bigger, [5, 95])
print(f"90% of 160-day estimates fall between {low:.0f} and {high:.0f}")
print(estimates.std().round(1), bigger.std().round(1))
```

```
90% of 160-day estimates fall between 263 and 287
14.1 7.0
```

With four times as many days the range is half as wide (263 to 287), and the spread of the estimates halves from 14.1 to 7.0. More data helps, with diminishing returns. Module 4 gives this a name and a formula.

◆ GUIDED LAB

Lab A (everyone): open `labs/module-03/l1_4-random-generation/mystery_shopper_starter.py`. Let the generator choose 2 pilot stores and 12 different Flagstaff days in 2025, prove the same seed gives the same schedule, and split all the rows 70/30 at random. Use the seed 7 and make the draws in the order the file gives them: your output should then match your neighbor's exactly.

Lab B (pairs): open `labs/module-03/l1_4-random-generation/returns_desk_starter.py`. The question: **on how many days should customer service expect 4 or more returns from one day's orders?** Model, repeat, count; then check your simulation against the 731 real days; then try the same model on orders per day and see it fail.

Before you start Lab B, fill in the three steps for the returns question. Model: one day is _____. Repeat: _____ days, in one call. Count: the mean of the mask _____ :

Finished early? `labs/module-03/l1_4-random-generation/debug_random.py`: four bugs in random code, three of them silent.

◆ PREDICT, THEN RUN

Write what you think each one prints **before** you run it. Being wrong here is useful: it shows you exactly which idea to look at again.

P1

What does this print?

```
r = np.random.default_rng(9)
first = r.random(3)
r = np.random.default_rng(9)
second = r.random(3)
third = r.random(3)
print(np.array_equal(first, second), np.array_equal(second, third))
```

P2

What does this print?

```
r = np.random.default_rng(1)
picks = r.choice(4, size=4, replace=False)
print(np.sort(picks), picks.sum())
```

P3

This line fails. Predict the kind of error and what the message says.

```
np.random.default_rng(2).choice(np.array(["A", "B", "C"]), size=5, replace=Fal
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Project: how much could a month vary? (lesson page: "Project: How Much Could a Month Vary?"). You build 10,000 possible months from real Boise days, then check the simulation against the 24 real months.

In class: Step 1 (726 Boise days, mean $10,144.21, and ONE generator with seed 42) and Step 2 (two simulated months: $324,963.33 and $303,054.24 if you draw nothing else in between).

At home: Steps 3 to 7. Check yourself: `sims` has shape `(10000, 30)`; the middle 90% of months runs from about $271,698 to $339,193; `simulate_months(7)` gives identical results twice. In Step 7 only 10 of the 24 real months fall inside your 90% range. That is not a mistake in your code. Work out why.

SELF-CHECK

Six questions on this lesson. Circle or write your answer, then check with the class.

Q1

What does this print?

```
a = np.random.default_rng(5)
b = np.random.default_rng(5)
x = a.integers(0, 100, size=3)
y = a.integers(0, 100, size=3)
z = b.integers(0, 100, size=3)
print(np.array_equal(x, y), np.array_equal(x, z))
```

A) `True True`

B) `False True`

C) `True False`

D) `False False`

Q2

Which line rolls ten ordinary six-sided dice (values 1 to 6)?

A) `rng.integers(1, 6, size=10)`

B) `rng.random(6, size=10)`

C) `rng.choice(6, size=10)`

D) `rng.integers(1, 7, size=10)`

Q3

What does this print?

```
d = np.arange(5)
r = np.random.default_rng(0).shuffle(d)
print(r, np.sort(d))
```

Q4

`orders` **has** `n` **rows. Which line chooses 50 different rows to audit?**

A) `rng.choice(n, size=50, replace=False)`

B) `rng.choice(n, size=50)`

C) `rng.integers(0, 50, size=n)`

D) `rng.permutation(50)`

Q5

A simulation of 1,000,000 days says a store runs out of gift bags on 4.5% of days, and the estimate barely moves from run to run. The model assumed exactly 485 visitors every day. What is the best reading?

A) The answer is correct, because a million simulations cannot be wrong

B) The answer is useless, because the numbers were random

C) The estimate is precise, but it is only as true as the model; a better model of visitors could change it a lot

D) Run ten million simulations to remove the effect of the assumption

Q6

What does this print?

```
s = np.random.default_rng(3).integers(1, 4, size=(1000, 2))
print(s.shape, s.min(), s.max())
```

JOURNAL

This lesson's prompt (from the lesson page): the gift-bag question got three different answers from three models. In your own words, what is the difference between an estimate being precise and being right? Then think of a "what are the chances?" question from your own work or life, and sketch the three steps (model, repeat, count) you would use to simulate it. What assumption in your model worries you most?

1.5 Linear Algebra Essentials

BY THE END YOU CAN

- Compute a dot product and explain it as a weighted sum
- Multiply matrices and vectors with `@`, predict the shape of the result, and tell `@` apart from `*`
- Solve a system of linear equations with `np.linalg.solve` and check the answer
- Fit a line or a multi-feature linear model with `np.linalg.lstsq`, and interpret its coefficients and its error
- Measure length and distance with `np.linalg.norm`, and standardize features before comparing them

KEY IDEAS

- **A dot product** multiplies matching positions and adds them up: a weighted sum. `a @ b`. Two vectors in, one number out.
- **matrix @ vector** takes the dot product of every row with the vector: many totals, or many predictions, at once.
- **The shape rule:** `(n, k) @ (k, m)` gives `(n, m)`. The inner numbers must match, and order matters.
- ** `*` is position by position and keeps the shape. `@` multiplies and adds.** When shapes broadcast, `*` gives the wrong thing with no error: check the shape.
- ** `.T` ** swaps rows and columns.
- ** `np.linalg.solve(A, b)` ** finds the unknowns of an exact system. Check with `np.allclose(A @ x, b)`.
- ** `np.linalg.lstsq(A, y)` ** finds the best-fitting weights when there are more equations than unknowns. That is linear regression. A column of ones gives the intercept; more features are more columns.
- **RMSE** is the typical miss, in the units of what you predict. **R squared** is the share of the variation the model accounts for.
- **A fitted line describes; it does not prove.** Read a coefficient as "goes with", with the other columns held fixed.
- ** `np.linalg.norm` ** measures length; the norm of a difference is a distance. Standardize the columns first.

YOU WANT TO...	WRITE
A weighted sum	`quantities @ prices`
A weighted average	`ratings @ reviews / reviews.sum()`
One total (or prediction) per row	`table @ weights`
Rows become columns	`M.T`
Solve an exact system	`np.linalg.solve(A, b)`
Build the matrix for a line	`A = np.column_stack([np.ones(x.size), x])`
Fit by least squares	`coef, *_ = np.linalg.lstsq(A, y)`
Predictions and residuals	`A @ coef` and `y - A @ coef`
Typical miss (RMSE)	`np.sqrt((residuals ** 2).mean())`
Distance from row 0 to every row	`np.linalg.norm(z - z[0], axis=1)`

WORKED EXAMPLE: WHICH PRODUCT IS MOST LIKE THIS ONE?

A shopper is looking at a pair of trail runners. Which other footwear product is most like it, judged by price and weight? `products.csv` has both columns (5 is price, 7 is weight in kilograms; 2 is the category and 3 the product type).

```
import numpy as np

def col(k, dtype=float):
    return np.genfromtxt("data/products.csv", delimiter=",", skip_header=1, usecol

category, kind = col(2, dtype=str), col(3, dtype=str)
price, weight = col(5), col(7)

keep = (category == "Footwear") & ~np.isnan(weight)
shoes = np.column_stack([price[keep], weight[keep]])     # one row per product: pr
kinds = kind[keep]
print(shoes.shape)
print(kinds[1], shoes[1])
```

```
(26, 2)
Trail Runners [135.95   0.57]
```

One mask (footwear, and a weight present) applied to every column keeps them parallel. `np.column_stack` puts price and weight side by side: 26 products, 2 features. Row 1 is a pair of trail

runners at $135.95 and 0.57 kg.

```
raw = np.linalg.norm(shoes - shoes[1], axis=1)           # a table minus a row, th
raw[1] = np.inf                                          # a product is not its ow
print("closest on raw numbers:", kinds[raw.argmin()], shoes[raw.argmin()])
```

```
closest on raw numbers: Hiking Boots [135.95   1.21]
```

`shoes - shoes[1]` is a table minus a row, and `axis=1` asks for one norm per row. The "closest" product is a pair of hiking boots: exactly the same price, and more than twice the weight. Price is measured in tens and hundreds, weight in fractions of a kilogram, so price decided everything.

```
z = (shoes - shoes.mean(axis=0)) / shoes.std(axis=0)     # every column on one sca
dist = np.linalg.norm(z - z[1], axis=1)
dist[1] = np.inf
print("closest after standardizing:", kinds[dist.argmin()], shoes[dist.argmin()])
```

```
closest after standardizing: Trail Runners [132.95   0.62]
```

After standardizing (the one-line step from Lesson 1.3), both columns pull equal weight, and the nearest product is another pair of trail runners: $3 cheaper and 50 grams heavier. That is what a shopper would call similar.

GUIDED LAB

Lab A (everyone): open `labs/module-03/l1_5-linear-algebra/payroll_receipt_starter.py`. Staff counts times pay rates, added up: the lesson's receipt with different things on it. Five parts, each one `@` (or one `solve`). The starter leaves a blank for the result shape: fill it in before you run the line.

Lab B (pairs): open `labs/module-03/l1_5-linear-algebra/basket_model_starter.py`. Fit Flagstaff's revenue against the number of purchases, read the slope as a sentence, reach the same number a second way, add a sale-day column and decide whether it earned its place, then forecast three days with one `@`.

Shape practice before Lab A. Write the result shape, or "error": `(7, 4) @ (4,)` gives _____ · `(7, 4) @ (4, 2)` gives _____ · `(4,) @ (7, 4)` gives _____ · `(4, 7) @ (7,)` gives _____ · `(3, 3)` and `(3,)` into `np.linalg.solve` gives _____ :

Finished early? `labs/module-03/l1_5-linear-algebra/debug_linalg.py`: four bugs, two of which print wrong numbers with no error.

PREDICT, THEN RUN

Write what you think each one prints **before** you run it. Being wrong here is useful: it shows you exactly which idea to look at again.

P1

What does this print?

```
M = np.array([[1, 0, 2],
              [0, 3, 1]])
w = np.array([10, 1, 5])
print(M @ w, (M * w).shape)
```

P2

What does this print?

```
a = np.array([[2.0, 0.0],
              [0.0, 4.0]])
print(np.linalg.solve(a, np.array([6.0, 8.0])))
```

P3

This line fails. Say why, using the shape rule, before you run it.

```
np.ones((2, 3)) @ np.ones((2, 3))
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Project: a first revenue model (lesson page: "Project: A First Revenue Model"): turn a forecast of visitors into a forecast of revenue for the Asheville store.

In class: Step 1 (one mask, 726 complete days, mean visitors 275, mean revenue $5,703.83) and Step 2 (the fit: `revenue = 4.73 + 20.73 * visitors`). Write the slope as a sentence:

At home: Steps 3 to 6. Check yourself: RMSE $902.58 and R squared 0.809 for visitors only; RMSE $890.88 with four columns; a normal Saturday with 380 visitors forecast at $7,818.61, give or take about $891; the five biggest misses are all days that did far better than predicted.

Do Exercise 4 (the staffing trap). It shows a coefficient that looks like a business recommendation and is not.

SELF-CHECK

Six questions on this lesson. Circle or write your answer, then check with the class.

Q7

What does this print?

```
u = np.array([2, 0, 1])
v = np.array([10, 5, 3])
print(u @ v, u * v)
```

A) `23 [20 0 3]`

B) `[20 0 3] 23`

C) `23 23`

D) A `ValueError`, because `@` needs 2-D arrays

Q8

`X` has shape `(100, 3)` (100 days, 3 features) and `w` has shape `(3,)`. What is the shape of `X @ w`?

A) `(100, 3)`

B) `(100,)`

C) `(3,)`

D) It fails, because the shapes are different

Q9

What does this print?

```
A = np.array([[1, 2],
              [3, 4]])
print(A @ np.array([1, 1]), A.T[0])
```

Q10

A least-squares fit with the columns (ones, visitors, sale day) returns the coefficients `[189.11, 20.34, 2502.17]`. What does the model predict for a sale day with 400 visitors?

```
print((np.array([1, 400, 1]) @ np.array([189.11, 20.34, 2502.17])).round(2))
```

A) `8325.11`

B) `8136.0`

C) `10827.28`

D) `2711.62`

Q11

What does this print?

```
print(np.linalg.norm(np.array([6.0, 8.0])),
      np.linalg.norm(np.array([1.0, 1.0]) - np.array([4.0, 5.0])))
```

Q12

You look for the product most like a given one by distance over two columns: price (in the hundreds) and weight in kilograms (single digits). The "nearest" product has almost the same price and twice the weight. What is the fix?

A) Use `*` in place of `@` when computing the distance

B) Add a column of ones to the table

C) Sort the table by price first

D) Standardize each column (subtract its mean, divide by its standard deviation) before measuring distance

JOURNAL

This lesson's prompt (from the lesson page): name three weighted sums from your own life or work (a grade, a budget, a score of some kind) and say what the weights are in each. Then look back over Module 1: write one sentence for each of the five lessons about the single idea you most want to keep. Which of them did you find hardest, and what would you tell someone about to start it?

EXIT TICKET

Three questions, no notes. Write your answers, hand them in or keep them: they tell you and your instructor what to revisit.

E1

You run a cell that draws from `rng` twice and get different numbers. Is the code broken? What makes a random result reproducible?

E2

`orders` has shape `(3, 3)` (orders by products) and `prices` has shape `(3,)`. What is the difference between `orders * prices` and `orders @ prices`?

E3

A fit of revenue on visitors gives a slope of 21.39. Write the sentence a manager should hear, and one caution.

NOTES

HOMEWORK CHECKLIST

- ☐ Finish the month simulation (Steps 3 to 7). Real months inside my simulated 90% range: _____ of 24. Why so few?
- ☐ Finish the Asheville revenue model (Steps 3 to 6). My forecast for a normal Saturday: $ _____ give or take $ _____
- ☐ Read "Distributions: Shaped Randomness" in Lesson 1.4
- ☐ Lesson 1.4 exercises 1 to 3 (4 is a stretch)
- ☐ Lesson 1.5 exercises 1 to 3, and Exercise 4 (the staffing trap). The staff coefficient before and after: _____ → _____
- ☐ Both Quick Quizzes on the lesson pages
- ☐ Journal: both entries, including one sentence for each of the five Module 1 lessons
- ☐ Before Session 4: read the header row of `data/daily_sales.csv`, and skim my `first_look.py` from Lesson 0.1

BRING TO SESSION 4

- Your laptop with the same working environment (nothing new to install)
- The whole unzipped `data/` folder in `pinecrest-analysis` (Session 4 opens many of its files, not only `daily_sales.csv`)
- Your finished Module 1 projects and `first_look.py` from Lesson 0.1
- Your `labs/` folder, this workbook, and your journal with the Module 1 review

GLOSSARY

Generator (`rng`)	The object that produces random numbers: `rng = np.random.default_rng(seed)`.
Seed	A whole number that fixes where the generator's sequence starts, so results can be reproduced.
Pseudo-random	Produced by a fixed recipe, yet statistically indistinguishable from random for sampling and simulation.
With / without replacement	Whether a drawn value can be drawn again. `replace=False` gives different items.
Sampling variation	The gap between what the probabilities say and what one particular sample shows.
Distribution	A description of which values are likely: normal, Poisson, binomial, and many more.

Monte Carlo simulation	Answering a question by modeling one case with random draws, repeating it many times, and counting.
Law of large numbers	The average of more and more random draws settles toward the true underlying value.
Vector / matrix	A 1-D array / a 2-D array.
Dot product	Multiply matching positions and add them up: a weighted sum. Written `a @ b`.
Transpose (`.T`)	The same matrix with rows and columns swapped.
Least squares	Choosing the weights that make the sum of squared misses as small as possible. Linear regression.
Coefficient	A fitted weight: how much the prediction changes per unit of one feature, with the others held fixed.
Residual	An actual value minus the model's prediction for it.
RMSE	Root mean squared error: the typical size of a miss, in the units of what is predicted.
R squared	The share of the variation that a model accounts for: 1 is a perfect fit, 0 no better than the mean.
Norm / distance	The length of a vector / the norm of the difference between two vectors.

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 4: DataFrames and Loading Data

Your working pages for the session where arrays become tables. Before the break: the Series and the DataFrame, and the six moves you will type hundreds of times. After the break: getting data out of CSV files, spreadsheets and nested JSON with the right types, and saving your results.

Lessons 2.1–2.2 | Session 4 · 3 hours | Name: _____

HOW TO USE THIS WORKBOOK

Each lesson has **By the end you can** (your goals), **Key ideas** (one page to keep), a **Worked example** with its real output, the **Guided lab** you do in class, three **Predict, then run** boxes, the lesson project's class and home parts, space for notes, and a six-question **Self-check**. Lab files are in `labs/module-04/`: open the `_starter.py` files. One rule for the whole session: **run it, then look** (`shape`, `dtypes`, `head()`).

SESSION GOALS

- Explain what a Series and a DataFrame are, and greet any new table with four lines
- Select by label with `.loc` and by position with `.iloc`, and filter rows with masks
- Add and change columns the copy-on-write way, and say why `df[mask]["col"] = value` changes nothing
- Load CSV, Excel and nested JSON files with the right dtypes, and check every load
- Save a table so that its types survive, and say what a CSV loses
- Start both lesson projects: the Catalog Health Check and the Pinecrest Snapshot

LESSONS

2.1 DataFrames and Series

90 min lesson

BY THE END YOU CAN

- Describe a Series as a labeled array and a DataFrame as a set of Series that share one index
- Inspect any table with `shape`, `head()`, `info()`, `dtypes` and `describe()`, and read the dtypes pandas reports
- Select columns with `[]`, rows and cells by label with `.loc` and by position with `.iloc`, and filter rows with boolean masks
- Add and change columns in a way that works under copy-on-write, and explain why chained assignment never does
- Sort a table and count the values in a column with `sort_values()` and `value_counts()`

KEY IDEAS

- **The labeled shelf.** A Series is an array of values plus an index of labels (plus a name). A DataFrame is several Series side by side that share one index. Each column keeps its own dtype.
- **Alignment.** Arithmetic between two Series matches labels, not positions. A label with no partner gives `NaN`. Unexpected NaNs? Compare the two indexes.
- **The four-line greeting.** `df.shape`, `df.head()`, `df.info()`, `df.describe()` on every new table before anything else. In `info()`, a Non-Null Count below the row count means missing values.
- **dtypes.** `int64` whole numbers, `float64` decimals (and any numeric column with gaps), `str` text, `bool`, `datetime64[us]` dates. A date read from a file is `str` until you convert it. `object` on a column today usually means mixed types: investigate.
- **Three selection tools.** `df["col"]` is a Series; `df[["a", "b"]]` is a DataFrame. `.loc[rows, cols]` uses labels and its slices include the end. `.iloc[rows, cols]` uses positions and its slices exclude the stop, like NumPy.
- **Masks.** A comparison makes a True/False Series; `df[mask]` keeps the True rows. Combine with `&`, `|`, `~` and put every condition in its own parentheses. `.sum()` of a mask counts, `.mean()` gives the share. Helpers: `.isin([...])`, `.between(a, b)`, `.str.contains(...)`, `.isna()`.
- **Changing a table.** New column: `df["new"] = ...` (whole-column arithmetic, no loop). Some rows: `df.loc[mask, "col"] = value`. If/else: `np.where(cond, a, b)`.
- **Copy-on-write.** Every piece you take from a table behaves as an independent copy. So `df[mask]["col"] = value` (chained assignment) never changes `df`. It only warns.

- **Sort and count.** `sort_values("col", ascending=False)` returns a new table and the row labels travel with their rows. `value_counts()` counts categories (missing values left out unless `dropna=False`); `normalize=True` gives shares.

WORKED EXAMPLE: HOW RAINY IS EACH STORE'S CITY?

A table you have not been walked through: `store_weather.csv`, one row per city per day. The question is small on purpose, so you can watch each move: greet, add a column from a comparison, use a mask for the rows and a name for the column, find a row by its label.

Greet the table. Four cities times 731 days is 2,924 rows; the date is text for now.

```
weather = pd.read_csv("data/store_weather.csv")
print(weather.shape)
print(weather.head(3))
print(weather.dtypes)
```

```
(2924, 5)
         date   city  high_f  low_f  precip_in
0  2024-01-01  Boise      37     24        0.0
1  2024-01-02  Boise      44     32        0.0
2  2024-01-03  Boise      30     14        0.0
date             str
city             str
high_f         int64
low_f          int64
precip_in    float64
dtype: object
```

A comparison is a True/False column. Store it, then count it (`sum`) and take its share (`mean`).

```
weather["rainy"] = weather["precip_in"] >= 0.10
print(weather["rainy"].sum(), round(weather["rainy"].mean() * 100, 1))
```

```
389 13.3
```

`.loc[mask, "rainy"]` means: these rows, that column. One mask per city.

```
for city in ["Boise", "Bend", "Flagstaff", "Asheville"]:
    in_city = weather["city"] == city
    print(f"{city:<10}{weather.loc[in_city, 'rainy'].mean() * 100:5.1f}% rainy day
```

```
Boise       9.3% rainy days
Bend        6.8% rainy days
Flagstaff  13.5% rainy days
Asheville  23.5% rainy days
```

Where is the hottest day? `idxmax()` gives its row label, and `.loc` reads that row. Then a new column from two others, and the three widest daily swings.

```
hot = weather["high_f"].idxmax()
print(hot, weather.loc[hot, "date"], weather.loc[hot, "city"], weather.loc[hot, "h
weather["swing_f"] = weather["high_f"] - weather["low_f"]
print(weather.nlargest(3, "swing_f")[["date", "city", "high_f", "low_f", "swing_f"
```

```
559 2025-07-13 Boise 112
            date   city  high_f  low_f  swing_f
1338  2025-08-30   Bend      74     38       36
1233  2025-05-17   Bend      59     24       35
1300  2025-07-23   Bend      93     58       35
```

Asheville has rain on about one day in four; Bend on about one day in fifteen. You answered that with a new column, a mask and `.loc`, and no loop over rows (the short `for` loop runs four times, once per city, not once per row). In Lesson 2.4 `groupby` will replace even that loop.

GUIDED LAB

- `labs/module-04/l2_1-dataframes-series/busiest_days_starter.py` (about 15 min): Which were the five biggest store-days by revenue, were they promo days, and how often does a store pass $15,000 in a day?
- `labs/module-04/l2_1-dataframes-series/recommend_groups_starter.py` (about 15 min): Split the 0-to-10 'would you recommend us?' answers into promoters, passives and detractors, and compare Asheville shoppers with everyone.

Extra: `labs/module-04/l2_1-dataframes-series/debug_filters.py`. A file that is broken on purpose: five bugs, each with a 'should print' note. Fix them one at a time, reading the last line of each error.

1. Open the starter file in VS Code. The `# %%` lines are cells: run them one at a time with **Run Cell**, or run the whole file with `python busiest_days_starter.py`. It runs as it is and tells you which TODOs are left.
2. Do the TODOs in order and run after each one. Read every printed number and ask: is that plausible?
3. When an error appears, read the **last** line first. Most of today's errors are a missing pair of parentheses or a name that is almost right.

4. Check: the 'promo days' and 'ordinary days' counts must add up to the number of days above $15,000.
5. Then `recommend_groups_starter.py`. Before TODO 4, write your prediction here: how many rows of `survey` will say `checked`? _____

PREDICT, THEN RUN

Write what you think each one prints **before** you run it. Being wrong here is useful: it shows you exactly which rule to look at again.

P1. Labels travel with their values.

```
s = pd.Series([3, 1, 2], index=["c", "a", "b"])
print(s.sort_values().index.tolist(), s.iloc[0], s.loc["a"])
```

P2. A piece of a table, changed.

```
df = pd.DataFrame({"x": [1, 2, 3, 4]})
part = df[df["x"] > 2]
part["x"] = 0
print(df["x"].sum(), part["x"].sum(), part.index.tolist())
```

P3. Looking for missing values.

```
r = pd.Series([4.5, np.nan, 3.0])
print((r == np.nan).sum(), r.isna().sum(), r.mean())
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

- Project: **Catalog Health Check** (lesson page, six steps). In class: Steps 1 and 2 in a notebook called `lesson_2_1_project`.

- At home: Steps 3 to 6. You are done when your report says 8 star products in stock and a median margin of 45.5%, with no warnings anywhere.
- Then the lesson's Exercises 1 to 4 (customers table), the Quick Quiz and your journal entry. Exercise 5 is a stretch.

MY NOTES

SELF-CHECK

Six questions on Lesson 2.1. No notes on the first pass; then check with your instructor.

Q1

What does this print?

```
s = pd.Series([5, 6, 7], index=["x", "y", "z"])
t = pd.Series([1, 1], index=["z", "x"])
print((s + t).tolist())
```

A) `[6, 7, 7]`

B) `[6.0, 7.0, nan]`

C) `[6.0, nan, 8.0]`

D) A `ValueError`, because the lengths differ

Q2

What does this print?

```
df = pd.DataFrame({"n": [10, 20, 30, 40]}, index=["a", "b", "c", "d"])
print(len(df.loc["a":"c"]), len(df.iloc[0:2]), df.iloc[-1, 0])
```

Q3

In a table of 242 products, `products.info()` shows the line `rating 234 non-null float64`. What does that line tell you?

A) 8 products have no rating

B) There are 234 different ratings

C) The ratings are stored as text

D) 8 products have a rating of 0

Q4

Which line correctly counts the products that are in the Tents category or cost less than $20?

A) `(products["category"] == "Tents" | products["price"] < 20).sum()`

B) `(products["category"] == "Tents" or products["price"] < 20).sum()`

C) `products[("category" == "Tents") | ("price" < 20)].sum()`

D) `((products["category"] == "Tents") | (products["price"] < 20)).sum()`

Q5

What does the last line print? (The second line also produces a warning.)

```
df = pd.DataFrame({"price": [10, 50, 90]})
df[df["price"] > 40]["price"] = 0
print(df["price"].tolist())
```

Q6

After `top = products.sort_values("price", ascending=False)`, what is `top.index[0]`?

A) 0, because sorting renumbers the rows

B) The original row label of the most expensive product

C) The highest price

D) 241, the last position

2.2 Loading Data

85 min lesson

BY THE END YOU CAN

- Read a CSV file with the options that matter: columns, dtypes, dates, missing-value markers, separators and header position
- Read a sheet from an Excel workbook whose header is not on the first row
- Flatten nested JSON into tables with `pd.json_normalize()`
- Process a file in chunks and shrink its memory use with `usecols` and `dtype`
- Write a DataFrame to CSV, Excel, JSON and Parquet, and explain what each format keeps and loses

KEY IDEAS

- **Loading is translation.** A text file carries no types. The reader guesses; you brief it with options and then check the result: `shape`, `dtypes`, `head()` after every load.
- `read_csv` **options that earn their keep.** `parse_dates=[...]` real dates · `usecols=[...]` only these columns · `dtype={...}` set a type · `index_col` · `nrows=5` a peek · `na_values=[...]` extra words for missing · `sep=";"` · `skiprows` / `header` · `decimal`, `thousands` · `encoding`.
- **Three types to set yourself.** Dates (they stay `str` unless parsed). `"Int64"` (capital I) for whole numbers with gaps: ids, years, counts. `"category"` for text with few distinct values. Codes with leading zeros (ZIP codes) are text: `dtype={"zip": "str"}`.
- **Excel.** `pd.ExcelFile(path).sheet_names` lists the sheets. `Unnamed` columns mean the header is not where pandas looked: `skiprows=3` or `header=3`. To see the top of a sheet as it is: `header=None, nrows=8`.
- **Nested JSON.** `json.load` the file, find the list of records, `pd.json_normalize(records)`. Inner dictionaries become dotted columns (`device.type`). A missing key becomes NaN. For a list inside each record: `record_path="pages", meta=["session_id"]` gives one row per item.
- **Big files.** Peek (`nrows`), load fewer columns (`usecols`), use smaller dtypes, then `chunksize` and a running total. Sums and counts combine across chunks; a mean needs a sum and a count; a median cannot be done this way.
- **Writing.** Every reader has a writer. `to_csv(path, index=False)` for people and other tools: it forgets every dtype. `to_parquet(path)` for your future self: smaller, and dates, categories and `Int64` come back as they were.

WORKED EXAMPLE: A TYPED LOAD OF THE CHECKOUT EXPERIMENT

`ab_test.csv` has 28,000 rows, one per visitor in a checkout-page experiment (you will analyze it in Module 4). Here the only job is to get it in with the right types and see what that buys you. Follow the order: peek, load plainly, look, brief the reader, look again, save.

Peek at three rows. You learn the column names and can plan the real load.

```
print(pd.read_csv("data/ab_test.csv", nrows=3))
```

```
  visitor_id        date  variant   device  returning  converted  order_value
0    V024143  2025-09-08  control  desktop      False          0           NaN
1    V012790  2025-09-08  control  desktop      False          0           NaN
2    V004888  2025-09-08  control  desktop       True          0           NaN
```

Load it plainly and measure. Three text columns, and the date is one of them.

```
plain = pd.read_csv("data/ab_test.csv")
print(plain.dtypes)
print(round(plain.memory_usage(deep=True).sum() / 1024**2, 2), "MB")
```

```
visitor_id         str
date               str
variant            str
device             str
returning         bool
converted        int64
order_value    float64
dtype: object
2.15 MB
```

Brief the reader: the date is a date, and two columns hold only a few distinct values.

```
ab = pd.read_csv("data/ab_test.csv", parse_dates=["date"],
                 dtype={"variant": "category", "device": "category"})
print(ab.dtypes)
print(round(ab.memory_usage(deep=True).sum() / 1024**2, 2), "MB")
print(ab["date"].min().date(), "to", ab["date"].max().date())
```

```
visitor_id                str
date           datetime64[us]
variant              category
device               category
returning                bool
converted               int64
order_value           float64
dtype: object
1.12 MB
2025-09-08 to 2025-10-05
```

Check the translation with numbers you can reason about, then save a copy that remembers its types.

```
print(ab["converted"].sum(), ab["order_value"].notna().sum(), ab["order_value"].is
from pathlib import Path
Path("output").mkdir(exist_ok=True)
ab.to_parquet("output/ab_test.parquet")
print(pd.read_parquet("output/ab_test.parquet").equals(ab))
```

```
2645 2645 25355
True
```

The typed table uses far less memory than the plain one, the date has a real minimum and maximum, and the two counts agree: 2,645 visitors converted and exactly 2,645 rows have an order value. The other 25,355 order values are missing because those visitors bought nothing: missing here means 'not applicable', not 'lost'.

GUIDED LAB

- `labs/module-04/l2_2-loading-data/till_export_starter.py` (about 15 min): Load the till system's one-week export (`till_export_week08.txt`) so every column has the right dtype, then prove the load is right against `daily_sales.csv`.
- `labs/module-04/l2_2-loading-data/product_pages_starter.py` (about 15 min): In November 2025, which product pages were viewed most, and do visitors stay longer on product pages than on other pages?
- `labs/module-04/l2_2-loading-data/targets_vs_actuals_starter.py` (about 15 min): Which of the four stores beat its 2025 revenue target, and by how much?

1. Open `till_export_starter.py`. Run it once as it is and read the six raw lines it prints. On paper, list everything the reader needs to be told: _____
2. Write the `read_csv` call in TODO 1, one option at a time. Run after each option and watch `dtypes` change.

3. You are done when `revenue` is `float64`, `date` is `datetime64`, there are 28 rows, and the last lines say the totals match.
4. Then `product_pages_starter.py`: the answer is two levels down in the JSON. Write here which key holds the list you need: _____
5. Anything a lab saves goes to the `output/` folder next to the lab file (`OUTPUT`). Never write into the `data/` folder.
6. `targets_vs_actuals_starter.py` is homework (or a stretch task if you finish early).

◆ PREDICT, THEN RUN

Write what you think each one prints **before** you run it. Being wrong here is useful: it shows you exactly which rule to look at again.

P1. A code that looks like a number.

```
text = "sku,zip\nA1,02134\nB2,97701\n"
a = pd.read_csv(io.StringIO(text))
b = pd.read_csv(io.StringIO(text), dtype={"zip": "str"})
print(a["zip"].tolist(), b["zip"].tolist())
```

P2. A key that is not there.

```
recs = [{"id": 1, "geo": {"city": "Bend"}}, {"id": 2}]
flat = pd.json_normalize(recs)
print(flat.columns.tolist(), flat["geo.city"].isna().sum())
```

P3. A title above the header.

```
text = "Sales report,draft\nday,sales\nMon,10\nTue,12\n"
a = pd.read_csv(io.StringIO(text))
b = pd.read_csv(io.StringIO(text), skiprows=1)
print(a.columns.tolist(), a.shape, b.columns.tolist(), b["sales"].sum())
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

- Project: **The Pinecrest Snapshot** (lesson page, six steps). In class: Steps 1 and 2 in `lesson_2_2_project`.
- At home: Steps 3 to 6. You are done when the `snapshot` folder holds seven `.parquet` files and your verification table says `True` for every table.
- Then the lesson's Exercises 1 to 4, the Quick Quiz and your journal entry. Exercise 5 (what a round trip loses) is a stretch.
- Keep the `snapshot` folder.

MY NOTES

SELF-CHECK

Six questions on Lesson 2.2. No notes on the first pass; then check with your instructor.

Q7

What does this print?

```
f = io.StringIO("id,qty\n1,4\n2,\n3,6\n")
print(pd.read_csv(f, dtype={"qty": "Int64"})["qty"].tolist())
```

A) `[4.0, nan, 6.0]`

B) `[4, <NA>, 6]`

C) `[4, 0, 6]`

D) A `ValueError: Integer column has NA values`

Q8

A text export starts with two lines of notes, then the header `store;week;revenue`, then rows such as `Boise;2024-W01;55.400,22`. Which call loads `revenue` as a number?

A) `pd.read_csv(f, sep=";", skiprows=2, decimal=",", thousands=".")`

B) `pd.read_csv(f, sep=";", header=None)`

C) `pd.read_csv(f, skiprows=2, decimal=",")`

D) `pd.read_csv(f, sep=";", skiprows=3, decimal=",", thousands=".")`

Q9

What does this print?

```
recs = [{"id": 1, "tags": [{"t": "a"}, {"t": "b"}]},
        {"id": 2, "tags": [{"t": "c"}]}]
print(pd.json_normalize(recs, record_path="tags", meta=["id"]).shape)
```

Q10

You load `orders.csv` with no options. `orders["order_date"].dt.month` raises `AttributeError: Can only use .dt accessor with datetimelike values`. What is the right fix?

A) Add `index_col="order_date"` to the load

B) Convert the column with `.astype("int64")`

C) Add `encoding="latin-1"` to the load

D) Load with `parse_dates=["order_date"]`, or convert with `pd.to_datetime()`

Q11

What does this print?

```
df = pd.DataFrame({"a": [1, 2]})
df.to_csv("q11.csv")
print(pd.read_csv("q11.csv").columns.tolist())
```

Q12

A CSV file is larger than your computer's memory. You need the sum of one of its columns. Which approach is the right one?

A) Load the whole file, then delete the columns you do not need

B) Read the first 1,000 rows with `nrows` and multiply up

C) Read it with `usecols` and `chunksize`, keeping a running total

D) Open it in a spreadsheet program and save it as Excel first

NOTES

HOMEWORK CHECKLIST

- ☐ 2.1 project finished: my report says 8 star products and a median margin of 45.5%
- ☐ 2.1 exercises 1 to 4, Quick Quiz, journal entry
- ☐ 2.2 project finished: seven `.parquet` files, `True` on every row of the verification table

- ☐ 2.2 exercises 1 to 4, Quick Quiz, journal entry
- ☐ Labs finished: `busiest_days`, `recommend_groups`, `till_export`, `product_pages`; and `targets_vs_actuals`, `debug_filters.py` if I have time
- ☐ Peeked at `data/orders_messy.csv` with `nrows=8` and wrote down what looks wrong
- ☐ Bring to Session 5: laptop, this workbook, my notebooks. Nothing new to install

GLOSSARY FOR SESSION 4

- **Series:** a one-dimensional labeled array: values, an index and an optional name.
- **DataFrame:** a table: several Series side by side that share one index.
- **Index:** the row labels of a Series or DataFrame. The default is `RangeIndex`: 0, 1, 2, ...
- **dtype:** the type of every value in a column: `int64`, `float64`, `str`, `bool`, `datetime64[us]`, `category`, `Int64`.
- **NaN** / `<NA>`: pandas's markers for a missing value. Test for them with `.isna()`, never with `==`.
- **Alignment:** matching by label, not by position, when two Series or tables are combined.
- **Boolean mask:** a Series of True/False values used to keep the rows where it is True.
- `.loc` / `.iloc`: selection by label (slices include the end) / by integer position (slices exclude the stop).
- **Copy-on-write:** the rule that every object derived from another behaves as an independent copy.
- **Chained assignment:** `df[mask]["col"] = value`: two selections and then an assignment. It never changes `df`.
- **Nullable integer** (`Int64`): a whole-number dtype that can also hold a missing value.
- **Category:** a dtype that stores each distinct value once plus a small code per row; good for repetitive text.
- `json_normalize`: the pandas function that flattens nested dictionaries into dotted columns; `record_path` gives one row per item of an inner list.
- **Chunk:** one piece of a file read with `chunksize`, delivered as an ordinary DataFrame.
- **Parquet:** a binary, column-by-column file format that stores each column's dtype with the data.

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 5: Cleaning and Grouping

Your working pages for two of the most used skills in data work. Before the break: turning a damaged file into a table you can trust, and proving that you did. After the break: answering every question that starts with 'for each...'.

Lessons 2.3–2.4 | Session 5 · 3 hours | Name: _____

HOW TO USE THIS WORKBOOK

Each lesson has **By the end you can**, **Key ideas** (one page to keep), a **Worked example** with its real output, the **Guided lab** you do in class, three **Predict, then run** boxes, the lesson project's class and home parts, space for notes, and a six-question **Self-check**. Lab files are in `labs/module-05/`. Two habits for today: **count the rows before and after every step**, and, after the break, **ask what shape you want back** before you type a groupby.

SESSION GOALS

- Audit a table you have never seen and write a repair list before changing anything
- Repair structure, text, numbers and dates in code, on a copy, checking each step
- Decide what to do with each kind of missing or suspicious value, and say why
- Summarize 'for each...' questions with `groupby` and named aggregation
- Choose between `agg`, `transform` and `filter` by the shape of the answer
- Catch the quiet traps: lost rows, `count` versus `size`, averages of averages

LESSONS

2.3 Cleaning Data

100 min lesson

BY THE END YOU CAN

- Audit an unfamiliar table and list its problems before changing anything
- Repair structure: untidy headers, summary rows and duplicate rows
- Standardize inconsistent text and convert numbers and dates that were stored as text, including dates in several formats
- Decide, and defend, what to do with each kind of missing or suspicious value: repair, fill, flag or leave
- Verify a cleaning job with a before-and-after audit and assertions, and package it as a function you can rerun

KEY IDEAS

- **The repair bench.** Audit first and write the repair list. Fix one thing at a time, in code, on a copy. Check each repair. The raw file is read-only.
- **The order.** Structure (headers, rows that are not data, duplicates) → text → numbers → dates → missing values. Fixing the shape first makes every later step simpler.
- **Audit by counting.** `df.dtypes`, `df.isna().sum()`, `df.nunique()`, `df.duplicated().sum()`. Print `df.columns.tolist()` to see hidden spaces. A category column with more distinct values than the business has is damaged.
- **Text.** `.str.strip()`, then `.str.title()` / `.str.lower()` / `.str.upper()`, then a mapping dictionary with `.replace({...})`, built from `value_counts()` **after** the mechanical steps. Placeholders such as `none` and `-` become real missing values with `.where(...)`.
- **Numbers.** `pd.to_numeric(s, errors="coerce")` is a detector: what became NaN but was not empty? Remove symbols with `.str.replace("$", "")`, then convert without `errors=` as a guarantee. Then `.describe()`: is the minimum and maximum plausible?
- **An outlier is a question.** Never change a value for being extreme. Look for independent evidence (here: the order's item lines), then repair only the rows the evidence points to, with one `.loc`.
- **Dates.** One `pd.to_datetime(d, format=..., errors="coerce")` per known format, combined with `.fillna()`. Impossible dates become `NaT` and stay missing.
- **Missing values are decisions.** It is not missing (no promo code) · repair (a rule or another source) · leave and report · drop for one analysis with `dropna(subset=[...])`. Filling a number invents data. Write the decision down.

- **Prove it.** Run the audit again and compare. State the rules as `assert` lines. Package the job as a function so it can be rerun.

WORKED EXAMPLE: WHAT IS MISSING IN THE SURVEY, AND WHAT SHOULD WE DO ABOUT IT?

`survey.csv` is tidy: no bad headers, no duplicates. Its problem is the quiet one, missing answers. This example practices the part of cleaning that is judgment, not typing: look at what is missing, find out why, and decide.

Audit first. Seven columns have gaps, and two of them have hundreds.

```
survey = pd.read_csv("data/survey.csv")
print(len(survey), survey.duplicated(subset=["customer_id"]).sum())
missing = survey.isna().sum()
print(missing[missing > 0])
```

```
1800 0
sat_overall       53
sat_selection     53
sat_price         47
sat_staff        605
sat_website      472
recommend         54
comment          977
dtype: int64
```

What would `dropna()` do? Print the row count before and after every drop.

```
print(len(survey), "->", len(survey.dropna()))
print(len(survey), "->", len(survey.dropna(subset=["sat_overall", "recommend"])))
```

```
1800 -> 295
1800 -> 1699
```

Why are 605 staff ratings missing? Compare the share of missing answers for online shoppers and store shoppers (the mean of a True/False column is a share).

```
online = survey["store"] == "Online"
for label, rows in [("online shoppers", online), ("store shoppers ", ~online)]:
    print(label, round(survey.loc[rows, "sat_staff"].isna().mean() * 100, 1), "% m
          round(survey.loc[rows, "sat_website"].isna().mean() * 100, 1), "% missi
```

```
online shoppers 69.9 % missing staff rating, 2.9 % missing website rating
store shoppers  3.0 % missing staff rating, 45.9 % missing website rating
```

What would filling claim? Compare the mean as it is with the mean after filling every gap with a neutral 3.

```
print(survey["sat_staff"].mean().round(2), survey["sat_staff"].fillna(3).mean().ro
assert survey["sat_staff"].dropna().between(1, 5).all()
assert survey["recommend"].dropna().between(0, 10).all()
print("all checks passed")
```

```
4.02 3.68
all checks passed
```

Decision: leave the gaps and report them. Seven in ten online shoppers skipped the staff question because they never met staff: for them the honest value is 'does not apply', and filling it with 3 would pull the staff score from 4.02 down to 3.68 with invented answers. Dropping every row with any gap would keep 295 of 1,800 responses. What you write in the report: "Staff satisfaction is based on the 1,195 customers who answered, almost all of them store shoppers."

GUIDED LAB

- `labs/module-05/l2_3-cleaning-data/register_log_starter.py` (about 25 min): Clean the Asheville manager's hand-typed log of May 2025 (`asheville_may_log.csv`) and score it against the same 31 days of `daily_sales.csv`.
- `labs/module-05/l2_3-cleaning-data/total_row_check_starter.py` (about 20 min): If someone simply added up `Order Total` in `orders_messy.csv`, how far off would they be, and which flaw moves the sum the most?

1. Open `register_log_starter.py` and run it once. Read the audit it prints. **Before you fix anything**, write your repair list (in the file, or here): _____
2. Work in the lesson's order: structure, text, numbers, dates, missing values. Run after every TODO and check the row count.
3. TODO 3 has a trap: one revenue figure is about 100 times too large, and one large figure is genuine. What evidence in the same row tells them apart? _____
4. You are done when the last line says `cells compared: 217 | still blank: 2 | wrong: 0`. Two honest blanks are not a failure.
5. Never edit `asheville_may_log.csv` itself. Every fix is a line of code.
6. If you finish: TODO 6 (repair one blank using the TOTAL row), then `total_row_check_starter.py`.

PREDICT, THEN RUN

Write what you think each one prints **before** you run it. Being wrong here is useful: it shows you exactly which rule to look at again.

P1. Case without whitespace.

```
s = pd.Series([" Bend", "bend ", "BEND", "Bend OR"])
print(s.str.title().nunique(), s.str.strip().str.title().nunique())
```

P2. A detector for values that are not numbers.

```
s = pd.Series(["$1,200.50", "80", "free", None])
n = pd.to_numeric(s.str.replace("$", "").str.replace(",", ""), errors="coerce")
print(n.tolist(), n.isna().sum())
```

P3. Three ways to lose rows.

```
df = pd.DataFrame({"id": [1, 2, 2, 3], "promo": [None, "A", "A", None],
                   "amt": [5.0, 7.0, 7.0, None]})
print(len(df.drop_duplicates()), len(df.dropna()), len(df.dropna(subset=["amt"])))
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

- Project: **The Cleaning Pipeline** (lesson page, six steps). In class: Step 1 `fix_structure` and Step 2 `fix_text` in `lesson_2_3_project`.
- At home: Steps 3 to 6. You are done when `clean_orders()` runs from the raw file to a 20,302-row table, `validate()` passes, and your scorecard says 0 wrong and 623 still blank.
- Test `validate()`: temporarily remove one store fix and confirm that it fails loudly.
- Then the lesson's Exercises 1 to 3, the Quick Quiz and your journal entry. Exercise 4 is a stretch.

- Keep `output/orders_clean.parquet` and your `clean_orders()` function.

MY NOTES

SELF-CHECK

Six questions on Lesson 2.3. No notes on the first pass; then check with your instructor.

Q1

An audit shows that the `Store` column has 31 distinct values. Pinecrest has four stores and an online shop. What is the right first move?

A) Delete the rows whose store name is unusual

B) Print `value_counts()` to see the variants, then strip, fix the case, and map what is left

C) Convert the column to the `category` dtype

D) Fill the unusual names with the most common store

Q2

What does this print?

```
d = pd.Series(["2024-03-05", "03/05/2024", "March 5, 2024", "2024-02-30"])
iso = pd.to_datetime(d, format="%Y-%m-%d", errors="coerce")
us = pd.to_datetime(d, format="%m/%d/%Y", errors="coerce")
print(iso.notna().sum(), iso.fillna(us).notna().sum())
```

Q3

What does this print?

```
s = pd.Series(["none", "TRAIL15", "-", "trail15"])
p = s.str.upper()
print(p.where(~p.isin(["NONE", "-"])).tolist())
```

A) `['NONE', 'TRAIL15', '-', 'TRAIL15']`

B) `['TRAIL15', 'TRAIL15']`

C) `[nan, 'TRAIL15', nan, 'TRAIL15']`

D) `[nan, 'TRAIL15', nan, 'trail15']`

Q4

In `orders_messy.csv` one order total is $4,990.00, while the largest genuine order is $4,893.00. What justified dividing the $4,990.00 by 100?

A) It is above the 99th percentile of order totals

B) It is an online order, and online totals are less reliable

C) It is a round number, which suggests a typing error

D) Its item lines add up to $34.95, so the recorded total is over 100 times what was bought

Q5

What does this print?

```
df = pd.DataFrame({"total": [10.0, None, 50.0]})
print(df["total"].mean(), df["total"].fillna(0).mean())
```

Q6

`trial` has 406 missing payment methods. What happens when you run `trial["payment_method"].fillna("unknown", inplace=True)`?

A) `trial` is unchanged, and pandas warns with `ChainedAssignmentError`

B) The 406 gaps in `trial` are filled with `unknown`

C) A `TypeError`, because text cannot be used to fill a column

D) The gaps are filled, but only in the first 406 rows

2.4 GroupBy Operations

90 min lesson

BY THE END YOU CAN

- Explain split-apply-combine and read a groupby expression piece by piece
- Summarize groups with built-in methods and with named aggregation, on one key or several
- Choose correctly between `agg` (one row per group), `transform` (one row per original row) and `filter` (keep or drop whole groups)
- Control what happens to missing keys and unused categories with `dropna=` and `observed=`
- Recognize the common traps: lost rows, a key hidden in the index, and averages of averages

KEY IDEAS

- **Split, apply, combine.** `df.groupby("key")["column"].function()`: deal the rows into piles by key, compute on each pile, collect one answer per pile. Any question that starts 'for each...' is a groupby.
- **What comes back.** A Series or DataFrame whose **index is the group key**, sorted. Get the key back as a column with `.reset_index()` or `as_index=False`.
- **Counting.** `.size()` rows per group · `.count()` non-missing values of a column · `.nunique()` distinct values. They answer different questions.
- **Named aggregation.** `df.groupby("store").agg(orders=("order_id", "count"), revenue=("order_total", "sum"))`. New column name = (source column, function). Learn this form by heart.
- **Rates.** Make a True/False helper column; its mean within each group is the rate.
- **Several keys.** A list of keys gives one group per combination and a two-level index (MultiIndex). Flatten with `.reset_index()` or spread one level into columns with `.unstack()`. A key can be computed on the spot: `orders["order_date"].dt.year`.
- **What shape do I want back?** One row per group: `agg`. The same rows with a group number on each: `transform`. Fewer of the same rows: `filter` (or a `transform` and a mask, which is faster).
- **Quiet traps.** Rows whose key is missing vanish unless `dropna=False`: check that the groups add up to `len(df)`. Never average group means of unequal groups. `observed=False` shows categories that have no rows.

WORKED EXAMPLE: WHAT DO DIFFERENT AGE GROUPS THINK OF OUR PRICES?

`survey.csv` again, with a 'for each' question: for each age group, how satisfied are customers with prices and with the website (1 = very dissatisfied, 5 = very satisfied)? The example uses the four moves you will repeat in the lab: named aggregation, a check that the groups add up, `transform`, and `filter`.

One row per age group. `count` on `response_id` counts responses; `count` on `sat_price` counts answers, because `count` skips missing values.

```
by_age = survey.groupby("age_group").agg(
    responses=("response_id", "count"),
    answered_price=("sat_price", "count"),
    price=("sat_price", "mean"),
    website=("sat_website", "mean"),
)
print(by_age.round(2))
```

```
           responses  answered_price  price  website
age_group
18-24            192             186   2.80     3.69
25-34            280             273   3.32     3.67
35-44            318             309   3.38     3.67
45-54            339             332   3.38     3.66
55-64            279             273   3.44     3.72
65+              392             380   3.33     3.04
```

Do the groups add up? And is the average of the six group averages the overall average?

```
print(by_age["responses"].sum(), len(survey))
print(by_age["price"].mean().round(3), survey["sat_price"].mean().round(3))
```

```
1800 1800
3.276 3.309
```

`transform` gives one value per original row: each response next to its own age group's average.

```
survey["age_avg_price"] = survey.groupby("age_group")["sat_price"].transform("mean
print(survey["age_avg_price"].shape)
print(survey[["response_id", "age_group", "sat_price", "age_avg_price"]].head(3).r
```

```
(1800,)
   response_id age_group  sat_price  age_avg_price
0            1       65+        5.0           3.33
1            2     18-24        3.0           2.80
2            3     25-34        4.0           3.32
```

`filter` keeps whole groups: here, only the responses from stores with at least 250 responses.

```
busy = survey.groupby("store").filter(lambda g: len(g) >= 250)
print(len(busy), sorted(busy["store"].unique()))
```

```
1153 ['Boise', 'Online']
```

Two groups stand out. Customers aged 18 to 24 rate prices at 2.80 when every other group is between 3.32 and 3.44, and customers aged 65 and over rate the website at 3.04 when every other group is near 3.7. Those are leads to investigate, not conclusions: Module 4 tests whether gaps like these could be chance. The average of the six group averages (3.276) is not the overall average (3.309), because the groups are different sizes.

GUIDED LAB

- `labs/module-05/l2_4-groupby-operations/store_days_starter.py` (about 20 min): For each store: days, revenue and conversion; how much a promo day brings in; whether the lesson's growth claim (+19.7% Flagstaff, -5.0% Asheville) holds; and how many days were at least twice the store's own average.
- `labs/module-05/l2_4-groupby-operations/basket_sizes_starter.py` (about 20 min): How big is a typical basket, are the best sellers by units the best sellers by revenue, what share of its order is each line, and which orders have six or more lines?

Extra: `labs/module-05/l2_4-groupby-operations/debug_groupby.py`. A file that is broken on purpose. Two bugs stop the program; three are quiet. Each has a 'should print' note.

1. Open `store_days_starter.py`. Before each TODO, write the shape of the answer: how many rows will come back? TODO 1: ____ TODO 2: ____ TODO 3: ____ TODO 4: ____
2. TODO 1 has two counts, `days` and `days_with_revenue`. Why are they different?
   _____
3. TODO 2: before you run the last line, guess the average revenue of a store-day: _____
4. After every groupby, check that the groups add up to the number of rows in the table.
5. If you finish: `basket_sizes_starter.py`, then `debug_groupby.py`.

PREDICT, THEN RUN

Write what you think each one prints **before** you run it. Being wrong here is useful: it shows you exactly which rule to look at again.

P1. An average of averages.

```
df = pd.DataFrame({"team": ["a", "b", "a", "b", "a"], "pts": [2, 10, 4, 20, 6]})
means = df.groupby("team")["pts"].mean()
print(means.tolist(), means.mean(), df["pts"].mean())
```

P2. A key with gaps.

```
df = pd.DataFrame({"code": ["X", None, "X", None], "amt": [1, 2, 3, 4]})
print(len(df.groupby("code")), len(df.groupby("code", dropna=False)),
      df.groupby("code")["amt"].sum().sum())
```

P3. Three shapes.

```
df = pd.DataFrame({"k": ["a", "a", "b"], "v": [1, 2, 3]})
a = df.groupby("k")["v"].sum()
t = df.groupby("k")["v"].transform("sum")
f = df.groupby("k").filter(lambda g: len(g) > 1)
print(len(a), len(t), len(f))
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

- Project: **The Store Scorecard** (lesson page, six steps). In class: Step 1 (helper columns) and Step 2 (the scorecard) in `lesson_2_4_project`.
- At home: Steps 3 to 6. You are done when completed revenue totals $3,624,306.27, `customers` has 3,942 rows, and both CSV files are in `output/`.
- Then the lesson's Exercises 1 to 4, the Quick Quiz and your journal entry. Exercise 5 is a stretch.
- Keep `output/customer_summary.csv` for Session 6.

MY NOTES

SELF-CHECK

Six questions on Lesson 2.4. No notes on the first pass; then check with your instructor.

Q7

What does this print?

```
df = pd.DataFrame({"store": ["a", "a", "b"], "amt": [10, 30, 5]})
r = df.groupby("store")["amt"].agg(["count", "mean"])
print(r.shape, r.loc["a", "mean"])
```

A) `(3, 2) 20.0`

B) `(2, 2) 40.0`

C) `(2,) 20.0`

D) `(2, 2) 20.0`

Q8

What does this print?

```
df = pd.DataFrame({"k": ["x", "x", "y"], "v": [1, 3, 10]})
df["share"] = df["v"] / df.groupby("k")["v"].transform("sum")
print(df["share"].tolist())
```

Q9

`orders` has a True/False column `returned`. Which expression gives the fraction of each channel's orders that were returned?

A) `orders.groupby("channel")["returned"].mean()`

B) `orders.groupby("channel")["returned"].count()`

C) `orders.groupby("channel")["returned"].size()`

D) `orders.groupby("channel")["returned"].nunique()`

Q10

`orders.groupby("promo_code").size().sum()` gives 2,148, but `orders` has 20,302 rows. Why?

A) `size()` skips rows whose order total is missing

B) `groupby` removes duplicate rows before it counts

C) Rows whose key is missing are dropped by default; `dropna=False` keeps them as a group

D) `promo_code` is text and must be a category before grouping

Q11

What does this print?

```
df = pd.DataFrame({"g": ["a", "a", "b"], "x": [1.0, None, 3.0]})
print(df.groupby("g")["x"].size().tolist(), df.groupby("g")["x"].count().tolis
```

Q12

You want the original order rows of every customer who placed 25 or more orders. Which call returns them?

A) `orders.groupby("customer_id")["order_id"].count() >= 25`

B) `orders.groupby("customer_id").filter(lambda g: len(g) >= 25)`

C) `orders.groupby("customer_id").agg(n=("order_id", "count"))`

D) `orders.groupby("customer_id")["order_id"].transform("count")`

NOTES

HOMEWORK CHECKLIST

- ☐ 2.3 project finished: `clean_orders()` runs, `validate()` passes, 0 wrong cells, 623 still blank
- ☐ 2.3 exercises 1 to 3, Quick Quiz, journal entry
- ☐ 2.4 project finished: revenue $3,624,306.27, 3,942 customers, both CSV files saved
- ☐ 2.4 exercises 1 to 4, Quick Quiz, journal entry
- ☐ Labs: `register_log` (and TODO 6), `store_days`; then `total_row_check`, `basket_sizes`, `debug_groupby.py`
- ☐ Counted the distinct customer ids in `orders.csv` and the rows in `customers.csv`, and wrote down which is larger
- ☐ Bring to Session 6: laptop, this workbook, `output/customer_summary.csv`. Nothing new to install

GLOSSARY FOR SESSION 5

- **Audit:** a systematic count of what is in a table (dtypes, missing values, distinct values, duplicates) before changing anything.

- **Raw file:** the data as received. Read-only: all fixes are code applied to a copy.
- **Duplicate:** a row that is an exact copy of an earlier row (`duplicated()`, `drop_duplicates()`); `subset=` compares only some columns.
- **Placeholder:** text that stands for 'nothing', such as `none`, `-` or `pending`. It must become a real missing value.
- **Coerce:** `errors="coerce"`: convert what can be converted and turn the rest into NaN or NaT instead of raising an error.
- **NaT:** 'not a time': the missing value of a datetime column.
- **Mapping dictionary:** a dictionary of wrong spelling to right spelling, applied with `.replace()`; it documents each judgment call.
- **Outlier:** an extreme value. A question to investigate with independent evidence, not an error by itself.
- **Assertion:** an `assert` line that states a rule and stops the program when the rule is broken.
- **Pipeline:** cleaning steps packaged as functions and chained (`.pipe()`), so the whole job can be rerun.
- **Split-apply-combine:** the groupby pattern: split rows into groups by a key, compute on each group, combine the answers.
- **Aggregation:** reducing each group to one row (`sum`, `mean`, `agg`).
- **Named aggregation:** `agg(new_name=("column", "function"))`: one output column per argument.
- **Transform:** a group calculation that returns one value per original row.
- **Filter (groupby):** keeping or dropping whole groups according to a test on the group.
- **MultiIndex:** an index with more than one level, as produced by grouping on several keys.
- **Average of averages:** the mean of group means; it equals the overall mean only when the groups are the same size.

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 6: Merging and Reshaping

Your working pages for connecting tables and changing their shape. Before the break: `merge`, the four kinds of join, and the two silent ways a merge goes wrong. After the break: pivot tables, crosstabs and `melt`, and the checks to run before you trust a grid.

Lessons 2.5–2.6 | Session 6 · 3 hours | Name: _____

HOW TO USE THIS WORKBOOK

Each lesson has **By the end you can**, **Key ideas** (one page to keep), a **Worked example** with its real output, the **Guided lab** you do in class, three **Predict, then run** boxes, the lesson project's class and home parts, space for notes, and a six-question **Self-check**. Lab files are in `labs/module-06/`. Two habits for today: **count before and after every merge**, and **check that the corner of a pivot matches the source**.

SESSION GOALS

- Combine two tables on a key and choose the right kind of join
- Predict the row count of a merge, check it, and find the keys that did not match
- State the relationship between two tables and enforce it with `validate=`
- Avoid double counting after a one-to-many merge
- Build grids with `pivot_table` and `crosstab`, with totals and percentages
- Move between wide and tidy layouts with `melt`, and check a pivot before you trust it

LESSONS

2.5 Merging and Joining

95 min lesson

BY THE END YOU CAN

- Combine two tables on a key with `merge()` and choose between inner, left, right and outer joins
- Find the keys that have no match on either side with `indicator=True`, and decide what to do about them
- State the relationship between two tables (one-to-one, many-to-one, many-to-many) and enforce it with `validate=`
- Predict and check the row count of a merge, and avoid double counting after a one-to-many merge
- Merge on differently named or multiple keys, stack tables with `concat()`, and recognize when `merge_asof()` is the right tool

KEY IDEAS

- **The claim ticket.** A key is a column that appears in both tables. `left.merge(right, on="key")` finds, for each left row, the right rows with the same key. Two questions: what if a key has no match? What if it matches more than once?
- **Four joins.** `how="inner"` (the default) keeps only keys in both tables · `"left"` keeps every row of the left table · `"right"` every row of the right · `"outer"` every row of both. Missing partners become NaN.
- **Count before, count after.** Say what the row count should be, then check. Fewer rows: keys failed to match. More rows: keys matched more than once.
- **The working default.** Adding columns to a main table? `how="left"`. Select only the columns you need from the right table (key included) before merging.
- `indicator=True` adds `_merge`: `both`, `left_only`, `right_only`. Unmatched rows are findings with causes (closed accounts, discontinued products). Decide what to do with them; do not let the default decide.
- **Relationships.** one-to-one, many-to-one (safe: the row count cannot grow), one-to-many (rows multiply: one per match), many-to-many (usually a mistake). `validate="many_to_one"` makes pandas check that the key is unique where you believe it is.
- **Know what one row means.** After a one-to-many merge, columns from the 'one' side are repeated. Never sum or average them there. Aggregate first, then merge.
- **Awkward keys.** Different names: `left_on=`, `right_on=`. Several columns: lists. Clashing column names: `suffixes=`. Keys must have the same dtype. Missing keys match each other.

- **Other tools.** `pd.concat([a, b], ignore_index=True)` stacks tables with the same columns. `pd.merge_asof` matches the nearest earlier key (both tables sorted).

WORKED EXAMPLE: WHICH PRODUCTS NEVER SOLD, AND WHICH SALES HAVE NO PRODUCT?

A question that needs two tables: the catalog (`products.csv`, one row per product) and the order lines (`order_items.csv`, one row per line). The pattern is the one you will use in the lab: **aggregate** the long table to one row per key, **then merge**, and audit both sides.

Aggregate first: one row per SKU that was ever sold. Then check the keys before merging anything.

```
items["line_total"] = items["quantity"] * items["unit_price"]
sold = items.groupby("sku", as_index=False).agg(units=("quantity", "sum"), revenue
print(len(products), len(sold), products["sku"].is_unique, sold["sku"].is_unique)
```

```
242 240 True True
```

Prediction: catalog on the left, one-to-one, 242 rows. A left join keeps every product.

```
catalog = products[["sku", "name", "category", "launch_date"]].merge(
    sold, on="sku", how="left", validate="one_to_one", indicator=True)
print(len(catalog), catalog["_merge"].value_counts().to_dict())
print(catalog.loc[catalog["_merge"] == "left_only", ["sku", "name", "launch_date"]
```

```
242 {'both': 234, 'left_only': 8, 'right_only': 0}
         sku                                name launch_date
49  PCK-1002    Pinecrest Headwater 30L Daypack  2025-12-20
56  PCK-1009  Juniper Vane Sawtooth 30L Daypack  2025-12-28
89  FTW-1018     Granite Fork Wayfinder Insoles  2025-12-28
```

Now audit the other side with an outer join: are there sales with no product in the catalog?

```
audit = products[["sku"]].merge(sold, on="sku", how="outer", indicator=True)
print(audit["_merge"].value_counts().to_dict())
gone = audit[audit["_merge"] == "right_only"]
print(gone["sku"].tolist(), int(gone["units"].sum()), round(gone["revenue"].sum(),
```

```
{'both': 234, 'left_only': 8, 'right_only': 6}
['APL-1033', 'KIT-1016', 'PCK-1015', 'PCK-1025', 'SNW-1007', 'SNW-1018'] 1193 364
```

With names attached, the best sellers are readable.

```
print(catalog.nlargest(3, "revenue")[["sku", "name", "units", "revenue"]])
```

```
          sku                                name   units    revenue
224  ACC-1015  Cedar Notch Sawtooth First-Aid Kit  3475.0  159029.25
100  APL-1001    Granite Fork Cascade Rain Jacket   420.0  117579.00
48   PCK-1001        Wrenfield Canyon 22L Daypack   784.0  107368.80
```

Eight products have never sold: all of them launched on or after 2025-12-20, at the very end of the data, so that is no surprise. Six SKUs have sales but no catalog entry: discontinued products, last sold in 2024. Both groups of unmatched keys have a cause, and neither is an error. A default inner join would have hidden both without a word.

GUIDED LAB

- `labs/module-06/l2_5-merging-joining/customer_profiles_starter.py` (about 25 min): Who are the five biggest spenders by name, how many account holders never ordered (and in which tier), and which orders belong to nobody in `customers.csv`?
- `labs/module-06/l2_5-merging-joining/orders_weather_starter.py` (about 20 min): On rainy days, how many orders do signed-in customers place in each store, compared with dry days?

Extra: `labs/module-06/l2_5-merging-joining/debug_merges.py`. A file that is broken on purpose. Three bugs stop the program; two are quiet. Each has a 'should print' note.

1. Open `customer_profiles_starter.py`. The pattern is **aggregate first, then merge**.
2. Before TODO 2, write your prediction. Left table: _____ Relationship: _____ Rows after the merge: _____
3. After every merge, count the rows. Fewer than you predicted: keys failed to match. More: keys matched more than once.
4. TODO 3 fills missing order counts with 0. Why is that honest here, when filling a missing order total with 0 was wrong last session? _____
5. If you finish: `orders_weather_starter.py` (which table goes on the left, and why does it matter?), then `debug_merges.py`.

PREDICT, THEN RUN

Write what you think each one prints **before** you run it. Being wrong here is useful: it shows you exactly which rule to look at again.

P1. An outer join with an indicator.

```
left = pd.DataFrame({"k": [1, 2, 2], "a": ["x", "y", "z"]})
right = pd.DataFrame({"k": [2, 2, 3], "b": [10, 20, 30]})
m = left.merge(right, on="k", how="outer", indicator=True)
print(len(m), (m["_merge"] == "both").sum(), (m["_merge"] == "left_only").sum())
```

P2. Missing keys.

```
a = pd.DataFrame({"k": [1.0, np.nan], "a": ["x", "y"]})
b = pd.DataFrame({"k": [np.nan, np.nan], "b": ["p", "q"]})
print(len(a.merge(b, on="k")), len(a.merge(b, on="k", how="left")))
```

P3. An average after a merge.

```
o = pd.DataFrame({"order": [1, 2], "total": [300, 100]})
i = pd.DataFrame({"order": [1, 1, 1, 2], "qty": [1, 1, 1, 1]})
print(o["total"].mean(), o.merge(i, on="order")["total"].mean())
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

- Project: **Revenue by Category** (lesson page, seven steps). In class: Steps 1 to 3 in `lesson_2_5_project` (inspect the keys, lines + orders, + products with an indicator).
- At home: Steps 4 to 7. You are done when `lines` still has 45,490 rows, the six discontinued SKUs have their category back, completed merchandise revenue is $3,715,829.95, and you can explain the 1.7% pricing trap in one sentence.
- Then the lesson's Exercises 1 to 4, the Quick Quiz and your journal entry. Exercise 5 is a stretch.
- Keep `output/order_lines.parquet`.

MY NOTES

SELF-CHECK

Six questions on Lesson 2.5. No notes on the first pass; then check with your instructor.

Q1

What does this print (the lengths of an inner, a left and an outer merge)?

```
a = pd.DataFrame({"k": [1, 2, 3], "x": ["p", "q", "r"]})
b = pd.DataFrame({"k": [2, 3, 3, 4], "y": [10, 20, 30, 40]})
print(len(a.merge(b, on="k")), len(a.merge(b, on="k", how="left")),
      len(a.merge(b, on="k", how="outer")))
```

A) `2 3 4`

B) `3 3 5`

C) `3 4 5`

D) `3 4 4`

Q2

You want to add each customer's loyalty tier to `orders` without losing any order, and you want the merge to fail loudly if `customers` ever contains a duplicated id. Which call does both?

A) `orders.merge(customers, on="customer_id")`

B) `orders.merge(customers, on="customer_id", how="left", validate="many_to_one")`

C) `orders.merge(customers, on="customer_id", how="inner", validate="one_to_one")`

D) `pd.concat([orders, customers])`

Q3

What does this print?

```
o = pd.DataFrame({"order": [1, 2], "total": [100, 50]})
i = pd.DataFrame({"order": [1, 1, 1, 2], "sku": ["a", "b", "c", "d"]})
m = o.merge(i, on="order")
print(m["total"].sum(), m.drop_duplicates("order")["total"].sum())
```

Q4

`orders.merge(customers, on="customer_id", how="outer", indicator=True)` shows 739 rows marked `left_only`. What are they?

A) Orders whose `customer_id` has no row in `customers`

B) Customers who have never placed an order

C) Orders that matched more than one customer

D) Duplicate orders that the merge removed

Q5

What does this print?

```
a = pd.DataFrame({"store": ["Boise"], "rev": [100]})
b = pd.DataFrame({"store": ["Bend"], "rev": [80], "visitors": [300]})
c = pd.concat([a, b], ignore_index=True)
print(c.shape, c["visitors"].isna().sum())
```

Q6

`daily` has the columns `date` and `store`; `weather` has `date` and `city` (the same four place names). Each has one row per place per day. Which call attaches the weather to every sales row?

A) `daily.merge(weather, on=["date", "store"])`

B) `daily.merge(weather, on="date", how="left")`

C) `pd.concat([daily, weather], ignore_index=True)`

D) `daily.merge(weather, left_on=["date", "store"], right_on=["date", "city"], how="left", validate="one_to_one")`

2.6 Pivot Tables and Reshaping

85 min lesson

BY THE END YOU CAN

- Build a summary grid with `pivot_table()` by choosing its rows, columns, values and aggregation, and add totals with `margins=True`
- Count and compare categories with `pd.crosstab()`, including row and column percentages
- Choose between `pivot()` and `pivot_table()`, and explain the "duplicate entries" error
- Recognize wide and tidy (long) layouts and convert between them with `melt()`, `stack()` and `unstack()`
- Check a pivot before you trust it: its aggregation, its cell counts, its empty cells and its totals

KEY IDEAS

- **Logbook and scoreboard.** Long (tidy) data has one row per observation: it is for analysis. Wide data spreads one variable across the header: it is for reading. Tidy means each variable is a column, each observation a row, each cell one value.
- **Four questions.** `df.pivot_table(index=..., columns=..., values=..., aggfunc=...)`: what is each row, what is each column, which number goes in the cells, how are many rows combined? A pivot table is a groupby spread into a grid.
- **Always write `aggfunc=`.** The default is the mean, not the sum.
- **Totals and empty cells.** `margins=True` adds totals computed from all the underlying rows. A combination that never occurs is NaN: `fill_value=0` is right for counts and sums, wrong for averages. The corner of a count pivot should equal the number of source rows.
- `pd.crosstab(a, b)` counts combinations. `normalize="index"`: each row sums to 1 ('of online orders, what share were returned?'). `normalize="columns"`: each column sums to 1 ('of returned orders, what share were online?'). Different questions.
- `pivot()` **versus** `pivot_table()`. `pivot()` only rearranges and fails with 'duplicate entries' when a cell would get several rows. That error is information about your data.
- `melt(id_vars=..., value_vars=..., var_name=..., value_name=...)` turns header labels back into a column. `stack()` and `unstack()` do the same moves on index levels.
- **Labels are sorted.** Month and weekday names come out alphabetical. Pivot on numbers, or `reindex` in the order you want.
- **Before you trust a pivot.** What is one row of the source? What is the aggregation? How many rows are behind each cell? What do empty cells mean? Do the totals reconcile? Does a total hide a difference inside it?

WORKED EXAMPLE: ON WHICH DAYS DO PEOPLE ORDER, ONLINE AND IN STORE?

A counting question with a trap in it. The orders are a logbook; the answer is a small grid: weekdays down the side, the two channels across the top. Watch the label order, and watch which direction the percentages run.

Count the combinations with `crosstab`. The rows come out in alphabetical order: Friday first.

```
day = orders["order_date"].dt.day_name().rename("day")
counts = pd.crosstab(day, orders["channel"], margins=True)
print(counts)
```

```
channel    online  store    All
day
Friday       1677   1098   2775
Monday       2023    874   2897
Saturday     1789   1487   3276
Sunday       2004   1332   3336
Thursday     1727    913   2640
Tuesday      1853    859   2712
Wednesday    1781    885   2666
All         12854   7448  20302
```

Put the days in calendar order with `reindex`, and check that the corner is the number of orders.

```
week = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunda
counts = counts.reindex(week + ["All"])
print(counts.index.tolist())
print(counts.loc["All", "All"], len(orders))
```

```
['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday', 'A
20302 20302
```

Raw counts mislead, because there are more online orders on every day. Make each **column** add up to 100%: of each channel's orders, what share falls on each day?

```
share = pd.crosstab(day, orders["channel"], normalize="columns").reindex(week) * 1
print(share.round(1))
```

```
channel    online  store
day
Monday       15.7   11.7
Tuesday      14.4   11.5
Wednesday    13.9   11.9
Thursday     13.4   12.3
Friday       13.0   14.7
Saturday     13.9   20.0
Sunday       15.6   17.9
```

The grid is for people. Melt it back to a logbook for a program (a chart in Module 3 would want this shape).

```
tidy = share.round(1).reset_index().melt(id_vars="day", var_name="channel", value_
print(tidy.shape)
print(tidy.sort_values("pct", ascending=False).head(3))
```

```
(14, 3)
         day channel   pct
12  Saturday   store  20.0
13    Sunday   store  17.9
0     Monday  online  15.7
```

Stores are a weekend business: Saturday and Sunday bring 20.0% and 17.9% of in-store orders, when an even week would give 14.3% a day. Online ordering is flatter, with Monday and Sunday on top at 15.7% and 15.6%. The column percentages answer 'of store orders, what share is on a Saturday?'. Row percentages would answer a different question: 'of Saturday's orders, what share is in a store?'.

GUIDED LAB

- `labs/module-06/l2_6-pivot-tables/payment_returns_starter.py` (about 15 min): Are orders paid one way returned more often than orders paid another way?
- `labs/module-06/l2_6-pivot-tables/survey_by_age_starter.py` (about 15 min): Which survey question does each age group score lowest, and how many answers are behind each average?
- `labs/module-06/l2_6-pivot-tables/weather_grid_starter.py` (about 15 min): On how many days was each city the warmest, what is the average high by month, and what goes wrong when months are names?

1. Open `payment_returns_starter.py`. Before each pivot, say the four things: rows _____ columns _____ values _____ how to combine _____
2. TODO 2: one cell is empty. Should you fill it with 0? Yes / No, because _____
3. TODO 4: finish both sentences with your numbers. Which one answers the question 'which payment method has a returns problem?'

4. TODO 5: compare the `All` column with the two columns beside it. What do you now think about payment method and returns? _____

5. Then `survey_by_age_starter.py` (it saves one file to the `output/` folder next to the lab). `weather_grid_starter.py` is homework or a stretch.

PREDICT, THEN RUN

Write what you think each one prints **before** you run it. Being wrong here is useful: it shows you exactly which rule to look at again.

P1. An empty cell.

```
t = pd.DataFrame({"pay": ["card", "cash", "card"], "ch": ["web", "shop", "shop"],
                  "n": [1, 1, 1]})
p = t.pivot_table(index="pay", columns="ch", values="n", aggfunc="count")
print(p.shape, p.isna().sum().sum(), p.loc["card", "web"])
```

P2. Labels after a pivot.

```
t = pd.DataFrame({"month": ["Jan", "Feb", "Mar", "Apr"], "store": ["a"] * 4,
                  "rev": [1, 2, 3, 4]})
print(t.pivot(index="month", columns="store", values="rev").index.tolist())
```

P3. A margin of means.

```
t = pd.DataFrame({"store": ["a", "a", "a", "b"], "amt": [10, 10, 10, 50]})
p = t.pivot_table(index="store", values="amt", aggfunc="mean", margins=True)
print(p["amt"].tolist(), t.groupby("store")["amt"].mean().mean())
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

- Project: **The Seasonality Report** (lesson page, six steps). In class: Steps 1 to 3 in `lesson_2_6_project` (build the logbook, pivot with totals, find the missing revenue).
- At home: Steps 4 to 6. You are done when the pivot's `All` / `All` cell equals $3,715,829.95, the month columns run 1 to 12, and `category_seasonality.csv` has 108 rows and three columns.
- Then the lesson's Exercises 1 to 3, the Quick Quiz and your journal entry. Exercise 4 is a stretch.
- Keep `category_seasonality.csv` for Module 3.

MY NOTES

SELF-CHECK

Six questions on Lesson 2.6. No notes on the first pass; then check with your instructor.

Q7

What does this print?

```
t = pd.DataFrame({"store": ["a", "a", "b"], "year": [2024, 2024, 2025], "amt":
p = t.pivot_table(index="store", columns="year", values="amt")
print(p.loc["a", 2024], p.shape)
```

A) `20.0 (2, 2)`

B) `40 (2, 2)`

C) `20.0 (3, 2)`

D) A `ValueError: Index contains duplicate entries`

Q8

What does this print?

```
w = pd.DataFrame({"month": ["Jan", "Feb"], "Boise": [5, 6], "Bend": [3, 4], "C
tidy = w.melt(id_vars="month", var_name="store", value_name="rev")
print(tidy.shape, tidy["rev"].sum())
```

Q9

In `pd.crosstab(orders["channel"], orders["status"], normalize="index")`, the cell in row `online`, column `returned` is 0.069. What does it mean?

A) Of all returned orders, 6.9% were online

B) 6.9% of all orders are online orders that were returned

C) Online orders are 6.9% more likely to be returned than store orders

D) Of online orders, 6.9% were returned

Q10

A pivot that counts order lines by category and month, with `margins=True`, shows a grand total of 44,881. The source table has 45,490 rows. What is the most likely explanation?

A) `fill_value=0` removed the lines in empty cells

B) `margins=True` leaves the largest category out of the total

C) 609 lines have a missing category (or month), and rows with a missing key are dropped silently

D) `pivot_table` removes duplicate rows before counting

Q11

What does this print?

```
idx = pd.MultiIndex.from_product([["a", "b"], [2024, 2025]])
s = pd.Series([1, 2, 3, 4], index=idx)
u = s.unstack()
print(u.shape, u.loc["b", 2024])
```

Q12

`orders.pivot(index="store", columns="status", values="order_total")` raises `ValueError: Index contains duplicate entries, cannot reshape`. What is the right response?

A) Call `orders.drop_duplicates()` first, then `pivot()` again

B) Decide how to combine the rows and use `pivot_table(..., aggfunc="sum")`

C) Sort the table by store and status, then `pivot()` again

D) Convert `store` and `status` to the `category` dtype

NOTES

HOMEWORK CHECKLIST

- ☐ 2.5 project finished: `lines` has 45,490 rows after both merges, revenue $3,715,829.95, the pricing trap explained

- ☐ 2.5 exercises 1 to 4, Quick Quiz, journal entry
- ☐ 2.6 project finished: the corner reconciles, months run 1 to 12, `category_seasonality.csv` has 108 rows
- ☐ 2.6 exercises 1 to 3, Quick Quiz, journal entry
- ☐ Labs: `customer_profiles`, `payment_returns`, `survey_by_age`; then `orders_weather`, `weather_grid`, `debug_merges.py`
- ☐ Rebuilt `wide` (daily revenue, one column per store) from memory and looked at Bend, 10 to 16 August 2024
- ☐ Bring to Session 7: laptop, this workbook, my note about Bend. Nothing new to install

GLOSSARY FOR SESSION 6

- **Key:** a column (or set of columns) that appears in two tables and connects their rows.
- **Merge / join:** combining two tables side by side by matching rows on a key.
- **Inner join:** keeps only keys found in both tables. The default of `merge`.
- **Left join:** keeps every row of the left table; unmatched rows get NaN in the borrowed columns.
- **Outer join:** keeps every row of both tables.
- **Indicator:** the `_merge` column added by `indicator=True`: `both`, `left_only` or `right_only`.
- **Many-to-one:** a relationship in which the key is unique in the right table: each left row finds at most one partner.
- **One-to-many:** the key is unique in the left table: each left row can match several right rows, so rows multiply.
- **Many-to-many:** the key repeats on both sides: every pairing appears. Usually a mistake.
- `validate=`: a `merge` argument that states the expected relationship and raises `MergeError` if the keys say otherwise.
- **Concatenate:** stacking tables on top of each other with `pd.concat`.
- **Long (tidy) data:** one row per observation, one column per variable: the logbook.
- **Wide data:** one variable's values spread across the column headers: the scoreboard.
- **Pivot table:** a groupby on row and column keys whose result is spread into a grid.
- **Margins:** the totals row and column added by `margins=True`, computed from all the underlying rows.
- **Crosstab:** a table of counts of two categorical columns (`pd.crosstab`); `normalize` turns counts into shares.
- **Melt:** turning column headers back into the values of one column (`df.melt`): wide to tidy.

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 7: Time Series and Windows

Your working notebook for the session where the calendar becomes part of your data. Before the break you turn a column of date text into a ruler, find the days that are missing from two years of store sales, and roll days up into weeks, months and years. After the break you slide windows along that ruler: moving averages, running totals and ranks, and you learn to keep every calculation honest about what it could have known.

Lessons 2.7-2.8 | 2 lessons | Session 7 · 3 hours | Name: _____

HOW TO USE THIS WORKBOOK

Each lesson has **Key ideas** (the reference to keep), a **Worked example** with its real output, the **Guided lab** you do in class, **Predict, then run** boxes, the start of the **lesson project**, a six-question **Self-check** and space for notes. The lab files are in `labs/module-07/`: open the `_starter.py` file, run it as it is, then work through the TODOs in order. Every starter runs before you change anything and tells you which TODOs are still open.

SESSION GOALS

- Turn date text into real dates and select days, months and ranges with a date string
- Check a time series against the calendar, and decide whether each missing day is a true zero or an unknown
- Roll daily rows up to weeks, months and years with `resample()`, and print the count beside every total
- Measure change with `shift()`, `diff()` and `pct_change()`, including year over year
- Smooth a series with a rolling window, and build running totals, year-to-date figures and ranks
- Write a baseline that uses only the past, and explain look-ahead leakage in your own words

PROGRESS TRACKER

- ☐ **2.7 lab** `units_calendar`: I found Bend's incomplete weeks and its best complete week
- ☐ **2.7 project** The Store Trend Report: Step 1 runs in class; Steps 2 to 6 finished at home
- ☐ **2.8 lab** `foot_traffic`: I found the date from which Flagstaff has stayed ahead of Bend
- ☐ **2.8 project** The Sale-Day Detector: Step 1 runs in class; Steps 2 to 5 finished at home
- ☐ I did each lesson's exercises and Quick Quiz, and wrote both journal entries

LESSONS

2.7 Time Series

90 min solo

KEY IDEAS

- **Dates are a ruler, not labels.** Text that looks like a date has no order, no distances and no months. Parse it: `pd.read_csv(..., parse_dates=["date"])` or `pd.to_datetime(column)`. Check `df.dtypes` after every load.
- `.dt` **reads the parts** of a date column: `.dt.year`, `.dt.month`, `.dt.dayofweek` (Monday is 0), `.dt.day_name()`. The new columns are ordinary columns you can group by.
- **Put dates in the index** and `.loc` understands strings: `.loc["2025-07-04"]` is a day, `.loc["2025-07"]` a month, `.loc["2025"]` a year. A string range includes **both** ends.
- **Check against the calendar.** `pd.date_range(start, end, freq="D")` is every day; `calendar.difference(table.index)` is the days you do not have; `asfreq("D")` gives each of them a row of `NaN`.
- **A gap is a decision.** Closed store: a true 0. Export failed or value blank: unknown, so leave it `NaN`, or estimate it and say so. The data cannot tell you which is which; documentation can.
- `resample()` **is groupby for the calendar.** Sums skip `NaN` without telling you, so print `count` beside every `sum`.
- **Change means "compared with an earlier row".** `shift(1)` shows the previous row, `diff()` subtracts it, `pct_change()` divides by it. They count rows, so they mean "earlier in time" only on a complete calendar.
- **The order of operations:** parse → index → complete the calendar → decide each gap → then resample, shift or compare.

YOU WANT	WRITE
Real dates when loading	`pd.read_csv(path, parse_dates=["date"])`
Every row of one month	`s.loc["2025-07"]`
A range, both ends included	`s.loc["2024-12-30":"2025-01-02"]`
The days that are missing	`pd.date_range(a, b, freq="D").difference(df.index)`
One row per calendar day	`df.asfreq("D")`
Monthly totals with a check	`s.resample("ME").agg(["sum", "count"])`

YOU WANT	WRITE
Monthly totals from a date column	`df.resample("ME", on="order_date")["order_total"].sum()`
Per group and per quarter, long table	`df.groupby(["store", pd.Grouper(key="date", freq="QE")])`
Growth against last year (monthly rows)	`monthly.pct_change(12)`
Buckets	`"D"` day · `"W"` week ending Sunday · `"ME"` month · `"QE"` quarter · `"YE"` year

WORKED EXAMPLE: HOW MANY RAINY DAYS, AND IS THE CALENDAR COMPLETE?

The weather file has one row per city per day. Before counting anything by month, check the calendar. (This example uses `data/store_weather.csv`, which the lesson does not, so the numbers are new.)

Parse the dates, go wide, and compare with a full calendar.

```
weather = pd.read_csv("data/store_weather.csv", parse_dates=["date"])
rain = weather.pivot(index="date", columns="city", values="precip_in")
calendar = pd.date_range(rain.index.min(), rain.index.max(), freq="D")
print(len(rain), len(calendar), len(calendar.difference(rain.index)))
```

```
731 731 0
```

731 rows, 731 days, nothing missing: this table needs no `asfreq`. Now count the days with at least 0.10 inches of rain in each quarter. A comparison gives True/False, and the sum of True/False values is a count.

```
rainy = rain >= 0.10
by_quarter = rainy.resample("QE").agg(["sum", "count"])
print(by_quarter["Flagstaff"])
```

```
            sum  count
date
2024-03-31   15     91
2024-06-30    3     91
2024-09-30   26     92
2024-12-31    9     92
2025-03-31    9     90
2025-06-30    4     91
2025-09-30   25     92
2025-12-31    8     92
```

`count` confirms each quarter is complete (90 to 92 days). Flagstaff's rain arrives in the third quarter. Is that the same in both years? Compare each quarter with the same quarter a year earlier: 4 rows back.

```
print(rainy.resample("QE").sum().diff(4).loc["2025"])
```

```
city        Asheville  Bend  Boise  Flagstaff
date
2025-03-31       -3.0   3.0   -5.0       -6.0
2025-06-30        4.0  -2.0    1.0        1.0
2025-09-30        3.0  -1.0    4.0       -1.0
2025-12-31       -2.0   2.0    0.0       -1.0
```

Read one cell: Asheville had 3 more rainy days in the third quarter of 2025 than in the third quarter of 2024. The pattern for this example was the order of operations: parse, index, check the calendar, then resample and compare.

GUIDED LAB

Two labs practice this lesson on data the lesson did not use. Do Lab 2.7-A in class. Lab 2.7-B is for early finishers and for homework.

Lab 2.7-A · An honest weekly calendar (25 minutes, in class): open `labs/module-07/l2_7-time-series/units_calendar_starter.py`. Bend's manager wants unit sales by week with no misleading weeks. Which weekly totals are built from fewer than 7 days, what was the best complete week, and which week-over-week changes should not be reported?

1. **TODO 1:** build the full calendar and find the days on which every store was closed.
2. **TODO 2:** give the table a row for every day with `asfreq("D")` and count the missing days per store.
3. **TODO 3:** set the closed days to 0. Leave the failed exports as `NaN`.
4. **TODO 4:** resample Bend to weeks with `sum` and `count`, and list the weeks with fewer than 7 days.
5. **TODO 5:** find the best week among the complete ones.
6. **TODO 6:** print the week-over-week change for August 2024 and mark the changes you would not report.

Done looks like: a list of four closed dates; missing days of 4, 7, 4, 6 and then 0, 3, 0, 2; two incomplete weeks; a best complete week ending on a Sunday in late summer 2025; four changes, two of them caused by the gap.

Lab 2.7-B · Is the online return rate getting worse? (20 minutes, early finishers or homework): open `labs/module-07/l2_7-time-series/returns_trend_starter.py`. The support team says returns feel worse this year. Compute the share of online orders returned by month, quarter and year, compare each month with the same month a year earlier, and write one honest sentence.

1. **TODO 1:** resample the online orders by month on `order_date` and aggregate the True/False `returned` column with `mean`, `sum` and `count`.
2. **TODO 2:** name the highest and the lowest month.
3. **TODO 3:** the same rate by quarter and by year.
4. **TODO 4:** `diff(12)` for the year-over-year change in percentage points; count the 2025 months that were worse.
5. **TODO 5:** write your conclusion in one sentence.

Done looks like: 24 monthly rates between about 5.7% and 9.1%, 8 quarterly rates, 2 yearly rates, 12 year-over-year changes, and a sentence that says whether there is a trend.

◆ PREDICT, THEN RUN

Write your prediction on the line **before** you run the code. Being wrong here is useful: it shows you exactly which idea to revisit.

Predict 2.7.1 How many rows does each selection return? (Two numbers.)

```
days = pd.Series(1, index=pd.date_range("2024-01-01", "2024-12-31", freq="D"))
print(len(days.loc["2024-02"]), len(days.loc["2024-12-30":"2024-12-31"]))
```

Predict 2.7.2 One Sunday is missing from this small table. What are the two weekly sums, and the two counts?

```
s = pd.Series([5, 5, 5, 5, 5, 5, 9, 9, 9, 9, 9, 9, 9],
              index=pd.date_range("2025-03-03", periods=14, freq="D").delete(6))
print(s.resample("W").agg(["sum", "count"]).values.tolist())
```

Predict 2.7.3 What does `diff()` print for the last row, and why is that surprising?

```
s = pd.Series([100, 104, 120],
              index=pd.to_datetime(["2025-06-01", "2025-06-02", "2025-06-06"]))
print(s.diff().tolist())
print(s.asfreq("D").diff().tolist())
```

PROJECT START: THE STORE TREND REPORT

Lesson 2.7's project takes you from the raw daily file to a one-page trend report for each store. In class you do **Step 1** in a new notebook called `lesson_2_7_project`: load, go wide, complete the calendar, zero the closed days.

You are done with Step 1 when your output shows `(731, 4)`, `4 days closed`, and unknown days of Asheville 1, Bend 6, Boise 1, Flagstaff 3.

At home: Steps 2 to 6. Check yourself against two numbers in the headline table: Flagstaff +19.7% and Asheville -5.0%. Keep `store_monthly_revenue.csv`: Session 8 charts it.

SELF-CHECK (ANSWERS ARE WITH YOUR INSTRUCTOR)

Q1

After `raw = pd.read_csv("data/daily_sales.csv")`, the line `raw["date"].dt.month` raises `AttributeError: Can only use .dt accessor with datetimelike values`. What is the cause?

A) Some dates are missing from the file

B) The column was loaded as text; pass `parse_dates=["date"]` or convert with `pd.to_datetime`

C) `.dt` only works on the index, not on a column

D) `month` is a method and needs parentheses

Q2

What does this print?

```
s = pd.Series(range(10),
              index=pd.date_range("2025-01-28", periods=10, freq="D"))
print(len(s.loc["2025-01"]), len(s.loc["2025-01-30":"2025-02-02"]))
```

Q3

`wide` **has one row per date and 727 rows between 2024-01-01 and 2025-12-31, a span of 731 days. What does** `wide.asfreq("D")` **return?**

A) 727 rows; the four gaps are interpolated

B) 731 rows; the four new rows are filled with 0

C) An error, because the index has missing dates

D) 731 rows; the four new rows hold `NaN`

Q4

What does this print?

```
s = pd.Series([10.0, np.nan, 30.0],
              index=pd.to_datetime(["2025-05-01", "2025-05-02", "2025-05-03"]))
print(s.resample("ME").agg(["sum", "count"]).values.tolist())
```

A) `[[40.0, 2.0]]`

B) `[[nan, 3.0]]`

C) `[[40.0, 3.0]]`

D) `[[nan, 2.0]]`

Q5

Bend has no rows at all for three days in August 2024. On that rows-only Series, what does `bend_rows.diff(7)` compare the first Saturday after the gap with?

A) The previous Saturday, seven days earlier

B) Nothing: the result is `NaN` because the window crosses a gap

C) The value seven **rows** earlier, which is ten days back and not a Saturday

D) Nothing: pandas raises an error on an irregular index

Q6

What does this print?

```
m = pd.Series([200, 220, 264],
              index=pd.date_range("2025-01-31", periods=3, freq="ME"))
print((m.pct_change() * 100).round(1).tolist())
```

MY NOTES FOR THIS LESSON

2.8 Window Functions

90 min solo

KEY IDEAS

- **A window function returns one value for every row**, computed from nearby rows. `groupby().agg()` and `resample()` return one value per group.
- `rolling(n)` is a fixed pane sliding down the rows: this row and the `n - 1` before it. Follow it with `.mean()`, `.sum()`, `.max()`, `.std()`. Match `n` to the cycle you want to remove: 7 for a weekly rhythm.
- **A window must be full by default.** The first `n - 1` rows are `NaN`, and one missing value blanks `n` results. `min_periods=` relaxes that. Choose a threshold you can defend (6 of 7, 21 of 28), not 1.
- `center=True` puts the row in the middle of its window, so it uses future rows. Fine for describing history; never for predicting or flagging.
- **Expanding windows only grow:** `cumsum()` is a running total, `cummax()` the best so far, `expanding().mean()` the average so far. Year to date is a running total within each year: `groupby(index.year).cumsum()`.
- **On a long table: sort, group, window.** `groupby("store")["revenue"].transform(lambda s: s.rolling(7).mean())` keeps the rows lined up. Quick test: the first rows of **each** group are `NaN`.
- `rank()` gives positions. `ascending=False` makes the largest value rank 1; `method=` settles ties; `pct=True` gives percentiles; `groupby(...).rank()` ranks inside each group.
- **Look-ahead leakage:** a baseline must not contain the row it judges, or anything after it. Ask of every window: "On that day, could I have known every number in it?" Shift first: `s.shift(1).rolling(7).mean()`.

WINDOW	REMEMBERS	GOOD FOR
`rolling(n)`	The last n rows, equally	Removing a known cycle; easy to explain
`expanding()` / `cumsum()`	Everything so far, equally	Running totals, year to date, records
`ewm(span=n)`	Everything, recent rows more	A level that drifts; reacting quickly (read the lesson section)
`shift(1)` then a window	Only rows before this one	Anything used to predict, flag or score the current row

WORKED EXAMPLE: SMOOTHING TEMPERATURES, AND A RECORD THAT KEEPS BEING BROKEN

Daily high temperatures in Boise jump around from day to day. A window makes the season visible. (This example uses `data/store_weather.csv`, which has no gaps, so it needs no calendar repair.)

One row per day for Boise, then a trailing 7-day mean. The first six rows have no full window yet.

```
weather = pd.read_csv("data/store_weather.csv", parse_dates=["date"])
boise_high = weather[weather["city"] == "Boise"].set_index("date")["high_f"]
view = pd.DataFrame({"high_f": boise_high,
                     "avg_7d": boise_high.rolling(7).mean().round(1)})
print(view.head(8))
```

```
            high_f  avg_7d
date
2024-01-01      37     NaN
2024-01-02      44     NaN
2024-01-03      30     NaN
2024-01-04      26     NaN
2024-01-05      29     NaN
2024-01-06      27     NaN
2024-01-07      32    32.1
2024-01-08      26    30.6
```

A day sets a record when it equals the hottest day so far. `cummax()` is the hottest day so far.

```
records = boise_high[boise_high == boise_high.cummax()]
print(len(records), "record days; the last three:")
print(records.tail(3))
```

```
26 record days; the last three:
date
2024-07-02    109
2025-07-12    110
2025-07-13    112
Name: high_f, dtype: int64
```

Was July 13, 2025 unusual? Compare it with the 7 days **before** it, not with a window that contains it.

```
before = boise_high.shift(1).rolling(7).mean()
with_today = boise_high.rolling(7).mean()
day = "2025-07-13"
print(boise_high.loc[day], "| before:", round(before.loc[day], 1),
      "| with today:", round(with_today.loc[day], 1))
```

```
112 | before: 88.3 | with today: 93.0
```

The honest baseline (the seven days before) is lower than the one that includes the day itself, so the hot day stands out more against it. Any day you want to judge has to stay out of its own baseline.

◆ GUIDED LAB

Two labs practice this lesson on data the lesson did not use. Do Lab 2.8-A in class. Lab 2.8-B is for early finishers and for homework.

Lab 2.8-A · Foot traffic: when did Flagstaff pass Bend? (25 minutes, in class): open `labs/module-07/l2_8-window-functions/foot_traffic_starter.py`. Daily visitor counts zigzag with the week. Smooth them, then find when Flagstaff's foot traffic passed Bend's, how many visitor records Flagstaff set, and each store's busiest 7 consecutive days.

1. **TODO 1:** a 7-day rolling mean of visitors; look at Boise's first 8 rows.
2. **TODO 2:** a 28-day rolling mean that still answers when at least 21 of the 28 days are known.
3. **TODO 3:** compare Flagstaff's 28-day average with Bend's: the first day ahead, the number of days ahead, and the date from which Flagstaff has stayed ahead.
4. **TODO 4:** Flagstaff's record days with `cummax()`.
5. **TODO 5:** each store's busiest 7 consecutive days with a rolling sum and `idxmax()`.

Done looks like: six `NaN` then two numbers; a 4-by-4 table of 28-day averages; three lines about the crossover; a count of record days with the last three; four lines naming each store's busiest 7 days.

Lab 2.8-B · The online shop: milestones, ranks and a fair baseline (20 minutes, early finishers or homework): open `labs/module-07/l2_8-window-functions/online_milestones_starter.py`. When did completed online revenue pass $1,000,000 in each year, where does each month rank within its year, how often did the shop set a new best day, and how unusual was the best day when judged only by what was known that morning?

1. **TODO 1:** year-to-date revenue with `groupby(day.index.year).cumsum()`, and the first date at or above 1,000,000 in each year.
2. **TODO 2:** monthly totals, ranked within each year with `groupby(...).rank(ascending=False)`.
3. **TODO 3:** record days with `cummax()`.
4. **TODO 4:** a leak-free weekday baseline (the mean of the 4 same weekdays before) and the best day's ratio to it.
5. **TODO 5:** the same baseline without `shift(1)`, and the count of days at 2 times baseline under each version.

Done looks like: two million-dollar dates about two months apart in the calendar; a 24-row table with December ranked 1 in both years; a count of record days; and two ratios for the best day, the honest one clearly larger.

◆ PREDICT, THEN RUN

Write your prediction on the line **before** you run the code. Being wrong here is useful: it shows you exactly which idea to revisit.

Predict 2.8.1 How many `NaN` values, and what is the last number?

```
s = pd.Series([2, 4, 6, 8, 10, 12])
r = s.rolling(4).mean()
print(int(r.isna().sum()), r.iloc[-1])
```

Predict 2.8.2 Two groups stacked in one column. What does each line print?

```
t = pd.DataFrame({"g": ["a", "a", "a", "b", "b", "b"],
                  "v": [1, 2, 3, 10, 20, 30]})
print(t["v"].cumsum().tolist())
print(t.groupby("g")["v"].cumsum().tolist())
```

Predict 2.8.3 Which of the three columns could you have computed on the morning of each row? Predict the last row of each.

```
s = pd.Series([10, 10, 10, 40, 10])
print(s.rolling(3).mean().round(1).tolist())
print(s.rolling(3, center=True).mean().round(1).tolist())
print(s.shift(1).rolling(3).mean().round(1).tolist())
```

◆ PROJECT START: THE SALE-DAY DETECTOR

Lesson 2.8's project asks: Pinecrest ran six sale weekends in two years. If nobody had told you when, could you find them from revenue alone? In class you do **Step 1** in a new notebook called `lesson_2_8_project`: the clean revenue calendar, and a matching table of promotion flags to score against.

You are done with Step 1 when `revenue` and `promo` are both `(731, 4)` and `promo.sum()` shows 24 for every store.

At home: Steps 2 to 5. Your first detector will be wrong about three times in four. That is planned: Step 3 asks you to look at which days it gets wrong before you change anything. Check numbers: 260 store-days flagged at first, 142 with the weekday baseline. Keep `sale_day_flags.csv`.

◆ SELF-CHECK (ANSWERS ARE WITH YOUR INSTRUCTOR)

Q7

What does this print?

```
s = pd.Series([4, 8, 6, 10])
print(s.rolling(3, min_periods=2).mean().tolist())
```

A) `[nan, nan, 6.0, 8.0]`

B) `[4.0, 6.0, 6.0, 8.0]`

C) `[nan, 6.0, 6.0, 8.0]`

D) `[6.0, 6.0, 8.0, nan]`

Q8

What does this print?

```
s = pd.Series([5, 3, 9, 4, 9])
print((s == s.cummax()).tolist())
```

Q9

A long table has columns `store`, `date`, `revenue`, sorted by date and then store. Which code adds each store's own 7-day average, lined up with the right rows?

A) `df["avg"] = df["revenue"].rolling(7).mean()`

B) `df["avg"] = df.groupby("store")["revenue"].rolling(7).mean()`

C) `df["avg"] = df.groupby("date")["revenue"].transform(lambda s: s.rolling(7).mean())`

D) Sort by store and date, then `df["avg"] = df.groupby("store")["revenue"].transform(lambda s: s.rolling(7).mean())`

Q10

What does this print?

```
s = pd.Series([30, 10, 30, 20])
print(s.rank(ascending=False, method="dense").tolist())
```

A) `[1.0, 3.0, 1.0, 2.0]`

B) `[1.5, 4.0, 1.5, 3.0]`

C) `[1.0, 4.0, 1.0, 3.0]`

D) `[3.0, 1.0, 3.0, 2.0]`

Q11

To flag unusual days, an analyst computes `lift = rev / rev.rolling(28).mean()` and flags days where `lift >= 1.5`. What is wrong with the baseline?

A) A 28-day window is too long to react to a sale

B) Each day is inside its own baseline, so every spike is understated; use `rev.shift(1).rolling(28).mean()`

C) A rolling mean cannot be used as a divisor

D) The first 27 rows are `NaN`, so the division fails

Q12

What does this print?

```
s = pd.Series([1, 2, 3, 4, 5])
print(s.shift(1).rolling(2).sum().tolist())
```

MY NOTES FOR THIS LESSON

EXIT TICKET

Answer on a sticky note or a scrap of paper, and hand it in on the way out.

1. You resample daily revenue to monthly totals with `.sum()`. What do you print beside every total, and why?
2. Write an expression for a 7-day average of the Series `rev` that is safe to use for judging or predicting today.
3. List the order of operations for a time series, and name the step people skip most often.

HOMEWORK CHECKLIST

- ☐ Finish the Lesson 2.7 project, **The Store Trend Report** (Steps 2 to 6). Check: Flagstaff +19.7%, Asheville -5.0%, and `store_monthly_revenue.csv` saved.
- ☐ Finish the Lesson 2.8 project, **The Sale-Day Detector** (Steps 2 to 5). Check: 260 flags at first, then 142 with the weekday baseline, and `sale_day_flags.csv` with 2,924 rows.
- ☐ Lesson 2.7: Exercises 1 to 3 (Exercise 4 is a stretch), the Quick Quiz and the journal prompt. Read **Time Zones in Brief**: it was not covered in the room.
- ☐ Lesson 2.8: Exercises 1 to 3 (Exercise 4 is a stretch), the Quick Quiz and the journal prompt. Read **Exponentially Weighted Windows**: it was only mentioned in the room.

- ☐ Kit labs not reached in the room: `labs/module-07/l2_7-time-series/returns_trend_starter.py` and `labs/module-07/l2_8-window-functions/online_milestones_starter.py` (about 20 minutes each).
- ☐ From Lesson 2.8, "Before the Next Lesson": open `data/customers.csv` and list three new columns you could compute for each customer that might help predict whether they will order again.

> **BRING TO SESSION 8**
>
> Your two project notebooks, `store_monthly_revenue.csv` and `sale_day_flags.csv`, and your list of three customer features. Nothing new to install. At home, run `import matplotlib; print(matplotlib.__version__)` once and tell your instructor before the session if it fails.

GLOSSARY

TERM	MEANING
`Timestamp`	One point in time. A `datetime64` column holds one per row; it knows its year, month and weekday.
`Timedelta`	A length of time: what you get when you subtract two timestamps.
`NaT`	"Not a time": the missing value for dates, as `NaN` is for numbers.
`DatetimeIndex`	Row labels that are dates. It lets `.loc["2024-07"]` select a month and makes `resample` possible.
`asfreq`	Give the table one row for every period (`"D"` = every day). Rows that did not exist appear as `NaN`.
`Resample`	Groupby for the calendar: drop rows into weeks, months or quarters, then aggregate each bucket.
Frequency alias	The code for a bucket size: `"D"`, `"W"`, `"ME"`, `"QE"`, `"YE"`, `"h"`. The old `"M"`, `"Q"`, `"Y"`, `"H"` raise an error.
shift / diff / pct_change	The previous row's value beside this one; this row minus that; that change as a fraction.
Year over year	A period compared with the same period one year earlier: 12 rows back in monthly data, 4 in quarterly.
Window function	A calculation that returns one value for every row, computed from nearby rows.
Rolling window	A fixed-size window that slides down the rows: `rolling(7)` is this row and the six before it.

TERM	MEANING
`min_periods`	How many real values a window needs before it gives an answer. The default is all of them.
Expanding window	A window that starts at the first row and only grows: running totals, best so far.
`Year to date`	A running total that restarts each January 1: `groupby(index.year).cumsum()`.
`Rank`	A value's position in sorted order. `method=` settles ties; `pct=True` gives a percentile.
Look-ahead leakage	Using information in a calculation that was not available at the moment the calculation describes.

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 8: Features and First Charts

Your working notebook for the session that finishes pandas and starts charting. Before the break you build features: columns designed so a pattern is easy to see, one row per customer or per order, with nothing leaking in from the future. After the break you turn tables into pictures with Matplotlib: a line, a scatter, bars and a histogram, each chosen because of the question it answers.

Lessons 2.9-3.1 | 2 lessons | Session 8 · 3 hours | Name: _____

HOW TO USE THIS WORKBOOK

Each lesson has **Key ideas**, a **Worked example** with its real output, the **Guided lab** you do in class, **Predict, then run** boxes, the start of the **lesson project**, a six-question **Self-check** and space for notes. The lab files are in `labs/module-08/`. The two chart labs save their pictures as PNG files in an `output/` folder next to the lab file: open them from there, or run the file cell by cell in VS Code to see each chart as you go.

SESSION GOALS

- Decide what one row of a feature table represents, and build flags, ratios, date parts and bins for it
- Summarize many rows per customer or per order, join the summary back, and fill missing values honestly
- Encode an ordered category and an unordered one correctly
- Apply the cutoff rule: every feature must be computable on the cutoff date from what existed then
- Build any chart with `fig, ax = plt.subplots()` and methods on `ax`, with a title and labeled axes
- Choose line, scatter, bar or histogram from the question, and spot a chart that draws without error and is wrong

PROGRESS TRACKER

- ☐ **2.9 lab** `order_features`: I found the one combination of features where returns jump
- ☐ **2.9 project** The Customer Feature Table: Step 1 runs in class; Steps 2 to 6 finished at home
- ☐ **3.1 lab** `four_questions`: four PNG files in my output folder, each answering its question
- ☐ **3.1 project** Does Rain Keep Shoppers Away?: Step 1 runs in class; Steps 2 to 6 finished at home
- ☐ I did each lesson's exercises and Quick Quiz, and wrote both journal entries

LESSONS

2.9 Feature Engineering Basics

95 min solo

KEY IDEAS

- **A feature is a column you build so a pattern becomes visible.** First decide the **unit of analysis**: what one row is. Every feature must be one fact about that unit.
- **Derived columns** come from four patterns: a flag (`.notna()`, `.isin()`, a comparison), arithmetic between columns, a choice (`np.where`), a lookup (`.map`). **Ratios** remove size and leave behavior.
- **Dates** become features as parts that repeat (`.dt.month`, a weekend flag) or as elapsed time from a **fixed reference date**. Never use today's date.
- `pd.cut`: you choose the edges; each bin includes its right edge; values outside the edges become `NaN`. `pd.qcut`: equal-sized groups, edges chosen from the data.
- **An ordered categorical** (`pd.CategoricalDtype([...], ordered=True)`) sorts, compares and takes `max()` in the real order, not the alphabet.
- **Per-customer features:** group and aggregate to one row each, then **left-merge** onto the table that defines the unit. Fill counts and sums with 0. Leave averages missing and add a flag.
- **Encoding:** ordinal categories → ordered integers (`.cat.codes`). Nominal categories → one yes/no column each (`pd.get_dummies`).
- **The cutoff rule:** could I have computed this value on the cutoff date, using only what existed then? If not, it leaks. Leaks raise no error; they give results that are too good.

YOU WANT	WRITE
A yes/no flag	`df["col"].notna()` · `df["a"] == "x"` · `df["a"].isin([...])`
A two-way choice	`np.where(condition, "large", "regular")`
Days since a date	`(as_of - df["date"]).dt.days` with `as_of = pd.Timestamp("2025-12-31")`
Bands with edges you choose	`pd.cut(s, bins=[17, 24, 34, 120], labels=[...])`
Edges that cover everything	`bins=[-np.inf, 40, 60, 80, np.inf]`
Four equal-sized groups	`pd.qcut(s, q=4, labels=["low", "mid", "high", "top"])`
One row per customer	`orders.groupby("customer_id").agg(n_orders=("order_id", "count"), ...)`

YOU WANT	WRITE
Join without losing anyone	`base.merge(summary, on="customer_id", how="left", validate="one_to_one")`
One-hot columns	`pd.get_dummies(df["channel"], prefix="via", dtype=int)`
Rate and size per group	`df.groupby("feature")["target"].agg(["mean", "size"])`

WORKED EXAMPLE: PRODUCT FEATURES: DOES TIME ON SALE EXPLAIN REVIEW COUNTS?

One row is a product. The question: which products have many reviews? Two candidate features: how long the product has been on sale, and how expensive it is. (The lesson builds customer features; this example uses `data/products.csv`.)

Elapsed time from a fixed reference date, then bands with edges that mean something (under a year, one to three years, longer).

```
products = pd.read_csv("data/products.csv", parse_dates=["launch_date"])
as_of = pd.Timestamp("2025-12-31")
products["years_on_sale"] = (as_of - products["launch_date"]).dt.days / 365.25
products["age_band"] = pd.cut(products["years_on_sale"], bins=[0, 1, 3, np.inf],
                              labels=["new", "established", "veteran"])
print(products["age_band"].value_counts(sort=False, dropna=False))
```

```
age_band
new             13
established     49
veteran        180
Name: count, dtype: int64
```

No `NaN` row in that count, so every product landed in a band. Now check the feature against the thing we care about, with the group size beside it.

```
print(products.groupby("age_band", observed=True)["review_count"]
      .agg(["mean", "median", "size"]).round(1))
```

```
              mean  median  size
age_band
new           37.5     0.0    13
established   56.8    34.0    49
veteran       76.6    40.5   180
```

The second feature: three equal-sized price bands, made by `qcut`.

```
products["price_band"] = pd.qcut(products["price"], q=3,
                                 labels=["low", "mid", "high"])
print(products.groupby("price_band", observed=True)["review_count"]
      .agg(["mean", "median", "size"]).round(1))
```

```
             mean  median  size
price_band
low          99.5    61.0    81
mid          56.8    26.0    81
high         54.8    26.5    80
```

Both features separate the products: review counts rise with time on sale, and the cheapest third of the catalog collects far more reviews than the rest. Notice `observed=True`: these are categoricals, and it keeps the output to the bands that exist. And notice the 13 new products with a median of 0 reviews: for a product launched last month, "no reviews" says nothing about quality.

GUIDED LAB

Two labs practice this lesson with a different unit of analysis than the lesson's. Do Lab 2.9-A in class. Lab 2.9-B is for early finishers and for homework.

Lab 2.9-A · Which orders come back? (25 minutes, in class): open `labs/module-08/l2_9-feature-engineering/order_features_starter.py`. The returns desk asks which kinds of orders get returned. One row is an order. Build flags, per-order summaries from the order lines, and bins; check each against the return rate; then find the combination that stands out.

1. **TODO 1:** three flags: `is_online`, `used_promo`, `is_weekend`.
2. **TODO 2:** group the order lines by `order_id`: number of lines, number of units, and whether any line is footwear.
3. **TODO 3:** left-merge the summary onto the orders with `validate="one_to_one"`; the row count must stay 20,302.
4. **TODO 4:** `value_band` with `qcut`, `lines_band` with `cut`; check that nothing became `NaN`.
5. **TODO 5:** for each feature, the return rate and the group size.
6. **TODO 6:** combine two flags into `online_footwear` and look again.
7. **TODO 7:** name the column that must not be used as a feature, and say why.

Done looks like: an overall return rate near 5.6%; six small tables; one combination of two features with a return rate about three times any other; a sentence about `status`.

Lab 2.9-B · How busy will the store be tomorrow? (20 minutes, early finishers or homework): open `labs/module-08/l2_9-feature-engineering/store_day_features_starter.py`. One row is a

store on a day, and the thing to predict is `visitors`. The cutoff rule here is "known the evening before". Build a ratio, date parts, temperature bands and a same-weekday-last-week feature, then audit every candidate column.

1. **TODO 1:** `rel_visitors`: visitors divided by the store's own average (1.0 is a normal day).
2. **TODO 2:** `weekday`, `is_weekend`, `rainy`.
3. **TODO 3:** `temp_band` with `pd.cut` and outer edges of `-np.inf` and `np.inf`.
4. **TODO 4:** relative traffic, with group sizes, for each feature.
5. **TODO 5:** "the same weekday last week" two ways: 7 rows back, and 7 days back on the completed calendar. Count the rows where they differ.
6. **TODO 6:** correlate each candidate with `visitors` and give a verdict: feature or leak.

Done looks like: 2,903 rows throughout; four temperature bands with no missing values; relative traffic rising from cold to hot days; a count of rows where the two lag columns disagree; a verdict for seven columns.

◆ PREDICT, THEN RUN

Write your prediction on the line **before** you run the code. Being wrong here is useful: it shows you exactly which idea to revisit.

Predict 2.9.1 The same three values, binned twice. What does each line print?

```
v = pd.Series([10, 20, 30])
edges, names = [0, 10, 20, 30], ["a", "b", "c"]
print(pd.cut(v, bins=edges, labels=names).tolist())
print(pd.cut(v, bins=edges, labels=names, right=False).tolist())
```

Predict 2.9.2 Three customers, one of whom never ordered. What are the two lists?

```
people = pd.DataFrame({"id": [1, 2, 3]})
bought = pd.DataFrame({"id": [1, 1, 3], "total": [10.0, 20.0, 7.0]})
per = bought.groupby("id").agg(n=("total", "count"), avg=("total", "mean"))
t = people.merge(per, on="id", how="left")
t["n"] = t["n"].fillna(0).astype(int)
print(t["n"].tolist(), t["avg"].tolist())
```

Predict 2.9.3 What are the column names, and what does each column add up to?

```
d = pd.get_dummies(pd.Series(["web", "store", "web"]), dtype=int)
print(d.columns.tolist(), d.sum().tolist(), d.sum(axis=1).tolist())
```

PROJECT START: THE CUSTOMER FEATURE TABLE

Lesson 2.9's project builds the table that answers marketing's question: one row per customer, features describing everything known on July 1, 2025, and a target saying whether the customer ordered in the following six months. In class you do **Step 1** in a new notebook called `lesson_2_9_project`: choose the unit, the cutoff and the target.

You are done with Step 1 when your output shows `4850 4474`, then `14497 5805`, then `2025-06-30 2025-07-01`.

At home: Steps 2 to 6. Check numbers: `(4474, 19)` after Step 3 and a target rate of 0.569. In Step 5 you break the cutoff rule on purpose and measure what a leak does. Keep `customer_features.csv`.

SELF-CHECK (ANSWERS ARE WITH YOUR INSTRUCTOR)

Q1

Marketing asks which customers will order again, so one row of the feature table is a customer. Which of these is not a valid column for that table as it stands?

A) Days from the customer's signup to the cutoff date

B) The `order_total` of each of the customer's orders

C) The number of orders the customer placed before the cutoff

D) The share of the customer's orders that were placed online

Q2

What does this print?

```
ages = pd.Series([18, 24, 25, 40])
bands = pd.cut(ages, bins=[17, 24, 34, 120], labels=["18-24", "25-34", "35+"])
print(bands.tolist())
```

Q3

What does this print?

```
x = pd.Series([0, 10, 20, 30])
print(pd.cut(x, bins=[0, 10, 20]).isna().sum())
```

A) `0`

B) `1`

C) `2`

D) `4`

Q4

`customers` has 4,850 rows. `per_customer`, built with `orders.groupby("customer_id").agg(...)`, has 3,942 rows. What does `customers.merge(per_customer, on="customer_id")` return with the default settings?

A) 4,850 rows, with `NaN` for customers who never ordered

B) 3,942 rows, one for every customer id in the orders

C) 8,792 rows, every row of both tables

D) 3,792 rows: every customer who never ordered is gone

Q5

A method needs numbers. You have `loyalty_tier` (Basic, Silver, Gold) and `acquisition_channel` (five values with no order). Which encoding is appropriate?

A) Ordered integer codes for the tier; one yes/no column per channel with `pd.get_dummies`

B) Integer codes 0 to 4 for the channel, in alphabetical order; one yes/no column per tier

C) Integer codes for both, because they take the least space

D) Standardize both columns to mean 0 and standard deviation 1

Q6

You build features as of the cutoff 2025-07-01 to predict who orders in the following six months. A colleague's `n_orders` counts all of each customer's orders in the file. What happens?

A) An error: pandas refuses to aggregate rows dated after the cutoff

B) Nothing important: more data makes a better feature

C) The feature looks more predictive than it is, because orders after the cutoff are in both the feature and the target

D) The feature becomes useless, because the extra rows add noise

MY NOTES FOR THIS LESSON

3.1 Line, Scatter and Bar Charts

90 min solo

KEY IDEAS

- **A chart is an answer to a question.** Decide the question first; the chart type follows.
- **Figure** = the whole canvas (it has a size, and it is what gets saved). **Axes** = one chart on it. Create both with `fig, ax = plt.subplots(figsize=(7, 3.5), layout="constrained")`.
- **One rule:** `plt` creates figures and shows them. Everything else is a method on `ax` or `fig`: `ax.plot`, `ax.set_title`, `ax.set_xlabel`, `ax.legend`, `fig.savefig`.
- **Most of a chart is made in pandas first.** Aggregate to the level the question needs, then plot.
- **Line** (`ax.plot`): change over time. One call per line; `label=` feeds `ax.legend()`.
- **Scatter** (`ax.scatter`): do two numbers move together? A small `s` and an `alpha` below 1 fix overplotting. It shows a pattern, not a cause.
- **Bar** (`ax.bar`, `ax.barh`): which group is bigger? `ax.bar` **never aggregates**: one number per bar, and you compute it. Bars start at zero.
- **Histogram** (`ax.hist`): what is typical, how spread out? It **always** counts: give it the raw rows. `bins` and `range` are your decisions; say what you left out.

YOUR QUESTION	CHART	METHOD	PREPARE WITH
How did it change over time?	Line	`ax.plot(x, y)`	`resample` or group by a period
Do two numbers move together?	Scatter	`ax.scatter(x, y, s=8, alpha=0.3)`	Usually nothing: plot the rows
Which group is bigger?	Bar	`ax.bar(names, values)` · `ax.barh`	`groupby` or `value_counts`, then sort
What is typical? How spread out?	Histogram	`ax.hist(column, bins=40)`	Nothing: it counts for you
You see `plt.title(...)`		`ax.set_title(...)`	
You see `plt.savefig(...)`		`fig.savefig(...)`	

WORKED EXAMPLE: HOW DID THE NUMBER OF ORDERS PER MONTH CHANGE, ONLINE AND IN STORES?

The question is about change over time, so the answer is a line chart, and most of the work happens in pandas before any plotting. (The lesson charts store revenue; this example counts orders by channel.)

Aggregate first: one row per month, one column per channel, each cell a count of orders.

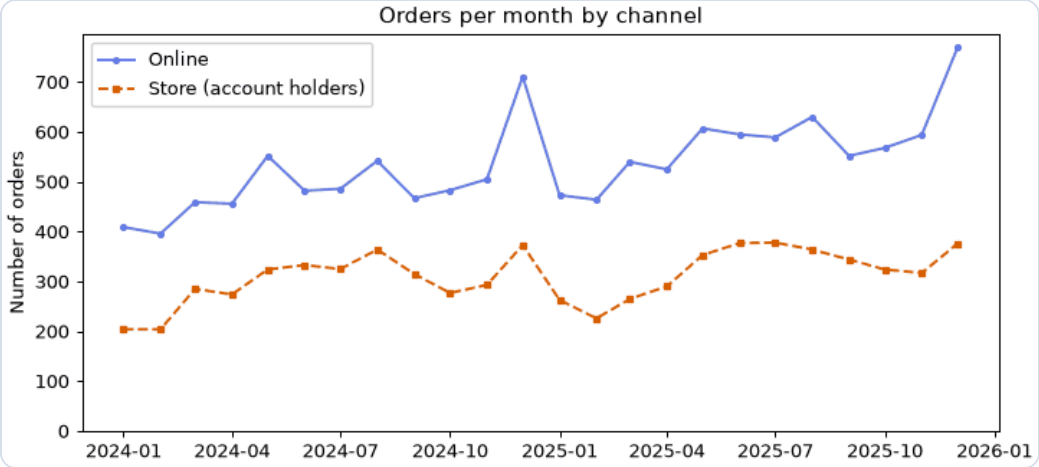
```
orders = pd.read_csv("data/orders.csv", parse_dates=["order_date"])
orders["month"] = orders["order_date"].dt.to_period("M").dt.to_timestamp()
per_month = orders.pivot_table(index="month", columns="channel",
                               values="order_id", aggfunc="count")
print(per_month.iloc[[0, -1]])
```

```
channel     online  store
month
2024-01-01     409    204
2025-12-01     769    376
```

Then the five-line pattern, with one `ax.plot` call per column. A solid line and a dashed line, so the chart still works without color.

```
fig, ax = plt.subplots(figsize=(8, 3.6), layout="constrained")
ax.plot(per_month.index, per_month["online"], marker="o", markersize=3,
        color="#667eea", label="Online")
ax.plot(per_month.index, per_month["store"], marker="s", markersize=3,
        color="#d95f02", linestyle="--", label="Store (account holders)")
ax.set_title("Orders per month by channel")
ax.set_ylabel("Number of orders")
ax.set_ylim(bottom=0)
ax.legend()
plt.show()
print(len(fig.axes), len(ax.lines))
```

```
1 2
```



The chart this code draws.

Both channels grow, and both spike in December. Online runs at about twice the store count throughout. The label on the store line matters: `orders.csv` holds store orders by signed-in account holders only, and a chart should say what it is counting.

GUIDED LAB

Two labs practice this lesson on questions the lesson did not ask. Do Lab 3.1-A in class. Lab 3.1-B is for early finishers and for homework. Chart labs save PNG files to an `output/` folder next to the lab file and never call `plt.show()`.

Lab 3.1-A · Four questions, four charts (25 minutes, in class): open `labs/module-08/l3_1-line-scatter-bar/four_questions_starter.py`. Four questions about Pinecrest's customers and products. For each one, decide the chart type from the question before you read the TODO, draw it with a title and labeled axes, and save it.

1. **Question 1:** how did the number of new accounts change from year to year? Save `q1_signups.png`.
2. **Question 2:** which acquisition channel brought in the most customers? Save `q2_channels.png`.
3. **Question 3:** do heavier products cost more? Save `q3_weight_price.png` and print the correlation.
4. **Question 4:** what does a typical product cost, and how spread out are the prices? Save `q4_prices.png`.

Done looks like: four PNG files in `output/`; a line with 7 points; 5 horizontal bars with the longest on top; a scatter of the products that have a weight; a histogram with a dashed line at the median price.

Lab 3.1-B · Chart repair (20 minutes, early finishers or homework): open `labs/module-08/l3_1-line-scatter-bar/chart_repair_starter.py`. The starter runs with no error and draws three charts that are wrong. For each, read the printed clue, say what the chart is really showing, fix the cause and save the fixed chart.

1. **Chart 1:** December 2025 revenue by payment method. 1,145 bars were drawn for 4 methods. Save `fixed_1_payment.png`.
2. **Chart 2:** Flagstaff visitors in late January 2025. 17 rows for 19 days. Save `fixed_2_visitors.png`.
3. **Chart 3:** Boise's daily high temperature. 731 tick labels. Save `fixed_3_temperature.png`.

Done looks like: 4 sorted bars showing totals; a line that breaks on January 21 and 22; a handful of date ticks.

◆ PREDICT, THEN RUN

Write your prediction on the line **before** you run the code. Being wrong here is useful: it shows you exactly which idea to revisit.

Predict 3.1.1 One figure, a loop that plots three columns. What are the two numbers?

```
t = pd.DataFrame({"a": [1, 2], "b": [2, 1], "c": [3, 3]})
fig, ax = plt.subplots()
for col in t.columns:
    ax.plot(t.index, t[col], label=col)
print(len(fig.axes), len(ax.lines))
plt.close(fig)
```

Predict 3.1.2 No error. How many bars, and how tall is the tallest thing you can see?

```
fig, ax = plt.subplots()
bars = ax.bar(["x", "y", "x"], [5, 2, 9])
print(len(bars), max(bar.get_height() for bar in bars))
plt.close(fig)
```

Predict 3.1.3 Three dates given as text, out of order. Where does Matplotlib put them on the x axis?

```
fig, ax = plt.subplots()
ax.plot(["2025-01-03", "2025-01-01", "2025-01-02"], [3, 1, 2])
print(list(ax.get_xticks()))
plt.close(fig)
```

PROJECT START: DOES RAIN KEEP SHOPPERS AWAY?

Lesson 3.1's project answers one question with all four chart types. Store managers believe rainy days are slow days; you have door-counter data and a weather file for every store city. In class you do **Step 1** in a new notebook called `lesson_3_1_project`: merge the two files and add a `rainy` column.

You are done with Step 1 when your output shows `(2903, 13)` and a rainy share of 0.134.

At home: Steps 2 to 6: a line with the rainy days marked, a scatter, two histograms on one Axes, grouped bars for every store, and a written finding. Check: a rainy-day drop at all four stores, between -17.6% and -26.3%. Your finding must include one thing the charts do **not** prove.

SELF-CHECK (ANSWERS ARE WITH YOUR INSTRUCTOR)

Q7

Question: "Do products with more reviews tend to have higher ratings?" Which chart answers it?

A) A line chart of rating against review count

B) A scatter plot: one dot per product, review count against rating

C) A bar chart with one bar per product

D) A histogram of the ratings

Q8

What does this print?

```
fig, ax = plt.subplots()
bars = ax.bar(["a", "b", "a", "b", "a"], [1, 5, 3, 2, 2])
print(len(bars), len(fig.axes))
plt.close(fig)
```

Q9

A tutorial writes `plt.title("Revenue")`. You are holding the chart in a variable `ax`. What do you write?

A) `ax.set_title("Revenue")`

B) `ax.title("Revenue")`

C) `fig.set_title("Revenue")`

D) `ax.title = "Revenue"`

Q10

A histogram of 20,302 order totals with `bins=40` shows almost everything in three bars, on an axis that runs to $5,000. What is the best fix?

A) `bins=400`, so that the bars are narrower

B) Delete the large orders from the DataFrame before plotting

C) `range=(0, 1000)`, and say in the title how many orders are not shown

D) Switch to a bar chart with one bar per order

Q11

What does this print?

```
fig, ax = plt.subplots()
counts, edges, patches = ax.hist([1, 2, 2, 3, 3, 3, 9], bins=4, range=(0, 8))
print(counts.tolist(), len(edges))
plt.close(fig)
```

Q12

A scatter plot of 2,903 store-days shows a solid blob in the lower left: you cannot tell 5 overlapping dots from 500. Which change fixes it?

A) A larger `figsize`

B) `linewidth=0.8` and `linestyle="--"`

C) `bins=40` and `range=(0, 1000)`

D) A smaller `s` and an `alpha` below 1

MY NOTES FOR THIS LESSON

EXIT TICKET

Answer on a sticky note or a scrap of paper, and hand it in on the way out.

1. Before you build any feature, what is the first thing you decide?
2. State the cutoff rule in one sentence.

3. Name the chart for each question: (a) how did it change over time, (b) do two numbers move together, (c) which group is bigger, (d) what values are typical and how spread out are they?

HOMEWORK CHECKLIST

- ☐ Finish the Lesson 2.9 project, **The Customer Feature Table** (Steps 2 to 6). Check: `(4474, 19)` after Step 3, a target rate of 0.569, and `customer_features.csv` with 20 columns.
- ☐ Finish the Lesson 3.1 project, **Does Rain Keep Shoppers Away?** (Steps 2 to 6). Check: a rainy-day drop at all four stores, between -17.6% and -26.3%, and a written finding that says one thing the charts do not prove.
- ☐ Lesson 2.9: Exercises 1 to 3 (Exercise 4 is a stretch), the Quick Quiz and the journal prompt. Read **Scaling by Hand**: it was only mentioned in the room.
- ☐ Lesson 3.1: Exercises 1 to 3 (Exercise 4 is a stretch), the Quick Quiz and the journal prompt. Read **What a Line Does with Missing Days** and **Points with a Range**.
- ☐ Kit labs not reached in the room: `labs/module-08/l2_9-feature-engineering/store_day_features_starter.py` and `labs/module-08/l3_1-line-scatter-bar/chart_repair_starter.py` (about 20 minutes each).
- ☐ From Lesson 3.1, "Before the Next Lesson": find one pyplot-style example (`plt.plot`, `plt.title`) and rewrite it with `fig, ax = plt.subplots()`.

> **BRING TO SESSION 9**
>
> Your Lesson 3.1 notebook (Lesson 3.2 uses its `daily` and `monthly` tables again) and the pyplot-style example you rewrote. Nothing new to install.

GLOSSARY

TERM	MEANING
`Feature`	A column you build so that a pattern becomes visible: orders so far, days since last order, share bought online.
Unit of analysis	What one row of a table represents. The question decides it: a customer, an order, a store-day.
Derived column	A column computed from other columns in the same row: a flag, a difference, a ratio, a choice.
Reference date	A fixed date, written down, from which elapsed time is measured. Never today's date.
`Binning`	Turning a number into a group. `pd.cut`: you choose the edges. `pd.qcut`: equal-sized groups from the data.

TERM	MEANING
`Categorical`	A column that can hold only values from a fixed list. Ordered categoricals sort and compare correctly.
Ordinal / nominal	A category with a meaningful order (tiers, age bands) / one without (store, channel).
One-hot encoding	One yes/no column per category, made with `pd.get_dummies`. For nominal categories.
`Scaling`	Putting numeric columns on a common footing: min-max (0 to 1), z-score (mean 0, std 1), `np.log1p` for skew.
`Cutoff date`	The date you pretend to stand on. Features come from before it; the target comes from after it.
`Leakage`	A feature that uses information not available on the cutoff date. No error; results that are too good.
`Figure`	The whole picture: the sheet of paper. It has a size and it is what gets saved.
`Axes`	One chart on a Figure, with its own x axis, y axis, title and data. Not the plural of axis.
`Overplotting`	So many dots on top of each other that density is invisible. Fix with a small `s` and `alpha`.
Bin (histogram)	One of the equal intervals a histogram cuts the range into; the bar shows how many rows fall in it.
Object-oriented interface	Drawing with methods on `ax` and `fig`. `plt` is used only to create and show figures.

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 9: Composing and Customizing Figures

Your working notebook for the session where charts become figures for other people. Before the break you put several charts on one page: grids, shared scales, named layouts, and an honest alternative to the two-axis chart. After the break you edit one chart from Matplotlib's first draft to a finished figure: a headline that states the finding, readable axes, color that points at the story, labels where the eye already is, and nothing that does not earn its place.

Lessons 3.2-3.3 | 2 lessons | Session 9 · 3 hours | Name: _____

HOW TO USE THIS WORKBOOK

Each lesson has **Key ideas**, a **Worked example** with its real output and chart, the **Guided lab** you do in class, **Predict, then run** boxes, the start of the **lesson project**, a six-question **Self-check** and space for notes. The lab files are in `labs/module-09/`. Every lab saves its charts as PNG files in an `output/` folder next to the lab file: open them there, or run the file cell by cell in VS Code to see each figure as you go.

SESSION GOALS

- Create a grid of charts with `plt.subplots(rows, cols)`, loop over it with `zip(axs.flat, ...)`, and title the whole figure
- Decide whether panels share a scale, and tell the reader when they do not
- Build an uneven layout with `plt.subplot_mosaic` and fill each panel by name
- Replace a twin-axis chart of two different measures with stacked panels or a ratio
- Edit a default chart: a finding headline checked with a number, formatted ticks, gray plus one, direct labels, an annotation, no junk
- Capture repeated choices in a house style with `plt.rc_context`

PROGRESS TRACKER

- ☐ **3.2 lab** `traffic_panels`: four PNG files, including small multiples and a named layout
- ☐ **3.2 project** Nine Categories, Twelve Months: Step 1 runs in class; Steps 2 to 5 finished at home
- ☐ **3.3 lab** `finish_chart`: my finished chart says its point in five seconds, and I checked the headline

- ☐ **3.3 project** The Footwear Returns Chart: Step 1 runs in class; Steps 2 to 5 finished at home
- ☐ I saved `pinecrest_style` and my helper functions where Session 10 can find them

LESSONS

3.2 Subplots and Composition

85 min solo

KEY IDEAS

- **Panels on a page.** The Figure is the page; each Axes is a panel. The arrangement tells the reader how to read them.
- `plt.subplots(rows, cols)` **returns an array of Axes**: 2-D for a grid, 1-D for one row or column, a single Axes for 1 by 1. `squeeze=False` always gives 2-D.
- **Loop, don't repeat:** `for ax, item in zip(axs.flat, items):` works for every grid shape.
- `sharey=True` makes heights comparable; separate scales show each shape and hide differences in size. Choose for the question, and say so when panels do not share.
- **Label the figure, not four panels:** `fig.suptitle`, `fig.supylabel`, `fig.supxlabel`. Use `layout="constrained"` so nothing collides.
- `plt.subplot_mosaic` builds an uneven layout from a picture of names and returns a dictionary: `axd["trend"]`. A repeated name spans cells.
- **A twin axis with two different measures can be made to say anything.** Stack panels with `sharex=True`, or plot the ratio. A second axis is fine for one quantity in two units (`secondary_yaxis`).
- **Small multiples:** one function that draws a panel on the `ax` it is given, plus a loop. Gray for the other groups, color for the panel's own.

YOU ASK FOR	YOU GET	INDEX IT WITH
`plt.subplots()`	A single Axes	`ax`
`plt.subplots(1, 3)` or `(3, 1)`	A 1-D array	`axs[0]`, `axs[2]`
`plt.subplots(2, 3)`	A 2-D array	`axs[row, col]`
`plt.subplots(1, 3, squeeze=False)`	A 2-D array of shape (1, 3)	`axs[0, 2]`
`plt.subplot_mosaic([["a", "a"], ["b", "c"]])`	A dictionary	`axd["a"]`
`plt.subplot_mosaic("AAB;CCB")`	A dictionary (one letter per cell)	`axd["B"]`

WORKED EXAMPLE: FOUR CITIES' TEMPERATURES AS SMALL MULTIPLES

Question: how does each store city's year of temperatures compare with the others? Four groups, the same chart for each: small multiples on a shared scale, with the other cities in gray behind each panel. (The lesson draws store revenue; this example uses `data/store_weather.csv`.)

One row per month, one column per city: the shape that one line per panel wants.

```
weather = pd.read_csv("data/store_weather.csv", parse_dates=["date"])
weather["month"] = weather["date"].dt.to_period("M").dt.to_timestamp()
highs = weather.pivot_table(index="month", columns="city", values="high_f",
                            aggfunc="mean")
print(highs.round(1).iloc[[0, 6]])
```

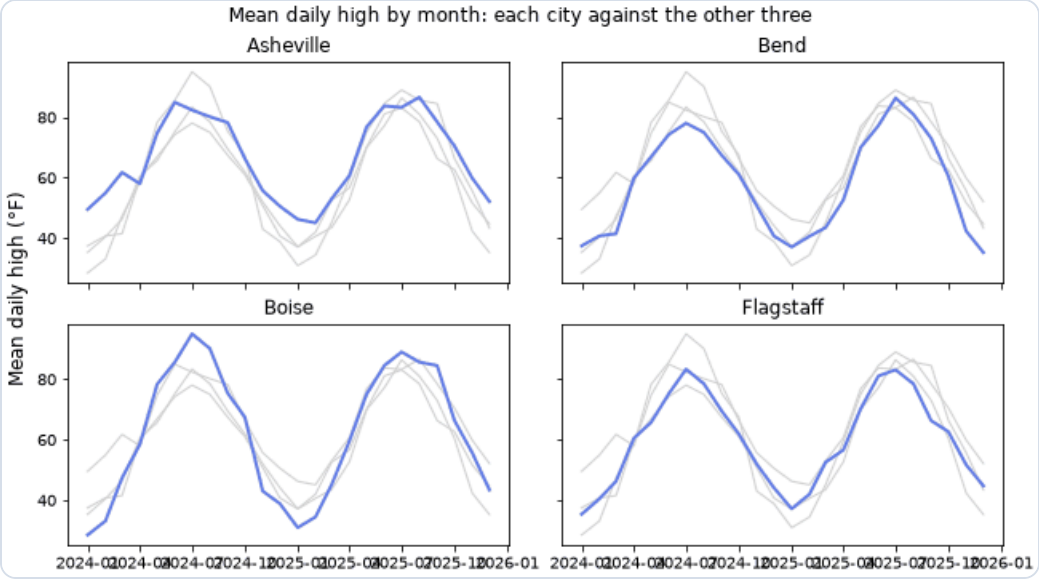
```
city        Asheville  Bend  Boise  Flagstaff
month
2024-01-01       49.5  37.5   28.5       35.3
2024-07-01       82.4  78.0   95.0       83.3
```

A function that draws one panel on the Axes it is given, then a loop over a 2 by 2 grid that shares both axes.

```
def draw_city(ax, table, city):
    for other in table.columns:
        if other != city:
            ax.plot(table.index, table[other], color="lightgray", linewidth=1)
    ax.plot(table.index, table[city], color="#667eea", linewidth=2)
    ax.set_title(city)

fig, axs = plt.subplots(2, 2, figsize=(9, 5), sharex=True, sharey=True,
                        layout="constrained")
for ax, city in zip(axs.flat, highs.columns):
    draw_city(ax, highs, city)
fig.suptitle("Mean daily high by month: each city against the other three")
fig.supylabel("Mean daily high (°F)")
plt.show()
print(axs.shape, [len(ax.lines) for ax in axs.flat])
```

```
(2, 2) [4, 4, 4, 4]
```



The chart this code draws.

Every panel holds four lines, and only one is in color. Because the scale is shared, you can see at once that Boise has the widest swing between winter and summer (its mean monthly high runs from about 28 °F in January 2024 to 95 °F in July 2024) and Asheville the narrowest. With separate scales the four shapes would look almost alike.

◆ GUIDED LAB

Two labs practice this lesson on questions the lesson did not ask. Do Lab 3.2-A in class. Lab 3.2-B is for early finishers and for homework. Both save PNG files to `output/` and never call `plt.show()`.

Lab 3.2-A · Foot traffic on one page (25 minutes, in class): open `labs/module-09/l3_2-subplots-composition/traffic_panels_starter.py`. Which store's foot traffic is growing, and how big is each store's traffic next to the others? Build the same grid twice (own scales, then shared), then small multiples with gray context, then a one-store summary with a named layout.

1. **TODO 1:** a 2 by 2 grid of mean visitors per day, each panel with its own scale, and a suptitle that says so. Print the four y ranges.
2. **TODO 2:** the same grid with `sharex=True, sharey=True` and one `supylabel`. Print whether two panels have the same y range.
3. **TODO 3:** `draw_store(ax, table, store)` with the other stores in gray; a 1 by 4 row of panels.
4. **TODO 4:** a Flagstaff summary with `subplot_mosaic`: a wide trend panel, weekday bars and a histogram.

Done looks like: four PNG files in `output/`; four different y ranges, then `True`; four lines in every small-multiples panel; the panel names `['trend', 'week', 'spread']`.

Lab 3.2-B · Two measures, honestly (20 minutes, early finishers or homework): open `labs/module-09/l3_2-subplots-composition/honest_axes_starter.py`. A manager wants Asheville's visitors

and the number of rainy days on one chart. Make the twin-axis version tell two different stories, then show the data honestly, compute the relationship directly, and finish with the one honest use of a second axis.

1. **TODO 1:** one figure with two twin-axis charts of the same data; only the right-hand limits differ.
2. **TODO 2:** stacked panels sharing the x axis: visitors on top, rainy days as bars below.
3. **TODO 3:** the monthly correlation, and mean visitors on rainy against dry **days**.
4. **TODO 4:** mean daily high with Fahrenheit on the left and Celsius on the right (`secondary_yaxis`).

Done looks like: three PNG files; a correlation near 0.3; a clear per-day difference; a temperature chart with two agreeing scales.

◆ PREDICT, THEN RUN

Write your prediction on the line **before** you run the code. Being wrong here is useful: it shows you exactly which idea to revisit.

Predict 3.2.1 Three grids. What type or shape does each give back?

```
a = plt.subplots(1, 1)[1]
b = plt.subplots(2, 1)[1]
c = plt.subplots(2, 1, squeeze=False)[1]
print(type(a).__name__, b.shape, c.shape)
plt.close("all")
```

Predict 3.2.2 Two panels with very different data and `sharey=True` . Same y range or not?

```
fig, axs = plt.subplots(1, 2, sharey=True)
axs[0].plot([0, 1], [0, 10])
axs[1].plot([0, 1], [0, 2])
print(axs[0].get_ylim() == axs[1].get_ylim())
plt.close(fig)
```

Predict 3.2.3 A period in the layout picture. How many panels, with which names?

```
fig, axd = plt.subplot_mosaic([["main", "side"],
                               ["main", "."]])
print(list(axd), len(fig.axes))
plt.close(fig)
```

PROJECT START: NINE CATEGORIES, TWELVE MONTHS

Lesson 3.2's project builds a 3 by 3 figure of small multiples showing when each product category has its season, from completed orders. In class you do **Step 1** in a new notebook called `lesson_3_2_project`: merge the order lines, the completed orders and the product categories, and pivot to one row per month and one column per category.

You are done with Step 1 when your output shows 41,788 lines on completed orders and 567 with no category, then `(12, 9)`, then the Snow/Tents/Apparel table.

At home: Steps 2 to 5: a first grid, percent of each category's year on a shared scale, panels ordered by peak month, and a finished figure. Check: Snow peaks in December at 28.0% of its year.

SELF-CHECK (ANSWERS ARE WITH YOUR INSTRUCTOR)

Q1

What does this print?

```
fig, axs = plt.subplots(2, 3)
print(axs.shape, len(fig.axes), type(axs[1, 2]).__name__)
plt.close(fig)
```

Q2

After `fig, axs = plt.subplots(1, 4)`, the line `axs[0, 2].set_title("Boise")` fails. Why?

A) A grid must have at least two rows before panels can be titled

B) A one-row grid comes back as a 1-D array, so it takes one index: `axs[2]`

C) `set_title` only works on the figure: `fig.set_title`

D) The panels are numbered from 1, so the third panel is `axs[1, 3]`

Q3

Four panels show monthly revenue for four stores. When should you pass `sharey=True`?

A) Always: panels without a shared scale are wrong

B) Never: sharing squashes the smaller stores

C) When the reader should compare sizes across panels; separate scales show each shape but hide the differences in size

D) Only when the panels are in one row

Q4

What does this print?

```
fig, axd = plt.subplot_mosaic("AAB;CCB")
print(sorted(axd), len(fig.axes))
plt.close(fig)
```

Q5

Which chart is an honest use of a second y axis?

A) Revenue on the left and visitors on the right, made with `twinx()`

B) Revenue on the left and staff on duty on the right

C) Boise's revenue on the left and Asheville's revenue on a separate right-hand scale

D) Temperature in °F on the left and the same values in °C on the right, with `secondary_yaxis`

Q6

Seven categories are drawn on a 3 by 3 grid with `for ax, name in zip(axs.flat, categories):`. What does the reader see in the last two cells?

A) Two empty frames with ticks from 0 to 1, until you remove them with `ax.remove()`

B) Nothing: `zip` deletes the unused panels

C) Copies of the last category

D) An `IndexError`, because the grid has more panels than names

MY NOTES FOR THIS LESSON

3.3 Customizing Plots

90 min solo

KEY IDEAS

- **A default chart is a first draft; you are the editor.** Every edit should help the reader get the point faster.
- **The title states the finding; the subtitle says what is plotted**, the units and the period. Check the finding with a number before you write it, and keep causes out unless they were tested.
- **Locators** choose where ticks go; **formatters** what they say: `"${x:,.0f}"`, `FuncFormatter(thousands)`, `PercentFormatter(xmax=1)`, `mdates.MonthLocator`, `mdates.DateFormatter("%b %Y")`.
- **Give color one job:** highlight (gray plus one), categories (qualitative, such as Okabe-Ito), magnitude (sequential, such as viridis), above or below a midpoint (diverging, centered).
- **Design for color-blind readers:** a safe palette, no red against green as the only difference, and never color as the only cue.
- **Direct labels beat a legend** when lines can be labeled at their ends: nothing to look up.
- **Annotate sparingly:** `ax.annotate` with an arrow, `ax.axvspan` for a period, `ax.axhline` for a reference. One to three per chart.
- **Delete chart junk** (extra spines, tick marks, heavy grids, repeated labels). **Bars start at zero.** Put repeated choices in a style dictionary and use it with `plt.rc_context`.

YOU WANT	WRITE
A finding headline and a gray subtitle	`ax.set_title(h, loc="left", fontweight="bold", pad=24)` + `ax.text(0, 1.03, s, transform=ax.transAxes, color="0.35")`
Dollars with commas	`ax.yaxis.set_major_formatter("${x:,.0f}")`
Thousands of dollars	`FuncFormatter(lambda x, pos: f"${x / 1000:,.0f}K")`
A fraction as a percent	`ax.yaxis.set_major_formatter(PercentFormatter(xmax=1))`
Ticks every January and July	`ax.xaxis.set_major_locator(mdates.MonthLocator(bymonth=[1, 7]))`
Positions and labels together	`ax.set_xticks(range(1, 13), labels=MONTHS)`
A label at the end of a line	`ax.annotate(name, xy=(x, y), xytext=(6, 0), textcoords="offset points")`

YOU WANT	WRITE
A shaded period	`ax.axvspan(start, end, color="0.93", zorder=0)`
No top or right frame	`ax.spines[["top", "right"]].set_visible(False)`
A style for one block	`with plt.rc_context(pinecrest_style): ...`

WORKED EXAMPLE: ONE SHRINKING STORE, SAID IN FIVE SECONDS

The finding from Session 7: three stores grew in 2025 and one shrank. A finished chart should say that before anyone reads the axis. (The lesson edits a line chart; this example edits a bar chart.)

The numbers first: revenue growth per store, 2025 against 2024, sorted.

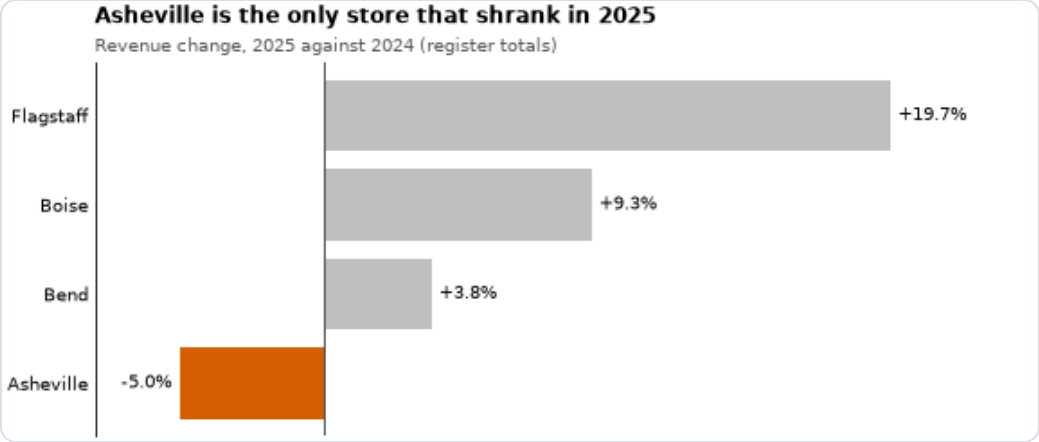
```
daily = pd.read_csv("data/daily_sales.csv", parse_dates=["date"])
year = daily["date"].dt.year
yearly = daily.groupby(["store", year])["revenue"].sum().unstack()
growth = ((yearly[2025] / yearly[2024] - 1) * 100).sort_values()
print(growth.round(1))
```

```
store
Asheville    -5.0
Bend          3.8
Boise         9.3
Flagstaff    19.7
dtype: float64
```

Horizontal bars, gray plus one accent for the store that shrank, each value written on its bar, a zero line, a finding headline, and the junk removed.

```
colors = ["#d55e00" if g < 0 else "0.75" for g in growth]
fig, ax = plt.subplots(figsize=(8, 3.4), layout="constrained")
bars = ax.barh(growth.index, growth.values, color=colors)
ax.bar_label(bars, labels=[f"{g:+.1f}%" for g in growth], padding=4)
ax.axvline(0, color="0.3", linewidth=1)
ax.set_title("Asheville is the only store that shrank in 2025", loc="left",
             fontweight="bold", pad=22)
ax.text(0, 1.03, "Revenue change, 2025 against 2024 (register totals)",
        transform=ax.transAxes, color="0.35")
ax.xaxis.set_visible(False)
ax.spines[["top", "right", "bottom"]].set_visible(False)
ax.tick_params(length=0)
ax.set_xlim(growth.min() * 1.6, growth.max() * 1.25)
plt.show()
print([b.get_facecolor()[:3] != (0.75, 0.75, 0.75) for b in bars])
```

```
[True, False, False, False]
```



The chart this code draws.

The printed list marks which bars are not gray: only the first, Asheville. The labels make the x axis unnecessary, so it is hidden; the line at zero is the one reference the reader needs. The headline is a claim the bars prove: -5.0% for Asheville, growth everywhere else.

◆ GUIDED LAB

Two labs practice this lesson on data the lesson did not use. Do Lab 3.3-A in class. Lab 3.3-B is for early finishers and for homework.

Lab 3.3-A · From first draft to finished chart (25 minutes, in class): open `labs/module-09/l3_3-customizing-plots/finish_chart_starter.py`. The online team will show one chart at the year-end meeting: completed online revenue by month, 2024 against 2025. Check the story with numbers, then edit the first draft until a stranger gets the point in five seconds.

1. **TODO 1:** count the months in which 2025 beat 2024 and find the largest gap, **before** writing a headline.
2. **TODO 2:** `add_titles`: a finding headline and a gray subtitle.
3. **TODO 3:** y ticks as thousands of dollars, x ticks as month names, y from zero.
4. **TODO 4:** 2024 in gray, 2025 in color, the years written at the line ends, no legend.
5. **TODO 5:** an arrow at the largest gap, the junk removed, `finished.png` saved.

Done looks like: `draft.png` and `finished.png`; 12 of 12 months; y labels like `$20K`; month names on the x axis; only the left and bottom spines; no legend.

Lab 3.3-B · Give color a job (20 minutes, early finishers or homework): open `labs/module-09/l3_3-customizing-plots/color_jobs_starter.py`. Two store-by-month tables need two different kinds of color: mean visitors (low to high: sequential) and revenue change against 2024 (above or below zero: diverging, centered). Draw both as heatmaps and check what the colors claim.

1. **TODO 1:** a viridis heatmap of mean visitors with a colorbar and month and store tick labels.
2. **TODO 2:** an RdBu heatmap of the change table with `vmin=-limit, vmax=limit`, so that zero is the neutral color.
3. **TODO 3:** count the months each store shrank against 2024.

Done looks like: two PNG files; the change map's color limits symmetric around zero; shrinking months per store printed.

◆ PREDICT, THEN RUN

Write your prediction on the line **before** you run the code. Being wrong here is useful: it shows you exactly which idea to revisit.

Predict 3.3.1 Two format strings. What does each produce?

```
print("{x:,.0f}".format(x=1234.6), "${x:,.2f}".format(x=7))
```

Predict 3.3.2 Which sides of the frame are still visible?

```
fig, ax = plt.subplots()
ax.spines[["top", "right"]].set_visible(False)
print([side for side in ax.spines if ax.spines[side].get_visible()])
plt.close(fig)
```

Predict 3.3.3 Three positions and three labels. What do the tick labels say?

```
fig, ax = plt.subplots()
ax.plot([0, 1], [0, 1])
ax.set_xticks([0, 0.5, 1], labels=["start", "middle", "end"])
print([t.get_text() for t in ax.get_xticklabels()])
plt.close(fig)
```

PROJECT START: THE FOOTWEAR RETURNS CHART

Lesson 3.3's project finds which product categories are returned most often online and edits the chart until someone who sees it for five seconds gets the point. Continue in your lesson notebook: the project reuses `add_titles`. In class you do **Step 1**: the return rate of online orders containing each category.

You are done with Step 1 when nine sorted rates print with Footwear last at 0.144, and `All online orders: 0.069`.

At home: Steps 2 to 5: the first draft, shape and color, a reference line, and a headline checked against the numbers. Then save `pinecrest_style` and your helper functions: Session 10 uses them.

SELF-CHECK (ANSWERS ARE WITH YOUR INSTRUCTOR)

Q7

A chart shows the monthly online return rate for 2024 and 2025; both lines wobble between about 5.7% and 9.1%. Which title serves a reader best?

A) Online return rate by month

B) Online returns are rising

C) The online return rate held near 7% in both years

D) Why customers send things back

Q8

What does this print?

```
x = 1234567
print(f"${x / 1000:,.0f}K", "{x:.1%}".format(x=0.0712))
```

Q9

A heatmap shows each store's revenue change against the same month last year, from -12.6% to +29.0%. Which color choice fits?

A) A qualitative palette such as Okabe-Ito, one color per cell

B) A sequential map such as viridis, from the lowest change to the highest

C) A diverging map such as RdBu, with the limits set symmetric around zero

D) A rainbow map such as jet, to show as many levels as possible

Q10

A line chart has four series whose ends are well apart. Why replace the legend with direct labels?

A) The reader no longer has to look a color up in a box, and the chart works even when colors are hard to tell apart

B) Matplotlib raises a warning when a legend has more than three entries

C) Direct labels make the figure render faster

D) Legends cannot be used with gray lines

Q11

What does this print?

```
before = plt.rcParams["axes.spines.top"]
with plt.rc_context({"axes.spines.top": False}):
    fig, ax = plt.subplots()
inside = ax.spines["top"].get_visible()
plt.close(fig)
print(before, inside, plt.rcParams["axes.spines.top"])
```

Q12

Which edit makes a bar chart of 2025 store revenue misleading?

A) Removing the top and right spines

B) Starting the y axis at $1.5 million to "show the differences better"

C) Adding a faint horizontal grid behind the bars

D) Writing each bar's value on it with `bar_label`

MY NOTES FOR THIS LESSON

EXIT TICKET

Answer on a sticky note or a scrap of paper, and hand it in on the way out.

1. What does `plt.subplots(2, 2)` give you in `axs`, and what is the safest way to loop over the panels?
2. Why is a twin-axis chart of two different measures risky, and what do you draw instead?
3. Rewrite the title "Monthly online revenue, 2024 and 2025" as a finding, and say what you checked first.

HOMEWORK CHECKLIST

- ☐ Finish the Lesson 3.2 project, **Nine Categories, Twelve Months** (Steps 2 to 5). Check: `by_month` is `(12, 9)`, every column of `share` adds up to 100, and Snow peaks in December at 28.0%.
- ☐ Finish the Lesson 3.3 project, **The Footwear Returns Chart** (Steps 2 to 5). Check: Footwear at 14.4%, 2.1 times the overall online rate of 6.9%, and a headline you compared with the numbers.
- ☐ Lesson 3.2: Exercises 1 to 3 (Exercise 4 is a stretch), the Quick Quiz and the journal prompt. Read **Layout: Size and Spacing** and the secondary-axis example.
- ☐ Lesson 3.3: Exercises 1 to 3 (Exercise 4 is a stretch), the Quick Quiz and the journal prompt. Read the **heatmap** section and **Ready-Made Styles**.

- ☐ Kit labs not reached in the room: `labs/module-09/l3_2-subplots-composition/honest_axes_starter.py` and `labs/module-09/l3_3-customizing-plots/color_jobs_starter.py` (about 20 minutes each).
- ☐ From Lesson 3.3, "Before the Next Lesson": save `pinecrest_style` and your helper functions (`add_titles`, `format_axes`, `label_line_ends`, `clean_up`) in a file or a clearly marked cell. Session 10 uses them.

> **BRING TO SESSION 10**
> Your Lesson 3.3 notebook with `pinecrest_style` and the helper functions saved in a clearly marked cell or a `.py` file, and your two finished project figures. Nothing new to install.

GLOSSARY

TERM	MEANING
`Subplots`	Several Axes on one Figure. `plt.subplots(nrows, ncols)` returns the Figure and an array of Axes.
`axs.flat`	Walks through every panel of the array row by row, whatever its shape.
`Squeeze`	`plt.subplots` drops dimensions of length one: one row gives a 1-D array, one panel a single Axes. `squeeze=False` keeps 2-D.
`Shared axes`	`sharex=True` / `sharey=True`: panels use one scale, so positions compare across panels.
suptitle / supylabel	A title or axis label for the whole figure, not one panel.
subplot_mosaic	An uneven layout drawn as a picture of names; returns a dictionary of Axes.
`Twin axis`	`ax.twinx()`: a second y axis on the right with its own, arbitrary scale.
Secondary axis	`ax.secondary_yaxis("right", functions=(f, g))`: the same quantity in a second unit, tied by a formula.
Small multiples	The same small chart repeated once per group, with the same axes.
`Finding title`	A title that states what the chart shows, with a subtitle that says what is plotted.
Locator / formatter	Where the ticks go / what text each tick shows.
`Gray plus one`	One series in color, the rest in gray as context: color used to direct attention.

TERM	MEANING
Qualitative / sequential / diverging	Palettes for unordered groups / low-to-high numbers / numbers with a meaningful midpoint.
`Direct label`	A series' name written next to the series itself, replacing a legend.
`Chart junk`	Ink that carries no information: extra spines, tick marks, heavy grids, redundant labels.
rcParams / rc_context	Matplotlib's defaults, and a `with` block that changes them for the charts drawn inside it only.

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 10: Saving Figures; Center and Spread

Your working notebook for the session that finishes charts and starts statistics. First you get figures out of the notebook at the right size, in the right format, with text people can read. Then you boil 20,302 orders down to two honest numbers: one for what is typical and one for how much orders vary.

Lessons 3.4-4.1 | Session 10 · 3 hours | Name: _____

HOW TO USE THIS WORKBOOK

Each lesson has **By the end you can** (your goals), **Key ideas** (the page to keep), a **Worked example** (new code with its real output), the **Guided lab** you do in class, **Predict, then run** boxes, where the lesson project starts, space for notes, and a six-question **Self-check**. Lab files are in `labs/module-10/` : open the `_starter.py` files. Type the examples yourself and run after every small step. This session changes subject at the break, from charts to statistics. That is on purpose: the charts you can now make are how you will check that the numbers are telling the truth.

SESSION GOALS

- Save a figure with `fig.savefig()` and predict its pixel size from `figsize` and `dpi`.
- Explain why text comes out tiny, and fix it with the figure size, not the dpi.
- Choose PNG, SVG or PDF for a destination, and save a whole set of charts through one function that closes each figure.
- Compute mean, median and mode, and say which one a sentence needs.
- Compute range, variance, standard deviation and IQR, and know whether your library divided by n or n - 1.
- Pair mean with SD for symmetric data and median with IQR for skewed data.

PROGRESS TRACKER

- ☐ **3.4** I saved one chart for three destinations and checked each file with `plt.imread(path).shape` (lab `pixel_budget`)
- ☐ **3.4** My Chart Pack has 9 files and every web PNG is 990 x 495 (lesson project)
- ☐ **4.1** I can explain why the mean order ($194.60) and the median order ($146.85) differ (lab `orders_per_buyer` shows the same pattern)

- ☐ **4.1** My `center_and_spread()` function works and my Typical Order Memo is generated from variables (lesson project)
- ☐ I finished the exercises and the Quick Quiz for each lesson at home
- ☐ I wrote a journal entry after each lesson

LESSONS

3.4 Saving Figures

75 min alone

BY THE END YOU CAN

- Save a figure with `fig.savefig()` and predict its size in pixels from `figsize` and `dpi`
- Choose between raster formats (PNG) and vector formats (SVG, PDF) for a given destination
- Explain why text looks tiny or huge in a saved chart, and fix it with the figure size
- Control the saved area and background with `bbox_inches`, `facecolor` and `transparent`
- Write one function that applies a house style and saves for web, slides or print, then use it in a loop that closes each figure

KEY IDEAS

- **The sheet and the scanner.** `figsize` is the sheet of paper, in inches. `dpi` is the scanner. **pixels = inches x dpi**, once for the width and once for the height.
- **Say which figure.** `fig.savefig(path)` saves the figure you name. `plt.savefig()` saves whatever pyplot thinks is current, which can be an empty one.
- **The format comes from the extension.** `.png` is a grid of pixels (raster). `.svg` and `.pdf` are drawing instructions (vector) and stay sharp at any size.
- **Text is sized in points**, and a point is 1/72 of an inch on the sheet. A smaller sheet makes the same text look bigger. Raising the dpi never does.
- **Destinations.** Web or email: PNG, about 110 dpi. A 16:9 slide: PNG at 9.6 x 5.4 inches and 200 dpi (1,920 x 1,080). Print or anything that will be resized: PDF or SVG. Never JPEG for charts.
- `bbox_inches="tight"` stops labels being cut off and changes the saved size. With `layout="constrained"` you rarely need it.
- `transparent=True` removes the background only. Black text on a dark slide vanishes. A solid `facecolor="white"` is the safe default.
- **Presets and one save function.** Put sizes, dpi and formats in one dictionary; create the figure at the right size before drawing; save through one function.
- **Create, draw, save, close.** In a loop, `plt.close(fig)` after every save. `PdfPages` puts many figures in one PDF.
- **Do not trust, check.** `plt.imread(path).shape` is (rows, columns, channels). Then open the file at the size it will really be shown.

WORKED EXAMPLE

The intranet home page has a banner slot exactly 1,200 pixels wide and 300 high. Work backward: 8 inches x 150 dpi = 1,200 and 2 inches x 150 dpi = 300. Create the figure at that size, draw, save, close, check.

```
from pathlib import Path
import matplotlib.pyplot as plt
import pandas as pd

daily = pd.read_csv("data/daily_sales.csv", parse_dates=["date"])
weekly = daily.set_index("date")["visitors"].resample("W").sum()

fig, ax = plt.subplots(figsize=(8, 2), layout="constrained")
ax.plot(weekly.index, weekly.values, color="#667eea")
ax.set_title("All stores: visitors per week", loc="left")

Path("charts").mkdir(exist_ok=True)
banner = Path("charts") / "visitors_banner.png"
fig.savefig(banner, dpi=150, facecolor="white")     # 8 x 150 by 2 x 150
plt.close(fig)

print(plt.imread(banner).shape)
print("open figures:", len(plt.get_fignums()))
```

```
(300, 1200, 4)
open figures: 0
```

300 rows, 1,200 columns, 4 values per pixel (red, green, blue, alpha). Zero open figures, because the figure was closed after saving. The same 1,200 x 300 pixels from a 12 x 3 inch sheet at 100 dpi would have a title two thirds the size.

GUIDED LAB

Lab 3.4-A: open `labs/module-10/l3_4-saving-figures/pixel_budget_starter.py`. One chart of Bend's visitors by weekday, saved for an email, a newsletter and a poster. Write your pixel prediction in the comment before you run Step 1. You are done when the last lines say `5 files` and `open figures: 0`.

Lab 3.4-B: open `labs/module-10/l3_4-saving-figures/category_price_pack_starter.py`. In class: TODO 1 and 2 (the presets and the save line). At home: the loop, the PDF and the manifest. Done means 7 files, web images 960 x 480, slide images 1920 x 1080.

Both labs save into an `output` folder next to the lab file. Open the saved files to look at your charts: judging the file, not the notebook preview, is the habit this lesson builds.

Finished early? `labs/module-10/l3_4-saving-figures/debug_blank_slide.py` runs without an error and is wrong in three ways. Find them.

PREDICT, THEN RUN

Write your prediction first. Then type the code and run it. A wrong prediction is useful: it shows exactly which idea to revisit.

Predict 3.4-1

What does this print? (Careful: which number comes first?)

```
fig, ax = plt.subplots(figsize=(5, 2))
fig.savefig("charts/p1.png", dpi=300)
plt.close(fig)
print(plt.imread("charts/p1.png").shape[:2])
```

Predict 3.4-2

What two numbers does this print?

```
for i in range(3):
    fig, ax = plt.subplots()
    fig.savefig(f"charts/p2_{i}.png")
print(len(plt.get_fignums()))
plt.close("all")
print(len(plt.get_fignums()))
```

LESSON PROJECT: WHERE YOU START IN CLASS

The Pinecrest Chart Pack. In class you do Step 1 in your demo notebook: build `yearly` and `growth` and write `draw_store_page()`. Your `growth` should print Asheville -5.0, Bend 3.8, Boise 9.3, Flagstaff 19.7. At home: Steps 2 to 4 (save every store two ways, one PDF, the manifest). You are finished when the manifest shows 9 files and every web PNG is 990 x 495.

MY NOTES

SELF-CHECK (Q1 TO Q6)

Q1

A figure is created with `figsize=(9, 5)` and saved with `fig.savefig("sales.png", dpi=120)`. How many pixels is the image?

A) 900 wide by 500 high

B) 1,080 wide by 600 high

C) 600 wide by 1,080 high

D) 129 wide by 125 high

Q2

What does this print?

```
fig, ax = plt.subplots(figsize=(4, 3))
fig.savefig("q2.png", dpi=50)
print(plt.imread("q2.png").shape[:2])
```

Q3

Two PNG files are both 1,600 by 800 pixels. File A came from `figsize=(16, 8)` at `dpi=100`, file B from `figsize=(8, 4)` at `dpi=200`. Which statement is true?

A) The two images are identical, because the pixel counts are identical

B) B is blurrier, because its sheet was smaller

C) A holds more detail, because its figure was larger

D) In B the text, lines and markers are twice as large relative to the chart

Q4

A scatter plot of 200,000 points must go into a printed report as a PDF. The file is huge and slow to open. What is the best fix?

A) Pass `rasterized=True` to the scatter layer and keep saving as PDF

B) Save as JPEG instead, because JPEG files are small

C) Lower the `dpi` of the PDF to 50

D) Add `bbox_inches="tight"` to the save call

Q5

A chart is saved with `transparent=True` and pasted onto a dark slide. The title and tick labels cannot be seen. Why?

A) Transparent PNG files cannot contain text

B) The slide program removed the labels when it resized the image

C) Only the background became transparent; the text is still black, on a dark slide

D) The dpi was too low for the labels to be drawn

Q6

What does this print?

```
plt.close("all")
first, ax = plt.subplots()
second, ax = plt.subplots()
first.savefig("q6.png")
plt.close(first)
print(len(plt.get_fignums()))
```

4.1 Descriptive Statistics I: Center and Spread

85 min alone

BY THE END YOU CAN

- Compute the mean, median and mode of a column with pandas and say what each one measures
- Recognize when the mean misleads, and choose the median instead, using Pinecrest's skewed order totals
- Compute range, variance, standard deviation and interquartile range (IQR), and explain each in the units of the data
- Explain the difference between dividing by `n` and by `n - 1`, and predict which one NumPy and pandas use by default
- Pair the right measure of center with the right measure of spread for a given column

KEY IDEAS

- **Two questions about any column.** Center: what is typical? Spread: how much does it vary? One number is never enough.
- **Mean** = sum / count: the balance point. It uses every value, multiplies back to the total, and is pulled by extremes.
- **Median** = the middle value of the sorted list. Half the values are below it. It is **robust**: an extreme value cannot move it far.
- **Mode** = the most common value. The only center for categories. `.mode()` returns a Series because ties happen; take `[0]` for one value.
- **The mean of a True/False column is a proportion.** `(totals < totals.mean()).mean()` is the share of orders below the mean: 0.626.
- **Right-skewed** data (a long tail to the right, like anything about money) has its mean above its median. The gap is information, not an error.
- **Variance** = the average squared distance from the mean. **Standard deviation (SD)** = its square root, back in the data's own units: "a typical distance from the mean".
- **n or n - 1?** NumPy divides by n (`ddof=0`, a population). pandas divides by n - 1 (`ddof=1`, a sample). If unsure, `ddof=1`.
- **IQR** = Q3 - Q1, the width of the middle half of the data. Robust, like the median.
- **Two standard pairs.** Symmetric data: mean and SD. Skewed data, or data that may hold errors: median and IQR. A histogram tells you which you have.
- **Missing values.** pandas skips them (print `.count()` next to `len()`); `np.mean` returns `nan`

without an error.

WORKED EXAMPLE

Two columns from the product catalog, summarized the same way: the star rating (1 decimal) and the number of reviews.

```
import pandas as pd

products = pd.read_csv("data/products.csv")
for column in ["rating", "review_count"]:
    values = products[column]
    iqr = values.quantile(0.75) - values.quantile(0.25)
    gap = (values.mean() - values.median()) / values.median() * 100
    print(column, "| n", values.count())
    print(f"  mean {values.mean():.2f}  median {values.median():.2f}")
    print(f"  mean vs median {gap:+.1f}%")
    print(f"  SD {values.std():.2f}  IQR {iqr:.2f}")
```

```
rating | n 234
  mean 4.14  median 4.10
  mean vs median +0.9%
  SD 0.39  IQR 0.50
review_count | n 242
  mean 70.45  median 39.00
  mean vs median +80.7%
  SD 88.57  IQR 75.75
```

Read the gap first (how far the mean sits above the median, as a percentage of the median). For `rating` the mean and the median almost agree (the mean is 0.9% higher), so there is no long tail: report **mean and SD**, "ratings average 4.14 with an SD of 0.39". For `review_count` the mean is 81% above the median: a few products have hundreds of reviews. Report **median and IQR**, "a typical product has 39 reviews; the middle half have between 14 and about 90". Notice also `n`: 8 products have no rating yet, and pandas left them out without saying so.

GUIDED LAB

Lab 4.1-A: open `labs/module-10/l4_1-center-and-spread/orders_per_buyer_starter.py`. The loyalty team says "our customers order about five times". Find the mean, median and mode of orders per buyer, and decide which number belongs in which sentence. Write your guess for their order before TODO 1.

Lab 4.1-B: open `labs/module-10/l4_1-center-and-spread/visitor_spread_starter.py`. In class: Steps 1 to 3 (a variance by hand, matching the libraries, a spread table for the four stores). At home: Steps 4 and 5.

Finished early? `labs/module-10/l4_1-center-and-spread/debug_average_of_averages.py`: three summaries that run without an error and are quietly wrong.

PREDICT, THEN RUN

Write your prediction first. Then type the code and run it. A wrong prediction is useful: it shows exactly which idea to revisit.

Predict 4.1-1

Three numbers: the mean, the median and the mode. Predict all three.

```
s = pd.Series([3, 5, 5, 8, 50])
print(s.mean(), s.median(), s.mode()[0])
```

Predict 4.1-2

Same three numbers, two libraries. What does each print?

```
x = np.array([2, 4, 9])
print(np.var(x), pd.Series(x).var())
```

LESSON PROJECT: WHERE YOU START IN CLASS

The Typical Order Memo. New notebook `lesson_4_1_project`. In class you write `center_and_spread(values)` and run it on the order totals: n 20,302, mean 194.60, median 146.85, sd 178.76, iqr 186.10, and the mean 32.51% above the median. At home: Steps 2 to 5 (three columns in one table, the bulk-order check, the two-panel chart, the memo). Every number in your memo must come from a variable.

MY NOTES

SELF-CHECK (Q7 TO Q12)

Q7

What does this print?

```
print(pd.Series([10, 20, 30, 40, 400]).median())
```

A) `100.0`

B) `30.0`

C) `25.0`

D) `400`

Q8

`orders["payment_method"]` **holds the words card, digital_wallet, cash and gift_card. Which summary of its center makes sense?**

A) The mean

B) The median

C) The mode

D) The standard deviation

Q9

What does this print?

```
x = np.array([1, 2, 3, 4, 5])
print(np.var(x), pd.Series(x).var())
```

Q10

Pinecrest's order totals have a mean of $194.60, a median of $146.85 and a mode of $53.90. Finance wants a revenue forecast for 1,000 orders next month. Which number do you multiply by 1,000?

A) The mean, $194.60

B) The median, $146.85

C) The mode, $53.90

D) The midpoint of the mean and the median

Q11

One order in a column of 20,000 order totals was typed as $1,000,000 by mistake. Which pair of summaries barely changes?

A) Mean and standard deviation

B) Range and standard deviation

C) Mean and range

D) Median and interquartile range

Q12

What does this print?

```
revenue = pd.Series([100.0, np.nan, 300.0])
print(revenue.mean(), np.mean(revenue.to_numpy()), revenue.count())
```

A) `200.0 nan 2`

B) `133.33 nan 3`

C) `200.0 200.0 3`

D) `nan nan 2`

EXIT TICKET

Three questions at the end of class. No notes: this is for you and your instructor to see what stuck.

Exit 1

A figure has `figsize=(6, 4)` and is saved with `dpi=150`. How many pixels wide and high is the PNG?

Exit 2

A column of order totals is strongly right-skewed. Which measure of center and which measure of spread do you report for a typical order?

Exit 3

`np.std(x)` and `pd.Series(x).std()` give different answers for the same small list. Why?

HOMEWORK CHECKLIST

- ☐ **Lesson 3.4 project:** finish the Pinecrest Chart Pack (Steps 2 to 4): 9 files, every web PNG 990 x 495, a 4-page PDF, 0 open figures.
- ☐ **Lesson 3.4 practice:** Exercises 1 to 3 (Exercise 4 is a stretch), the five-question Quick Quiz, and the journal prompt (it also asks for one line on Module 3 as a whole).
- ☐ **Lesson 4.1 project:** finish the Typical Order Memo (Steps 2 to 5): the three-row table, the bulk-order check, the two-panel figure saved as `typical_order.png`, and the memo built from variables.
- ☐ **Lesson 4.1 practice:** Exercises 1 to 3 (Exercise 4 is a stretch), the Quick Quiz, and the journal prompt. Read the section "Sample or Population? The n - 1 Question" and run its simulation once.
- ☐ **Kit labs:** finish `category_price_pack` (TODO 3 and 4) and `visitor_spread` (Steps 4 and 5).

- ☐ **Workbook:** the 12 self-check items and anything left in Predict, then run.
- ☐ **Before Lesson 4.2** (from the lesson page): run `orders["order_total"].describe()` and name every line you can; guess, then check, whether `visitors` in `daily_sales.csv` is skewed by comparing its mean and median.

BRING TO SESSION 11

- Your Lesson 4.1 notebook with a working `center_and_spread()` function (Lesson 4.2 extends it).
- Your `pinecrest_style`, `new_figure()` and `save_chart()` code, somewhere you can copy it from.
- Nothing new to install, no accounts, no servers. Check once at home that `import scipy` works in the course environment.
- This workbook.

GLOSSARY

- **dpi:** dots per inch: how many pixels the saved image has for each inch of the figure.
- **figsize:** the size of the figure in inches, (width, height). The sheet of paper.
- **Raster image:** a grid of colored pixels with a fixed resolution (PNG, JPEG).
- **Vector image:** a list of drawing instructions that stays sharp at any size (SVG, PDF).
- **Point:** the unit of font sizes and line widths: 1/72 of an inch on the sheet.
- **Bounding box:** the area of the figure that is saved. `bbox_inches="tight"` fits it to what is drawn, which changes the size.
- **facecolor:** the figure's background color. `transparent=True` removes it.
- **Preset:** a named set of saved settings (size, dpi, format) for one destination.
- **Manifest:** a small table listing every file you produced, used to verify an export.
- **Center (central tendency):** what is typical in a column: mean, median or mode.
- **Spread (variability):** how much the values vary: range, variance, SD or IQR.
- **Mean:** the sum divided by the count; the balance point of the data.
- **Median:** the middle value of the sorted data; half the values are below it.
- **Mode:** the most common value; the only center for categories.
- **Robust:** barely affected by extreme values. The median and the IQR are robust; the mean and the SD are not.
- **Right-skewed:** having a long tail of large values to the right; the mean sits above the median.

- **Range:** the largest value minus the smallest.
- **Variance:** the average squared deviation from the mean.
- **Standard deviation (SD):** the square root of the variance, in the data's own units; a typical distance from the mean.
- **ddof:** "delta degrees of freedom": the number subtracted from n before dividing. NumPy's default is 0, pandas's is 1.
- **Population / sample:** all of the group you care about / a part of it used to estimate the whole.
- **Quartiles, IQR:** Q1 and Q3 have a quarter and three quarters of the values below them; the IQR is Q3 - Q1, the width of the middle half.

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 11: Shape, Outliers and Distributions

Your working notebook for the session that finishes describing data and starts reasoning about chance. First you measure the shape of a column, locate values inside it and decide what to do with the ones that stand apart. Then you model uncertainty with four distributions, simulate when the math gets hard, and put an honest margin of error on an average.

Lessons 4.2-4.3 | Session 11 · 3 hours | Name: _____

HOW TO USE THIS WORKBOOK

Each lesson has **By the end you can** (your goals), **Key ideas** (the page to keep), a **Worked example** (new code with its real output), the **Guided lab** you do in class, **Predict, then run** boxes, where the lesson project starts, space for notes, and a six-question **Self-check**. Lab files are in `labs/module-11/`: open the `_starter.py` files. This is a full session. You are not expected to remember four distributions by tonight. Aim to leave with two habits: look at what a rule flags before you remove anything, and check a model against the data before you trust it.

SESSION GOALS

- Measure the shape of a column with `.skew()` and say what it means for your choice of summary.
- Locate a value with percentiles and with a z-score.
- Run the IQR and z-score outlier rules, explain why they disagree, and decide what to do with a flagged value.
- Read `describe()` and a box plot, and summarize survey answers by group with their n.
- Choose a distribution (uniform, normal, binomial, Poisson), freeze it with `scipy.stats`, and ask it questions with `cdf`, `sf`, `ppf` and `pmf`.
- Compute a standard error and a 95% confidence interval for a mean, and say what the 95% describes.

PROGRESS TRACKER

- ☐ **4.2** I ran both outlier rules on a column and investigated the flagged values before deciding anything (lab `catalog_weights`)
- ☐ **4.2** My Store Profile Sheet explains what the 91 flagged days are and what I did about them (lesson project)

- ☐ **4.3** I froze a normal distribution, checked it against data, and answered questions with `cdf`, `sf` and `ppf` (lab `conversion_model`)
- ☐ **4.3** My Returns Forecast has a checked Poisson model, capacity numbers and a confidence interval (lesson project)
- ☐ I finished the exercises and the Quick Quiz for each lesson at home
- ☐ I wrote a journal entry after each lesson

LESSONS

4.2 Descriptive Statistics II: Shape, Position and Outliers

90 min alone

BY THE END YOU CAN

- Describe the shape of a distribution with a histogram and a skewness value, and say what the shape implies for your choice of summary
- Locate a value within its column using percentiles, quantiles and z-scores
- Apply the IQR and z-score outlier rules, explain why they disagree, and choose a defensible action for flagged values
- Read every line of `describe()` and every part of a box plot
- Summarize by group with `groupby().agg()`, and summarize 1-to-5 survey answers as the ordered categories they are

KEY IDEAS

- **Four things to know about a column:** center, spread, shape, and one value's position in it. Shape decides which summaries are fair.
- **Skewness** (`.skew()`): 0 for a symmetric pile, positive for a long right tail. Between -0.5 and 0.5: mean and SD are fair. Beyond 1: median and IQR. It is not robust and does not show humps: always look at a histogram.
- **A logarithm tames a right tail.** Order totals: skew 5.48; `np.log(totals)` : -0.41.
- **Percentiles and quantiles locate by rank.** `totals.quantile(0.90)` takes a fraction (0 to 1). `np.percentile(totals, 90)` takes 0 to 100. The reverse question: `(totals <= 500).mean()` .
- **A z-score locates by distance:** z = (value - mean) / SD. It makes different scales comparable. Standardizing gives mean 0 and SD 1 and does not change the shape.
- **Two outlier rules.** IQR fences: below Q1 - 1.5 x IQR or above Q3 + 1.5 x IQR. z-scores: beyond 3. They disagree, because the mean and SD are inflated by the values being judged.
- **A flag is a question, not a verdict.** Provable error? Fix or remove it, and note what you did. Real and part of your question? Keep it and use the median and IQR. Real but a different kind of thing? Analyze separately and say so.
- `describe()` : count (non-missing), mean, std (sample SD), min, 25%, 50%, 75%, max. Compare mean with 50%, and the two halves of the box.
- **A box plot**: the line is the median; the box is Q1 to Q3; each whisker ends at the last data point inside the fence; separate points are the flagged values. Good for comparing groups; hides humps.

- **Grouped summaries always carry their n.** `count` is the number of values, `size` the number of rows.
- **Survey answers on a 1-to-5 scale are ordinal:** ordered categories. Lead with the distribution, the median and the share satisfied (4 or 5), and check who did not answer.

WORKED EXAMPLE

Was Black Friday 2025 an unusual day for foot traffic at the Flagstaff store? Locate it two ways: by distance (a z-score) and by rank (a percentile).

```
import pandas as pd

daily = pd.read_csv("data/daily_sales.csv", parse_dates=["date"])
flagstaff = daily.loc[daily["store"] == "Flagstaff"].set_index("date")
visitors = flagstaff["visitors"]

day = visitors.loc["2025-11-28"]                  # Black Friday 2025
z = (day - visitors.mean()) / visitors.std()
rank = (visitors <= day).mean()

print("visitors that day:", day)
print("store mean:", round(visitors.mean(), 1))
print("store SD  :", round(visitors.std(), 1))
print("z-score:", round(z, 2))
print(f"at or below this on {rank:.1%} of Flagstaff's days")

boise = daily.loc[daily["store"] == "Boise", "visitors"]
print("the same count in Boise: z =",
      round((day - boise.mean()) / boise.std(), 2))
```

```
visitors that day: 596
store mean: 328.5
store SD  : 117.4
z-score: 2.28
at or below this on 96.8% of Flagstaff's days
the same count in Boise: z = 0.68
```

596 visitors is 2.28 standard deviations above Flagstaff's average, and only about 3 days in 100 were busier. The same 596 visitors in Boise would be 0.68 SDs above average: a good day, nothing more. A z-score is always measured against a particular group's mean and SD, so say which group.

GUIDED LAB

Lab 4.2-A: open `labs/module-11/l4_2-shape-position-outliers/catalog_weights_starter.py`. The shipping team wants you to "remove the outlier weights" before computing a typical product weight. Describe the shape, run both rules, look at what they flag, then decide. Write your guess (mean above or below the median?) before TODO 1.

Lab 4.2-B: open `labs/module-11/l4_2-shape-position-outliers/price_satisfaction_starter.py`. In class: TODO 1 to 3 (the distribution, `bottom_two()`, the table by age group). At home: who skipped the question, the saved chart, and your one-sentence finding with its n. The chart is saved into an `output` folder next to the lab file; open it from there.

Finished early? `labs/module-11/l4_2-shape-position-outliers/debug_summary_report.py`: four results that run without an error and are quietly wrong.

PREDICT, THEN RUN

Write your prediction first. Then type the code and run it. A wrong prediction is useful: it shows exactly which idea to revisit.

Predict 4.2-1

One number. Careful: is the answer one of the values in the list?

```
print(pd.Series([10, 20, 30, 40, 50]).quantile(0.25))
```

Predict 4.2-2

Three z-scores. Predict the list.

```
s = pd.Series([2, 4, 6])
z = (s - s.mean()) / s.std()
print(z.tolist())
```

LESSON PROJECT: WHERE YOU START IN CLASS

The Store Profile Sheet. New notebook `lesson_4_2_project`. In class you do Step 1: drop the 6 store-days with no revenue, add the weekday name, and build the profile table (n, median, q1, q3, p95, max, iqr) for the four stores. Check: Boise's median is 9,360 and its maximum 27,677. At home: Steps 2 to 6 (z-scores within each store, the flagged days, what they turn out to be, the box plot, the written profile).

MY NOTES

SELF-CHECK (Q1 TO Q6)

Q1

Pinecrest's order totals have a skewness of 5.48. After `np.log(totals)` the skewness is -0.41. What does that tell you?

A) The logarithm deleted the twelve bulk orders

B) The original totals contained errors that the logarithm repaired

C) On the log scale the pile is roughly symmetric, so methods that prefer symmetric data will behave better there

D) The mean of the logs is the log of the mean

Q2

What does this print?

```
print(pd.Series([10, 20, 30, 40]).quantile(0.5))
```

Q3

A $12,000 day has a z-score of 3.05 at the Asheville store and 0.50 at the Boise store. What does that mean?

A) The day is far above Asheville's own average, measured in Asheville's SDs, and only a little above Boise's

B) Asheville took 3.05 times as much revenue as Boise that day

C) The day was in the top 3.05% of Asheville's days

D) z-scores from different stores cannot be compared

Q4

On the order totals the IQR rule flags 848 orders and the rule "|z| above 3" flags 217. Why does the z-score rule flag so many fewer?

A) The z-score rule only looks at the smallest values

B) The IQR rule counts duplicated orders twice

C) z-scores cannot be computed for skewed data

D) The mean and SD are inflated by the same large orders being judged, which pushes the z-score line outward; the quartiles are not

Q5

What does this print?

```
s = pd.Series([1, 2, 3, 4, 5, 6, 7, 8, 9, 50])
q1, q3 = s.quantile(0.25), s.quantile(0.75)
upper = q3 + 1.5 * (q3 - q1)
print(upper, (s > upper).sum())
```

Q6

In the survey, Online shoppers' mean `sat_staff` is 4.07, computed from 248 of the 824 Online respondents. What is the honest way to report it?

A) Report 4.07 as the staff satisfaction of Online shoppers

B) Report it with its n and say who answered: 70% of Online shoppers skipped the question, so it describes the minority who dealt with staff

C) Fill the 576 missing answers with 4.07 so that the n is 824

D) Leave the Online row out of the table without comment

4.3 Probability Distributions

100 min alone

BY THE END YOU CAN

- Explain what a random variable and a probability distribution are, and tell discrete from continuous
- Choose between the uniform, normal, binomial and Poisson distributions for a situation, and check the choice against data
- Answer probability questions with `scipy.stats` using `pmf` / `pdf`, `cdf`, `sf` and `ppf`
- Simulate random outcomes reproducibly with `np.random.default_rng` and use simulation to check a calculation
- Demonstrate the central limit theorem by simulation, compute a standard error, and build and correctly interpret a 95% confidence interval for a mean

KEY IDEAS

- **A random variable** is a number decided by chance. Its **distribution** lists the possible values with their probabilities: the vending machine's stocking list.
- **Discrete** variables are counted (returns in a day). **Continuous** ones are measured (spend per visitor), and only ranges have probability.
- **Four workhorses.** Uniform: equally likely anywhere in a range. Normal: sums and averages of many small effects (mean, SD). Binomial: successes in n independent tries with the same p. Poisson: events in an interval at a steady rate (mean about equal to variance).
- **Freeze, then ask.** `model = stats.norm(loc=mean, scale=sd)`. `cdf(x)`: this or less. `sf(x)`: more than this. `ppf(q)`: which value has q below it. `pmf(k)`: exactly k (discrete). `rvs(size, random_state=rng)`: random draws.
- `scale` **is the SD, not the variance.** For `stats.uniform`, `loc` is the start and `scale` is the width.
- **For a count, "5 or more" is** `sf(4)`.
- **Simulation checks any formula.** Create one seeded generator at the top (`rng = np.random.default_rng(42)`) and keep drawing from it. Never create it inside a loop.
- **68-95-99.7:** in a normal distribution about 68% of values are within 1 SD of the mean, 95% within 2, 99.7% within 3. It is a property of the normal shape only.
- **A model's assumptions are part of it.** Check the fit against data before trusting its answers.
- **Central limit theorem:** means of random samples are approximately normal, centered on the population mean, with SD sigma / sqrt(n), whatever the population's shape. The data itself does not become normal.

- **Standard error:** SE = s / sqrt(n): how much a sample mean varies from sample to sample. The SD describes single values and does not shrink with more data.
- **95% confidence interval:** mean plus or minus about 2 SE (`stats.t.interval`). The 95% describes the method: about 95 of 100 intervals built this way contain the true mean. It is not a range for individual values.

WORKED EXAMPLE

About 7% of Pinecrest's orders are paid with a gift card. Out of the next 30 orders, how many will be? A fixed number of tries (30), each with the same probability: a binomial model. Compute, then check by simulation.

```
import numpy as np
import pandas as pd
from scipy import stats

rng = np.random.default_rng(3)
orders = pd.read_csv("data/orders.csv")
p_gift = (orders["payment_method"] == "gift_card").mean()

batch = stats.binom(n=30, p=p_gift)              # gift-card orders out of 30
print("share paid by gift card:", round(p_gift, 4))
print("expected in a batch of 30:", round(batch.mean(), 2))
print(f"P(none)      = {batch.pmf(0):.3f}")
print(f"P(5 or more) = {batch.sf(4):.3f}")

sim = rng.binomial(n=30, p=p_gift, size=100_000)  # check by counting
print("simulated P(5 or more):", round((sim >= 5).mean(), 3))
```

```
share paid by gift card: 0.074
expected in a batch of 30: 2.22
P(none)      = 0.100
P(5 or more) = 0.067
simulated P(5 or more): 0.067
```

About 2 gift-card orders per batch on average; one batch in ten has none; five or more happens in about 7% of batches. "5 or more" is `sf(4)`: more than 4. The simulation, which only counts, agrees with the formula to the third decimal. The model assumes each order is independent with the same probability, which is worth doubting in December.

GUIDED LAB

Lab 4.3-A: open `labs/module-11/l4_3-probability-distributions/conversion_model_starter.py`. What share of walk-in visitors buy something on a typical store-day, and what do a bad day and a great day look like? Fit a normal model, check it against the data, then use `cdf`, `sf` and `ppf`. Write the method next to each question before you code it.

Lab 4.3-B: open `labs/module-11/l4_3-probability-distributions/lines_per_order_ci_starter.py`. In class: Steps 1 to 4 (one sample of 150 orders, its standard error, a 95% interval, and a prediction). At home: Steps 5 and 6 (is "95%" honest? what does four times the sample buy?).

Both files create one seeded generator at the top. Run them from top to bottom and your numbers match your neighbor's.

Finished early? `labs/module-11/l4_3-probability-distributions/debug_delivery_model.py`: four model answers, all quietly wrong.

PREDICT, THEN RUN

Write your prediction first. Then type the code and run it. A wrong prediction is useful: it shows exactly which idea to revisit.

Predict 4.3-1

Two numbers. No calculator needed.

```
model = stats.norm(loc=50, scale=10)
print(model.cdf(50), model.ppf(0.5))
```

Predict 4.3-2

Ten coin flips. Three numbers: predict each.

```
flips = stats.binom(n=10, p=0.5)
print(flips.mean(), flips.cdf(10), flips.sf(10))
```

LESSON PROJECT: WHERE YOU START IN CLASS

The Returns Forecast. New notebook `lesson_4_3_project`. In class you do Steps 1 and 2: count returned orders per day including the zero days (731 days, 1,143 returns, 151 days with none), then check that a Poisson model fits (mean 1.564, variance 1.526, and an observed-against-expected table with no large

differences). At home: Steps 3 to 6 (planning questions, a simulated year, a week, a confidence interval for the rate, the forecast).

MY NOTES

SELF-CHECK (Q7 TO Q12)

Q7

You want to model the number of returned orders in a day: events scattered through time at a steady average rate, with no fixed number of tries. Which distribution do you try first, and what do you check?

A) Uniform; check that the histogram is flat

B) Normal; check that the 68-95-99.7 rule holds

C) Binomial with n = 731; check that p is the same every day

D) Poisson; check that the mean and the variance are about equal

Q8

What does this print?

```
print(stats.norm(loc=20, scale=3).cdf(20))
```

Q9

`batch = stats.binom(n=25, p=0.07)` models returns in a batch of 25 orders. Which expression is the probability of 3 or more returns?

A) `batch.sf(2)`

B) `batch.sf(3)`

C) `batch.pmf(3)`

D) `batch.cdf(3)`

Q10

The standard deviation of order totals is about $178.76. What is the standard error of the mean of a random sample of 400 orders?

A) About $178.76

B) About $8.94

C) About $0.45

D) About $3,575

Q11

From 200 orders an auditor gets a 95% confidence interval for the mean order total that is about $44 wide. Roughly how many orders are needed for an interval half as wide?

A) 100

B) 400

C) 800

D) 20,000

Q12

What is true of the two lists this prints?

```
for _ in range(2):
    rng = np.random.default_rng(7)
    print(rng.integers(1, 7, size=3).tolist())
```

A) They are identical: each pass builds a new generator from the same seed, which replays the same numbers

B) They differ, because a generator never repeats itself

C) The second list continues where the first one stopped

D) The code raises an error, because a seed can be used only once

EXIT TICKET

Three questions at the end of class. No notes: this is for you and your instructor to see what stuck.

Exit 1

The IQR rule flags 91 store-days as unusually high. What do you do before removing any of them?

Exit 2

You have a frozen distribution called `model`. Which method returns the value that 95% of the distribution lies below?

Exit 3

A 95% confidence interval for the mean order total is $165 to $209. Does that mean 95% of orders are between $165 and $209? If not, what does the 95% describe?

HOMEWORK CHECKLIST

- ☐ **Lesson 4.2 project:** finish the Store Profile Sheet (Steps 2 to 6): z-scores within each store, the flagged days and what they turn out to be, the box plot saved as `store_profile.png`, the written profile.
- ☐ **Lesson 4.2 practice:** Exercises 1 to 3 (Exercise 4 is a stretch), the five-question Quick Quiz, and the journal prompt. Read the survey section in full, including "Who Did Not Answer?"
- ☐ **Lesson 4.3 project:** finish the Returns Forecast (Steps 3 to 6): planning questions with `pmf`, `sf` and `ppf`, a simulated year, a week, a confidence interval for the rate.
- ☐ **Lesson 4.3 practice:** Exercises 1 to 3 (Exercise 4 is a stretch), the Quick Quiz, and the journal prompt. Read "Common Mistakes" in full: all six are silent.
- ☐ **Kit labs:** finish `price_satisfaction` (TODO 4 to 6) and `lines_per_order_ci` (Steps 5 and 6).
- ☐ **Workbook:** the 12 self-check items and anything left in Predict, then run.
- ☐ **Before Lesson 4.4** (from the lesson page): make a scatter plot of `visitors` against `revenue` from `daily_sales.csv` and describe the pattern in one sentence. Then predict: do stores take more revenue on days when more staff are on duty? Why might that be true even if adding staff did nothing?

BRING TO SESSION 12

- Your scatter plot of visitors against revenue, and your written prediction about staffing and revenue.
- The pattern mean plus or minus t x s / sqrt(n) in your notes.
- Nothing new to install, no accounts, no servers. Check once at home that `import statsmodels` works in the course environment.
- This workbook.

GLOSSARY

- **Distribution:** how the values of a column (or the outcomes of a random variable) are spread: which values

occur and how often.

- **Skewness:** a number for lopsidedness: 0 for symmetric, positive for a long right tail, negative for a long left tail.
- **Percentile / quantile:** the value with a given share of the data at or below it, on a 0-to-100 scale (percentile) or a 0-to-1 scale (quantile).
- **z-score:** (value - mean) / SD: how many standard deviations a value lies above or below the mean.
- **Outlier:** a value that sits far from the rest of its column. A candidate for a closer look, not automatically an error.
- **IQR fences:** Q1 - 1.5 x IQR and Q3 + 1.5 x IQR. Values beyond them are flagged; box plots use this rule.
- **Box plot:** a drawing of the median, the quartiles, whiskers to the last points inside the fences, and the flagged points.
- **Ordinal data:** ordered categories with no guaranteed distances between them, such as 1-to-5 survey answers (a Likert item).
- **Top-two-box:** the share of answers that are 4 or 5 on a 1-to-5 scale.
- **Missing not at random:** missing for a reason connected to what is being measured, so the answers you have may not represent the group.
- **Random variable:** a number whose value is decided by chance.
- **Discrete / continuous:** counted in separate values / measured on a continuous range.
- **Expected value:** the long-run average of a random variable.
- **Frozen distribution:** a `scipy.stats` distribution with its parameters fixed, ready to answer questions.
- **pmf / pdf:** the probability of exactly k (discrete) / the height of the curve at x (continuous; not a probability).
- **cdf / sf / ppf:** the probability of x or less / of more than x / the value with probability q below it.
- **Normal distribution:** the bell curve, described by its mean and SD.
- **Binomial distribution:** the number of successes in n independent tries with the same probability p.
- **Poisson distribution:** the number of events in a fixed interval at a steady average rate; its mean equals its variance.
- **Overdispersion:** more day-to-day variation than a single-rate Poisson allows: variance well above the mean.
- **Central limit theorem (CLT):** means of random samples are approximately normal, centered on the population mean, with SD sigma / sqrt(n).
- **Standard error (SE):** s / sqrt(n): how much a sample mean varies from sample to sample.
- **Confidence interval:** an estimate with a margin of error, from a method that captures the true value a stated share of the time (95%).

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 12: Correlation and Hypothesis Tests

Your working notebook for the session that turns tables into claims you can defend. First you measure how two columns move together and learn why a strong correlation can point the wrong way. Then you ask whether a difference could be chance, answer it with a test you build yourself, and learn to report the answer honestly.

Lessons 4.4-4.5 | Session 12 · 3 hours | Name: _____

HOW TO USE THIS WORKBOOK

Each lesson has **By the end you can** (your goals), **Key ideas** (the page to keep), a **Worked example** (new code with its real output), the **Guided lab** you do in class, **Predict, then run** boxes, where the lesson project starts, space for notes, and a six-question **Self-check**. Lab files are in `labs/module-12/` : open the `_starter.py` files. Today the code is easy and the sentences are hard. Practice saying results out loud: "goes with", not "causes"; "not detected", not "the same"; and every p-value with how big and how sure.

SESSION GOALS

- Draw the scatter plot first, then measure association with Pearson and Spearman and choose between them.
- Read a correlation matrix, and test a suspected confounder by comparing like with like.
- Fit a regression line and read its slope, intercept and r squared in the units of the data, without reading the slope as a cause.
- Explain what a p-value is and what it is not, and build a test yourself by shuffling.
- Choose and run a Welch, one-sample, paired, chi-square or Mann-Whitney test.
- Report every test with an effect size, a confidence interval and its caveats.

PROGRESS TRACKER

- ☐ **4.4** I plotted first, measured two ways, and wrote an answer about reviews and sales that does not claim a cause (lab `reviews_and_sales`)
- ☐ **4.4** My Rain Check compares like with like and gives the effect in visitors (lesson project)
- ☐ **4.5** I wrote my hypotheses before testing, and my report on returned orders says both "real" and "small" (lab `returned_order_size`)

- ☐ **4.5** My rain finding uses the reporting template, with Holm-adjusted p-values and caveats (lesson project)
- ☐ I finished the exercises and the Quick Quiz for each lesson at home
- ☐ I wrote a journal entry after each lesson, and I am ready for the Session 13 checkpoint

LESSONS

4.4 Correlation Analysis

95 min alone

BY THE END YOU CAN

- Read direction, form and strength from a scatter plot, and explain why the plot comes before the number
- Compute Pearson and Spearman correlations with pandas and scipy, and choose between them
- Build a correlation matrix and draw it as an annotated heatmap with Matplotlib
- Identify a confounding variable and test for it by comparing like with like
- Fit and interpret a simple regression line, and state what it does and does not tell you

KEY IDEAS

- **Plot first.** A scatter plot shows direction (rising or falling), form (straight, curved, clusters) and strength (how tightly points hug the form). r summarizes one kind of pattern only.
- **Pearson's r** measures straight-line association from -1 to +1. It is the average product of z-scores: no units, symmetric, and sensitive to extreme points. `x.corr(y)`.
- **r is not a slope and not a percentage.** The share of variance a straight line accounts for is r squared: r = 0.5 is 25%.
- **r = 0 means no linear trend**, not no relationship. A perfect curve can score 0.
- **Spearman's rho** is Pearson on ranks (`method="spearman"`): consistent up or down, straight or not. Use it for curves, outliers and ordinal data. A gap between the two means a curve or an influential point. Compute both.
- **A correlation matrix** (`df[cols].corr()`) shows every pair. A block of high correlations usually has one common driver. Heatmaps: a diverging colormap with the scale fixed at -1 to 1.
- **Correlation is not causation.** A causes B, B causes A, a confounder causes both, selection, or chance. Ask how the data came to be.
- **A confounder** drives both variables. Test for it by comparing like with like: subtract each group's mean (`groupby().transform("mean")`) and correlate the deviations, or use a partial correlation.
- **The staffing verdict:** staff and revenue r = 0.88 pooled, 0.14 among comparable days, 0.07 holding visitors constant. The calendar drives both.
- **A regression line** (`stats.linregress(x, y)`, predictor first) gives the slope in real units. Predict only within the data's range. A slope describes; it is not the effect of changing x.

WORKED EXAMPLE

Do orders with more product lines cost more? Measure two ways, then fit a line to get the answer in dollars. (No test here: whether a pattern could be chance is Lesson 4.5's question.)

```
import pandas as pd
from scipy import stats

orders = pd.read_csv("data/orders.csv")
items = pd.read_csv("data/order_items.csv")
lines = items.groupby("order_id").size().rename("lines")
both = orders.merge(lines, on="order_id")

print("orders:", len(both))
print("Pearson :", round(both["lines"].corr(both["order_total"]), 3))
print("Spearman:", round(both["lines"].corr(both["order_total"],
                                            method="spearman"), 3))
fit = stats.linregress(both["lines"], both["order_total"])
print("slope   :", round(fit.slope, 2), "dollars per extra product line")
print("r squared:", round(fit.rvalue ** 2, 3))
```

```
orders: 20302
Pearson : 0.54
Spearman: 0.646
slope   : 86.46 dollars per extra product line
r squared: 0.291
```

Spearman (0.65) is clearly above Pearson (0.54): the relationship is consistent but not a straight band, because a few orders with one bulk line are very large. Each extra product line goes with about $86 more, on average, and the line accounts for about 30% of the variance: the number of lines is far from the whole story (what is on them matters more). Notice the reading: "goes with", not "adds".

◆ GUIDED LAB

Lab 4.4-A: open `labs/module-12/l4_4-correlation-analysis/reviews_and_sales_starter.py`. Marketing says "products with more reviews sell more". Plot first (write a sentence about each panel before you compute anything), then measure two ways, draw a heatmap, and decide what you can and cannot tell marketing. Your charts are saved into an `output` folder next to the lab file.

Lab 4.4-B: open `labs/module-12/l4_4-correlation-analysis/staffing_rule_starter.py`. In class: Steps 1 and 2 (the line and its typical miss). At home: one line per store, the reversed line, and what the line does not say.

Finished early? `labs/module-12/l4_4-correlation-analysis/debug_correlation_claims.py`: four claims about r, all wrong.

◆ PREDICT, THEN RUN

Write your prediction first. Then type the code and run it. A wrong prediction is useful: it shows exactly which idea to revisit.

Predict 4.4-1

Two correlations. Predict both before you run.

```
u = pd.Series([-2, -1, 0, 1, 2])
print(u.corr(u ** 2), u.corr(u * 3 + 1))
```

Predict 4.4-2

Pearson, then Spearman. Which one is 1?

```
a = pd.Series([1, 2, 3, 4, 5])
b = pd.Series([1, 4, 9, 16, 100])
print(round(a.corr(b), 2), round(a.corr(b, method="spearman"), 2))
```

LESSON PROJECT: WHERE YOU START IN CLASS

The Rain Check. New notebook `lesson_4_4_project`. In class you do Step 1: merge `store_weather.csv` into `daily_sales.csv` (the location column is `city` in one file and `store` in the other) and check that you still have 2,903 rows with no missing weather. Keep this table: the 4.5 project uses the same merge. At home: Steps 2 to 6 (look, measure two ways, compare like with like, put the effect in visitors, write the finding).

MY NOTES

SELF-CHECK (Q1 TO Q6)

Q1

Across the 234 products that have sold, the number of reviews and the units sold have r = 0.46. Which reading is soundest?

A) Asking customers for more reviews will raise sales

B) Products with more reviews tend to have sold more; the arrow may run from sales to reviews, and the data alone cannot say

C) 46% of a product's sales are explained by its reviews

D) The correlation is too weak to mean anything

Q2

What does this print?

```
x = pd.Series([1, 2, 3, 4])
y = pd.Series([10, 8, 6, 4])
print(round(x.corr(y), 2))
```

Q3

To check whether staffing and revenue are linked beyond the calendar, which calculation compares like with like?

A) Correlate the z-scores of staff and revenue instead of the raw values

B) Use Spearman's correlation instead of Pearson's

C) Subtract each group's mean (same store, month, weekday and promotion status) from both columns with `groupby().transform("mean")`, then correlate the deviations

D) Remove the outlier days first, then recompute r

Q4

For which pair of columns is Spearman clearly the better choice than Pearson?

A) Visitors and revenue, which form a straight, tight band

B) Revenue in dollars and the same revenue in cents

C) Two columns that are identical

D) 1-to-5 overall satisfaction and the 0-to-10 recommend score

Q5

A line fitted to two columns with r = 0.5. What does this print?

```
r = 0.5
print(f"the line accounts for {r ** 2:.0%} of the variance")
```

Q6

A regression of revenue on staff on duty gives a slope of $1,154 per extra staff member. A manager plans to add two people every Tuesday to gain about $2,300 a day. What is the problem?

A) A slope describes how revenue differs across days with different staffing, not what changing the staffing would do; here it is mostly the calendar acting on both

B) The slope must first be multiplied by r

C) The intercept was left out of the calculation

D) Regression cannot be used when one column holds whole numbers

4.5 Hypothesis Testing

110 min alone

BY THE END YOU CAN

- Explain the logic of a hypothesis test in plain words, and say what a p-value is and what it is not
- Build a permutation test by simulation and match it to SciPy's ready-made tests
- Choose and run a one-sample, Welch two-sample or paired t-test, a chi-square test, or a rank-based test for the data in front of you
- Report every test with an effect size and a confidence interval, and check its assumptions on the data
- Describe Type I and Type II errors and power, and correct for multiple comparisons

KEY IDEAS

- **A hypothesis test asks one question:** how surprising would these data be if nothing were going on?
- **The four steps:** state what "nothing going on" means (the null, H0); work out what that would produce; see where the real result falls; report how rare it is.
- **The p-value** is the probability, if the null hypothesis were true, of a result at least as extreme as the one observed. It is **not** the probability that the null is true, and it is **not** a measure of size.
- **Verdicts:** p below alpha (usually 0.05, chosen before looking): reject the null. Otherwise: fail to reject. Never "accept". 0.049 and 0.051 are the same evidence: report the number.
- **A permutation test** builds "nothing going on" by shuffling the labels many times. The named tests reach the same answer with theory.
- **Choosing:** two independent groups: Welch (`ttest_ind(x, y, equal_var=False)`). One group against a target: `ttest_1samp`. The same units twice: paired (`ttest_rel`). Counts in categories: chi-square (`chi2_contingency(crosstab)`). Ratings or ranks: Mann-Whitney (`mannwhitneyu`).
- **Every p-value travels with an effect size and a confidence interval.** Cohen's d (0.2 small, 0.5 medium, 0.8 large); for tables, the rates and their ratio.
- **Statistically significant** = unlikely to be chance alone. **Practically significant** = big enough to matter. Large samples make tiny effects significant.
- **Two ways to be wrong:** a false alarm (Type I, rate alpha) and a miss (Type II). **Power** is the chance of catching a real effect; it rises with sample size. "Not significant" means "not detected".
- **Many tests, many false alarms.** Count every test you ran, correct with Holm (`multipletests(..., method="holm")`), and label scanned results as exploratory.
- **Every test assumes independent observations.** SciPy does not skip missing values: `.dropna()` first.

- **Report template:** what was compared (and n) · the estimate · its 95% CI · the test and p · the effect size in words · caveats.

WORKED EXAMPLE

Do Bend and Flagstaff shoppers differ in how likely they are to recommend Pinecrest? Hypotheses first, then the test, the interval and a careful sentence.

```
import pandas as pd
from scipy import stats

survey = pd.read_csv("data/survey.csv")
bend = survey.loc[survey["store"] == "Bend", "recommend"].dropna()
flag = survey.loc[survey["store"] == "Flagstaff", "recommend"].dropna()
# H0: Bend and Flagstaff shoppers have the same mean recommend score.
# H1: the means differ (two-sided). alpha = 0.05. Welch t-test.

res = stats.ttest_ind(bend, flag, equal_var=False)
ci = res.confidence_interval()
print(f"Bend n = {len(bend)}, mean {bend.mean():.2f}; "
      f"Flagstaff n = {len(flag)}, mean {flag.mean():.2f}")
print(f"difference {bend.mean() - flag.mean():.2f}, "
      f"95% CI {ci.low:.2f} to {ci.high:.2f}")
print(f"t = {res.statistic:.2f}, p = {res.pvalue:.2f}")
```

```
Bend n = 223, mean 7.27; Flagstaff n = 207, mean 7.39
difference -0.11, 95% CI -0.51 to 0.28
t = -0.57, p = 0.57
```

p = 0.57: a gap this size (about a tenth of a point) turns up by chance more than half the time, so you fail to reject the null. The careful sentence: "We did not detect a difference between Bend and Flagstaff shoppers; the data are consistent with Bend being anywhere from about half a point lower to about a quarter of a point higher." It does not say the two stores are the same.

GUIDED LAB

Lab 4.5-A: open `labs/module-12/l4_5-hypothesis-testing/returned_order_size_starter.py`. Customer service says "people send back the big orders". Write your hypotheses in the TODO 0 comment **before** you run any test. Then Welch, the interval, Cohen's d, a permutation test, and a challenge to your own result. Your report must say how big the difference is, not only whether it is significant.

Lab 4.5-B: open `labs/module-12/l4_5-hypothesis-testing/footwear_returns_starter.py`. In class: Step 1 (online orders with and without footwear: a table, the rates, a chi-square test). At home: the same question for store orders, the Holm correction, and the report.

Finished early? `labs/module-12/l4_5-hypothesis-testing/debug_test_report.py`: five conclusions, each one misreading a test.

PREDICT, THEN RUN

Write your prediction first. Then type the code and run it. A wrong prediction is useful: it shows exactly which idea to revisit.

Predict 4.5-1

Two identical groups. What are t and p?

```
res = stats.ttest_ind([1, 2, 3], [1, 2, 3], equal_var=False)
print(res.statistic, res.pvalue)
```

Predict 4.5-2

A table in which both rows have the same proportions. Predict the statistic, the p-value and the expected counts.

```
table = pd.DataFrame([[30, 10], [30, 10]])
chi = stats.chi2_contingency(table)
print(chi.statistic, chi.pvalue, chi.expected_freq.tolist())
```

LESSON PROJECT: WHERE YOU START IN CLASS

Does Rain Keep Shoppers Away? New notebook `lesson_4_5_project`. In class you do Steps 1 and 2: the merge from the Rain Check plus a `rainy` column (0.10 inches or more), the summary of visitors by store and rain, and your hypotheses and alpha written as a comment **before** the first test. At home: Steps 3 to 7 (Asheville in full, a permutation check, all four stores with Holm, a challenge for season and weekday, the written finding).

MY NOTES

SELF-CHECK (Q7 TO Q12)

Q7

Shuffling the store labels 10,000 times gives a p-value of 0.030 for Asheville's lower recommend score. What does 0.030 measure?

A) In a world where the store makes no difference, about 3% of shuffles produced a gap at least as large as the real one

B) There is a 3% probability that the null hypothesis is true

C) There is a 97% probability that Asheville really is worse

D) The gap is 3% of the average score

Q8

You want to compare 1-to-5 staff-satisfaction ratings at Asheville (193 answers) with the other stores (754 answers). Which test fits best?

A) A Welch t-test on the two means

B) A paired t-test

C) A one-sample t-test against 4

D) A Mann-Whitney U test

Q9

What does this print?

```
print(stats.ttest_ind([1, 2, 3], [1, 2, np.nan], equal_var=False).pvalue)
```

Q10

A Welch t-test reports a 95% confidence interval for the difference in means of -0.68 to -0.03. Without computing it, what do you know about the two-sided p-value at alpha = 0.05?

A) It is above 0.05

B) It is exactly 0.05

C) It is below 0.05

D) It cannot be known from the interval

Q11

A pilot with 10 Asheville answers and 50 others gives p = 0.82 for a staff-rating gap that the full survey shows clearly (p near 3 in a billion). How should the pilot's result be worded?

A) The stores' staff ratings are the same

B) We could not detect a difference with this sample; it is too small to say much

C) The pilot proves the null hypothesis

D) The full survey must contain an error

Q12

Five independent tests, all of true null hypotheses, each at alpha = 0.05. What does this print?

```
print(f"{1 - 0.95 ** 5:.0%}")
```

EXIT TICKET

Three questions at the end of class. No notes: this is for you and your instructor to see what stuck.

Exit 1

Staff on duty and revenue correlate at r = 0.88. Give two explanations other than "more staff cause more revenue".

Exit 2

Complete the sentence: "A p-value is the probability of _____, assuming _____."

Exit 3

Besides the p-value, which two numbers belong in every test report?

HOMEWORK CHECKLIST

- ☐ **Lesson 4.4 project:** finish the Rain Check (Steps 2 to 6): look, measure two ways, compare like with like, the effect in visitors, the finding with its limits.
- ☐ **Lesson 4.4 practice:** Exercises 1 to 3 (Exercise 4 is a stretch), the Quick Quiz, and the journal prompt.
- ☐ **Lesson 4.5 project:** finish Does Rain Keep Shoppers Away? (Steps 3 to 7), ending with a written finding in the reporting template.
- ☐ **Lesson 4.5 practice:** Exercises 1 to 4 (Exercise 5 is a stretch), the Quick Quiz, and the journal prompt. Reread the "What a p-value Is Not" table tomorrow.
- ☐ **Kit labs:** finish `staffing_rule` (Steps 3 to 5) and `footwear_returns` (Steps 2 to 4).
- ☐ **Workbook:** the 12 self-check items and anything left in Predict, then run.
- ☐ **Before Lesson 4.6** (from the lesson page): open `data/ab_test.csv` and look at its columns. Before running anything, write down which test from Lesson 4.5 you would use to compare conversion between the

two variants, and why.

- ☐ **Prepare for the Session 13 checkpoint:** a short open-notebook task on Modules 1 to 4 (NumPy, pandas, Matplotlib, statistics). Tidy your lesson notebooks so you can find things, and skim the key ideas pages of this workbook from Session 1 on. No memorizing needed: it is open-notebook.

BRING TO SESSION 13

- **Session 13 includes a short open-notebook checkpoint on Modules 1 to 4.** Bring your lesson notebooks from Modules 1 to 4 and this workbook. You may use them, the lesson pages and the documentation.
- Your `cohens_d` function, the reporting template, and your note on which test you would use for `ab_test.csv`.
- Nothing new to install, no accounts, no servers.
- A charged laptop.

GLOSSARY

- **Scatter plot:** one point per row, one variable on each axis; read direction, form and strength from it.
- **Pearson's r:** a number from -1 to +1 for straight-line association: the average product of z-scores.
- **Spearman's rho:** Pearson's r on the ranks: consistent (monotonic) association, robust to outliers, suited to ordinal data.
- **Monotonic:** consistently rising or consistently falling, whether or not along a straight line.
- **Correlation matrix:** a square table of the correlation of every pair of columns.
- **Confounder:** a third variable that influences both variables being correlated.
- **Partial correlation:** the correlation of two variables after removing the straight-line effect of a third from each.
- **Simpson's paradox:** a pooled correlation with the opposite sign to the correlation inside every group.
- **Regression line:** the least-squares line y = intercept + slope x; predictor first in `stats.linregress(x, y)`.
- **r squared:** the share of the variance in y that the straight line accounts for.
- **Residual:** actual value minus the value the line predicts.
- **Null hypothesis (H0):** the "nothing is going on" claim assumed in order to test it.
- **Alternative hypothesis (H1):** what you conclude if the null does not hold up; usually two-sided.
- **p-value:** the probability, if the null hypothesis were true, of a result at least as extreme as the one observed.
- **Significance level (alpha):** the cutoff chosen before looking, usually 0.05.
- **Permutation test:** a test that builds the null by shuffling group labels many times.

- **Welch's t-test:** a two-sample t-test that does not assume equal variances (`equal_var=False`).
- **Paired t-test:** a test on the differences within matched pairs (`ttest_rel`).
- **Chi-square test:** a test of independence for a table of counts (`chi2_contingency`).
- **Mann-Whitney U:** a rank-based test for two independent groups; effect size U / (n1 x n2).
- **Effect size:** how big a difference is: Cohen's d, a rate ratio, Cramér's V, a probability of superiority.
- **Type I / Type II error:** a false alarm (rate alpha) / a miss.
- **Power:** the probability that a test detects an effect that truly exists; it grows with sample size.
- **Multiple comparisons:** running many tests, which multiplies false alarms; correct with Holm.

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 13: A/B Testing and the EDA Workflow

Your notes, worked examples, labs and self-check for the session that finishes Module 4, plus the task sheet for Checkpoint 1.

Lessons 4.6-4.7 | Session 13 · 3 hours | Checkpoint 1 | Name: _____

SESSION GOALS

Today closes Module 4. By the end of the session you will have:

- followed one real experiment, Pinecrest's new checkout page, from the plan written before launch to the decision to ship;
- sized a test, checked that its randomization worked, and reported a difference with its confidence interval;
- seen why looking early and slicing by segment both need care, and learned the word that keeps you honest: **exploratory**;
- run a seven-step routine on a dataset from first question to written findings, and used it to work a business rule out of raw orders;
- done **Checkpoint 1**: thirty minutes on your own with a small question. It is open notebook, it is not graded, and it is the best practice for the capstone you will get.

> **HOW TODAY IS DIFFERENT**
>
> Two lessons and a checkpoint fit in three hours only because the room does the core of each lesson and you finish the projects at home. That is normal for this session. The homework checklist at the end tells you what to do first.

LESSONS

4.6 A/B Testing

KEY IDEAS

- **Why experiment?** In observational data the groups you compare differ in more ways than the one you care about. Random assignment makes the groups alike in everything except the treatment, including things nobody measured. That is what lets you use the word "caused".
- **Plan before data.** Six lines, written first: the **unit** (what is randomized and counted), the **primary metric** (the one number that decides), the **guardrail** (what must not get worse), the **hypotheses** (two-sided, alpha 0.05), the **sample size and duration**, and the **decision rule**.
- **Sample size comes from four ingredients:** the baseline rate, the smallest lift worth catching, alpha and power. `proportion_effectsize(target, baseline)` turns the two rates into Cohen's h; `NormalIndPower().solve_power(...)` finds whichever one of effect size, sample size, alpha and power you leave out. Small lifts on low rates need many visitors: 8.9% to 10.0% needs 11,092 per arm.
- **Check the experiment before the result:** one row per unit; a split consistent with the design (`stats.binomtest` against 50/50); arms balanced on things fixed in advance. A failed check means stop and find the bug, not adjust.
- **Report four things:** the difference in **percentage points**, its 95% **confidence interval**, the **relative lift** (and say which is which), and the p-value. Then lay the interval next to what is worth having.
- **Peeking** (stopping the first time p dips below 0.05) is multiple testing in disguise: false alarms rise from 5% to about 12% with weekly looks and past 25% with daily looks. Fix the sample size and decide once.
- **Segments are exploratory.** "Significant here, not there" does not show that here and there differ. Test the difference between the two lifts, correct for the slices, and label unplanned results.
- **Judge on every randomized visitor.** Buyers are not a randomized group, because the page helps decide who buys. Revenue per visitor (a non-buyer counts as $0) keeps the protection of the coin flip.

THE PLAN IN SIX LINES (FILL IN FOR THE CHECKOUT TEST AS YOU WATCH THE DEMO)

LINE	FOR THE CHECKOUT TEST
Unit	

LINE	FOR THE CHECKOUT TEST
Primary metric	
Guardrail	
Hypotheses and alpha	
Sample size and duration	
Decision rule	

◆ WORKED EXAMPLE: DOES THE NEW PAGE WORK BETTER ON WEEKENDS?

A colleague says weekend shoppers are in less of a hurry, so the new page "must matter more on weekends". Check it the way the lesson checks devices: compare each slice, then test the difference between the lifts. (`ab` and `compare_rates` are the lesson's.)

```
ab["weekend"] = ab["date"].dt.dayofweek >= 5
weekday = compare_rates(ab[~ab["weekend"]])
weekend = compare_rates(ab[ab["weekend"]])
for label, r in [("Mon-Fri", weekday), ("Sat-Sun", weekend)]:
    print(f"{label}: {r['visitors']:>6,} visitors  "
          f"{r['control']:.2%} -> {r['treatment']:.2%}  {r['diff'] * 100:+.2f} pts
          f"(95% CI {r['ci_low'] * 100:+.2f} to {r['ci_high'] * 100:+.2f})  "
          f"p = {r['p']:.3f}")

gap = weekend["diff"] - weekday["diff"]
gap_se = np.sqrt(weekend["se"] ** 2 + weekday["se"] ** 2)
p_gap = 2 * stats.norm.sf(abs(gap / gap_se))
print(f"Weekend lift minus weekday lift: {gap * 100:+.2f} points, p = {p_gap:.2f}"
```

```
Mon-Fri: 20,082 visitors  8.88% -> 9.91%  +1.03 pts (95% CI +0.22 to +1.84)  p =
Sat-Sun:  7,918 visitors  8.80% -> 10.33%  +1.53 pts (95% CI +0.23 to +2.83)  p =
Weekend lift minus weekday lift: +0.50 points, p = 0.52
```

Reading it. Both slices show a lift, and both are significant by themselves. The weekend lift looks larger (+1.53 against +1.03 points), but the difference between them is half a point with p = 0.52: no evidence that the page works differently on weekends. Notice the weekend interval is wider. There are fewer weekend visitors, so the estimate is rougher. Nobody planned this comparison, so the sentence for a memo is: "The lift is similar on weekdays and weekends (exploratory)."

GUIDED LAB

Open `labs/module-13/l4_6-ab-testing/novelty_check_starter.py` and work through TODO 1 to 4. A product manager fears the new page won big in week 1 and has been fading since. You will look at each week by itself, compare the first half of the test with the second half, and write the sentence you would say to that manager. The file runs as it is and tells you which TODOs are still open.

If you finish early (or at home): `labs/module-13/l4_6-ab-testing/followup_test_plan_starter.py` sizes the mobile-only follow-up test the memo asked for. Stretch: `labs/module-13/l4_6-ab-testing/aa_test_starter.py` splits the control arm at random 2,000 times to see how often a test with no real difference looks "significant".

PREDICT, THEN RUN

Write your prediction first. Then run the line and write what you saw. The imports are the ones from the demo.

P1

Ten of 100 visitors convert in one arm and twenty of 100 in the other: twice the rate. Is the p-value below 0.05? Write yes or no and a guess for the number.

```
print(round(proportions_ztest([20, 10], [100, 100])[1], 3))
```

P2

The lesson needed 11,092 visitors per arm for 80% power. This asks for 90% power. Will the number be about 12,000, about 15,000 or about 22,000?

```
h = proportion_effectsize(0.100, 0.089)
n90 = NormalIndPower().solve_power(effect_size=h, alpha=0.05, power=0.90)
print(round(n90))
```

P3

Two splits out of 10,000 visitors: 5,100 / 4,900 and 5,080 / 4,920. Which of them, if either, fails the 50/50 check at 0.05?

```
print(round(stats.binomtest(5100, 10_000, p=0.5).pvalue, 3))
print(round(stats.binomtest(5080, 10_000, p=0.5).pvalue, 3))
```

NOTES

SELF-CHECK (DO THESE AT HOME, WITHOUT NOTES FIRST)

Q1

Pinecrest showed the old and the new checkout page during the same four weeks and let a coin flip decide who saw which. What does the coin flip buy?

A) Two groups of exactly the same size, which the z-test requires

B) A smaller sample size than a before/after comparison would need

C) Groups that are alike in everything except the page, including things nobody measured

D) A guarantee that the difference between the pages is not due to chance

Q2

A colleague sizes a test for a rise from 8.9% to 10.0% like this and gets 129,733 visitors per arm. What went wrong?

```
n = NormalIndPower().solve_power(effect_size=0.011, alpha=0.05, power=0.80)
```

A) `effect_size` expects Cohen's h from `proportion_effectsize(0.100, 0.089)`, not the raw difference in rates

B) `power` should be 0.20, the chance of missing the effect

C) `solve_power` needs `nobs1` as well, or it has nothing to solve

D) Nothing: a one-point lift really needs about 130,000 visitors per arm

Q3

Conversion went from 8.86% to 10.03%. What does this print? (First number: percentage points. Second: relative lift.)

```
print(round((0.1003 - 0.0886) * 100, 2), round(0.1003 / 0.0886 - 1, 3))
```

Q4

An experiment planned as 50/50 reports 14,400 visitors in treatment and 13,600 in control. `stats.binomtest(14400, 28000, p=0.5).pvalue` is `1.8e-06`. What do you do?

A) Carry on: 51.4% is close enough to half

B) Drop 800 treatment visitors at random so the arms match, then analyze

C) Report the conversion result with a note that the split was uneven

D) Stop and find where the control visitors went: the experiment did not run as planned, so its results cannot be read

Q5

A team looks at its four-week test every Monday and stops the first time p is below 0.05. In the lesson's simulation of experiments with no real effect, how often did that rule declare a winner?

A) About 5%, because alpha is 0.05 at each look

B) About 12%, because each look is another chance to cross the line by luck

C) About 1.25%, because the four looks share the 5%

D) About 50%, because the p-value wanders at random

Q6

By device: mobile +1.76 points (p = 0.00004), desktop +0.27 points (p = 0.67). The test of the difference between the two lifts gives p = 0.049, and nobody planned this comparison. Which sentence belongs in the memo?

A) The new page works on mobile and does nothing on desktop

B) The new page should be shipped to mobile visitors only

C) The gain appears concentrated on mobile; the evidence that devices differ is borderline and exploratory, and worth a follow-up test

D) The desktop result proves the page has no effect on desktop

4.7 An EDA Workflow

KEY IDEAS

- **EDA is a routine,** not "make charts until something looks interesting": **1 Questions · 2 Structure · 3 Quality · 4 One variable · 5 Two variables · 6 Segments · 7 Findings and caveats**, and then around again with the new questions.
- **Questions come first.** They decide which columns matter, which comparisons are planned and which are fishing, and when you are done. Changing a question openly is fine; inventing one afterward to fit a pretty chart is not.
- **Know the grain:** what one row is, and which column identifies it. Every count and every average depends on it. Make merges prove the relationship you expect with `validate=`.
- **For missing data ask "missing for whom?"** Values that are not missing at random change who a column describes. Decide per question; `dropna(subset=[...])`, never a blanket `dropna()`.
- **Ask who is in the data** compared with who you want to talk about. The survey speaks for active, fairly loyal customers.
- **Look before you summarize:** every distribution before its mean, the grouped values behind every correlation.
- **Scanning segments is multiple testing.** Two disciplines: (1) count every comparison and correct (Holm); (2) for what survives, look at the size of the gap and check the obvious competing explanation.
- **An analysis ends in writing:** findings (one sentence with a number each), caveats (who and what the data cannot speak for), next questions.

THE SEVEN STEPS (WRITE ONE LINE EACH ON WHAT YOU WOULD ACTUALLY DO)

STEP	WHAT I DO
1 Questions	
2 Structure	

STEP	WHAT I DO
3 Quality	
4 One variable	
5 Two variables	
6 Segments	
7 Findings and caveats	

◆ WORKED EXAMPLE: DO FREQUENT BUYERS RECOMMEND PINECREST MORE?

A new question for the survey, and a merge that would go wrong if you were careless. `orders` has one row per order, the survey one row per customer. Merging them directly would repeat each response once per order (1,800 rows become 9,230). So summarize the orders to one row per customer first, then merge, and let `validate` prove it. (`survey` is the lesson's.)

```
orders = pd.read_csv("data/orders.csv")
per_customer = (orders.groupby("customer_id")
                      .agg(n_orders=("order_id", "count"))
                      .reset_index())
print("orders:", len(orders), "rows -> one row per customer:", len(per_customer))

joined = survey.merge(per_customer, on="customer_id", how="left",
                      validate="one_to_one")
print("survey rows before:", len(survey), "after:", len(joined),
      "| no orders found:", joined["n_orders"].isna().sum())

rho = joined["n_orders"].corr(joined["recommend"], method="spearman")
print(f"Spearman correlation, number of orders and recommend score: {rho:.2f}")
joined["buyer"] = pd.cut(joined["n_orders"], [0, 2, 5, 10, 100],
                         labels=["1-2", "3-5", "6-10", "11+"])
by_buyer = joined.groupby("buyer", observed=True)["recommend"]
print(by_buyer.agg(["count", "median", "mean"]).round(2))
```

```
orders: 20302 rows -> one row per customer: 3942
survey rows before: 1800 after: 1800 | no orders found: 0
Spearman correlation, number of orders and recommend score: -0.02
       count  median  mean
buyer
1-2      559     8.0  7.32
3-5      572     8.0  7.48
6-10     414     7.0  7.30
11+      201     7.0  7.23
```

Reading it. The merge kept 1,800 rows and every respondent has orders (step 3 of the routine already said so). The correlation is -0.02: no relationship. The grouped values agree: the mean recommend score is between 7.2 and 7.5 whether someone has ordered twice or a dozen times. An honest finding: "Among survey respondents, how often a customer orders is unrelated to how likely they are to recommend Pinecrest (Spearman -0.02)." A caveat: respondents are all active customers, so this says nothing about people who stopped buying. A null result, checked and written down, is a result.

GUIDED LAB

Open `labs/module-13/l4_7-eda-workflow/shipping_rules_starter.py`. Nobody at Pinecrest can find the document that says how shipping is charged; the data dictionary only says the charge is 0.00, 6.95 or 14.95. Run the routine on that one column and work the rule out of the data. TODO 1 to 6 follow steps 3 to 7. Finish with a note: findings with numbers, at least one caveat, one next question.

At home: `labs/module-13/l4_7-eda-workflow/order_size_scan_starter.py` asks when online orders are bigger. You will scan 22 groups, correct for them, and then challenge the one result that survives.

Stretch: `labs/module-13/l4_7-eda-workflow/customer_overview_starter.py` runs steps 2 and 3 on the customer table and finds 739 orders that belong to nobody in it.

PREDICT, THEN RUN

Write your prediction first, then run it.

P1

Three rows on the left, three on the right. How many rows come out?

```
left = pd.DataFrame({"k": [1, 1, 2]})
right = pd.DataFrame({"k": [1, 1, 2], "v": [10, 20, 30]})
print(len(left.merge(right, on="k")))
```

P2

Two numbers print. Which is the honest mean?

```
s = pd.Series([4, None, 2])
print(s.mean(), s.fillna(0).mean())
```

P3

Three p-values, all below 0.05. How many are still below 0.05 after Holm?

```
print(multipletests([0.01, 0.04, 0.03], method="holm")[1].round(2).tolist())
```

NOTES

SELF-CHECK (DO THESE AT HOME, WITHOUT NOTES FIRST)

Q7

Someone hands you a table you have never seen. After writing your questions, what is the first fact to establish about it?

A) Which columns correlate most strongly

B) Its grain: what one row represents, and which column identifies a row

C) Whether the numeric columns are normally distributed

D) Which rows are outliers, so they can be removed

Q8

What does this print?

```
df = pd.DataFrame({"a": [1.0, None, 3.0],
                   "b": [None, 5.0, 6.0],
                   "c": ["x", "y", None]})
print(len(df.dropna()), len(df.dropna(subset=["a"])))
```

Q9

In the survey, `sat_website` is missing for 46% of store shoppers and 3% of online shoppers. Its mean is 3.54. What is the most accurate description?

A) Pinecrest customers rate the website 3.54

B) The mean is unusable until the gaps are filled with 3.54

C) Store shoppers rate the website 3.54

D) Customers who answered, mostly online shoppers, rate the website 3.54

Q10

What does this print?

```
left = pd.DataFrame({"id": [1, 2, 3]})
right = pd.DataFrame({"id": [1, 1, 1, 2], "x": [5, 6, 7, 8]})
print(len(left.merge(right, on="id")),
      len(left.merge(right, on="id", how="left")))
```

Q11

You compare online order totals across 22 groups (weekdays, payment methods, months). Three groups have p below 0.05; after a Holm correction one remains. What goes in the findings?

A) The one that survived, with the size of its gap, the number of comparisons, and the result of checking an obvious competing explanation; the other two are leads

B) All three, with their original p-values

C) None: results found by scanning are never reportable

D) Only the group with the largest gap in dollars, whatever its p-value

Q12

Which sentence from the survey note is a caveat, not a finding?

A) Only 42% of customers are satisfied with prices

B) Every respondent has placed an order, so customers who never ordered are not heard

C) Asheville's staff rating is 3.62 against about 4.1 at the other stores

D) Asheville is the only store whose revenue fell in 2025

CHECKPOINT 1: TASK SHEET

2:15 Checkpoint 1: analysis task on Modules 1 to 4

30 min

THE RULES

- **30 minutes, on your own.** This checkpoint is practice for the capstone. It is not graded. You get your rubric back with one sentence of advice.
- **Open notebook:** your notes, your earlier notebooks, your helper functions, the lesson pages and the library documentation are all allowed. Your neighbor and AI assistants are not.
- **A finished small answer beats an unfinished big one.** If you run out of time, write what you would do next. That counts.
- Work in a new notebook, or start from `labs/module-13/checkpoint-1/checkpoint1_starter.py` (it has the question, the load line and six empty headings).

THE QUESTION

Pinecrest's store buyers believe that **December shoppers buy more expensive things**. Finance wants a number before it plans next December's stock. Using `data/daily_sales.csv`: is the average price per unit sold (revenue ÷ units) higher in December than in the rest of the year? By how much, how sure are you, and is it the same in all four stores?

`daily_sales.csv` has one row per store per day: `date`, `store`, `visitors`, `transactions`, `units`, `revenue`, `staff_on_duty`, `promo_day`.

WHAT TO HAND IN (ONE NOTEBOOK OR .PY FILE)

1. **A plan, written before any code** (three lines): what you will compute for each row, which rows you will compare with which, and what you will check before trusting the numbers. *About 3 minutes.*
2. **Your checks:** what one row is, what is missing, what you did about it. *About 5 minutes.*
3. **The comparison:** the difference in dollars, a 95% confidence interval, a suitable test and an effect size. Then: is it the same in all four stores? *About 10 minutes.*
4. **One figure** with a title that states your finding, and labeled axes. *About 5 minutes.*
5. **A note:** two findings with numbers, two caveats that are specific to this data, one next question. *About 7 minutes. Start it by minute 25 whatever happens.*

HOW IT IS READ

Five rows, four levels each (Beginning, Developing, Proficient, Exemplary): the same levels as the capstone rubric in Lesson 8.1.

ROW	WHAT A PROFICIENT ANSWER SHOWS
Question and plan	Measure (revenue ÷ units per store-day), comparison (December against other days) and checks are written before the analysis, and the work stays on them.
Data handling	States the grain (one row per store per day), finds the 6 missing revenue values and says what was done with them; the new column is checked (range or describe).
Analysis	Difference in dollars with a 95% confidence interval, a suitable test (Welch or Mann-Whitney) and an effect size (Cohen's d or percent). Stores are compared.
Figure	One figure with a title that states the finding, labeled axes with units, and the comparison easy to see.
Communication	Leads with the answer and its size; two specific caveats; one next question; plain language.

◆ IF YOU FINISH EARLY

Is it the mix? `daily_sales.csv` cannot say whether December shoppers choose pricier products or whether the same products cost more. Which other files could? Write two lines on how you would check (you do not have to do it).

◆ YOUR PLAN (WRITE HERE FIRST IF YOU LIKE PAPER)

HOMEWORK CHECKLIST

About three and a half hours this week, the most of any week in the course. Do the two projects first.

- ☐ **Lesson 4.6 project:** The Decision Memo, Steps 2 to 5. Write your own memo before reading the lesson's.
- ☐ **Lesson 4.7 project:** Why Do Orders Come Back? All seven steps (about 45 minutes).
- ☐ Lesson 4.6: read "The Peeking Problem" and "Common Mistakes"; Exercises 1 to 4 (5 is a stretch); quiz; journal.
- ☐ Lesson 4.7: read Step 7 and "Common Mistakes"; Exercises 1 to 3 (4 is a stretch); quiz; journal.

- ☐ Kit labs: `labs/module-13/l4_6-ab-testing/followup_test_plan_starter.py` and `labs/module-13/l4_7-eda-workflow/order_size_scan_starter.py`. Stretch: `aa_test`, `customer_overview`.
- ☐ Self-check Q1 to Q12 in this workbook.
- ☐ Save `compare_rates`, `overview` and `rate_by` in a file of your own helpers.
- ☐ **For Session 14:** check that `data/pinecrest.db` is in your data folder and that `import sqlalchemy` works; sketch on paper how `customers`, `orders`, `order_items` and `products` connect. No SQL is assumed.

GLOSSARY

A/B test: an experiment that shows two versions to randomly assigned groups at the same time and compares one planned metric. **Arm:** one of the groups in an experiment (control, treatment). **Unit:** what is randomized and counted: here, one visitor. **Primary metric:** the single number, chosen in advance, that decides the test. **Guardrail metric:** a number that must not get worse while the primary metric improves. **Minimum detectable effect:** the smallest improvement the test is sized to have a good chance of catching. **Power:** the chance of detecting an effect of a given size if it is real (0.80 by convention). **Cohen's h:** the effect size for a difference between two proportions; what statsmodels power functions expect. **Sample ratio mismatch:** a split between arms that differs from the design by more than chance allows; a sign of a bug. **Percentage points:** the absolute difference between two rates (10.03% - 8.86% = 1.17 points). **Relative lift:** the difference as a share of the baseline (1.17 / 8.86 = 13.2%). **Peeking:** checking a running test and stopping when it first looks significant; inflates false alarms. **Exploratory:** found by looking, not planned in advance; a lead to confirm, not a confirmed result. **EDA:** exploratory data analysis: getting to know a dataset before relying on it. **Grain:** what one row of a table represents. **Key:** the column (or columns) whose values identify a row. **Missing not at random:** gaps that are more common for some kinds of rows, so the column describes only those who answered. **Holm correction:** an adjustment to p-values that accounts for having run several tests. **Caveat:** a specific statement of who or what the data cannot speak for.

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 14: SQL from Python: Queries and Joins

Your notes, worked examples, labs and self-check for the first SQL session: one table, asked safely, then several tables joined without losing a row.

Lessons 5.1–5.2 | Session 14 · 3 hours | Name: _____

SESSION GOALS

Today you start Module 5: data that lives in a database instead of a file. By the end of the session you will have:

- connected to Pinecrest's database from Python and explored its tables and columns;
- written your first SQL: filters, sorts, aggregates, groups and group filters;
- passed values into a query safely, and watched what happens when it is done the unsafe way;
- moved results into pandas with `pd.read_sql` and back with `to_sql`;
- joined tables and accounted for every row that did not match;
- named the steps of a longer query with `WITH`, walked a reporting tree, and met the logic of NULL.

YOU ALREADY SPEAK THIS

Filtering, sorting, grouping and merging are what you did in Module 2. SQL is a second language for the same ideas. When a query feels foreign, translate it into pandas in your head. The table below is your phrase book.

SQL	WHAT IT DOES	PANDAS
`SELECT a, b`	Choose columns	`df[["a", "b"]]`
`WHERE x > 5`	Keep rows	`df[df["x"] > 5]`
`GROUP BY g` with aggregates	One row per group	`df.groupby("g").agg(...)`
`HAVING COUNT(*) > 5`	Keep groups	a mask on the aggregated result
`ORDER BY x DESC`	Sort	`df.sort_values("x", ascending=False)`
`LIMIT 10`	First rows only	`df.head(10)`
`A JOIN B ON ...`	Matched pairs only	`a.merge(b, how="inner")`
`A LEFT JOIN B ON ...`	All of A	`a.merge(b, how="left")`
`LEFT JOIN ... WHERE b.key IS NULL`	Rows of A with no partner	`indicator=True`, keep `left_only`
`COALESCE(x, 0)`	Replace NULL	`s.fillna(0)`

LESSONS

5.1 Database Connections

KEY IDEAS

- **A database answers questions** and returns only what you ask for. SQL is the language of the request. SQLite is a complete database engine in one file; Python includes it as `sqlite3`.
- **Work on a copy** (`shutil.copy`) or open the file read-only, so the original stays as documented.
- **Connection, cursor, rows.** `sqlite3.connect()` gives a connection. Running a statement gives a cursor that walks forward through the result. Rows come back as tuples; `fetchone()`, `fetchmany(n)`, `fetchall()`.
- **One statement does most of the work:** `SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ... LIMIT`, always written in that order. The engine runs it starting from `FROM`: FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY, LIMIT.
- **WHERE filters rows before grouping; HAVING filters groups after it.** Every selected column is either in the `GROUP BY` or inside an aggregate.
- **Three COUNTs:** `COUNT(*)` rows, `COUNT(col)` rows where `col` is not NULL, `COUNT(DISTINCT col)` different values.
- **Values go in through placeholders** (`?` with a tuple, or `:name` with a dictionary). Never with an f-string, `.format()`, `%` or `+`. A one-item tuple needs its comma: `("Boise",)`.
- **Changes are saved only on commit.** `with conn:` commits if the block finishes and rolls back if it raises: all or nothing.
- **SQLAlchemy** gives one way to talk to any database: `create_engine(url)`, `with engine.connect() as c:`, `text("...")` with `:name` parameters.
- `pd.read_sql(query, connection, params=...)` brings a result into a DataFrame; `df.to_sql(name, engine, if_exists=..., index=False)` saves one as a table. Let the database do the heavy filtering and grouping.

WORKED EXAMPLE: A REPORT FUNCTION WITH PARAMETERS

Marketing asks, for any year: how many completed orders used each promo code, and what was the average order? The year changes, so it travels through `params`. (The f-strings build the two date VALUES from an integer; the SQL text itself is fixed. `conn` is the lesson's connection.)

```
def promo_report(conn, year):
    """Orders and average total for each promo code in one calendar year."""
    sql = """
        SELECT promo_code,
               MAX(discount_pct)          AS discount_pct,
               COUNT(*)                   AS orders,
               ROUND(AVG(order_total), 2) AS avg_total
        FROM orders
        WHERE status = 'completed'
          AND order_date >= :start AND order_date < :end
        GROUP BY promo_code
        HAVING COUNT(*) >= 100
        ORDER BY orders DESC
    """
    params = {"start": f"{year}-01-01", "end": f"{year + 1}-01-01"}
    return pd.read_sql(sql, conn, params=params)

print(promo_report(conn, 2025))
```

```
  promo_code  discount_pct  orders  avg_total
0        NaN             0    9162     197.15
1  WELCOME10            10     262     183.36
2    GEARUP5             5     235     159.03
3    TRAIL15            15     183     170.37
4   SUMMIT20            20     128     141.76
```

Reading it. The first row has no promo code: NULL is its own group, and pandas shows it as `NaN`. Most orders use no code. `HAVING COUNT(*) >= 100` removed the smaller groups after grouping. The averages fall as the discount rises (SUMMIT20 orders average $141.76), partly because `order_total` is the amount AFTER the discount: compare like with like before you conclude anything about basket size. Call `promo_report(conn, 2024)` and the same function answers for another year, with no SQL rewritten.

◆ GUIDED LAB

Open `labs/module-14/l5_1-database-connections/staff_report_starter.py`. The people team wants a quick picture of Pinecrest's 55 employees, a table that exists only in the database. TODO 1 to 6 take you from the list of columns to a grouped report with `HAVING`, and a check in pandas.

Then `labs/module-14/l5_1-database-connections/safe_lookup_starter.py`: store managers want to look up low stock by location. An unsafe function is given; you write the safe one and attack both with the same input.

At home: `labs/module-14/l5_1-database-connections/save_summary_starter.py` builds a small summary table and saves it with `to_sql`, on a copy of the database.

◆ PREDICT, THEN RUN

Write your prediction first, then run it. `conn` is the connection from the demo.

P1

18 or 0? The product names say "Tent", the pattern says "TENT".

```
sql = "SELECT COUNT(*) FROM products WHERE name LIKE '%TENT%'"
print(conn.execute(sql).fetchone()[0])
```

P2

Five rows were asked for. Two are fetched one at a time. What are the two things printed?

```
cur = conn.execute("SELECT order_id FROM orders ORDER BY order_id LIMIT 5")
cur.fetchone()
cur.fetchone()
print(len(cur.fetchall()), cur.fetchone())
```

P3

20,302 orders; 18,154 have no promo code. What is the tuple?

```
print(conn.execute("SELECT COUNT(*), COUNT(promo_code) FROM orders").fetchone(
```

NOTES

SELF-CHECK (DO THESE AT HOME, WITHOUT NOTES FIRST)

Q1

A query is written `SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ... LIMIT`. Which clause does the engine work on first?

A) `SELECT`, because it is written first

B) `WHERE`, because rows must be filtered before anything else

C) `FROM`, because the engine has to pick the table before it can filter, group or select

D) `ORDER BY`, because sorted rows are faster to filter

Q2

What does this print?

```
conn = sqlite3.connect(":memory:")
conn.execute("CREATE TABLE t (x INTEGER, y TEXT)")
conn.executemany("INSERT INTO t VALUES (?, ?)",
                 [(1, "a"), (2, None), (3, "c")])
print(conn.execute("SELECT COUNT(*), COUNT(y), SUM(x) FROM t").fetchone())
```

Q3

This line raises `ProgrammingError: Incorrect number of bindings supplied. The current statement uses 1, and there are 5 supplied.` What is the fix?

```
conn.execute("SELECT COUNT(*) FROM orders WHERE store = ?", "Boise")
```

A) Pass a one-item tuple: `("Boise",)`

B) Write five placeholders: `?????`

C) Put the value in the text with an f-string

D) Use double quotes around the `?`

Q4

You want the average order per store for completed orders in 2025, but only for stores whose average is above $190. Which condition belongs in `HAVING`?

A) `status = 'completed'`

B) `AVG(order_total) > 190`

C) `order_date >= '2025-01-01'`

D) `store IN ('Boise', 'Bend')`

Q5

A store name typed into a web form arrives in the variable `store`. Which line passes it to the database safely?

A) `conn.execute(f"SELECT * FROM orders WHERE store = '{store}'")`

B) `conn.execute("SELECT * FROM orders WHERE store = '" + store + "'")`

C) `conn.execute("SELECT * FROM orders WHERE store = '{}'".format(store))`

D) `conn.execute("SELECT * FROM orders WHERE store = ?", (store,))`

Q6

What does this print?

```
conn = sqlite3.connect(":memory:")
conn.execute("CREATE TABLE t (k INTEGER PRIMARY KEY)")
conn.commit()
try:
    with conn:
        conn.execute("INSERT INTO t VALUES (1)")
        conn.execute("INSERT INTO t VALUES (1)")
except sqlite3.IntegrityError:
    pass
print(conn.execute("SELECT COUNT(*) FROM t").fetchone()[0])
```

5.2 Complex Queries

KEY IDEAS

- **A join pairs rows by key.** Join types differ only in what happens to a row with no partner.
- `JOIN` **(inner) keeps matched pairs only** and drops the rest without telling you. `LEFT JOIN` **keeps every row of the first table**, with NULL where there is no partner.
- **The anti-join** answers "which ones are missing?": `LEFT JOIN ... WHERE right.key IS NULL`. `NOT EXISTS` does the same job.
- **Count after every join.** Four numbers to remember: 739 orders have no customer row; 1,058 customers never ordered; 609 order lines have no product row; 8 products never sold.
- **The fan-out trap:** joining to a "many" side repeats the "one" side. Sum a column at the level where it lives; when you need two levels, aggregate first (a subquery or CTE), then join.
- **Subqueries** return a value, a list or a table depending on where they sit. **CTEs** (`WITH name AS (...)`) give the steps names, so a long query reads top to bottom.
- **A recursive CTE** (anchor, `UNION ALL`, recursive part) walks a hierarchy of any depth. It stops when a round finds no new rows.
- `CASE` is if / elif / else. Inside `SUM` or `COUNT` it makes side-by-side columns. Multiply by `100.0`, not `100`, or integer division gives you zeros.
- **NULL means unknown.** `= NULL` never matches (use `IS NULL`); a condition and its opposite do not add up to everything; `NOT IN` with a NULL in the list returns nothing. `COALESCE(x, y)` is the SQL `fillna`.

HOW THE FOUR TABLES CONNECT (FINISH THE SKETCH YOU BROUGHT)

LINK	KEY	WHICH SIDE IS "ONE"?	UNMATCHED ROWS
customers and orders	customer_id		
orders and order_items	order_id		

LINK	KEY	WHICH SIDE IS "ONE"?	UNMATCHED ROWS
products and order_items	sku		

◆ WORKED EXAMPLE: RETURN RATE BY LOYALTY TIER, WITH EVERY ORDER ACCOUNTED FOR

Does the return rate differ by the customer's loyalty tier? The tier is in `customers`, the status in `orders`. An inner join would quietly leave out the 739 orders from closed accounts, so this uses a LEFT JOIN and names those rows with `COALESCE`. (`conn` and `show` are the lesson's.)

```
show("""
    SELECT COALESCE(c.loyalty_tier, '(closed account)') AS tier,
           COUNT(*)                                     AS orders,
           SUM(CASE WHEN o.status = 'returned' THEN 1 ELSE 0 END) AS returned,
           ROUND(100.0 * SUM(CASE WHEN o.status = 'returned' THEN 1 ELSE 0 END)
                 / COUNT(*), 1)                         AS return_pct
    FROM orders AS o
    LEFT JOIN customers AS c ON c.customer_id = o.customer_id
    GROUP BY tier
    ORDER BY orders DESC
""")
show("""
    SELECT COUNT(*) AS rows_with_inner_join
    FROM orders AS o
    JOIN customers AS c ON c.customer_id = o.customer_id
""")
```

```
            tier  orders  returned  return_pct
            Gold    8491       403         4.7
          Silver    7132       383         5.4
           Basic    3940       316         8.0
(closed account)     739        41         5.5
 rows_with_inner_join
                19563
```

Reading it. The `orders` column adds up to 20,302: every order is counted once. The second query shows what an inner join would have kept: 19,563. Basic customers' orders come back 8.0% of the time against 4.7% for Gold. Before calling that a finding, ask how the tier is made: it is earned from COMPLETED spend, and a returned order does not count. A customer who returns a lot is held in a lower tier, so the tier partly reflects returns rather than causing them. SQL got the numbers right; the reading needs Lesson 4.4's

question: how was this data generated? The pattern to keep: LEFT JOIN from the table you care about, COALESCE to name the unmatched, conditional aggregation for the rate, and a row-count check at the end.

GUIDED LAB

Open `labs/module-14/l5_2-complex-queries/join_row_counts_starter.py`. The buying team wants item revenue by brand for completed orders. Before you write a join, write down how many rows you expect; the starter has a line for it. You will see what the tempting inner join loses, give the lost rows a name, and list them with an anti-join.

Then start `labs/module-14/l5_2-complex-queries/stock_vs_sales_starter.py`: units sold against units on hand, by category. Steps 1 and 2 in the room (the trap), Steps 3 to 5 at home (the fix, with CTEs).

Stretch: `labs/module-14/l5_2-complex-queries/org_payroll_starter.py` asks how many people and how much salary sit under each of the CEO's direct reports, however deep the tree goes.

PREDICT, THEN RUN

Write your prediction first, then run it. `conn` is the read-only connection from the second demo.

P1

A JOIN with no ON. `employees` has 55 rows. How many rows come back?

```
sql = "SELECT COUNT(*) FROM employees AS e JOIN employees AS m"
print(conn.execute(sql).fetchone()[0])
```

P2

Three values. Which of them is `None`?

```
print(conn.execute("SELECT NULL = NULL, NULL IS NULL, COUNT(NULL)").fetchone()
```

P3

`orders` has 20,302 rows and two channels. What are the two counts?

```
q = "SELECT channel FROM orders"
for op in ["UNION", "UNION ALL"]:
    print(op, conn.execute(f"SELECT COUNT(*) FROM ({q} {op} {q})").fetchone()[
```

NOTES

SELF-CHECK (DO THESE AT HOME, WITHOUT NOTES FIRST)

Q7

`orders` has 20,302 rows; 739 of them have a `customer_id` that is not in `customers`; 1,058 customers have never ordered. How many rows does `orders JOIN customers ON c.customer_id = o.customer_id` return?

A) 20,302

B) 19,563

C) 21,360

D) 19,244

Q8

What does this print?

```
conn = sqlite3.connect(":memory:")
conn.executescript("""
    CREATE TABLE o (id INTEGER, total REAL);
    CREATE TABLE i (order_id INTEGER, sku TEXT);
    INSERT INTO o VALUES (1, 100), (2, 50);
    INSERT INTO i VALUES (1, 'a'), (1, 'b'), (1, 'c'), (2, 'd');
""")
sql = "SELECT SUM(o.total) FROM o JOIN i ON i.order_id = o.id"
print(conn.execute(sql).fetchone()[0])
```

Q9

Which query lists the customers who have never placed an order?

A) `SELECT c.* FROM customers AS c LEFT JOIN orders AS o ON o.customer_id = c.customer_id WHERE o.order_id IS NULL`

B) `SELECT c.* FROM customers AS c JOIN orders AS o ON o.customer_id = c.customer_id WHERE o.order_id IS NULL`

C) `SELECT c.* FROM customers AS c LEFT JOIN orders AS o ON o.customer_id = c.customer_id WHERE o.order_id = NULL`

D) `SELECT c.* FROM orders AS o LEFT JOIN customers AS c ON c.customer_id = o.customer_id WHERE c.customer_id IS NULL`

Q10

359 of the 20,302 orders used the code TRAIL15. `SELECT COUNT(*) FROM orders WHERE promo_code <> 'TRAIL15'` returns 1,789, not 19,943. Why?

A) `<>` is not a valid operator in SQLite

B) The query needs `DISTINCT`

C) Text comparison ignores case, so some codes were double counted

D) For the 18,154 orders with no code the comparison with NULL is unknown, and WHERE keeps only rows where the condition is true

Q11

What does this print?

```
conn = sqlite3.connect(":memory:")
print(conn.execute("SELECT 7 / 2, 7 / 2.0, 100 * 1 / 4").fetchone())
```

Q12

A recursive CTE walks the employee tree from the CEO down. What makes the recursion stop?

A) The `LIMIT` that every recursive CTE must have

B) It reaches the row whose `manager_id` is NULL

C) A round of the recursive part finds no new rows

D) It has run once for every row in the table

HOMEWORK CHECKLIST

- ☐ **Lesson 5.1 project:** The Store Scorecard, Steps 3 to 6. Your check: revenue from SQL matches revenue from pandas for all five stores.

- ☐ **Lesson 5.2 project:** The Returns Investigation, Steps 2 to 6. Your check: the four rows of the footwear table add up to 20,302 orders.
- ☐ Lesson 5.1: read "Where PostgreSQL Differs" and "Common Mistakes"; Exercises 1 to 3 (4 is a stretch); quiz; journal.
- ☐ Lesson 5.2: read the sections on set operations and outer joins; Exercises 1 to 4 (5 is a stretch); quiz; journal.
- ☐ Kit labs: `labs/module-14/l5_1-database-connections/save_summary_starter.py`; Steps 3 to 5 of `labs/module-14/l5_2-complex-queries/stock_vs_sales_starter.py`. Stretch: `org_payroll`.
- ☐ Self-check Q1 to Q12 in this workbook.
- ☐ **For Session 15:** keep `show()` and `ORG_CTE`; check that `python -c "import sqlite3; print(sqlite3.sqlite_version)"` prints 3.25 or later; glance back at Lesson 2.8 (`rolling`, `shift`, `cumsum`, `rank`).

GLOSSARY

SQL: the language for asking a relational database for data. **SQLite:** a complete SQL database engine that lives in one file; included in Python as sqlite3. **Connection:** an open line from your program to a database. **Cursor:** a pointer that runs one statement and walks forward through its result. **Aggregate function:** a function that turns many rows into one value: COUNT, SUM, AVG, MIN, MAX. **GROUP BY:** split rows into groups and return one row per group. **HAVING:** a filter on groups, applied after GROUP BY. **Placeholder:** a ? or :name in the SQL text where a value will be supplied separately. **SQL injection:** an attack in which text supplied as a value becomes part of the SQL command. **Transaction:** a batch of changes that is saved all together (commit) or not at all (rollback). **Engine (SQLAlchemy):** an object made from a database URL that hands out connections. **Key:** a column whose value identifies a row, or points at a row in another table. **Inner join:** keeps only rows that found a partner. **Left join:** keeps every row of the first table; unmatched rows get NULL on the other side. **Anti-join:** the rows of one table that have no partner in another. **Fan-out:** the repetition of "one"-side rows when joined to a "many" side; it inflates sums. **Subquery:** a SELECT in parentheses inside another query. **CTE:** common table expression: a named query defined with WITH and used like a table in the statement that follows. **Recursive CTE:** a CTE that refers to itself to walk a hierarchy level by level. **CASE:** SQL's if / elif / else expression. **NULL:** the marker for an unknown or missing value; comparisons with it are unknown. **COALESCE:** returns its first argument that is not NULL.

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 15: SQL Windows and Query Speed

Your notes, worked examples, labs and self-check for window functions and query speed.

Lessons 5.3–5.4 | Session 15 · 3 hours | Name: _____

SESSION GOALS

Today finishes the SQL half of Module 5. By the end of the session you will have:

- put group averages, shares, ranks, running totals and moving averages on every row, without losing the rows;
- picked the top N rows of every group with the three-step recipe;
- compared each row with the one before it, within its own series;
- read the engine's plan for a query, added an index, and shown its effect on the plan and on a fair timing;
- seen the cases where an index cannot help, and replaced a loop of queries with one;
- checked every answer by a second route: pandas, or a number you already knew.

> **THREE QUESTIONS FOR EVERY WINDOW**
> **Partition:** which rows belong together? **Order:** in what order, and is it unique? **Frame:** how many of those rows does the function look at? Say them before you type the OVER.

5.3 SQL Window Functions

KEY IDEAS

- **A window function computes over a set of related rows and keeps every row;** GROUP BY collapses them. Any aggregate becomes a window function with `OVER (...)`.
- `OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN ...)`: which rows belong together, in what order, and how many of them to look at. An empty `OVER ()` is one window holding every row.
- `ROW_NUMBER`, `RANK`, `DENSE_RANK` differ only in ties: 1, 2, 3, 4 · 1, 1, 3, 4 · 1, 1, 2, 3.
- **Top N per group:** number the rows in a CTE, then filter outside it. WHERE runs before window functions exist.
- `LAG` **and** `LEAD` read the previous and next row. Partition by the series, or one series reads another's rows.
- **Running totals and moving averages need an explicit** `ROWS` **frame.** The default frame treats rows that tie in the order as one.
- **SQL uses partial frames** at the start of a series; pandas `rolling` gives NaN there unless `min_periods=1`. A frame ending at `1 PRECEDING` never sees the current row.
- **End every window's ORDER BY with a unique column** so results are the same every run.

WINDOW FUNCTIONS AND THEIR PANDAS TWINS

NEED	SQL	PANDAS
Group value on every row	`AVG(x) OVER (PARTITION BY g)`	`groupby("g")["x"].transform("mean")`
Number or rank rows	`ROW_NUMBER(), RANK(), DENSE_RANK()`	`rank(method="first" / "min" / "dense")`
Previous or next row	`LAG(x), LEAD(x)`	`shift(1), shift(-1)`
Running total	`SUM(x) OVER (ORDER BY t ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`	`cumsum()`
Moving average	`AVG(x) OVER (ORDER BY t ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)`	`rolling(3, min_periods=1).mean()`

NEED	SQL	PANDAS
Earlier rows only (no leakage)	`... ROWS BETWEEN 3 PRECEDING AND 1 PRECEDING`	`shift(1).rolling(3, min_periods=1)`
Equal-sized buckets	`NTILE(4) OVER (ORDER BY x)`	`pd.qcut(x, 4)`
Share of group	`x / SUM(x) OVER (PARTITION BY g)`	`x / groupby("g")["x"].transform("sum")`

WORKED EXAMPLE: EACH STORE'S WEAKEST MONTH, AND HOW WEAK IT WAS

Store managers ask: "Which month was our worst, and how far below our own average was it?" That is a bottom-1 per group (ROW_NUMBER ordered by revenue ascending) and a group average on the same row. (`conn` and `show` are the lesson's read-only connection and helper.)

```
show("""
    WITH store_month AS (
        SELECT store, strftime('%Y-%m', order_date) AS month,
               ROUND(SUM(order_total), 2) AS revenue
        FROM orders
        WHERE status = 'completed' AND store <> 'Online'
        GROUP BY store, month
    ),
    ranked AS (
        SELECT store, month, revenue,
               ROW_NUMBER() OVER (PARTITION BY store ORDER BY revenue, month) AS r
               ROUND(AVG(revenue) OVER (PARTITION BY store), 2) AS store_avg
        FROM store_month
    )
    SELECT store, month AS weakest_month, revenue, store_avg,
           ROUND(100.0 * revenue / store_avg, 1) AS pct_of_avg
    FROM ranked
    WHERE rn = 1
    ORDER BY store
""")
```

```
    store weakest_month  revenue  store_avg  pct_of_avg
Asheville       2025-07  5754.39    9907.47        58.1
     Bend       2025-02  6874.57   11857.30        58.0
    Boise       2024-01 11237.18   18474.45        60.8
Flagstaff       2024-01  6660.61   12524.18        53.2
```

Reading it. Each store's weakest month ran at 53% to 61% of that store's own monthly average. Three of the four are in January or February; Asheville's is July 2025. Ordering by `revenue, month` makes the choice

unique even if two months tied. The average and the rank sit on the same row because both are windows over the same partition; with GROUP BY you would have needed two queries and a join.

GUIDED LAB

Open `labs/module-15/l5_3-sql-window-functions/staff_rankings_starter.py`: the people team wants each employee compared with their location (average, share of payroll, rank) and the top two earners at every location. You finish with the same answers in pandas.

Then `labs/module-15/l5_3-sql-window-functions/silver_milestone_starter.py`: on which order does a customer's completed spend cross the $600 Silver line? A running total per customer, the first row that meets a condition, and a check against the customer table.

At home: `labs/module-15/l5_3-sql-window-functions/weekly_trend_starter.py` follows online orders week by week with two frames and finds the weeks that beat their own recent past.

PREDICT, THEN RUN

Write your prediction first, then run it. `MONTHLY` is the lesson's CTE string of monthly completed revenue.

P1

24 months. How many have a value twelve months earlier?

```
show(MONTHLY + """
    , lagged AS (
        SELECT month, LAG(revenue, 12) OVER (ORDER BY month) AS year_before
        FROM monthly
    )
    SELECT COUNT(*) AS months, COUNT(year_before) AS with_a_year_before
    FROM lagged
""")
```

P2

Seven rows into three buckets. Which bucket gets the extra row?

```
show("""
    SELECT NTILE(3) OVER (ORDER BY v) AS bucket, v
    FROM (SELECT 1 AS v UNION ALL SELECT 2 UNION ALL SELECT 3 UNION ALL SELECT
          UNION ALL SELECT 5 UNION ALL SELECT 6 UNION ALL SELECT 7)
""")
```

P3

LAST_VALUE with the default frame. What is in the last_value column for the first two months?

```
show(MONTHLY + """
    SELECT month, revenue,
           LAST_VALUE(revenue) OVER (ORDER BY month) AS last_value
    FROM monthly
    ORDER BY month
    LIMIT 2
""")
```

◆ NOTES

◆ SELF-CHECK (DO THESE AT HOME, WITHOUT NOTES FIRST)

Q1

Which pandas line gives the same column as `SUM(order_total) OVER (PARTITION BY store)`, one value on every order row?

A) `orders.groupby("store")["order_total"].sum()`

B) `orders.groupby("store")["order_total"].transform("sum")`

C) `orders["order_total"].cumsum()`

D) `orders["order_total"].rolling(5).sum()`

Q2

What does this print?

```
rows = conn.execute("""
    SELECT v, ROW_NUMBER() OVER (ORDER BY v DESC, id),
           RANK() OVER (ORDER BY v DESC), DENSE_RANK() OVER (ORDER BY v DESC)
    FROM (SELECT 1 AS id, 50 AS v UNION ALL SELECT 2, 70
          UNION ALL SELECT 3, 50 UNION ALL SELECT 4, 40)
    ORDER BY v DESC, id
""").fetchall()
print(rows[-1])
```

Q3

`SELECT ... ROW_NUMBER() OVER (PARTITION BY category ORDER BY revenue DESC) AS rn FROM ... WHERE rn <= 2` fails. Why, and what is the fix?

A) WHERE runs before window functions are computed; compute `rn` in a CTE and filter it in the outer query

B) ROW_NUMBER needs a ROWS BETWEEN frame; add one

C) `rn` is a reserved word; rename it

D) PARTITION BY cannot be combined with ORDER BY; remove the partition

Q4

Which frame averages the three months before the current one, without the current month (no leakage)?

A) `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW`

B) `ROWS BETWEEN 3 PRECEDING AND 1 PRECEDING`

C) `ROWS BETWEEN 3 PRECEDING AND CURRENT ROW`

D) `ROWS BETWEEN 1 FOLLOWING AND 3 FOLLOWING`

Q5

What does this print? (Two rows share d = 1.)

```
sql = """SELECT d, x, SUM(x) OVER (ORDER BY d)
         FROM (SELECT 1 AS d, 10 AS x UNION ALL SELECT 1, 20
               UNION ALL SELECT 2, 5)
         ORDER BY d, x"""
print([row[2] for row in conn.execute(sql)])
```

Q6

A SQL column is `AVG(revenue) OVER (ORDER BY month ROWS BETWEEN 3 PRECEDING AND CURRENT ROW)`. A colleague computes `df["revenue"].rolling(4).mean()`. Where do they disagree?

A) On every row: SQL counts rows, pandas counts days

B) Nowhere: the two are identical

C) Only on the last row, because SQL has no following rows

D) On the first three rows: SQL averages the rows that exist, pandas gives NaN until four rows exist

5.4 Query Optimization

KEY IDEAS

- **SQL is declarative:** you say what you want; the planner chooses how. `EXPLAIN QUERY PLAN` shows its choice without running the query.
- **SCAN reads every row; SEARCH jumps.** A scan is fine when you need most of the table, and a problem when you wanted a few rows.
- **An index is a sorted copy of some columns** plus a pointer to each row. Primary keys are indexed automatically; foreign keys and filter columns usually are not.
- **Joins run as nested loops.** The inner step runs once per outer row, so it should be a SEARCH. The planner chooses the table order, not your FROM clause.
- **Composite indexes follow the leftmost-prefix rule:** (a, b) serves a, or a and b, but not b alone. Equality columns first, the range or sort last. A covering index answers without opening the table.
- **An index cannot help** when the column is inside a function or arithmetic, a pattern starts with `%`, most rows match, or the table is tiny.
- `ANALYZE` gives the planner statistics. **Every index costs** write speed and file size.
- **Measure properly:** best of several runs, plan before and after, identical results. Work on a copy; open it with `cached_statements=0` when you drop indexes and re-plan.
- **Ask for less:** name your columns, aggregate in SQL, and send one query instead of a loop of queries.

READING A PLAN

THE PLAN SAYS	IT MEANS	COST GROWS WITH
`SCAN t`	Read every row and test each one	the size of the table
`SEARCH t USING INTEGER PRIMARY KEY (rowid=?)`	Jump straight to the row	almost nothing
`SEARCH t USING INDEX i (col=?)`	Find entries in the index, then fetch those rows	the number of matching rows
`SEARCH t USING COVERING INDEX i (...)`	The index alone answers; the table is not touched	the number of matching rows
`SCAN t USING (COVERING) INDEX i`	Still reads everything, but through the index	the size of the table

THE PLAN SAYS	IT MEANS	COST GROWS WITH
`USE TEMP B-TREE FOR ORDER BY`	A separate sorting step	the number of rows sorted

◆ WORKED EXAMPLE: SHOULD THE PRODUCT LOOKUP GET AN INDEX?

The shop's search box filters products by brand and price. One composite index, the plan before and after, and a fair timing. `NOT INDEXED` tells SQLite to ignore indexes, so the "without" timing runs on the same connection. (`conn`, `plan` and `timed` are the lesson's, on the working copy.)

```
LOOKUP = "SELECT name, price FROM products WHERE brand = ? AND price < ?"
plan(LOOKUP, ("Tallgrass", 50))
conn.execute("CREATE INDEX IF NOT EXISTS idx_products_brand_price "
             "ON products (brand, price)")
conn.commit()
plan(LOOKUP, ("Tallgrass", 50))
print("rows:", len(conn.execute(LOOKUP, ("Tallgrass", 50)).fetchall()), "of",
      conn.execute("SELECT COUNT(*) FROM products").fetchone()[0], "products")
NO_INDEX = "SELECT name, price FROM products NOT INDEXED WHERE brand = ? AND price
t_without = timed(NO_INDEX, [("Tallgrass", 50)] * 500)
t_with = timed(LOOKUP, [("Tallgrass", 50)] * 500)
print(f"500 lookups: {t_without:.1f} ms without the index, {t_with:.1f} ms with it
conn.execute("DROP INDEX idx_products_brand_price")
conn.commit()
```

```
    SCAN products
    SEARCH products USING INDEX idx_products_brand_price (brand=? AND price<?)
rows: 11 of 242 products
500 lookups: 7.4 ms without the index, 4.5 ms with it
```

Reading it. The plan changed from SCAN to SEARCH, and 500 lookups got a few milliseconds faster in total (your numbers will differ). With 242 products the whole table fits in a few pages: scanning it is nearly as fast as any index, and a few milliseconds over 500 searches is nothing a shop page would notice. This is the fourth case in "When an Index Does Not Help": a small table. The honest recommendation is **no index**, and the cell drops it again.

◆ GUIDED LAB

Open `labs/module-15/l5_4-query-optimization/customer_lookup_plans_starter.py`: a support screen finds customers by state and city. Predict three plans, add one composite index, read the plans again, add a column that removes a sort, and drop everything to confirm the plan goes back. The lab works on a copy of the database.

Then `labs/module-15/l5_4-query-optimization/n_plus_one_starter.py`: a script runs one query per customer for 300 customers. Read its plan, time it, replace the loop with one grouped query, and

prove the answers are identical.

◆ PREDICT, THEN RUN

Write SCAN or SEARCH (and which index) first, then run it on the working copy from the demo.

P1

A filter on `sku` in `order_items`, with no index on `sku`. SCAN or SEARCH?

```
plan("SELECT * FROM order_items WHERE sku = ?", ("TNT-1001",))
```

P2

A pattern that starts with `%`, on a table with only its primary key indexed. SCAN or SEARCH?

```
plan("SELECT sku FROM products WHERE name LIKE ?", ("%Tent%",))
```

P3

After the demo's index on `orders(customer_id)`: one customer's orders, sorted by date. How many plan lines?

```
plan("SELECT * FROM orders WHERE customer_id = ? ORDER BY order_date", (12673,
```

◆ NOTES

SELF-CHECK (DO THESE AT HOME, WITHOUT NOTES FIRST)

Q7

Which plan line means the engine answered from the index alone and never opened the table?

A) `SCAN orders`

B) `SEARCH orders USING INDEX idx_orders_store_date (store=?)`

C) `SEARCH orders USING COVERING INDEX idx_orders_store_date_total (store=? AND order_date>? AND order_date<?)`

D) `USE TEMP B-TREE FOR ORDER BY`

Q8

`orders` has an index on `(store, order_date)` and no other. Which filter can NOT seek in it?

A) `WHERE order_date = '2025-07-04'`

B) `WHERE store = 'Bend'`

C) `WHERE store = 'Bend' AND order_date = '2025-07-04'`

D) `WHERE store = 'Bend' AND order_date >= '2025-07-01'`

Q9

After `DROP INDEX`, you run the same `EXPLAIN QUERY PLAN` text again on the same connection and it still names the dropped index. What is going on?

A) The DROP did not happen until you commit

B) SQLite keeps dropped indexes until VACUUM

C) Python's sqlite3 reused its cached prepared statement, planned while the index existed; connect with `cached_statements=0` or open a new connection

D) EXPLAIN QUERY PLAN always shows the plan from the first time a query ran

Q10

What does this print?

```
conn = sqlite3.connect(":memory:")
conn.execute("CREATE TABLE t (a INTEGER, b TEXT)")
conn.execute("CREATE INDEX idx_t_a ON t (a)")
sql = "EXPLAIN QUERY PLAN SELECT * FROM t WHERE a + 0 = 5"
print(conn.execute(sql).fetchone()[3])
```

Q11

A table receives thousands of inserts a minute and has nine indexes, most added "to be safe". What does each extra index cost?

A) Nothing: indexes only affect reads

B) Wrong query results when two indexes cover the same column

C) A full table scan on every SELECT

D) Slower inserts and a larger file, because every index is a sorted copy that must be kept up to date

Q12

Before adding an index a query took 12 ms; after, 4 ms, each measured once. What should you do before reporting "3 times faster"?

A) Nothing: the numbers speak for themselves

B) Run ANALYZE and report the plan instead of times

C) Repeat each measurement several times, take the best, and check the plan changed from SCAN to SEARCH

D) Measure on a different computer

HOMEWORK CHECKLIST

- ☐ **Lesson 5.3 project:** the Store Performance Table, Steps 2 to 5. Your check: all five window columns match pandas.
- ☐ **Lesson 5.4 project:** the Tuning Report, Steps 3 to 6. Your check: `same_results` is True for all three queries, and the report states the file-size cost.
- ☐ Lesson 5.3: read the "pandas Way" blocks; Exercises 1 to 3 (4 is a stretch); quiz; journal.
- ☐ Lesson 5.4: read "What Indexes Cost" and "Where PostgreSQL Differs"; Exercises 1 to 3 (4 is a stretch); quiz; journal.
- ☐ Kit labs: `labs/module-15/l5_3-sql-window-functions/weekly_trend_starter.py`; TODO 4 of the Silver lab; `labs/module-15/l5_4-query-optimization/n_plus_one_starter.py` if not finished.
- ☐ Delete your working copies (`pinecrest_tuning.db`, `tuning_project.db`, the labs' `output/` folders).
- ☐ Self-check Q1 to Q12 in this workbook.
- ☐ **For Session 16:** check `import pymongo, mongomock, requests`; run `python data/mock_api.py` once, open http://127.0.0.1:8000/health, stop it with Ctrl+C.

GLOSSARY

Window function: a function computed over a set of rows related to the current row, returning one value per row. **Partition:** the group of rows a window function looks within (PARTITION BY). **Frame:** the part of the partition the function actually uses for the current row (ROWS BETWEEN ... AND ...). **ROW_NUMBER / RANK / DENSE_RANK:** numbering that never repeats / repeats on ties and skips / repeats on ties without skipping. **LAG / LEAD:** the value from the previous / next row of the window. **Running total:** a sum from the first row of the partition to the current row. **Moving average:** an average over a frame that slides with the current row. **NTILE:** deals ordered rows into a number of nearly equal buckets. **Query planner:** the part of the engine that chooses how to run a query. **EXPLAIN QUERY PLAN:** returns the planner's chosen strategy instead of running the query. **SCAN / SEARCH:** read every row / jump to matching rows using an index or the primary key. **Index:** a separate sorted structure of one or more columns that lets the engine find rows without reading the whole table. **Composite index:** an index on several columns, usable from the leftmost column onward. **Covering index:** an index that contains every column a query needs, so the table is not read. **Selectivity:** how small a share of the rows a filter keeps; indexes pay off on selective filters. **ANALYZE:** gathers statistics about tables and indexes for the planner. **N+1 problem:** running one query for a list and then one more query per item, instead of one combined query.

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 16: Documents and HTTP

Your pages for Session 16. Before the break: a database whose records are nested documents, not rows. After the break: the request-and-response conversation behind every web page and API, and how to take part in it politely.

Lessons 5.5–6.1 | Session 16 · 3 hours | Name: _____

SESSION GOALS

By the end of this session you will be able to:

- explain how a document differs from a row, and say when each kind of database fits
- query nested documents with pymongo: dot notation, operators, `$exists`, `$elemMatch`, projections
- build a small aggregation pipeline and name the SQL and pandas counterpart of each stage
- bring MongoDB results back into a pandas DataFrame
- describe an HTTP request and response, and read the status, headers and body from Python
- check `robots.txt`, identify yourself, set a timeout and check the status on every request
- fetch JSON from an API with query parameters and turn it into a DataFrame

> **TWO HALVES, ONE IDEA**
> Before the break the data is nested documents in a database. After the break it is nested documents arriving over the web. Both end where everything in this course ends: in a DataFrame you already know how to clean, summarize and plot.

LESSONS

5.5 NoSQL with MongoDB

KEY IDEAS

- A **document** is one record that can hold nested objects and lists: one website visit with its device, its pages and its cart. A **collection** is a set of documents, and they need not all have the same fields.
- A **query is a dictionary**: `{"converted": True}`. Several pairs mean AND. Reach inside nested objects with **dot notation** in quotes: `"device.type"`.
- Anything other than "equals" uses an **operator** that starts with `$`: `{"cart.value": {"$gte": 500}}`, `$in`, `$exists`, `$regex`.
- **Missing is not null.** In a query, `None` matches both. `{"$exists": False}` asks precisely "is the field absent?"
- A condition on a list field (`"pages.path"`) is true when **any** element matches. Two conditions on the **same** element need `$elemMatch`.
- The **aggregation pipeline** is a list of stages. `$match` filters (WHERE, or HAVING after a group), `$group` summarizes (GROUP BY), `$unwind` makes one document per list element (pandas `explode`), `$sort` and `$limit` order and cut.
- Inside a pipeline, a field's **value** is written with a leading dollar: `"$device.type"`. In a query, the field name is just the key, with no dollar.
- A wrong field name does not raise an error. It returns zero. **Be suspicious of zero:** look at one document with `find_one()` first.
- Finish in pandas: `pd.DataFrame(list(cursor))` for flat results, `pd.json_normalize(docs)` for nested ones.

SQL	MONGODB
Table	Collection
Row	Document
Column	Field
`WHERE a = 1 AND b > 2`	`{"a": 1, "b": {"$gt": 2}}`
`SELECT a, b`	projection `{"a": 1, "b": 1, "_id": 0}`

SQL	MONGODB
`GROUP BY` + `COUNT(*)`	`$group` with `{"$sum": 1}`
`ORDER BY x DESC LIMIT 3`	`.sort("x", -1).limit(3)`
join to a child table	an embedded list, or `$unwind`

◆ WORKED EXAMPLE: THE SEARCH AUDIENCE

Question: how do visitors who arrive from a search engine behave? Read each block, cover the output, and say what you expect before you look.

Step 1. Three counts with one base query. `{**news, ...}` copies the dictionary and adds a condition. `"cart.items.0"` is "element 0 of the list": if it exists, the cart is not empty.

```
with open("data/web_sessions.json", encoding="utf-8") as f:
    sessions = json.load(f)["sessions"]
web = mongomock.MongoClient()["shop"]["sessions"]
web.insert_many(sessions)

news = {"utm.source": "search"}
print("search sessions :", web.count_documents(news))
has_cart = {"cart.items.0": {"$exists": True}}
print("  with a cart   :", web.count_documents({**news, **has_cart}))
print("  that ordered  :", web.count_documents({**news, "converted": True}))
```

```
search sessions : 915
  with a cart   : 201
  that ordered  : 85
```

Step 2. The same audience, grouped in one pipeline. `$ifNull` supplies a label when the campaign is missing; `$cond` inside `$sum` counts only the sessions that converted. The percentage is finished in pandas.

```
rows = list(web.aggregate([
    {"$match": news},
    {"$group": {"_id": {"$ifNull": ["$utm.campaign", "(no campaign)"]},
                "sessions": {"$sum": 1},
                "orders": {"$sum": {"$cond": ["$converted", 1, 0]}},
                "avg_cart": {"$avg": "$cart.value"}}},
    {"$sort": {"sessions": -1}},
]))
table = pd.DataFrame(rows).rename(columns={"_id": "campaign"})
table = table[["campaign", "sessions", "orders", "avg_cart"]]
table["order_pct"] = (100 * table["orders"] / table["sessions"]).round(1)
print(table.round(2).to_string(index=False))
```

```
        campaign  sessions  orders  avg_cart  order_pct
   (no campaign)       601      52     56.88        8.7
fall_gear_search       314      33     47.37       10.5
```

Step 3. Check it a second way. Flatten the same documents and count in pandas. 601 + 314 = 915 and 52 + 33 = 85: the two routes agree. Notice which sessions have no campaign: the organic ones. Nobody ran a campaign for them.

```
docs = list(web.find(news, {"_id": 0, "session_id": 1, "converted": 1,
                            "utm": 1}))
flat = pd.json_normalize(docs)
print(flat.shape, list(flat.columns))
print(flat.groupby("utm.medium")["converted"].agg(["size", "sum"]).to_string())
print(flat["utm.campaign"].isna().sum(), "rows with no campaign")
```

```
(915, 5) ['session_id', 'converted', 'utm.source', 'utm.medium', 'utm.campaign']
            size  sum
utm.medium
cpc          314   33
organic      601   52
601 rows with no campaign
```

GUIDED LAB

Lab 5.5-A · The sale page audit. Open `labs/module-16/l5_5-nosql-mongodb/sale_page_audit_starter.py`. Run it once as it is: it prints the field names and several `None` values. Then work through TODO 1 to TODO 5. Each one is a single query.

- Run after every TODO. One small step, then look.
- TODO 3 is the thinking step: two queries that sound alike and count different things.

- When you finish, line 6 compares two order rates. Write one sentence: what may you conclude from it, and what may you not?

Lab 5.5-B · Landing pages and what is in the carts (if you finish, or at home): `labs/module-16/l5_5-nosql-mongodb/landing_pages_starter.py`. Two pipelines, then a pandas check that should agree with them.

◆ PREDICT, THEN RUN

Set up once: load the sessions into a collection called `web`, as in the worked example.

Write your prediction on the line **before** you run the code. A wrong prediction that you then explain is worth more than a right one.

1. The collection `web` holds the 2,400 sessions. 916 have no `utm` at all; another 737 have a `utm` object with no `campaign` in it. Predict the three numbers.

```
print(web.count_documents({"utm.campaign": None}))
print(web.count_documents({"utm.campaign": {"$exists": False}}))
print(web.count_documents({"utm": {"$exists": True},
                           "utm.campaign": {"$exists": False}}))
```

2. 15 converted sessions used a tablet. Predict the two numbers.

```
cursor = web.find({"device.type": "tablet", "converted": True},
                  {"_id": 0, "session_id": 1})
first = [d["session_id"] for d in cursor]
second = [d["session_id"] for d in cursor]
print(len(first), len(second))
```

3. 200 sessions have exactly one page. Every session has at least one page. Predict the first two numbers; then guess what the third line is counting.

```
print(web.count_documents({"pages": {"$size": 1}}))
print(web.count_documents({"pages.0": {"$exists": True}}))
print(web.count_documents({"cart.items": {"$size": 0}}))
```

SELF-CHECK (ANSWERS ARE IN YOUR INSTRUCTOR'S KEY)

Q1

In the sessions collection, `utm` is absent from direct visits and never set to null. Which query counts exactly the sessions where the field is not there, and would still be right if some documents held `utm: null`?

A) `{"utm": None}`

B) `{"utm": {"$exists": False}}`

C) `{"utm": ""}`

D) `{"utm": {"$size": 0}}`

Q2

What does this print?

```
c = mongomock.MongoClient()["demo"]["notes"]
c.insert_many([{"n": 1, "tags": ["a", "b"]},
               {"n": 2, "tags": ["b"]},
               {"n": 3}])
print(c.count_documents({"tags": "b"}), c.count_documents({"tags": None}))
```

A) `3 0`

B) `2 0`

C) `1 1`

D) `2 1`

Q3

What does this print?

```
c = mongomock.MongoClient()["demo"]["sales"]
c.insert_many([{"store": "Bend", "units": 2},
               {"store": "Boise", "units": 5},
               {"store": "Bend", "units": 4}])
rows = c.aggregate([{"$group": {"_id": "$store", "units": {"$sum": "$units"}}},
                    {"$sort": {"units": -1}}])
print([(r["_id"], r["units"]) for r in rows])
```

Q4

Which pipeline stage turns every element of a list into its own document, the way `explode` does in pandas?

A) `$unwind`

B) `$group`

C) `$project`

D) `$match`

Q5

What does this print?

```
c = mongomock.MongoClient()["demo"]["visits"]
c.insert_many([
    {"id": 1, "pages": [{"path": "/cart", "seconds": 10},
                        {"path": "/sale", "seconds": 200}]},
    {"id": 2, "pages": [{"path": "/cart", "seconds": 150}]},
])
loose = {"pages.path": "/cart", "pages.seconds": {"$gte": 100}}
strict = {"pages": {"$elemMatch": {"path": "/cart", "seconds": {"$gte": 100}}}}
print(c.count_documents(loose), c.count_documents(strict))
```

A) `1 1`

B) `2 2`

C) `2 1`

D) `1 2`

Q6

What does this print? (The collection holds the three sales documents from Q3.)

```
rows = list(c.aggregate([{"$group": {"_id": "store", "n": {"$sum": 1}}}]))
print(rows)
```

A) `[{'n': 2, '_id': 'Bend'}, {'n': 1, '_id': 'Boise'}]`

B) `[{'n': 3, '_id': 'store'}]`

C) `[]`

D) A `KeyError`: there is no field called `store`

6.1 HTTP with Requests

THE SIX GROUND RULES (EVERY SESSION OF MODULE 6 STARTS HERE)

1. **Read `robots.txt` and obey it.** It is the owner's published request to programs: where they may go and how fast.
2. **Read the terms of use.** If they forbid automated collection, the answer is no, however easy the code would be.
3. **Prefer an API or a download** when one exists. It is the front door.
4. **Go slowly and say who you are.** Pause between requests; send a User-Agent with a way to contact you.
5. **Never go around a login, a paywall or a block.** If you are asked to stop, stop.
6. **Do not collect personal data.** Names, emails, profiles and photos of real people are protected by law in many places.

KEY IDEAS

- **HTTP is request and response.** Out: a method (`GET` to read), a URL, headers, maybe a body. Back: a **status code**, headers, a body.
- **Status codes by first digit:** 2xx success, 3xx look elsewhere (handled for you), 4xx your request has a problem, 5xx the server has a problem.
- A 404 or a 500 is **not an exception** until you call `r.raise_for_status()`. The pattern is: **get → raise_for_status() → use the body.**
- **Rules first.** Fetch `robots.txt`, parse it with `RobotFileParser`, ask `can_fetch()` and `crawl_delay()`.
- Send a **User-Agent** that names your project and gives a contact address. Never pretend to be a browser.
- `r.content` is bytes; `r.text` is those bytes decoded with `r.encoding`. Junk like `â˜…` means the wrong encoding: set `r.encoding = "utf-8"`.
- Pass query parameters as a dictionary (`params=`) and read JSON with `r.json()`. A list of flat dictionaries goes straight into `pd.DataFrame`.
- **Every request gets a `timeout`.** Without one, `requests` waits forever.
- A `Session` reuses the connection and remembers your headers (not your timeout). Save `r.content` to disk once; parse the saved copy as often as you like.

RANGE	MEANING	WHAT TO DO
2xx	Success	Use the body
3xx	Look elsewhere	Nothing: `requests` follows redirects
4xx	Your request has a problem (400, 401, 403, 404, 429)	Fix the request (429: wait first)
5xx	The server has a problem (500, 503)	Wait and try again later

◆ WORKED EXAMPLE: DOES THIS PRODUCT EXIST?

Question: the shop has a page for SLP-1004. Does it have one for SLP-9004? Ask the website, then ask the API, and compare how each one says "no".

Step 1. Ask the website. The function checks the status before it touches the body. For a missing page it returns the status and `None`: it never tries to read a title out of an error page. (Setup: `BASE_URL`, `HEADERS`, `urljoin` and `time` as in the demo.)

```
def product_page(sku):
    """Return (status, title) for one product page; title None if missing."""
    url = urljoin(BASE_URL, f"products/{sku}.html")
    resp = requests.get(url, headers=HEADERS, timeout=10)
    try:
        resp.raise_for_status()
    except requests.exceptions.HTTPError:
        return resp.status_code, None
    resp.encoding = "utf-8"
    html = resp.text
    title = html[html.index("<title>") + 7 : html.index("</title>")]
    return resp.status_code, title

for sku in ["SLP-1004", "SLP-9004"]:
    print(sku, product_page(sku))
    time.sleep(1)
```

```
SLP-1004 (200, 'Larkspur Ridge Backcountry Sleeping Pad | Pinecrest Outfitters (p
SLP-9004 (404, None)
```

Step 2. Ask the API. Same question, one line of parsing. The API answers a missing product with status 404 **and** a JSON body that explains itself. (`API = "http://127.0.0.1:8000"`; start it with `python data/mock_api.py`.)

```
for sku in ["SLP-1004", "SLP-9004"]:
    r = requests.get(f"{API}/products/{sku}", headers=HEADERS, timeout=10)
    body = r.json()
    print(sku, r.status_code, body.get("name") or body["error"]["message"])
```

```
SLP-1004 200 Larkspur Ridge Backcountry Sleeping Pad
SLP-9004 404 No product with SKU 'SLP-9004'.
```

Both sources agree, and neither needed you to guess: the status code told the program what happened, and the message told you.

GUIDED LAB

Lab 6.1-A · Three category pages, one of them missing. Open `labs/module-16/l6_1-http-requests/category_check_starter.py` and run it as it is. It already fetches the pages (or, with no network, reads saved copies: your instructor will tell you which). Then:

- TODO 1: ask `robots.txt` whether each page may be fetched.
- TODO 2: check the status **before** counting anything on the page.
- TODO 4: decode the same bytes with the wrong table, then the right one, and watch the stars.
- Last line of the lab: which page must not get a count, and why? One sentence.

Lab 6.1-B · Ask the API: which category costs the most? (if you finish, or at home): `labs/module-16/l6_1-http-requests/api_price_check_starter.py`. Start the practice API first: `python data/mock_api.py` in a second terminal.

No network at all? Both labs run from saved responses: `LABS_OFFLINE=1 python category_check_starter.py` (PowerShell: `$env:LABS_OFFLINE = "1"`, then `python category_check_starter.py`).

PREDICT, THEN RUN

Set up once: `BASE_URL`, `HEADERS` and `urljoin` as in the demo.

Write your prediction on the line **before** you run the code. A wrong prediction that you then explain is worth more than a right one.

1. The catalog has 13 pages; the last one holds 2 products. Predict both lines.

```
for name in ["page-13.html", "page-14.html"]:
    r = requests.get(urljoin(BASE_URL, name), headers=HEADERS, timeout=10)
    print(r.status_code, r.ok, r.text.count('class="product-card"'))
```

2. Nothing is sent here: `prepare()` only builds the request. Predict the URL.

```
params = {"category": "Tents", "per_page": 5, "page": 2}
prepared = requests.Request("GET", "http://127.0.0.1:8000/products",
                            params=params).prepare()
print(prepared.url)
```

3. `BASE_URL` is the shop's front page. Predict the first line, then what the second line does.

```
r = requests.get(BASE_URL, headers=HEADERS, timeout=10)
print(r.headers["Content-Type"])
r.json()
```

◆ SELF-CHECK (ANSWERS ARE IN YOUR INSTRUCTOR'S KEY)

Q7

A script runs `r = requests.get(url, timeout=10)` and the server answers with status 404. What happens on that line?

A) `requests` raises `HTTPError`

B) `r` is `None`

C) The call is retried until it succeeds or 10 seconds pass

D) A normal response comes back; `r.ok` is `False` and nothing is raised

Q8

What does this print?

```
star = "★".encode("utf-8")
print(len("★"), len(star), star.decode("cp1252") == "★")
```

Q9

You print `r.text` from a category page and every rating shows `â˜… 4.5`. What is the first thing to check?

A) `r.encoding` and the `Content-Type` header

B) Whether the page needs JavaScript

C) `r.status_code`, because this means the page is a 404

D) Your `timeout` value

Q10

What does this print? (`prepare()` only builds the request; nothing is sent.)

```
params = {"status": "returned", "since": "2025-12-01"}
prepared = requests.Request("GET", "http://127.0.0.1:8000/orders",
                            params=params).prepare()
print(prepared.url)
```

Q11

A site's `robots.txt` says `Disallow: /staff/` and `Crawl-delay: 1`. The staff page has no login. What should your collection script do?

A) Fetch the staff page, but only once

B) Fetch it with a browser's User-Agent so the site cannot tell

C) Skip the staff page, and wait at least one second between its other requests

D) Skip `robots.txt`: it has no legal force

Q12

What does this print?

```
resp = requests.Response()
resp.status_code = 503
print(resp.ok)
try:
    resp.raise_for_status()
    print("fine")
except requests.exceptions.HTTPError:
    print("caught")
```

A) `True` then `fine`

B) `False` then `caught`

C) `False` then `fine`

D) `True` then `caught`

NOTES

HOMEWORK CHECKLIST

- ☐ Lesson 5.5: finish the Funnel Report. Your funnel should read 2,400 → 1,610 → 543 → 230 → 230.
- ☐ Lesson 5.5: the four exercises, the quiz and the journal prompt.
- ☐ Lesson 6.1: finish the Polite Snapshot. `raw/` should hold four saved files and `manifest.csv` with six rows.
- ☐ Lesson 6.1: the four exercises, the quiz and the journal prompt.
- ☐ Finish any kit lab you did not finish in the room.
- ☐ Put your real contact address in `USER_AGENT`.

- ☐ **For Session 17:** make sure Google Chrome is installed and run the one-minute Chrome check your instructor gave you, while you have internet. Bring the exact error message if it fails.
- ☐ **For Session 17:** bring your Lesson 6.1 notebook and `raw/` folder, and have `data/pinecrest-practice-site.zip` in your project folder.

GLOSSARY

Document: one record in MongoDB: like a Python dictionary, and it may hold nested objects and lists.

Collection: a set of documents (the counterpart of a table). Its documents need not share a shape.

Dot notation: `"device.type"`: the name of a field inside a nested object, written in quotes.

Operator: a key that starts with `$` (`$gt`, `$in`, `$exists`, `$set`) and says how to compare or change.

Projection: the second argument of `find`: which fields to return (`1`) or leave out (`0`).

Aggregation pipeline: a list of stages (`$match`, `$unwind`, `$group`, `$sort`...) that each transform the documents and pass them on.

mongomock: a library that imitates a MongoDB server inside Python, with pymongo's methods. Data lives in memory only.

HTTP: the request-and-response conversation every web page and API is built on.

Status code: the three-digit summary of a response: 2xx success, 3xx look elsewhere, 4xx your request, 5xx the server.

Header: a name-and-value note that travels with a request or a response (`User-Agent`, `Content-Type`).

robots.txt: a site's published request to programs: where they may go and how fast.

Query parameters: the `?name=value&name=value` part of a URL. Pass them to `requests` as a dictionary (`params=`).

Timeout: the longest `requests` will wait to connect, and between bytes. Without one it waits forever.

Mojibake: junk such as `â˜…` that appears when bytes are decoded with the wrong encoding.

API: addresses that return data (usually JSON) for programs instead of pages for people.

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 17: Scraping Static and Dynamic Pages

Your pages for Session 17. Before the break: turning a web page into a checked table with Beautiful Soup. After the break: pages that JavaScript fills in, and how to collect them honestly, with the data file first and a browser only when you must.

Lessons 6.2-6.3 | Session 17 · 3 hours | Name: _____

SESSION GOALS

- read a page as a tree and choose selectors that describe meaning, not position
- turn one repeated block of HTML into one dictionary, with missing values handled on purpose
- follow pagination to the end, check a scrape against the page's own count, and read real tables with `read_html`
- tell a static page from a dynamic one, and look for the data file before reaching for a browser
- drive headless Chrome with Selenium: wait for things by name, click, fill a form, and always quit

THE SIX GROUND RULES (EVERY MODULE 6 SESSION STARTS HERE)

1. **Read `robots.txt` and obey it.** It is the owner's published request to programs: where they may go and how fast.
2. **Read the terms of use.** If they forbid automated collection, the answer is no, however easy the code would be.
3. **Prefer an API or a download** when one exists. It is the front door.
4. **Go slowly and say who you are.** Pause between requests; send a User-Agent with a way to contact you.
5. **Never go around a login, a paywall or a block.** If you are asked to stop, stop.
6. **Do not collect personal data.** Names, emails, profiles and photos of real people are protected by law in many places.

LESSONS

6.2 Parsing HTML with Beautiful Soup

KEY IDEAS

- Pass `resp.content` (bytes) to `BeautifulSoup(..., "html.parser")`: it reads the page's own charset.
- `find()` / `select_one()` return one element or `None`. `find_all()` / `select()` return a list, maybe empty.
- Choose labels, not positions: `li.product-card p.price`, not `main > ul > li:nth-of-type(3)`.
- Prefer `data-` attributes (`data-price`) to formatted text (`$299.95`). Everything from HTML is a string: convert it.
- Find the repeated block, then write one function that turns one block into one dictionary.
- Missing attribute: `tag["x"]` raises `KeyError`, `tag.get("x")` gives `None`. Missing element: `select_one` gives `None`. Decide per field what missing should mean.
- Follow the "next" link until there is none, pause between requests, and keep a page limit.
- Check before you trust: the page's count, unique keys, plausible values.
- `pd.read_html(io.BytesIO(resp.content))` reads real tables; pick one with `attrs={"id": ...}`.

SELECTOR	MATCHES
`p`	every p element
`.price`	class includes price
`#price-list`	the element with that id
`[data-sku]`	has the attribute
`[data-sku="PCK-1003"]`	attribute has that value
`A B`	B anywhere inside A
`A > B`	B directly inside A

WORKED EXAMPLE: THE CLIMBING GEAR THAT IS RUNNING OUT

Question: which climbing products are low on stock or out of stock? The stock label is deep inside each card, so start from the label and walk **up** to the card with `find_parent("li")`.

```
climb = get_soup(urljoin(BASE_URL, "category-climbing.html"))
low = []
for label in climb.select("p.stock.low-stock, p.stock.out-of-stock"):
    card = label.find_parent("li")
    low.append({"sku": card["data-sku"],
                "name": card.select_one("h2.product-name a").get_text(strip=True),
                "price": float(card.select_one("p.price")["data-price"]),
                "stock": label["class"][1]})
print(pd.DataFrame(low).to_string(index=False))
```

```
     sku                                name  price        stock
CLM-1005        Larkspur Ridge Mesa Chalk Bag  28.95    low-stock
CLM-1010 Cedar Notch Tundra Locking Carabiner  12.50    low-stock
CLM-1014    Granite Fork Summit Quickdraw Set 133.95    low-stock
CLM-1022 Quillback Trailhead Climbing Harness  58.95 out-of-stock
```

Then one detail page for the rope, the specs table as a dictionary, and the breadcrumbs:

```
time.sleep(DELAY)
page = get_soup(urljoin(BASE_URL, "products/CLM-1004.html"))
specs = {tr.th.get_text(strip=True): tr.td.get_text(strip=True)
         for tr in page.select("table.specs tr")}
print(specs)
print([li.get_text(strip=True) for li in page.select("nav.breadcrumbs li")])
```

```
{'Brand': 'Wrenfield', 'Type': '60m Rope', 'Weight': '4.15 kg', 'Color': 'Plum',
['Shop', 'Climbing', 'Wrenfield Ember 60m Rope']
```

Four products, one request for the listing and one for the detail page. Every value came from a label: a class, a `data-` attribute or a table heading.

GUIDED LAB

Lab 6.2-A · The sleeping bag shortlist. Open `labs/module-17/l6_2-beautifulsoup/sleeping_bag_shortlist_starter.py` and run it as it is. Then TODO 2 (finish `parse_card()`), TODO 3 (the shortlist), TODO 4 (the specs table), TODO 5 (clean the text). Notice how many detail pages the lab fetches, and why.

Lab 6.2-B · Price list tables (if you finish, or at home): `labs/module-17/l6_2-beautifulsoup/price_list_tables_starter.py`.

◆ PREDICT, THEN RUN

Write your prediction on the line **before** you run the code.

1. Predict all three lines.

```
snippet = BeautifulSoup('<div><p class="price" data-price="5">$5</p>'
                        '<p class="price sale">$4</p></div>', "html.parser")
print(len(snippet.select("p.price")), len(snippet.select("p.sale")))
print(snippet.select_one("p.sale").get("data-price"))
print(snippet.select_one("p.gone"))
```

2. Predict the three addresses.

```
print(urljoin("https://example.com/shop/page-3.html", "page-4.html"))
print(urljoin("https://example.com/shop/page-3.html", "../robots.txt"))
print(urljoin("https://example.com/shop/", "/shop/products/A-1.html"))
```

3. Predict the three lines.

```
tag = BeautifulSoup('<p class="rating">★ 4.2 <span>(31 reviews)</span></p>',
                    "html.parser").p
print(repr(tag.get_text()))
print(repr(tag.get_text(" | ", strip=True)))
print(int(re.search(r"\d+", tag.span.get_text()).group()))
```

SELF-CHECK (ANSWERS ARE IN YOUR INSTRUCTOR'S KEY)

Q1

Which call gives you `None` (not an empty list, not an exception) when nothing on the page matches?

A) `soup.select("p.sale-price")`

B) `soup.find_all("p", class_="sale-price")`

C) `soup.select_one("p.sale-price")`

D) `card["data-sale-price"]` on a card without that attribute

Q2

What does this print?

```
soup = BeautifulSoup('<ul><li class="card" data-sku="A1"><b>Tent</b></li>'
                     '<li class="card"><b>Stove</b></li></ul>', "html.parser")
print([li.get("data-sku") for li in soup.select("li.card")])
```

Q3

Your loop follows `nav.pagination a.next` from page to page. How does it know it has reached the last page?

A) On the last page "Next" is a plain `span`, so `select_one("nav.pagination a.next")` returns `None`

B) The server answers the last page with a 404

C) The last page has no product cards

D) The loop counts to 13, the number of pages

Q4

What does this print?

```
soup = BeautifulSoup('<p class="stock low-stock">Low stock</p>', "html.parser"
print(soup.p["class"], soup.p["class"][1].replace("-", "_"))
```

A) `['stock low-stock'] low-stock`

B) `stock low_stock`

C) `['stock', 'low-stock'] low-stock`

D) `['stock', 'low-stock'] low_stock`

Q5

Lesson 6.2 fetches `price-list.html` with `requests` and passes `io.BytesIO(resp.content)` to `pd.read_html()`, instead of giving `read_html` the URL. Why?

A) `read_html` cannot read a URL

B) So that your User-Agent, your timeout and `raise_for_status()` apply, and pandas reads the exact bytes you checked

C) Bytes make `read_html` return numbers instead of text

D) `BytesIO` removes the second table

Q6

What does this print?

```
html = ('<table id="t"><tr><th>City</th><th>Price</th></tr>'
        '<tr><td>Bend</td><td>$1,299.00</td></tr></table>')
t = pd.read_html(io.StringIO(html), attrs={"id": "t"})[0]
print(t["Price"].dtype, t["Price"].str.replace("[$,]", "", regex=True).astype(
```

6.3 Dynamic Pages with Selenium

KEY IDEAS

- Some pages arrive **flat-packed**: an empty frame, a data file, and JavaScript that assembles them. `requests` never runs the JavaScript.
- **Look for the data first:** the Network tab (Fetch/XHR) or the page's script. If you may request the file, do that.
- Drive a browser only when you must: the data is assembled in the page, it takes interaction to reveal, or you need what a person sees.
- `webdriver.Chrome(options=...)` starts a real browser; Selenium Manager fetches the driver the first time.
- `find_element` returns one element or **raises**; `find_elements` returns a list, maybe empty.
- **Never sleep. Wait by name:** `WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.ID, "x")))`, with the locator as one tuple.
- Let Selenium render, then parse `driver.page_source` with Beautiful Soup.
- A GET form is a slow way of building a URL: `urlencode` builds it directly.
- `driver.quit()` in a `finally` block, every time.

CONDITION	TRUE WHEN
`presence_of_element_located`	it exists
`visibility_of_element_located`	it exists and shows
`element_to_be_clickable`	it shows and is enabled
`text_to_be_present_in_element`	its text contains ...
`url_contains`	the address contains ...

WORKED EXAMPLE: THE SEARCH PAGE, WITHOUT A BROWSER

Question: which packs cost $100 or less? The Search page answers it with JavaScript from `products.json`. Ask the file:

```
resp = requests.get(urljoin(BASE_URL, "products.json"), headers=HEADERS, timeout=1
resp.raise_for_status()
shop = pd.DataFrame(resp.json()["products"])
hits = shop[(shop["category"] == "Packs") & (shop["price"] <= 100)]
print(len(hits), "products")
print(hits[["sku", "price", "stock_status"]].head(4).to_string(index=False))
```

```
11 products
     sku  price stock_status
PCK-1005  82.95     in_stock
PCK-1006  84.95     in_stock
PCK-1007  41.95     in_stock
PCK-1008  70.95     in_stock
```

Eleven packs, the same 11 the browser's form search reported ("11 products found"), with no browser, no waits and no form. Before you automate a page, ask where its data comes from.

GUIDED LAB

Lab 6.3-A · The budget shelf. Open `labs/module-17/l6_3-selenium/budget_search_starter.py`. TODO 1 (the query string), TODO 2 (the wait: no sleep), TODO 3 and 4 (parse and count), TODO 5 (the same search in `products.json`). If Chrome cannot start on your machine, the lab says so and uses a saved copy of the finished page: carry on.

Lab 6.3-B · Page triage (if you finish, or at home): `labs/module-17/l6_3-selenium/page_triage_starter.py`. No network at all? `LABS_OFFLINE=1` runs every lab from saved pages.

PREDICT, THEN RUN

Write your prediction on the line **before** you run the code.

1. The browser has just loaded the Deals page. Predict the second line.

```
driver.get(DEALS_URL)
print(driver.title)
print(len(driver.find_elements(By.CSS_SELECTOR, "li.deal")), "deals right after ge
```

2. Predict what each part prints.

```
driver = make_driver()
try:
    driver.get(DEALS_URL)
    print(driver.find_elements(By.CSS_SELECTOR, "li.gone"))
    try:
        driver.find_element(By.CSS_SELECTOR, "li.gone")
    except NoSuchElementException:
        print("NoSuchElementException")
finally:
    driver.quit()
```

3. Predict the query string.

```
print(urlencode({"q": "rain jacket", "category": "Apparel", "in_stock": 1}))
```

◆ SELF-CHECK (ANSWERS ARE IN YOUR INSTRUCTOR'S KEY)

Q7

`requests` gets the Search page and finds 0 results in `#search-results`, while the browser shows 24. What should you try first?

- A) Drive a headless browser with `time.sleep(5)` after `get()`
- B) Find the data file the page loads (Network tab, or read its script) and, if you may, request it directly
- C) Send a browser's User-Agent so the server sends the full page
- D) Use `pd.read_html` on the page instead

Q8

What does this print?

```
print(urlencode({"q": "", "category": "Snow", "max_price": "", "in_stock": 1})
```

Q9

Which line waits correctly for the deals status line to exist?

A) `time.sleep(3)`

B) `WebDriverWait(driver, 10).until(EC.presence_of_element_located(By.ID, "deal-status"))`

C) `WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.ID, "deal-status")))`

D) `driver.find_element(By.ID, "deal-status")`

Q10

After clicking Submit on the GET search form, the demo waits twice: first `EC.url_contains("category=Packs")`, then for `#search-results[data-loaded="true"]`. Why not only the second wait?

A) The second condition is slower

B) `url_contains` is the only wait that raises on failure

C) The form submits twice

D) The OLD page's results are already marked loaded, so the second wait alone could pass instantly, before the new page arrives

Q11

You stored `status = driver.find_element(By.ID, "deal-status")`, called `driver.refresh()`, and now `status.text` raises `StaleElementReferenceException`. What happened?

A) The handle pointed at an element in the page that was replaced; find the element again after anything changes the page

B) The browser crashed and must be restarted

C) The element's text was empty

D) The wait limit was too short

Q12

What is in `driver.page_source` after the deals have loaded and Load more was clicked twice?

A) The HTML exactly as the server sent it, with an empty deal list

B) Only the 24 deal elements, without the rest of the page

C) The JSON from `deals.json`

D) The page's HTML as it is now, after its scripts ran: all 24 deals are in it

NOTES

HOMEWORK CHECKLIST

- ☐ Lesson 6.2: finish the Category Price Report (nine rows, Tents first by median price), the exercises, quiz and journal.
- ☐ Lesson 6.3: finish Deals Watch (24 rows that equal `deals.json`), the exercises, quiz and journal.
- ☐ Finish any kit lab you did not finish in the room.

- ☐ **For Session 18:** check that `python data/mock_api.py --token-ttl 5` starts and `http://127.0.0.1:8000/health` answers.
- ☐ **For Session 18:** set the three `PINECREST_*` environment variables in the terminal you start your notebook from.
- ☐ **For Session 18:** Checkpoint 2 (15 minutes, open notes): review how Lesson 5.1 opens `pinecrest.db` read-only and how Lesson 6.1 passes `params`.

GLOSSARY

Element: one piece of HTML: a tag name, attributes (`class`, `href`, `data-sku`) and content.

Beautiful Soup: the library (`bs4`) that turns HTML into a tree you can search.

CSS selector: a pattern such as `li.product-card p.price` that names elements by tag, class, id or attribute.

`select` / `select_one`: every match as a list / the first match or `None`.

`data-` **attribute:** a machine-readable value a site stores for its own scripts, such as `data-price="299.95"`.

Pagination: a listing split over pages, joined by "next" links.

`read_html`: pandas' reader for real `<table>` elements; it returns a list of DataFrames.

Dynamic page: a page whose content is added by JavaScript after it loads.

Selenium: a library that drives a real browser from Python.

Headless: a browser running without a window.

Explicit wait: `WebDriverWait(driver, 10).until(condition)`: wait for a named condition, up to a limit.

WebElement: Selenium's handle on one live element; stale after the page changes.

`page_source`: the browser's current HTML, after scripts have run.

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 18: APIs: Keys, Tokens and Polite Clients

Your pages for Session 18. Before the break: API keys, logins and tokens, kept out of your code. After the break: how to be a client a server is glad to have. Then Checkpoint 2: one database, one API, one honest answer.

Lessons 6.4-6.5 | Session 18 · 3 hours | Name: _____

SESSION GOALS

- keep keys and passwords out of code, output and logs
- send an API key in a header and tell a 401 from a 403
- log in for a bearer token, use it, and renew it once when it expires
- follow `next` to the end of a paginated API and check the result
- read rate-limit headers, pace requests, and honor 429 and `Retry-After`
- retry only what is temporary, with backoff, jitter and a limit; cache responses; make long jobs resumable
- show what you can do on your own in Checkpoint 2

THE SIX GROUND RULES (EVERY MODULE 6 SESSION STARTS HERE)

1. **Read `robots.txt` and obey it.** It is the owner's published request to programs: where they may go and how fast.
2. **Read the terms of use.** If they forbid automated collection, the answer is no, however easy the code would be.
3. **Prefer an API or a download** when one exists. It is the front door.
4. **Go slowly and say who you are.** Pause between requests; send a User-Agent with a way to contact you.
5. **Never go around a login, a paywall or a block.** If you are asked to stop, stop.
6. **Do not collect personal data.** Names, emails, profiles and photos of real people are protected by law in many places.

LESSONS

6.4 API Authentication

KEY IDEAS

- **Secrets live in the environment**, not in code: `os.environ.get("PINECREST_API_KEY")`. Print a hint (`pine...`), never the value.
- The key goes in the header the documentation names (`X-API-Key`), never in the URL.
- **401:** "I cannot tell who you are" (missing, malformed or expired credential). **403:** "I understood, and the answer is no." Never retry a 403.
- Turn error bodies into an exception that keeps `status`, `code` and `message` (`ApiError`), so code can decide by status what to do next.
- Log in with `requests.post(url, json={...})`. Send the token as `Authorization: Bearer <token>`.
- Tokens expire. On `token_expired`, log in again and repeat the request **once**; anything else surfaces.
- Ask for one record first to learn the size of the job. Then follow `next` until it is `None`, sending the filters only with the first request.
- Check what you collected: the promised total, unique ids, the date range, and the source when you have it.

STATUS	MEANS	YOUR PROGRAM
400	malformed request	fix the code
401	no valid credential	renew an expired token once; otherwise stop
403	recognized, not allowed	stop
404	no such thing	stop, or record "does not exist"
429	too fast	wait (Retry-After), retry
5xx	the server failed	wait, retry a few times

WORKED EXAMPLE: CANCELLATIONS IN THE WEEK BEFORE CHRISTMAS

Question: how many orders were cancelled from December 18 to December 24, 2025, and what were they worth? The API filters by `since` only, so ask for the size first, pull everything since the 18th, and cut the end date in pandas.

```
since = "2025-12-18"
probe = check(requests.get(f"{API}/orders", headers=KEY_HEADERS, timeout=10,
                           params={"status": "cancelled", "since": since,
                                   "per_page": 1})).json()
print(probe["total"], "cancelled orders since", since, "| pages at 5 per page:",
      -(-probe["total"] // 5))
```

```
14 cancelled orders since 2025-12-18 | pages at 5 per page: 3
```

```
week = pd.DataFrame(get_all("/orders", {"status": "cancelled", "since": since,
                                        "per_page": 5}))
week = week[week["order_date"] <= "2025-12-24"]
print(week[["order_id", "order_date", "store", "payment_method", "order_total"]]
      .to_string(index=False))
print("total value:", round(week["order_total"].sum(), 2))
```

```
 order_id order_date  store payment_method  order_total
   119840 2025-12-18 Online           card        53.90
   119860 2025-12-18 Online digital_wallet        62.90
   119900 2025-12-20 Online digital_wallet       176.40
   119921 2025-12-20 Online digital_wallet        93.90
   119991 2025-12-21 Online digital_wallet       317.65
   120039 2025-12-23 Online digital_wallet       194.35
   120071 2025-12-24 Online           card        48.85
total value: 947.95
```

Fourteen cancellations since December 18, seven of them in the week up to the 24th, all online, worth $947.95. Three requests, one probe, one filter in pandas.

GUIDED LAB

Lab 6.4-A · Cancelled orders. Open `labs/module-18/l6_4-api-authentication/cancelled_orders_starter.py`. Set the three environment variables and start the API first (`python data/mock_api.py --token-ttl 5`). TODO 1: the key; TODO 3: the probe; TODO 4: follow `next`; TODO 5: the summary; TODO 6: a date the API cannot read.

Lab 6.4-B · Token lifecycle (if you finish, or at home): `labs/module-18/l6_4-api-authentication/token_lifecycle_starter.py`.

◆ PREDICT, THEN RUN

Write your prediction on the line **before** you run the code.

1. The three header sets are: no key, a wrong key, the right key. Predict the three status codes.

```
for headers in [HEADERS, {**HEADERS, "X-API-Key": "wrong"}, KEY_HEADERS]:
    r = requests.get(f"{API}/orders", headers=headers, params={"per_page": 1},
                     timeout=10)
    print(r.status_code, end=" ")
print()
```

2. 397 orders were cancelled in the two years. Predict the three values printed.

```
data = check(requests.get(f"{API}/orders", headers=KEY_HEADERS, timeout=10,
                          params={"status": "cancelled", "per_page": 100})).json()
print(data["total"], data["total_pages"], data["next"])
```

◆ SELF-CHECK (ANSWERS ARE IN YOUR INSTRUCTOR'S KEY)

Q1

The orders endpoint wants a key in the `X-API-Key` header. What do you get with no key at all, and with a misspelled key?

A) 401 for no key, 403 for the misspelled one

B) 403 for both

C) 401 for both

D) 404 for no key, 401 for the misspelled one

Q2

What does this print? (`ApiError` is the class from the demo.)

```
err = ApiError(401, "token_expired", "Log in again.")
print(err)
print(err.code == "token_expired", isinstance(err, Exception))
```

Q3

What does this print?

```
pages = {"/o?page=1": {"items": ["a", "b"], "next": "/o?page=2"},
         "/o?page=2": {"items": ["c"], "next": "/o?page=3"},
         "/o?page=3": {"items": [], "next": None}}
items, url, calls = [], "/o?page=1", 0
while url:
    page = pages[url]
    calls += 1
    items.extend(page["items"])
    url = page["next"]
print(items, calls)
```

A) `['a', 'b'] 1`

B) `['a', 'b', 'c'] 2`

C) `['a', 'b', 'c'] 3`

D) `['a', 'b', 'c', None] 3`

Q4

In `get_all()` the first request sends the filters in `params`, then the loop sets `params = None`. Why?

A) To save memory

B) The `next` address already carries the filters; sending them again would attach every filter twice

C) The server rejects a second page that has parameters

D) To force the server to start again at page 1

Q5

Which header sends a bearer token correctly?

A) `X-API-Key: <token>`

B) `Authorization: <token>`

C) The token as `?token=` in the URL

D) `Authorization: Bearer <token>`

Q6

You will share this notebook. Which line may stay in it?

A) `print(API_KEY)`

B) `print("key loaded:", API_KEY[:4] + "...")`

C) `API_KEY = "pinecrest-demo-key"`

D) `url = f"{API}/orders?api_key={API_KEY}"`

6.5 Rate Limiting and Retries

KEY IDEAS

- A **rate limit** is the owner's rule: so many requests per window per client. `X-RateLimit-Limit`, `-Remaining`, `-Reset` tell you where you stand.
- **Plan A: pace yourself.** Interval = window ÷ allowance × 1.1. A Throttle sleeps only for what is left.
- **Plan B: obey 429.** Wait `Retry-After` seconds (with a default), retry, with a maximum number of tries.
- **Retry only what is temporary:** 429, 500, 502, 503, 504, timeouts, dropped connections. Never 400, 403, 404.
- **Exponential backoff** (0.5, 1, 2, 4, 8 s, capped) plus **jitter** (a random wait below that ceiling).
- urllib3's `Retry` on an `HTTPAdapter` mounted on a Session does this for you; retry only GET and HEAD.
- **Cache:** same request, same file. Check the status before writing, so errors are never cached.
- **Resumable jobs:** one file per page, skip files that exist, write to `.part` then rename.
- **Log** milestones and problems; never log a key, a token or a password.

WORKED EXAMPLE: PLAN THE JOB BEFORE YOU RUN IT

Question: how long will a download take at a polite pace? Three services, three limits:

```
for limit, per, pages in [(10, 5, 21), (30, 10, 204), (5000, 3600, 100_000)]:
    gap = per / limit * 1.1
    print(f"{limit:>5} per {per:>4} s -> one every {gap:.2f} s; "
          f"{pages:>7,} requests: {pages * gap:>7,.0f} s ({pages * gap / 3600:.1f}
```

```
   10 per    5 s -> one every 0.55 s;      21 requests:      12 s (0.0 h)
   30 per   10 s -> one every 0.37 s;     204 requests:      75 s (0.0 h)
 5000 per 3600 s -> one every 0.79 s; 100,000 requests:  79,200 s (22.0 h)
```

Twenty-one requests take twelve seconds; 100,000 requests at 5,000 an hour take 22 hours. Know that before you press run.

GUIDED LAB

Lab 6.5-A · Stock check under a rate limit. Open `labs/module-18/l6_5-rate-limiting/stock_check_starter.py`, with the API restarted as `python data/mock_api.py --rate 10 --per 5`. TODO 1: read the limit and pick a pace; TODO 2: read `Retry-After`; TODO 3: plan B; TODO 4: plan A; TODO 5: one sentence.

Lab 6.5-B · Retry and cache (if you finish, or at home): `labs/module-18/l6_5-rate-limiting/retry_and_cache_starter.py`. No network? `LABS_OFFLINE=1` replays saved answers on a pretend clock.

◆ PREDICT, THEN RUN

Write your prediction on the line **before** you run the code.

1. Predict True or False for each status.

```
for status in [400, 401, 404, 429, 500, 503]:
    print(status, status in RETRY_STATUSES, end=" | ")
print()
```

2. Same parameters in a different order, then a different spelling. Predict both comparisons.

```
a = cache_path(f"{API}/products", {"category": "Tents", "per_page": 5})
b = cache_path(f"{API}/products", {"per_page": 5, "category": "Tents"})
c = cache_path(f"{API}/products", {"category": "tents", "per_page": 5})
print(a == b, a == c)
```

◆ SELF-CHECK (ANSWERS ARE IN YOUR INSTRUCTOR'S KEY)

Q7

A response carries `X-RateLimit-Remaining: 0` and `X-RateLimit-Reset: 3`. What should your next request do?

A) Go at once: the request itself is valid

B) Switch to a second API key

C) Wait about 3 seconds, until the window starts over

D) Stop the job: the limit is used up for the day

Q8

What does this print? (`CaseInsensitiveDict` is the type of `resp.headers`.)

```
print(int(requests.utils.CaseInsensitiveDict({"retry-after": "3"}).get("Retry-
```

Q9

What does this print?

```
print([min(2.0, 0.2 * 2 ** n) for n in range(5)])
```

Q10

The demo's `Retry` policy has `allowed_methods=["GET", "HEAD"]`. Why not retry POST too?

A) Repeating a POST can repeat its effect, such as placing an order twice

B) urllib3 cannot retry POST

C) POST requests never fail with 503

D) A POST is always faster

Q11

A cache function writes `resp.text` to disk and only then calls `check(resp)`. One request gets a 429. What happens on every later run?

A) Nothing: the cache refreshes itself

B) The next run raises `FileNotFoundError`

C) The 429 is retried automatically

D) The saved error body is served from the cache, as if it were data

Q12

A job is slow because the API allows 10 requests per 5 seconds per client. A colleague suggests running three copies of the script at once. What happens?

A) It finishes three times faster

B) The limit applies to you, so the copies hit 429 three times as often and the job is no faster

C) Each copy gets its own 10 per 5 seconds

D) The server queues the extra requests politely

CHECKPOINT 2: TASK SHEET

CP2 Checkpoint 2: Do the API and the Database Agree? 15 min

1. **Time:** 15 minutes. **Allowed:** the lessons, your own notebooks and notes. **Not allowed:** a neighbor's file, or anyone else's code. Work alone.
2. **File:** `labs/module-18/checkpoint-2/checkpoint2_starter.py`. It already runs: setup, the session and the mode are done for you.
3. **Question from Finance:** for orders with status `returned` placed on or after 2025-11-01, do the company database (`data/pinecrest.db`) and the orders API agree?
4. **Part A (database):** open `pinecrest.db` read-only and run ONE SQL query with parameters (no values glued into the SQL text). Print the number of orders and their total value.
5. **Part B (API):** read the key from the environment (never type it), send it in the header the API asks for, follow `next` to the last page, and check the status of every response. Print the number of pages, the number of orders and their total value.
6. **Part C (answer):** are the order ids the same in both sources? Are the totals? Print ONE sentence that answers Finance.
7. **Hand in:** the file with its printed output. A partial answer with an honest sentence about what is missing is worth more than silence.

What good looks like: a read-only connection and one parameterized query; the key from the environment and every page of the API; numbers from both sources; and a sentence a person in Finance could act on.

My answer in one sentence:

NOTES

HOMEWORK CHECKLIST

- ☐ Lesson 6.4: finish the December Returns Brief (online 6.6%, store 4.0% returned), the exercises, quiz and journal.
- ☐ Lesson 6.5: finish the Resumable Order Pull (2,056 orders, $367,810.59), the exercises, quiz and journal.
- ☐ Finish any kit lab you did not finish in the room, and Checkpoint 2 if your answer was partial.
- ☐ **For Session 19:** stop the practice API; run `python -c "import seaborn, plotly; print(seaborn.__version__, plotly.__version__)"`.
- ☐ **For Session 19:** reopen a Matplotlib notebook and remind yourself how `fig, ax = plt.subplots()` works.

GLOSSARY

API key: a long secret string that identifies your program to an API; sent in a header such as `X-API-Key`.

Environment variable: a named setting of the terminal session (`PINECREST_API_KEY`), read with `os.environ`.

401 Unauthorized: no valid credential: missing, malformed or expired.

403 Forbidden: the credential was understood and refused.

POST: the method that sends data in the request body; `json=` encodes a dictionary as JSON.

Bearer token: a short-lived credential from a login, sent as `Authorization: Bearer <token>`.

`ApiError`: the lesson's exception carrying `status`, `code` and `message` from the API's error body.

Rate limit: the owner's rule for how many requests one client may send per window.

429 Too Many Requests: over the limit; `Retry-After` says how long to wait.

Throttle: an object that keeps requests at least a set interval apart.

Exponential backoff: waiting longer after each failure (0.5, 1, 2, 4 s...), up to a cap.

Jitter: randomness added to backoff so clients do not retry in waves.

Cache: a saved copy of a response, used instead of asking again.

Resumable job: a download that saves each piece as it arrives and skips finished pieces on restart.

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 19: Seaborn and Plotly

Your working pages for the session where charts start doing statistics, and then start answering back. First seaborn: one call draws a grouped chart, with means and intervals built in. Then Plotly: a chart your reader can hover, zoom and filter, saved as one HTML file.

Lessons 7.1–7.2 | Session 19 · 3 hours | Name: _____

HOW TO USE THIS WORKBOOK

TWO LESSONS, TWO HABITS

Each lesson below has your goals, the key ideas to keep, one worked example with its real output, the guided lab, two "predict, then run" items, space for notes, a six-question self-check and the journal prompt. Starter files are in `labs/module-19/`. Today's two habits: **in seaborn, ask what the bar shows; in Plotly, aggregate first.**

SESSION GOALS

- ☐ I can draw a grouped chart with one seaborn call and say which kind of function I used
- ☐ I can say what an error bar shows and change it with `errorbar=`
- ☐ I can build a Plotly Express figure from an aggregated table and design its hover
- ☐ I can save a chart as one HTML file and say what data is inside it
- ☐ I started the Store Performance Brief (7.1) and the Sales Explorer (7.2)

LESSONS

7.1 Statistical Graphics with Seaborn

BY THE END YOU CAN

- Draw scatter, line, histogram, ECDF, box, bar and point charts from a tidy table with one call each
- Tell an axes-level function from a figure-level one and pick the right one
- Split a chart with `hue`, `col` and `row` instead of a loop
- Say what an error bar or band shows, and set it with `errorbar=`
- Finish a seaborn chart with Matplotlib

KEY IDEAS

- **Seaborn sits on Matplotlib.** Every seaborn chart is a Matplotlib figure. You name the table and the roles: `sns.something(data=table, x="col", y="col", hue="col")`.
- **Tidy in, chart out.** One row per observation, one column per variable. If a chart is hard to describe, reshape with `melt` first.
- **Axes-level** functions (`scatterplot`, `lineplot`, `histplot`, `ecdfplot`, `boxplot`, `barplot`, `pointplot`, `heatmap`) take `ax=` and return that Axes. Use them inside your own `plt.subplots()` layout.
- **Figure-level** functions (`relplot`, `displot`, `catplot`) make their own figure, build panels from `col=` and `row=`, and return a FacetGrid. Size them with `height=` and `aspect=`.
- **Seaborn aggregates.** `lineplot`, `barplot` and `pointplot` draw the mean of the rows that share an x, plus a bootstrapped 95% confidence interval. A band you did not expect means several rows per x.
- **Two kinds of error bar.** Spread of the data (`"sd"`, `("pi", 50)`) or uncertainty of the estimate (`("ci", 95)`, `"se"`). Uncertainty shrinks as the sample grows; spread does not. Always say which one you drew.
- **Order is your job.** Categories appear in first-seen order. Pass `order=`, `hue_order=`, `col_order=`.
- `sns.set_theme(style="whitegrid", palette="colorblind")` once, at the top. Pass `seed=` to make bootstrapped bars repeatable.

You want to...	Write
One color per group	`hue="store"`
One panel per group	`sns.relplot(..., col="store", col_wrap=2)`

A chart inside my own layout	`sns.barplot(..., ax=axes[0])`
Bars that show spread	`errorbar="sd"`
No bars at all	`errorbar=None`
A title on a FacetGrid	`g.figure.suptitle("...")` or `g.set_titles("{col_name}")`
Find out what I am holding	`print(type(obj).__name__)`

WORKED EXAMPLE: ONE LINE, TWO BANDS

The weather table has 731 days for each of the four store cities. The same `lineplot` call is drawn twice; only `errorbar=` changes. The printed table is July in numbers.

```
weather = pd.read_csv("data/store_weather.csv", parse_dates=["date"])
weather["month"] = weather["date"].dt.month
fig, axes = plt.subplots(1, 2, figsize=(10, 3.6), sharey=True, layout="constrained
sns.lineplot(data=weather, x="month", y="high_f", hue="city",
             errorbar=("ci", 95), seed=42, ax=axes[0])
axes[0].set_title("Band: 95% CI of the mean", loc="left")
sns.lineplot(data=weather, x="month", y="high_f", hue="city",
             errorbar="sd", legend=False, ax=axes[1])
axes[1].set_title("Band: mean +/- 1 standard deviation", loc="left")
fig.savefig("highs_two_bands.png", dpi=150)
july = weather[weather["month"] == 7].groupby("city")["high_f"]
july = july.agg(n="count", mean="mean", sd="std", se="sem")
print(july.round(2))
```

```
             n   mean    sd    se
city
Asheville   62  82.81  7.48  0.95
Bend        62  82.21  7.51  0.95
Boise       62  92.00  8.73  1.11
Flagstaff   62  83.21  6.94  0.88
```

Read July in the table. Each city has 62 July days. The standard deviation is about 7 to 9 degrees: that is how much one July day differs from another, and it is the width of the right-hand band. The standard error is about 1 degree: that is how well we know the July average, and it is why the left-hand band is almost too thin to see. Same data, same line, two honest bands that answer different questions.

GUIDED LAB

Lab 7.1-A: open `labs/module-19/l7_1-seaborn/tier_order_sizes_starter.py`. Run it once as it is, then work through TODO 1 to 6. You will draw two seaborn charts side by side and decide whether Gold members place bigger orders or more of them. Charts are saved in an `output` folder next to the lab file.

Lab 7.1-B: open `labs/module-19/l7_1-seaborn/website_by_age_starter.py`. The same means with two kinds of error bar, a hand check of the numbers, and a figure-level chart split by shopper type. Finish it at home if the room runs out of time.

When a lab is done, its last printed lines match the "Expected when you finish" note at the top of the file.

PREDICT, THEN RUN

P1

Write what you expect, then run it. Were you right? If not, what did you miss?

```
g = sns.catplot(data=sales, x="store", y="visitors", col="year", kind="bar", h
print(type(g).__name__, g.axes.shape)
```

P2

Write what you expect, then run it. Were you right? If not, what did you miss?

```
print(sales.groupby("store")["revenue"].agg(["std", "sem"]).round(0))
```

MY NOTES

SELF-CHECK

Q1

Which of these is a figure-level seaborn function?

A) `sns.histplot`

B) `sns.boxplot`

C) `sns.displot`

D) `sns.heatmap`

Q2

What does this print?

```
ax = sns.histplot(data=sales, x="revenue")
print(type(ax).__name__)
```

A) `Axes`

B) `FacetGrid`

C) `Figure`

D) `Histogram`

Q3

What does this print?

```
g = sns.relplot(data=sales, x="visitors", y="revenue",
                col="store", col_wrap=2, height=2)
print(g.axes.shape)
```

Q4

`sales` has about 60 rows for each store and month. What does `sns.lineplot(data=sales, x="month", y="revenue", hue="store", errorbar=None)` draw?

A) A zigzag through every daily row, in file order

B) One line per store through the monthly means, with no band

C) One line per store through the monthly totals

D) Nothing: it raises an error because `x` has repeated values

Q5

A chart shows the mean staff rating (1 to 5) for four stores, about 200 answers each. Every bar runs from roughly 3.2 to 5.0. What do the bars most likely show?

A) A 95% confidence interval for each mean

B) One standard error either side of each mean

C) The lowest and highest rating given

D) One standard deviation either side of each mean

Q6

What does this print? (Two lists.)

```
t = pd.DataFrame({"size": ["M", "S", "L", "S"], "n": [3, 1, 2, 5]})
fig, ax = plt.subplots()
sns.barplot(data=t, x="size", y="n", errorbar=None, ax=ax)
print([lab.get_text() for lab in ax.get_xticklabels()],
      [float(b.get_height()) for b in ax.patches])
```

◆ JOURNAL PROMPT (FROM THE LESSON)

Think of a chart with error bars that you have seen in a news article, a report or a presentation. Did it say what the bars were? Write two sentences on how your reading of that chart changes depending on whether

they showed spread or uncertainty.

7.2 Interactive Charts with Plotly

BY THE END YOU CAN

- Build line, scatter, bar, box and heatmap charts with one Plotly Express call each
- Design what a reader sees on hover
- Finish a figure with `update_layout`, `update_xaxes`, `update_yaxes` and `update_traces`
- Add reference lines, shaded regions and annotations
- Save a standalone HTML file and say what data is inside it

KEY IDEAS

- **A figure is a description:** traces (the data) plus a layout (titles, axes, legend). Python builds it; plotly.js draws it in a browser. Inspect it: `fig.data[0].type`, `len(fig.data)`, `fig.layout.title.text`.
- **Nothing shows until you ask:** `fig.show()` in a notebook, or `fig.write_html("chart.html")` and open the file.
- **Plotly Express follows the seaborn pattern** with different words (table below).
- **`px.bar` and `px.line` do not aggregate.** They draw one mark per row. Summarize with `groupby` first, then plot the summary.
- **Design the hover.** `hover_name=` is the bold first line; `hover_data={"revenue": ":$,.0f"}` adds and formats columns; `hovertemplate` gives full control; `hovermode="x unified"` suits time series.
- **The title is still the message.** Most readers look at the default view for a few seconds.
- **`write_html` choices:** `include_plotlyjs=True` (default, about 4.7 MB, works offline) or `"cdn"` (a few KB, reader must be online).
- **The data is in the file.** Every x, y and hover column is readable text inside the HTML. Aggregate first and pass only the columns the chart needs.

seaborn	Plotly Express
`data=table`	first argument
`hue=`	`color=`
`col=`, `row=`, `col_wrap=`	`facet_col=`, `facet_row=`, `facet_col_wrap=`
`order=`, `hue_order=`	`category_orders={"col": [...]}`

axis labels set on the Axes	`labels={"col": "Nice name"}`
(no hover)	`hover_name=`, `hover_data=`
`barplot` averages for you	`px.bar` never does: aggregate first

◆ WORKED EXAMPLE: AGGREGATE FIRST, THEN ONE CALL

How do people pay, online and in the stores? The orders table has 20,302 rows; the chart needs eight numbers. Count with pandas, turn counts into shares within each channel, and hand Plotly that small table.

```
orders = pd.read_csv("data/orders.csv")
mix = orders.groupby(["channel", "payment_method"], as_index=False).size()
mix["share"] = mix["size"] / mix.groupby("channel")["size"].transform("sum")
print(mix.round(3).to_string(index=False))
methods = ["card", "digital_wallet", "gift_card", "cash"]
fig = px.bar(mix, x="payment_method", y="share", color="channel", barmode="group",
             text_auto=".0%", category_orders={"payment_method": methods},
             labels={"share": "Share of orders", "payment_method": ""},
             title="Cash is a store-only habit; wallets are an online one")
fig.update_yaxes(tickformat=".0%")
fig.write_html("payment_mix.html", include_plotlyjs="cdn")
print(len(fig.data), "traces |", [len(t.x) for t in fig.data], "bars each")
```

```
channel payment_method  size  share
 online           card  7953  0.619
 online digital_wallet  3876  0.302
 online      gift_card  1025  0.080
  store           card  4439  0.596
  store           cash  1614  0.217
  store digital_wallet   918  0.123
  store      gift_card   477  0.064
2 traces | [3, 4] bars each
```

Seven rows went in (cash does not exist online), so the figure has two traces with three bars (online) and four (store). The file `payment_mix.html` holds those seven shares and nothing else: no order ids, no customers. `text_auto=".0%"` prints each share on its bar, and `category_orders` puts the methods in the order you chose instead of the order of the rows.

◆ GUIDED LAB

Lab 7.2-A: open `labs/module-19/l7_2-plotly/product_map_starter.py`. Run it once, then do TODO 1 to 5: a scatter of price against rating for the catalog, with a hover that names each product. The lab writes `output/product_map.html`; open that file from your file manager and explore it. Do not add `fig.show()` to the lab file.

Lab 7.2-B: open `labs/module-19/l7_2-plotly/promo_share_trend_starter.py`. Aggregate orders by week, draw the share that used a promo code, unify the hover and shade the six sale weekends.

If your room has no internet, write the HTML with `include_plotlyjs=True` instead of `"cdn"`.

PREDICT, THEN RUN

P1

Write what you expect, then run it. Were you right? If not, what did you miss?

```
fig = px.line(monthly, x="month_start", y="revenue", facet_col="store")
print(len(fig.data), len(fig.layout.annotations), fig.layout.annotations[0].te
```

P2

Write what you expect, then run it. Were you right? If not, what did you miss?

```
fig = px.histogram(sales, x="store", y="revenue", histfunc="avg")
print(fig.data[0].type, len(fig.data[0].x))
```

MY NOTES

SELF-CHECK

Q7

You run `fig = px.line(monthly, x="month_start", y="revenue", color="store")` in a script and nothing appears. Why?

A) The call only builds a Figure object; it is displayed when you call `fig.show()` (or write it to a file)

B) `px.line` needs `kind="line"` before it draws

C) Plotly needs an account before it can display a chart

D) `monthly` has more than one row per month, so Plotly refuses to draw

Q8

`sales` has four stores and `promo_day` is 0 or 1. What does this print?

```
fig = px.scatter(sales, x="visitors", y="revenue",
                 color="store", facet_col="promo_day")
print(len(fig.data))
```

Q9

Which `hover_data` entry shows revenue as `$27,677` in the hover label?

A) `"revenue": True`

B) `"revenue": "$%d"`

C) `"revenue": ":$,.0f"`

D) `"revenue": ":.2%"`

Q10

What does `fig.update_layout(hovermode="x unified")` change on a line chart with four stores?

A) Zooming one panel now zooms every panel

B) One hover box appears for the x position under the mouse, listing all four stores

C) The four lines are merged into one trace

D) The x axes of all facets are forced to the same range

Q11

What does this print?

```
totals = sales.groupby("store", as_index=False)["revenue"].sum()
fig = px.bar(totals, x="store", y="revenue")
print(fig.data[0].type, len(fig.data[0].x))
```

Q12

You will email one chart to a manager who is online. Your table has one row per customer, including an email column the chart does not use. What is the right way to prepare the file?

A) `fig.write_html("chart.html")` from the full table: unused columns are left out automatically

B) `fig.write_image("chart.html")`, so that no data is included

C) Put `email` in `hover_data` so the manager can check individual customers

D) Aggregate to what you mean to publish, pass only the columns the chart needs, then `fig.write_html("chart.html", include_plotlyjs="cdn")`

◆ JOURNAL PROMPT (FROM THE LESSON)

Open your sales_explorer.html and spend three minutes exploring it as if you had never seen the data. What did you find by hovering or zooming that the static store brief from Lesson 7.1 could not have shown you?

And what did the static brief say more clearly?

NOTES

HOMEWORK CHECKLIST

- ☐ Lesson 7.1 project finished: `store_brief.png` and three sentences for the manager
- ☐ Lesson 7.2 project finished: `sales_explorer.html` opens and shows three charts; I opened it on a phone
- ☐ Lesson 7.1 Exercises 1 to 4 and Lesson 7.2 Exercises 1 to 3 (the last exercise of each is a stretch)
- ☐ Both lesson quizzes and both journal entries
- ☐ The kit lab I did not finish in class
- ☐ I read the four capstone questions in Lesson 8.1 and wrote one sentence about the one I am leaning toward
- ☐ For Session 20: `python -c "import bokeh, streamlit; print(bokeh.__version__, streamlit.__version__)"` prints two versions

GLOSSARY

- **Tidy (long) table:** one row per observation, one column per variable: the shape seaborn and Plotly Express expect.
- **Semantic mapping:** giving a column a role in the chart: x, y, hue (color), size, style.
- **Axes-level function:** a seaborn function that draws on one Matplotlib Axes (takes `ax=`, returns the Axes).
- **Figure-level function:** `relplot`, `displot` or `catplot`: makes its own figure and returns a FacetGrid.

- **FacetGrid:** seaborn's object for a grid of panels; `g.figure` and `g.axes` reach the Matplotlib parts.
- **Small multiples:** the same chart repeated once per group with shared axes.
- **Estimator:** the statistic a chart draws for each group; the mean unless you change it.
- **Confidence interval:** a range for an estimate such as a mean; it narrows as the sample grows.
- **Bootstrap:** resampling your own rows with replacement many times to see how much an estimate moves.
- **ECDF:** empirical cumulative distribution: the share of values at or below each x; no bins to choose.
- **Trace:** one data series in a Plotly figure (a line, a set of bars, a set of points).
- **Layout:** everything in a Plotly figure that is not data: title, axes, legend, shapes, annotations.
- **Hover template:** the small text pattern, with placeholders such as `%{y:$,.0f}`, that fills the hover label.
- **CDN:** a public server the reader's browser downloads plotly.js from when the file was written with `include_plotlyjs="cdn"`.
- **Kaleido:** the helper package `write_image` uses to turn a Plotly figure into a PNG, SVG or PDF; it needs Chrome.

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 20: Bokeh and Streamlit

Your working pages for the last tools session. First Bokeh: several charts that look at one table, so a selection in one shows up in all. Then Streamlit: an app whose controls run your pandas code. At the end you choose your capstone question and write your plan.

Lessons 7.3–7.4 | Session 20 · 3 hours | Name: _____

HOW TO USE THIS WORKBOOK

TWO LESSONS AND A DECISION

Each lesson has your goals, the key ideas, a worked example with real output, the guided lab, two "predict, then run" items, a self-check and the journal prompt. The last section is new: the **capstone kickoff**, where you choose your question and write your plan. Starter files are in `labs/module-20/`. Today's two sentences: **share a source, share the rows** and **the whole script reruns**.

SESSION GOALS

- ☐ I can build a Bokeh figure from a `ColumnDataSource` with a formatted hover
- ☐ I can link two plots by a range and by a source, and say which link does what
- ☐ I can explain what happens when a reader changes a Streamlit widget
- ☐ I can keep a value across reruns and cache a slow step
- ☐ I chose my capstone question and wrote a one-paragraph plan

LESSONS

7.3 Interactive Charts with Bokeh

BY THE END YOU CAN

- Build a figure by adding glyphs and save it with `output_file` and `save`
- Put data in a `ColumnDataSource` and name its columns in glyph calls
- Write a hover tool with formatted fields
- Link plots by a shared range (panning) and by a shared source (brushing)
- Add a dropdown or slider that works in a plain HTML file

KEY IDEAS

- **Two moves:** `p = figure(...)` makes an empty plot; `p.line`, `p.scatter`, `p.vbar` add glyphs. Each glyph call adds one renderer.
- **Everything is an object with properties:** `p.yaxis.formatter`, `p.legend.location`, `p.title.text`.
- **`output_file(name)` then `save(p)`** writes the HTML; `show(p)` also opens it in a browser.
- **A `ColumnDataSource` is the table behind the page.** With `source=source`, a string argument is a column name; a number is the same value for every mark.
- **Hover:** `("Revenue", "@revenue{$0,0}")`. `@name` is this row's value; the format goes in braces. Dates need `formatters={"@date": "datetime"}`.
- **Share a range, share the view:** `figure(x_range=top.x_range)`. **Share a source, share the rows:** pass the same source variable to both glyphs and add `box_select`.
- **In a saved file only JavaScript can react:** `js_link` copies a widget value into a property; `CustomJS` runs a few lines. A Python callback needs a server.
- **Every column of every source travels in the file.** Build the source from a table that holds only what the page needs. A new page needs new objects (a source belongs to one document).

You want to...	Write
Dollars on an axis	`p.yaxis.formatter = NumeralTickFormatter(format="$0,0")`
Color by a category column	`color=factor_cmap("store", palette=..., factors=...)`

Plots side by side / stacked	`row(a, b)` / `column(a, b)`
A selection that stands out	`nonselection_alpha=0.15, selection_color="#d97706"`
Show only one group	`view=CDSView(filter=GroupFilter(column_name="store", group="Boise"))`
A file that works offline	`output_file(name, mode="inline")`
Find a blank plot's cause	`print(source.column_names)` and read the terminal for E-1001

WORKED EXAMPLE: A SOURCE, BARS AND A FORMATTED HOVER

Asheville is the rainy store city. Summarize two years of weather to twelve rows, put them in a source, and draw bars whose hover shows the share as a percentage. The source holds three columns besides the index, and nothing else travels in the file.

```
import pandas as pd
from bokeh.models import ColumnDataSource, NumeralTickFormatter
from bokeh.plotting import figure, output_file, save

weather = pd.read_csv("data/store_weather.csv", parse_dates=["date"])
ash = weather[weather["city"] == "Asheville"].copy()
ash["month"] = ash["date"].dt.strftime("%b")
ash["rainy"] = ash["precip_in"] >= 0.10
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun",
          "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]
table = ash.groupby("month").agg(high=("high_f", "mean"), rainy=("rainy", "mean"))
table = table.loc[months].reset_index()
source = ColumnDataSource(table)
tips = [("Month", "@month"), ("Rainy days", "@rainy{0%}"),
        ("Mean high", "@high{0} F")]
p = figure(width=700, height=300, x_range=months, tools="save", tooltips=tips,
           title="Asheville: July is the rainiest month")
p.vbar(x="month", top="rainy", width=0.8, source=source, color="#667eea")
p.yaxis.formatter = NumeralTickFormatter(format="0%")
p.y_range.start = 0
output_file("asheville_rain.html", title="Asheville rain")
save(p)
print(source.column_names, "|", len(p.renderers), "renderer,", len(table), "bars")
print(table.round(2).iloc[[0, 5, 6, 7, 11]].to_string(index=False))
```

```
['index', 'month', 'high', 'rainy'] | 1 renderer, 12 bars
month  high  rainy
  Jan 47.87   0.21
  Jun 84.33   0.28
  Jul 82.81   0.39
  Aug 83.39   0.34
  Dec 51.32   0.19
```

One `vbar` call drew twelve bars and added one renderer. The strings `"month"` and `"rainy"` are column names in the source; `width=0.8` is a plain number, the same for every bar. The tooltip reads `@rainy{0%}` from the same row. Open `asheville_rain.html` and hover July: 39% of July days had at least a tenth of an inch of rain.

◆ GUIDED LAB

Lab 7.3-A: open `labs/module-20/l7_3-bokeh/margin_brush_starter.py`. Run it once, then do TODO 1 to 6: two scatter plots of the product catalog that share one source. The lab writes `output/margin_brush.html`: open it from your file manager, drag a box in the left plot and watch the right one. Use `save`, not `show`, in the lab file.

Lab 7.3-B: open `labs/module-20/l7_3-bokeh/category_picker_starter.py`. A dropdown that filters the chart inside the HTML file, a slider that moves a reference line, and a counter the callback rewrites. The numbers your page should show are printed at the end of the starter.

If a callback does nothing, press F12 in the browser and read the Console tab: JavaScript errors appear there.

◆ PREDICT, THEN RUN

P1

Write what you expect, then run it. Were you right? If not, what did you miss?

```
p = figure(width=300, height=200)
p.vbar(x=[1, 2, 3], top=[4, 5, 6], width=0.8)
p.line([1, 2, 3], [4, 5, 6])
print(len(p.renderers), [type(r.glyph).__name__ for r in p.renderers])
```

P2

Write what you expect, then run it. Were you right? If not, what did you miss?

```
a = ColumnDataSource(pd.DataFrame({"x": [1, 2, 3]}))
b = ColumnDataSource(pd.DataFrame({"x": [1, 2, 3]}))
a.selected.indices = [0, 2]
print(a.selected.indices, b.selected.indices, a is b)
```

MY NOTES

SELF-CHECK

Q1

A glyph is drawn with `p.scatter("visitors", "revenue", source=source, size="visitors")`. What does `size="visitors"` do?

A) Raises an error: a size must be a number

B) Reads each marker's size from the `visitors` column of the source

C) Writes the word visitors next to each marker

D) Sets every marker to the average number of visitors

Q2

What does this print?

```
p = figure(width=300, height=200)
for store in ["Boise", "Bend"]:
    p.line([1, 2], [3, 4], legend_label=store)
    p.scatter([1, 2], [3, 4], legend_label=store)
print(len(p.renderers), len(p.legend.items))
```

Q3

Two stacked time-series plots draw different columns. You want panning or zooming either one to move both along the date axis. What do you write?

A) Put both plots in one `gridplot`

B) Give both plots the same `title`

C) Build each plot from its own copy of the source

D) Create the second plot with `x_range=top.x_range`

Q4

On one page you wrote `ColumnDataSource(df)` twice, once for each plot. A box selection in the left plot changes nothing in the right one, and there is no error. Why?

A) The selection is stored in a source, and the two plots have two separate sources

B) Selections only link plots that also share an `x_range`

C) `box_select` works on one plot per page

D) The DataFrame was not sorted the same way in both plots

Q5

What does this print?

```
source = ColumnDataSource(pd.DataFrame({"x": [1, 2, 3], "y": [4, 5, 6]}))
print(source.column_names, len(source.data["x"]))
```

Q6

Which Bokeh tooltip shows a row's revenue as `$12,345`?

A) `("Revenue", "$revenue")`

B) `("Revenue", "@revenue:$,.0f")`

C) `("Revenue", "@revenue{$0,0}")`

D) `("Revenue", "%{revenue}")`

◆ JOURNAL PROMPT (FROM THE LESSON)

Use your store explorer for five minutes on a store other than Boise. Write down one thing you noticed by brushing that you had not seen in any earlier lesson's chart, and one question it raised that the page cannot answer.

7.4 Dashboards with Streamlit

BY THE END YOU CAN

- Write an app as a plain script and start it with `streamlit run`
- Explain the rerun model and predict what a variable holds after a click
- Keep values with `st.session_state` and avoid repeated work with `@st.cache_data`
- Lay out a page with a sidebar, columns and tabs
- Test an app without a browser, and say what sharing one takes

KEY IDEAS

- **An app is a script.** Start it with `streamlit run app.py` from the folder that holds `data/`. Stop it with Ctrl+C. Not `python app.py`, and not in a notebook.
- **The whole script reruns** from the top every time a reader changes a widget. Ordinary variables start over. Widgets keep their values.
- **Showing and asking.** `st.title`, `st.metric`, `st.dataframe`, `st.plotly_chart` show things. `st.selectbox`, `st.multiselect`, `st.slider`, `st.date_input`, `st.toggle` ask, and return the answer.
- **`st.button` is True for one rerun only.** It is not a switch.
- **`st.session_state`** is a dictionary kept per browser tab across reruns. A widget with `key="x"` lives there as `st.session_state["x"]`. Change a widget's value only in a callback (`on_click=`).
- **`@st.cache_data`** stores a function's result per set of arguments. Use it for slow loading that does not depend on widgets. The cache is shared by all users.
- **`st.stop()`** ends the current run after a message: for an empty selection or a half-chosen date range.
- **Sharing** means a Python process kept running where readers can reach it, with the packages, the data, access control and no secrets in the code.

Where a value lives	How long it lasts
an ordinary variable	one run of the script
a widget's value	across reruns, in that browser tab
`st.session_state["x"]`	across reruns, in that browser tab
a `@st.cache_data` result	for every user, until the app restarts or the cache is cleared

Filters / side by side / views	`with st.sidebar:` / `st.columns(3)` / `st.tabs([...])`
Test without a browser	`AppTest.from_file("app.py").run()` then `assert not at.exception`

WORKED EXAMPLE: A SMALL APP AND ITS TEST

How many days count as rainy depends on where you draw the line. This app lets the reader choose the city and the line. The loading is cached; the filtering is not, so it always reflects the widgets.

File `rain_app.py`:

```
import pandas as pd
import streamlit as st

@st.cache_data
def load_weather(path):
    return pd.read_csv(path, parse_dates=["date"])

weather = load_weather("data/store_weather.csv")
city = st.selectbox("City", ["Boise", "Bend", "Flagstaff", "Asheville"])
inches = st.slider("A rainy day has at least this many inches",
                   0.05, 1.00, 0.10, step=0.05)
one = weather[weather["city"] == city]
rainy = one["precip_in"] >= inches
st.metric("Rainy days", f"{rainy.mean():.0%}",
          help="Share of the 731 days in 2024 and 2025")
st.caption(f"{int(rainy.sum())} of {len(one)} days in {city}")
```

Check it without a browser:

```
from streamlit.testing.v1 import AppTest

def report():
    print(at.selectbox[0].value, at.slider[0].value, "->", at.metric[0].value,
          "|", at.caption[0].value)

at = AppTest.from_file("rain_app.py", default_timeout=30).run()
report()
at.selectbox[0].select("Asheville").run()
report()
at.slider[0].set_value(0.5).run()
report()
```

```
Boise 0.1 -> 9% | 68 of 731 days in Boise
Asheville 0.1 -> 24% | 172 of 731 days in Asheville
Asheville 0.5 -> 6% | 44 of 731 days in Asheville
```

Three runs of the script, three different pages. `AppTest` ran the script in memory, changed a widget and ran it again, exactly as a reader's click would. Boise is rainy on about one day in ten at the default line; Asheville on about one in four; raise the line to half an inch and Asheville's share falls to 6%. No value was carried from one run to the next except the widgets' own.

GUIDED LAB

Lab 7.4-B (first): open `labs/module-20/l7_4-streamlit/gear_shortlist_starter.py`. This file is an app and its own test. `streamlit run gear_shortlist_starter.py` opens it; `python gear_shortlist_starter.py` checks it without a browser. Before you change anything, predict what the check prints. Then do TODO 1 to 5 so the shortlist remembers.

Lab 7.4-A: open `labs/module-20/l7_4-streamlit/catalog_browser_starter.py`. A catalog browser for store staff: sidebar filters with keys, a reset callback, a friendly message when nothing matches, three metrics, a table and a download. Run the check after each TODO; it tells you how far you have got.

Keep the app open in a browser tab with Always rerun switched on, and stop it with Ctrl+C when you finish.

PREDICT, THEN RUN

P1

Write what you expect, then run it. Were you right? If not, what did you miss?

```
at = AppTest.from_string("""
import streamlit as st
total = st.session_state.get("total", 0)
if st.button("Add 10"):
    total += 10
st.write("Total:", total)
""").run()
for _ in range(3):
    at.button[0].click().run()
print(at.markdown[-1].value)
```

P2

Write what you expect, then run it. Were you right? If not, what did you miss?

```
at = AppTest.from_string("""
import streamlit as st
@st.cache_data
def square(n):
    st.session_state["runs"] = st.session_state.get("runs", 0) + 1
    return n * n
answers = [square(3), square(3), square(4)]
st.write(st.session_state["runs"])
""").run()
print(at.markdown[0].value)
```

MY NOTES

SELF-CHECK

Q7

Line 5 of your app is `sales = pd.read_csv(...)` on a 200 MB file, with no decorator. A reader opens the page and then ticks a checkbox 10 times. How many times is the file read?

A) 1

B) 11

C) 10

D) 0

Q8

What does this print? (The check clicks the button twice.)

```
at = AppTest.from_string("""
import streamlit as st
if "n" not in st.session_state:
    st.session_state["n"] = 5
if st.button("Double"):
    st.session_state["n"] *= 2
st.write(st.session_state["n"])
""").run()
at.button[0].click().run()
at.button[0].click().run()
print(at.markdown[0].value)
```

Q9

A Reset button must put a slider (which has a key) back to 0. Where does the line that sets the slider's value belong?

A) In a function passed as `on_click=`, which Streamlit runs before the rerun starts

B) In an `if st.button("Reset"):` block placed after the slider

C) Inside the function decorated with `@st.cache_data`

D) Nowhere: a widget's value cannot be changed from code

Q10

Which step of a dashboard is the right one to wrap with `@st.cache_data`?

A) Filtering the table by the stores and dates the reader chose

B) The whole app, so that nothing ever runs twice

C) A function that draws the sidebar widgets

D) Loading and preparing the CSV file, which is slow and the same whatever the widgets say

Q11

What does this print? (Two numbers.)

```
at = AppTest.from_string("""
import streamlit as st
stores = st.multiselect("Stores", ["Boise", "Bend"], default=["Boise"])
if not stores:
    st.warning("Pick a store.")
    st.stop()
st.metric("Stores chosen", len(stores))
""").run()
at.multiselect[0].set_value([]).run()
print(len(at.warning), len(at.metric))
```

Q12

What does `st.button("Go")` return?

A) True from the first click onward

B) The number of times it has been clicked

C) True only during the one rerun caused by a click, otherwise False

D) The label of the button

◆ JOURNAL PROMPT (FROM THE LESSON)

You have now made four kinds of deliverable from the same sales data: a static figure (7.1), an interactive file (7.2), a linked explorer (7.3) and an app (7.4). For each, name one real person or situation it would suit best, and one where it would be the wrong choice.

8.1 The Pinecrest Report: choose and plan

THE FOUR QUESTIONS (READ THE FULL TEXT IN LESSON 8.1, CHOOSE YOUR QUESTION)

Q	Question	Main data
A	Why is Asheville falling behind?	daily_sales.csv (store_weather.csv, survey.csv, sales_2025.xlsx possibly useful)
B	Does it matter how we found the customer?	customers.csv, orders.csv
C	Does the welcome discount bring people back?	orders.csv (customers.csv possibly useful)
D	What did the March 2025 price increase do to sales?	order_items.csv, orders.csv, products.csv

Choose the one you are most curious about: curiosity will carry you through the tedious parts better than ease will. An honest "no clear difference" is a full answer and scores as well as a dramatic discovery.

MY QUESTION

MY ONE-PARAGRAPH PLAN

My question is ____ (A, B, C or D, in my own words), which matters because the decision is ____. I will use the tables ____ and one row of my analysis table will be ____. I will compare ____ with ____ using ____ (the measure, and the method that fits it). The comparison could be unfair because ____, and I will check that by ____. The chart I expect to make is ____, and it would change my mind if ____.

◆ PARTNER CHECK

Swap workbooks. Your partner completes this sentence about your plan: "From this plan, I predict your notebook will contain ..."

◆ BEFORE SESSION 21: CAPSTONE MILESTONES 1 TO 4

- ☐ **Milestone 1 · Question and plan (30 min).** The paragraph above becomes the first Markdown cell of `pinecrest_report.ipynb`. Done when someone else could read it and predict roughly what the notebook will contain.
- ☐ **Milestone 2 · Load and check (45 min).** For every file: shape, dtypes, date range, missing values, duplicate keys. Done when you can say what one row is and how many rows there are.
- ☐ **Milestone 3 · Clean and prepare (45 min).** One function returns your analysis table, with a row-count check around every merge and filter. Done when you can explain every row that was dropped.
- ☐ **Milestone 4 · Explore (75 min).** One variable, then pairs, then segments; a running list headed "What I have noticed". Done when you can say in two sentences what you expect the answer to be and what could still prove you wrong.
- ☐ Restart and Run All works on what I have so far.

NOTES

HOMEWORK CHECKLIST

- ☐ Capstone Milestones 1 to 4 (see the kickoff section): about 3 hours 15 minutes. Do these first

- ☐ Lesson 7.3 project: `store_explorer.html` opens by double-click and all three charts respond to a selection
- ☐ Lesson 7.4 project: `streamlit run dashboard.py` shows no red boxes, and the AppTest check runs clean
- ☐ Lesson exercises 7.3 (1 to 3) and 7.4 (1 to 3), both quizzes and both journal entries
- ☐ I read the rest of Lesson 8.1: the worked mini-example, the report template, the rubric, the five common mistakes
- ☐ For Session 21: my capstone notebook runs from a fresh kernel as far as I have got

GLOSSARY

- **Glyph:** a family of marks added to a Bokeh figure: `line`, `scatter`, `vbar`.
- **Renderer:** what one glyph call adds to a figure; `p.renderers` lists them.
- **ColumnDataSource (CDS):** the table behind a Bokeh page: a dictionary of equal-length columns.
- **Linked panning:** two plots that share a range object, so they always show the same view.
- **Linked brushing:** two plots that share a source, so a selection in one highlights the same rows in the other.
- **View (CDSView):** a filter between a source and a glyph that shows only some rows.
- **CustomJS:** a few lines of JavaScript attached to a widget or property, run in the reader's browser.
- **Rerun:** one top-to-bottom run of a Streamlit script, triggered by opening the page or changing a widget.
- **Widget:** a control (`selectbox`, `slider`, `button`) whose function returns the reader's input.
- **Session state:** `st.session_state`: a dictionary Streamlit keeps for one browser tab across reruns.
- **Callback:** a function passed as `on_click=` or `on_change=`; Streamlit runs it before the rerun.
- **Cache:** stored results of a function decorated with `@st.cache_data`, reused when the arguments repeat.
- **AppTest:** Streamlit's testing tool: runs an app in memory and lets code read the page and operate widgets.
- **Deploying:** keeping an app running on a machine other people can reach.

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 21: Capstone Studio and Showcase

Your pages for the last session: from a first finding to a tested answer, one figure and a three-minute showcase, then the week to finish the report.

Lesson 8.1 | Session 21 · 3 hours | Name: _____

HOW TO USE THIS WORKBOOK

> **TODAY IS YOURS**
>
> No new tools. You bring the capstone you started after Session 20, and today you take it from a first finding to a tested answer and one figure, then show it to the room. The pages below hold your check-ins, your main sentence, your three showcase sentences and the self-check.

SESSION GOALS

- ☐ My main result is one sentence with all five parts
- ☐ I tried to make my result disappear, and I wrote down what happened
- ☐ One figure carries my answer and is saved from code
- ☐ I presented my answer in three minutes
- ☐ My notebook runs with Restart and Run All

WHERE THE 420 MINUTES GO

When	Milestones	Min
Before Session 21	Milestones 1 to 4: question and plan (30), load and check (45), clean and prepare (45), explore (75)	195
In the room, Session 21	Milestone 5 test and quantify (45), Milestone 6 visualize (40 of 45), first draft of the Answer (5)	90
After Session 21, within one week	Finish Milestone 6 (5), the rest of Milestone 7 write the page (40), Milestone 8 review and reproduce (30)	75
Optional	A dashboard or interactive file, after the report is done	60

LESSON

8.1 The Pinecrest Report

BY THE END OF TODAY YOU CAN

- State your main result as one sentence with an estimate, an interval, a test and an effect size
- Try to break your own result with one robustness check
- Draw one figure that carries your answer on its own
- Present your answer in three minutes, and finish the report within a week

KEY IDEAS

- **Compare like with like.** Before you believe a difference, ask what else differs between the groups: the season, the time each customer had, the weekdays, the products.
- **Fit the method to the data.** Two proportions: a z-test with Cohen's h. A skewed amount: medians with a bootstrap interval, means alongside. Several groups: one overall test first, then a small number of planned pairs, adjusted.
- **Five parts in every claim:** the estimates, the difference, the 95% interval, the test, the effect size.
- **An honest null is a full answer.** Say what the interval rules out: "no difference larger than about X".
- **Words about cause** ("causes", "drives", "because") only where the design supports them.
- **One figure, one message.** A title that states the finding, axes with units, error bars named in the caption, saved from code.
- **If it does not rerun from a fresh start, it is not finished.**

WORKED EXAMPLE: A STRONG SENTENCE FOR A NULL RESULT

A practice question (not a capstone question): do customers who opted in to marketing recommend Pinecrest more? "Promoter" means a recommend score of 9 or 10.

```
from statsmodels.stats.proportion import confint_proportions_2indep, proportions_z

survey = pd.read_csv("data/survey.csv")
customers = pd.read_csv("data/customers.csv")
t = survey.merge(customers[["customer_id", "marketing_opt_in"]], on="customer_id",
                 validate="one_to_one").dropna(subset=["recommend"])
t["promoter"] = t["recommend"] >= 9
g = t.groupby("marketing_opt_in")["promoter"].agg(["sum", "count"])
yes, no = g.loc[True], g.loc[False]
p1, p0 = yes["sum"] / yes["count"], no["sum"] / no["count"]
z, p = proportions_ztest([yes["sum"], no["sum"]], [yes["count"], no["count"]])
low, high = confint_proportions_2indep(yes["sum"], yes["count"], no["sum"], no["co
h = 2 * np.arcsin(np.sqrt(p1)) - 2 * np.arcsin(np.sqrt(p0))
print(f"Promoters (recommend 9 or 10): {p1:.1%} of opted-in customers (n={yes['cou
      f"against {p0:.1%} of the others (n={no['count']}), a difference of {(p1 - p
      f"(95% CI {low * 100:+.1f} to {high * 100:+.1f}; z-test p = {p:.2f}; h = {h:
```

```
Promoters (recommend 9 or 10): 32.3% of opted-in customers (n=1020) against 31.3%
```

Every part is there: two rates with their group sizes, the difference, the interval, the test and the effect size. The interval runs from about 3.5 points lower to 5.4 points higher: the data cannot tell the groups apart, and they rule out a difference much larger than 5 points. That is the sentence a report would carry, word for word.

◆ GUIDED LAB (OPTIONAL)

Open `labs/module-21/l8_1-capstone/capstone_scaffold_starter.py`. Sections 1 to 4 of the capstone outline are filled in for a practice question; you write sections 5 to 9. Use it for its shape: the helpers `overview()`, `compare_rates()` and the one-function table build are yours to copy into your own notebook.

◆ STUDIO CHECK-INS: MY STATUS

Time	Status (green / amber / red) and one word
Studio 1 · 0:00	
Studio 1 · 0:15	
Studio 1 · 0:30	
Studio 2 · 0:00	
Studio 2 · 0:20	
Studio 2 · 0:35	

MY MAIN RESULT AS ONE STRONG SENTENCE

HOW I TRIED TO BREAK IT, AND WHAT HAPPENED

MY FIGURE: TITLE, AXES, CAPTION

Title (states the finding):

What the error bars are:

SHOWCASE: THREE SENTENCES

The question · the answer with its uncertainty · one specific caveat

QUESTIONS THE AUDIENCE GAVE ME (FOR NEXT QUESTIONS)

PREDICT, THEN RUN

P1

Write what you expect, then run it. Were you right?

```
first = pd.to_datetime(pd.Series(["2024-03-01", "2025-11-20"]))
print(((pd.Timestamp("2025-12-31") - first).dt.days >= 180).tolist())
```

P2

Write what you expect, then run it. Were you right?

```
sales = pd.read_csv("data/daily_sales.csv", parse_dates=["date"])
by_month = sales.groupby(sales["date"].dt.month)["revenue"].mean()
print(by_month.loc[[2, 8, 12]].round(0).to_dict())
```

SELF-CHECK

Q1

Which of these is a capstone question rather than a topic?

A) Asheville store performance

B) An exploration of customer acquisition channels

C) Did products repriced on 2025-03-01 sell fewer units, March to December, than comparable products whose price did not change?

D) Everything about the WELCOME10 code

Q2

You compare a strongly right-skewed amount (customer spend) between two groups of about 500 people each. Which summary and interval suit it best?

A) The difference in medians with a bootstrap 95% confidence interval, reported next to the means

B) The difference in means with no interval, since the groups are large

C) Whichever of mean or median gives the smaller p-value

D) The maximum of each group, since large customers matter most

Q3

What does this print?

```
orders = pd.DataFrame({"order_id": [1, 2], "order_total": [100.0, 50.0]})
lines = pd.DataFrame({"order_id": [1, 1, 1, 2], "sku": ["A", "B", "C", "A"]})
merged = orders.merge(lines, on="order_id")
print(len(merged), merged["order_total"].sum())
```

Q4

Your main result: a difference of +1.2 percentage points (95% CI −1.5 to +4.0), p = 0.39. Which sentence belongs in the report?

A) The new approach works: the rate went up by 1.2 points

B) We found no clear difference: the data are consistent with anything from 1.5 points lower to 4.0 points higher, so a gain larger than about 4 points is unlikely

C) There is no difference between the groups

D) The result is not significant, so the analysis failed

Q5

You compare how many customers "came back" after their first order. Some first orders were in November 2025 and the data end on 2025-12-31. What makes the comparison fair?

A) Count anyone who ever ordered again, whenever their first order was

B) Drop every customer whose first order was in 2025

C) Use months since signup as the outcome instead

D) Give everyone the same window (for example 180 days) and keep only customers whose first order was at least 180 days before the data end

Q6

What does this print?

```
from itertools import combinations
channels = ["organic", "paid_search", "social", "referral", "in_store"]
print(len(list(combinations(channels, 2))))
```

Q7

Sales are seasonal and the company is growing. A change happened on a known date to some products only. Which comparison is fairest?

A) The change in the affected products, against the change in unaffected products over the same months a year earlier (a difference in differences)

B) The two months before the date against the ten months after it, for the affected products

C) Affected against unaffected products, after the date only

D) The year before against the year after, for all products together

Q8

Two conversion rates, 10% and 9%. What does this print? (It is Cohen's h, the effect size for two proportions.)

```
print(round(2 * np.arcsin(np.sqrt(0.10)) - 2 * np.arcsin(np.sqrt(0.09)), 3))
```

Q9

Which caveat is specific enough to earn credit in the Communication row of the rubric?

A) More data is needed

B) Correlation is not causation

C) Customers chose whether to use the code, so those who used it may differ from those who did not in ways the data do not record

D) The results may not be accurate

Q10

What is the first sentence after the question in report.md?

A) A description of the files you loaded

B) The answer, with the main number and its uncertainty

C) A list of the methods you used

D) A thank-you to the reader

Q11

What does this print? (Run it twice: does the answer change?)

```
rng = np.random.default_rng(42)
print(rng.choice([10, 20, 30, 40], size=3).tolist())
```

Q12

You have three minutes in the showcase. What do you say first?

A) How long the analysis took

B) Every chart you made while exploring

C) The cleaning steps, in order

D) The question, then the answer with its uncertainty, pointing at the one figure

◆ JOURNAL PROMPT (FROM THE LESSON): AN END-OF-COURSE REFLECTION

Open your very first entry, from Lesson 0.1, and read it. Then write to the person who wrote it. What can you do now that you could not do then? Name three specific things. Which lesson was hardest at the time, and how does it look from here? What did the capstone teach you about how you work: where you rush, where you stall, what you enjoy? What is one question, about any data in your own life or interests, that you now know how to start answering?

SUBMISSION CHECKLIST (WITHIN ONE WEEK)

- ☐ `pinecrest_report.ipynb`: Restart and Run All succeeds from the project folder; relative paths; seeds set
- ☐ One or two figures, saved by the notebook, each with a finding-first title
- ☐ `report.md`: one page, Answer first, specific Caveats, Next questions, three-line Method
- ☐ `README.md`: what this is, how to run it, where the data is from (fictional course data), and what help I used (AI assistants, web code, classmates)
- ☐ My rubric self-score, one line per row
- ☐ Every number in the report can be found in the notebook's output
- ☐ The self-review checklist from Lesson 8.1, every box ticked honestly

NOTES

GLOSSARY

- **Unit of analysis:** what one row of your analysis table is: a store-day, a customer, a product.
- **Fair comparison:** groups that differ in the thing you care about and, as far as you can arrange, in nothing else.

- **Follow-up window:** a fixed period after an event (for example 180 days) in which an outcome is counted, the same for everyone.
- **Difference in differences:** the change in an affected group minus (or divided by) the change in an unaffected group over the same period.
- **Effect size:** how big a difference is, on a scale that does not depend on sample size: Cohen's d for means, h for proportions.
- **Multiple comparisons:** running many tests; some will look significant by chance, so plan the main one and adjust or report the rest.
- **Honest null:** a carefully measured "no clear difference", reported with the size of difference the data rule out.
- **Robustness check:** repeating the main comparison with another reasonable definition or subset to see whether it holds.
- **Reproducible:** the notebook gives the same numbers after Restart and Run All, on another machine, from relative paths.

Complete Participant Workbook · all 21 sessions · v1.0 · © 2026 Ray de la Paz